

Chapter 7. Security

Preventing unauthorized access to sensitive data is essential in any environment in which multiple users have access to the same physical or network resources. An operating system, as well as individual users, must be able to protect files, memory, and configuration settings from unwanted viewing and modification. Operating system security includes obvious mechanisms such as accounts, passwords, and file protection. It also includes less obvious mechanisms, such as protecting the operating system from corruption, preventing less privileged users from performing actions (rebooting the computer, for example), and not allowing user programs to adversely affect the programs of other users or the operating system.

In this chapter, we explain how every aspect of the design and implementation of Microsoft Windows was influenced in some way by the stringent requirements of providing robust security.

Security ratings

Having software, including operating systems, rated against well-defined standards helps the government, corporations, and home users protect proprietary and personal data stored in computer systems. The current security rating standard used by the United States and many other countries is the Common Criteria (CC). To understand the security capabilities designed into Windows, however, it's useful to know the history of the security ratings system that influenced the design of Windows: the Trusted Computer System Evaluation Criteria (TCSEC).

Trusted Computer System Evaluation Criteria

The National Computer Security Center (NCSC) was established in 1981 as part of the U.S. Department of Defense's (DoD) National Security Agency (NSA). One goal of the NCSC was to create a range of security ratings, listed in [Table 7-1](#), to indicate the degree of protection commercial operating systems, network components, and trusted applications offer. These security ratings, which can be found at <http://csrc.nist.gov/publications/history/dod85.pdf>, were defined in 1983 and are commonly referred to as the *Orange Book*.

Rating	Description
A1	Verified design
B3	Security domains
B2	Structured protection
B1	Labeled security protection
C2	Controlled access protection
C1	Discretionary access protection (obsolete)
D	Minimal protection

TABLE 7-1 TCSEC rating levels

The TCSEC standard consists of levels-of-trust ratings, where higher levels build on lower levels by adding more rigorous protection and validation requirements. No operating system meets the A1 (verified design) rating. Although a few operating systems have earned one of the B-level ratings, C2 is considered sufficient and the highest rating practical for a general-purpose operating system.

The following were the key requirements for a C2 security rating, and they are still considered the core requirements for any secure operating system:

- **A secure logon facility** This requires that users be able to be uniquely identified and that they must be granted access to the computer only after they have been authenticated in some way.
- **Discretionary access control** This allows the owner of a resource (such as a file) to determine who can access the resource and what they can do with it. The owner grants rights that permit various kinds of access to a user or to a group of users.
- **Security auditing** This affords the ability to detect and record security-related events or any attempts to create, access, or delete system resources. Logon identifiers record the identities of all users, making it easy to trace anyone who performs an unauthorized action.
- **Object reuse protection** This prevents users from seeing data that another user has deleted or from accessing memory that another user previously used and then released. For example, in some operating systems, it's possible to create a new file of a certain length and then examine the contents of the file to see data that happens to have occupied the location on the disk where the file is allocated. This data might be sensitive information that was stored in another user's file but had been deleted. Object reuse protection prevents this potential security hole by initializing all objects, including files and memory, before they are allocated to a user.

Windows also meets two requirements of B-level security:

- **Trusted path functionality** This prevents Trojan horse programs from being able to intercept users' names and passwords as they try to log on. The trusted path functionality in Windows comes in the form of its Ctrl+Alt+Delete logon-attention sequence, which cannot be intercepted by nonprivileged applications. This sequence of keystrokes, which is also known as the secure attention sequence (SAS), always displays a system-controlled Windows security screen (if a user is already logged on) or the logon screen so that would-be Trojan horses can easily be recognized. (The SAS can also be sent programmatically via the SendSAS API if Group Policy and other restrictions allow it.) A Trojan horse presenting a fake logon dialog box will be bypassed when the SAS is entered.
- **Trusted facility management** This requires support for separate account roles for administrative functions. For example, separate accounts are provided for administration (Administrators), user accounts charged with backing up the computer, and standard users.

Windows meets all these requirements through its security subsystem and related components.

The Common Criteria

In January 1996, the United States, United Kingdom, Germany, France, Canada, and the Netherlands released the jointly developed Common Criteria for Information Technology Security Evaluation (CCITSE) specification. CCITSE, usually referred to as the Common Criteria (CC), is the recognized multinational standard for product security evaluation. The CC home page is at <http://www.niap-ccevs.org/cc-scheme>.

The CC is more flexible than the TCSEC trust ratings and has a structure closer to the ITSEC standard than to the TCSEC standard. The CC includes the concept of a Protection Profile (PP), used to collect security requirements into easily specified and compared sets, and the concept of a Security Target (ST), which contains a set of security requirements that can be made by reference to a PP. The CC also defines a range of seven Evaluation Assurance Levels (EALs), which indicate a level of confidence in the certification. In this way, the CC (like the ITSEC standard before it) removes the link between functionality and assurance level that was present in TCSEC and earlier certification schemes.

Windows 2000, Windows XP, Windows Server 2003, and Windows Vista Enterprise all achieved Common Criteria certification under the Controlled Access Protection Profile (CAPP). This is roughly equivalent to a TCSEC C2 rating. All received a rating of EAL 4+, the “plus” denoting “flaw remediation.” EAL 4 is the highest level recognized across national boundaries.

In March 2011, Windows 7 and Windows Server 2008 R2 were evaluated as meeting the requirements of the US Government Protection Profile for General-Purpose Operating Systems in a Networked Environment, version 1.0 (GPOSPP)

(http://www.commoncriteriaportal.org/files/ppfiles/pp_gpospp_v1.0.pdf). The certification includes the Hyper-V hypervisor. Again, Windows achieved Evaluation Assurance Level 4 with flaw remediation (EAL 4+). The validation report can be found at

http://www.commoncriteriaportal.org/files/epfiles/st_vid10390-vr.pdf, and the description of the security target, giving details of the requirements satisfied, can be found at

http://www.commoncriteriaportal.org/files/epfiles/st_vid10390-st.pdf. Similar certifications were achieved by Windows 10 and Windows Server 2012 R2 in June 2016. The report can be found at http://www.commoncriteriaportal.org/files/epfiles/cr_windows10.pdf.

Security system components

These are the core components and databases that implement Windows security. (All files mentioned are in the %SystemRoot%\System32 directory unless otherwise specified.)

- **Security reference monitor (SRM)** This component in the Windows executive (Ntoskrnl.exe) is responsible for defining the access token data structure to represent a security context, performing security access checks on objects, manipulating privileges (user rights), and generating any resulting security audit messages.
- **Local Security Authority Subsystem Service (Lsass)** This user-mode process runs the image Lsass.exe that is responsible for the local system security policy (such as which users are allowed to log on to the machine, password policies, privileges granted to users and groups, and the system security auditing settings), user authentication, and sending security audit messages to the event log. The Local Security Authority service (Lsasrv.dll), a library that Lsass loads, implements most of this functionality.
- **LsIso.exe** This is used by Lsass (if so configured on supported Windows 10 and Server 2016 systems), also known as Credential Guard (see the upcoming “[Credential Guard](#)” section for more on Credential Guard), to store users’ token hashes instead of keeping them in Lsass’s memory. Because LsIso.exe is a Trustlet (Isolated User Mode process) running in VTL 1, no normal process—not even the normal kernel—can access the address space of this process. Lsass itself stores an encrypted blob of the password hash needed when it communicates with LsIso (via ALPC).
- **Lsass policy database** This database contains the local system security policy settings. It is stored in the registry in an ACL-protected area under HKLM\SECURITY. It includes such information as what domains are entrusted to authenticate logon attempts, who has permission to access the system and how (interactive, network, and service logons), who is assigned which privileges, and what kind of security auditing is to be performed. The Lsass policy database also stores “secrets” that include logon information used for cached domain logons and Windows service user-account logons. (See Chapter 9, “Management mechanisms,” in *Windows Internals Part 2* for more information on Windows services.)
- **Security Accounts Manager (SAM)** This service is responsible for managing the database that contains the user names and groups defined on the local machine. The SAM service, which is

implemented in Samsrv.dll, is loaded into the Lsass process.

- **SAM database** This database contains the defined local users and groups along with their passwords and other attributes. On domain controllers, the SAM does not store the domain-defined users, but stores the system's administrator recovery account definition and password. This database is stored in the registry under HKLM\SAM.
- **Active Directory** This is a directory service that contains a database that stores information about objects in a domain. A *domain* is a collection of computers and their associated security groups that are managed as a single entity. Active Directory stores information about the objects in the domain, including users, groups, and computers. Password information and privileges for domain users and groups are stored in Active Directory, which is replicated across the computers that are designated as domain controllers of the domain. The Active Directory server, implemented as Ntdsa.dll, runs in the Lsass process. For more information on Active Directory, see Chapter 10, "Networking," in Part 2.
- **Authentication packages** These include dynamic link libraries (DLLs) that run in the context of both Lsass process and client processes and implement Windows authentication policy. An authentication DLL is responsible for authenticating a user by checking whether a given user name and password match (or whatever mechanism was used to provide credentials), and if so, returning to Lsass information detailing the user's security identity, which Lsass uses to generate a token.
- **Interactive logon manager (Winlogon)** This is a user-mode process running Winlogon.exe that is responsible for responding to the SAS and for managing interactive logon sessions. Winlogon creates a user's first process when the user logs on, for example.
- **Logon user interface (LogonUI)** This is a user-mode process running the image LogonUI.exe that presents users with the user interface they can use to authenticate themselves on the system. LogonUI uses credential providers to query user credentials through various methods.
- **Credential providers (CPs)** These are in-process COM objects that run in the LogonUI process (started on demand by Winlogon when the SAS is performed) and used to obtain a user's name and password, smartcard PIN, biometric data (such as a fingerprint), or other identification mechanism. The standard CPs are authui.dll, SmartcardCredentialProvider.dll, BioCredProv.Dll, and FaceCredentialProvider.dll, a face-detection provider added in Windows 10.
- **Network logon service (Netlogon)** This is a Windows service (Netlogon.dll, hosted in a standard SvcHost) that sets up the secure channel to a domain controller, over which security requests—such as an interactive logon (if the domain controller is running Windows NT 4) or LAN Manager and NT LAN Manager (v1 and v2) authentication validation—are sent. Netlogon is also used for Active Directory logons.
- **Kernel Security Device Driver (KSecDD)** This is a kernel-mode library (%SystemRoot%\System32\Drivers\Ksecdd.sys) of functions that implement the advanced local procedure call (ALPC) interfaces that other kernel mode security components, including the Encrypting File System (EFS), use to communicate with Lsass in user mode.
- **AppLocker** This mechanism allows administrators to specify which executable files, DLLs, and scripts can be used by specified users and groups. AppLocker consists of a driver (%SystemRoot%\System32\Drivers\AppId.sys) and a service (AppIdSvc.dll) running in a standard SvcHost process.

[Figure 7-1](#) shows the relationships among some of these components and the databases they manage.

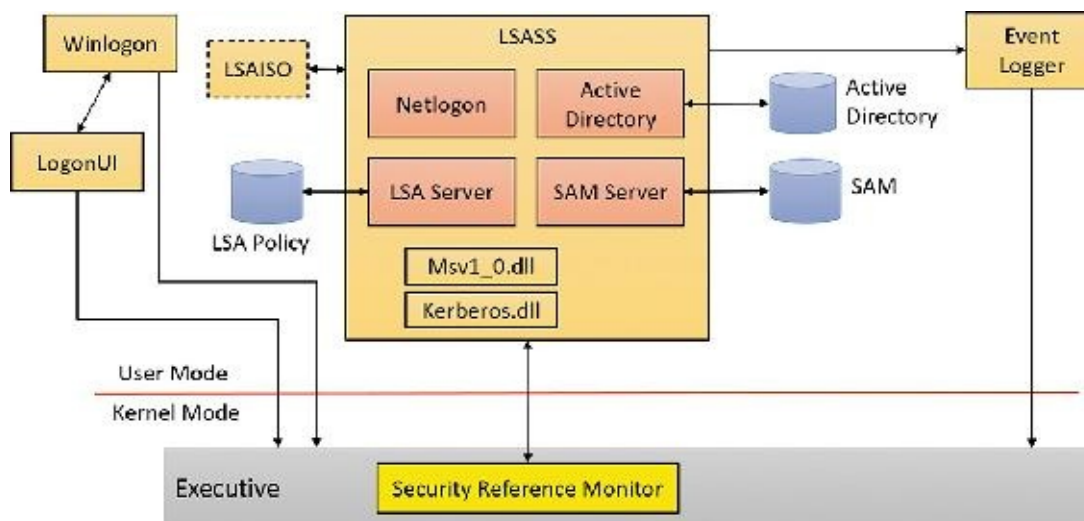


FIGURE 7-1 Windows security components.

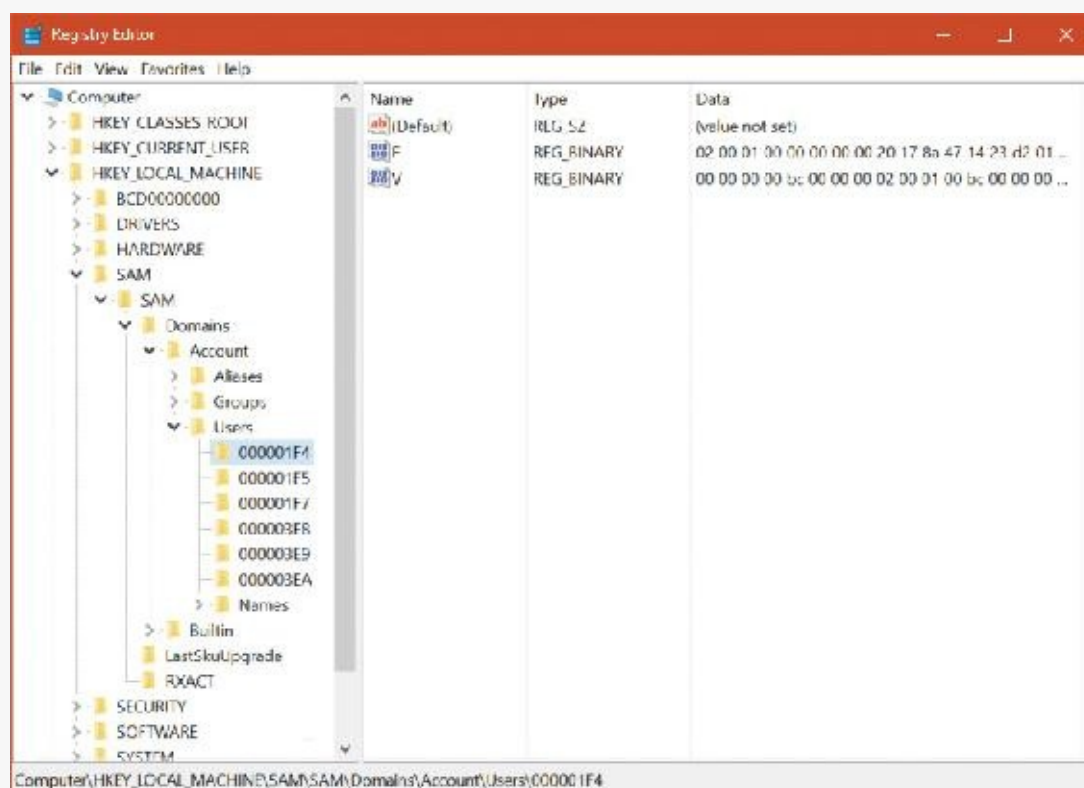
EXPERIMENT: Looking inside HKLM\SAM and HKLM\Security

The security descriptors associated with the SAM and Security keys in the registry prevent access by any account other than the local system account. One way to gain access to these keys for exploration is to reset their security, but that can weaken the system's security. Another way is to execute Regedit.exe while running as the local system account. This can be done using the PSEXec tool from Sysinternals with the -S option, as shown here:

[Click here to view code image](#)

```
C:\>psexec -s -i -d c:\windows\regedit.exe
```

The -i switch instructs PSEXec to run the target executable under the interactive window station. Without it, the process would run in a non-interactive window station, on an invisible desktop. The -d switch just indicates PSEXec should not wait until the target process exits.



The SRM, which runs in kernel mode, and Lsass, which runs in user mode, communicate using the ALPC facility described in Chapter 8, “System mechanisms,” in Part 2. During system initialization, the SRM creates a port, named `SeRmCommandPort`, to which Lsass connects. When the Lsass process starts, it creates an ALPC port named `SeLsaCommandPort`. The SRM connects to this port, resulting in the creation of private communication ports. The SRM creates a shared memory section for messages longer than 256 bytes, passing a handle in the connect call. Once the SRM and Lsass connect to each other during system initialization, they no longer listen on their respective connect ports. Therefore, a later user process has no way to connect successfully to either of these ports for malicious purposes. The connect request will never complete.

Virtualization-based security

It is common to refer to the kernel as *trusted*, due to its inherently higher level of privilege and isolation from user-mode applications. Yet, countless third-party drivers are written each month—Microsoft has stated that a million unique driver hashes are seen through telemetry, monthly! Each of these can contain any number of vulnerabilities, not to mention purposefully malicious kernel-mode code. In such a reality, the idea that the kernel is a small, protected component, and that user-mode applications are “safe” from attack, is clearly an unrealized ideal. This state of affairs leads to an inability to fully trust the kernel, and leaves key user-mode applications, which may contain highly private user data, open to compromise from other malicious user-mode applications (which exploit buggy kernel-mode components) or malicious kernel-mode programs.

As discussed in [Chapter 2](#), “[System architecture](#),” Windows 10 and Server 2016 include a virtualization-based security (VBS) architecture that enables an additional orthogonal level of trust: the virtual trust level (VTL). In this section, you will see how Credential Guard and Device Guard leverage VTLs to protect user data and provide an additional hardware-trust-based layer of security for digital code-signing purposes. At the end of this chapter, you will also see how Kernel Patch Protection (KPP) is provided through the PatchGuard component and enhanced by the VBS-powered HyperGuard technology.

As a reminder, normal user-mode and kernel code runs in VTL 0 and is unaware of the existence of VTL 1. This means anything placed at VTL 1 is hidden and inaccessible to VTL 0 code. If malware is able to penetrate the normal kernel, it still cannot gain access to anything stored in VTL 1, including even user-mode code running in VTL 1 (which is called Isolated User Mode). [Figure 7-2](#) shows the main VBS components we’ll be looking at in this section:

- Hypervisor-Based Code Integrity (HVCI) and Kernel-Mode Code Integrity (KMCI), which power Device Guard
- LSA (Lsass.exe) and isolated LSA (LsaIso.exe), which power Credential Guard

Additionally, recall that the implementation of Trustlets, which run in IUM, was shown in [Chapter 3](#), “[Process and jobs](#).”

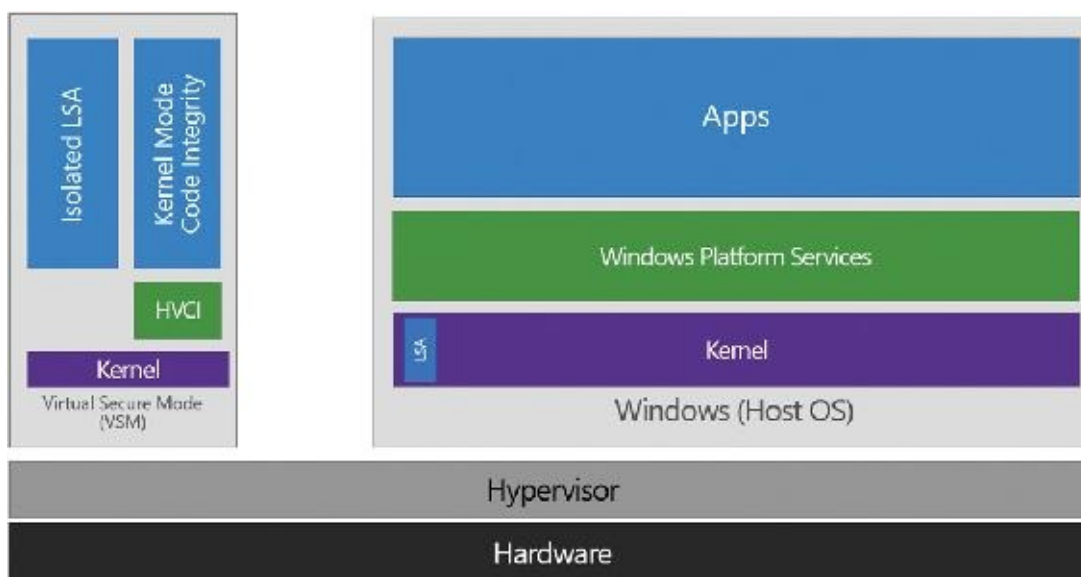


FIGURE 7-2 VBS components.

Of course, like any trusted component, VTL 1 also makes certain assumptions that the components it depends on can also be trusted. As such, VTL 1 requires Secure Boot (and thus, firmware) to function correctly, the hypervisor to not have been compromised, and hardware elements such as the IOMMU and Intel Management Engine to be free of VTL 0–accessible vulnerabilities. For more information on the hardware chain of trust and boot-related security technologies, see Chapter 11, “Startup and shutdown,” in Part 2.

Credential Guard

To understand the security boundary and protection that Credential Guard provides, it is important to understand the various components that provide access to a user’s resources and data or login capabilities on a networked environment:

- **Password** This is the primary credential used by interactive users to identify themselves on the machine. This credential is used for authentication and to derive the other components of the credential model. It is the most highly sought after piece of a user’s identity.
- **NT one-way function (NT OWF)** This is a hash used by legacy components to identify the user (after a successful password logon) using the NT LAN Manager (NTLM) protocol. While modern networked systems no longer use NTLM to authenticate the user, many local components still do, as do some types of legacy network components (such as NTLM-based authenticating proxies). Because NTOWF is an MD4 hash, its algorithmic complexity in the face of today’s hardware, and its lack of anti-repeatability protection, means that intercepting the hash leads to instant compromise and even possible recovery of the password.
- **Ticket-granting ticket (TGT)** This is the equivalent of the NTOWF when a much more modern remote authentication mechanism is used: Kerberos. This is the default on Windows Active Directory–based domains and is enforced on Server 2016. The TGT and a corresponding key are provided to the local machine after a successful logon (just like the NTOWF on NTLM), and intercepting both components will result in instant compromise of the user’s credentials, although reuse and password recovery will not be possible.

Without Credential Guard enabled, some or all of these components of a user’s authentication credentials are present in the memory of Lsass.



Note

To enable Credential Guard on Windows 10 Enterprise and Server 2016 editions, open the Group Policy editor (**gpedit.msc**), choose **Computer Configuration**, select **Administrative Templates**, choose **System**, choose **Device Guard**, and select **Turn on Virtualization Based Security**. In the top-left part of the dialog box that appears, select **Enabled**. Finally, select one of the **Enabled** options in the Credential Guard Configuration combo box.

Protecting the password

The password, encrypted with a local symmetric key, is stored to provide single sign-on (SSO) capabilities over protocols such as digest authentication (WDigest, used for HTTP-based authentication since Windows XP) or Terminal Services/RDP. As these protocols use plaintext authentication, the password must be kept in memory, which is then accessible through code injection, debugger, or other exploit techniques, and decrypted. Credential Guard cannot change the nature of these inherently unsafe protocols. Therefore, the only possible solution, which Credential Guard employs, is to disable SSO functionality for such protocols. This causes a loss of compatibility and forces the user to re-authenticate.

Obviously, a preferable solution is to remove the usage of a password completely, which Windows Hello, described in the “Windows Hello” section later in this chapter, allows. Authenticating with biometric credentials such as a user’s face or fingerprint removes the need to ever type a password, securing the interactive credential against hardware key loggers, kernel sniffing/hooks tools, and user mode-based spoofing applications. If the user never has a password to type, there is no password to steal. Another similar secure credential is the combination of a smart card and associated PIN. While a PIN may be stolen as its typed in, the smart card is a physical element whose key cannot be intercepted without a complex hardware-based attack. This is a type of two-factor authentication (TFA), of which many other implementations exist.

Protecting the NTOWF/TGT key

Even with protected interactive credentials, a successful login results in a domain controller’s key distribution center (KDC) returning the TGT and its key, as well as the NTOWF for legacy applications. Later, the user simply uses the NTOWF for accessing legacy resources and uses the TGT and its key to generate a service ticket. This can then be used to access remote resources (such as files on a share), as shown in [Figure 7-3](#).

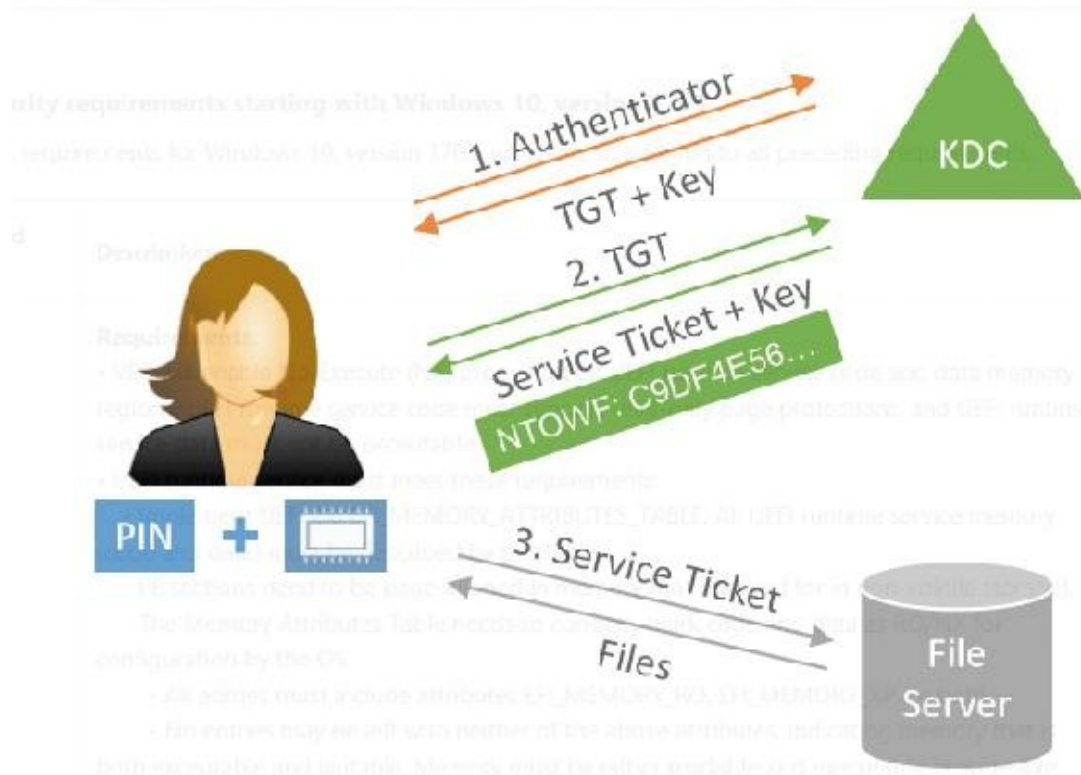


FIGURE 7-3 Accessing remote resources.

Thus, with either the NTOWF or the TGT and its key (stored in Lsass) in the attacker’s hands, access to resources is possible even without the smart card, PIN, or user’s face or fingerprint. Protecting Lsass from access by an attacker is thus one option that can be used, and which is possible using the Protected Process Light (PPL) architecture described in [Chapter 3](#).

Lsass can be configured to run protected by setting the DWORD value RunAsPPL in the HKLM\System\CurrentControlSet\Control\Lsa registry key to 1. (This is not a default option, as legitimate third-party authentication providers [DLLs] load and execute in the context of Lsass, which would not be possible if Lsass would run protected.) Unfortunately, while this protection does guard the NTOWF and TGT key from user-mode attackers, it does not protect against kernel attackers or user-mode attackers that leverage vulnerabilities in any of the millions of drivers that are produced monthly. Credential Guard solves this problem by using another process, LsaIso.exe, which runs as a Trustlet in VTL 1. This process therefore stores the user’s secrets in its memory, not in Lsass.

Secure communication

As shown in [Chapter 2](#), VTL 1 has a minimal attack surface, as it does not have the full regular “NT” kernel, nor does it have any drivers or access to I/O of hardware of any kind. As such, isolated LSA, which is a VTL 1 Trustlet, cannot directly communicate with the KDC. This is still the responsibility of the Lsass process, which serves as a proxy and protocol implementer, communicating with the KDC to authenticate the user and to receive the TGT and the key and NTOWF, as well as communicating with the file server by using service ticket. This seemingly results in a problem: the TGT and its key/NTOWF transiently pass through Lsass during authentication, and the TGT and its key are somehow available to Lsass for the generation of service tickets. This leads to two questions: How does Lsass send and receive the secrets from isolated ISA, and how can we prevent an attacker from doing the same?

To answer the first question, recall that [Chapter 3](#), “[Processes and jobs](#),” described which services are available to Trustlets. One was the Advanced Local Procedure Call (ALPC), which the Secure Kernel supports by proxying the NtAlpc* calls to the Normal Kernel. Then, the Isolated User Mode

environment implements support for the RPC runtime library (Rpcrt4.dll) over the ALPC protocol, which allows a VTL 0 and VTL 1 application to communicate using local RPC just like any other application and service. In [Figure 7-4](#), which shows Process Explorer, you can see the LsaIso.exe process, which has a handle to the LSA_ISO_RPC_SERVER ALPC port. This is used to communicate with the Lsass.exe process. (See Chapter 8 in Part 2 for more information on ALPC.)

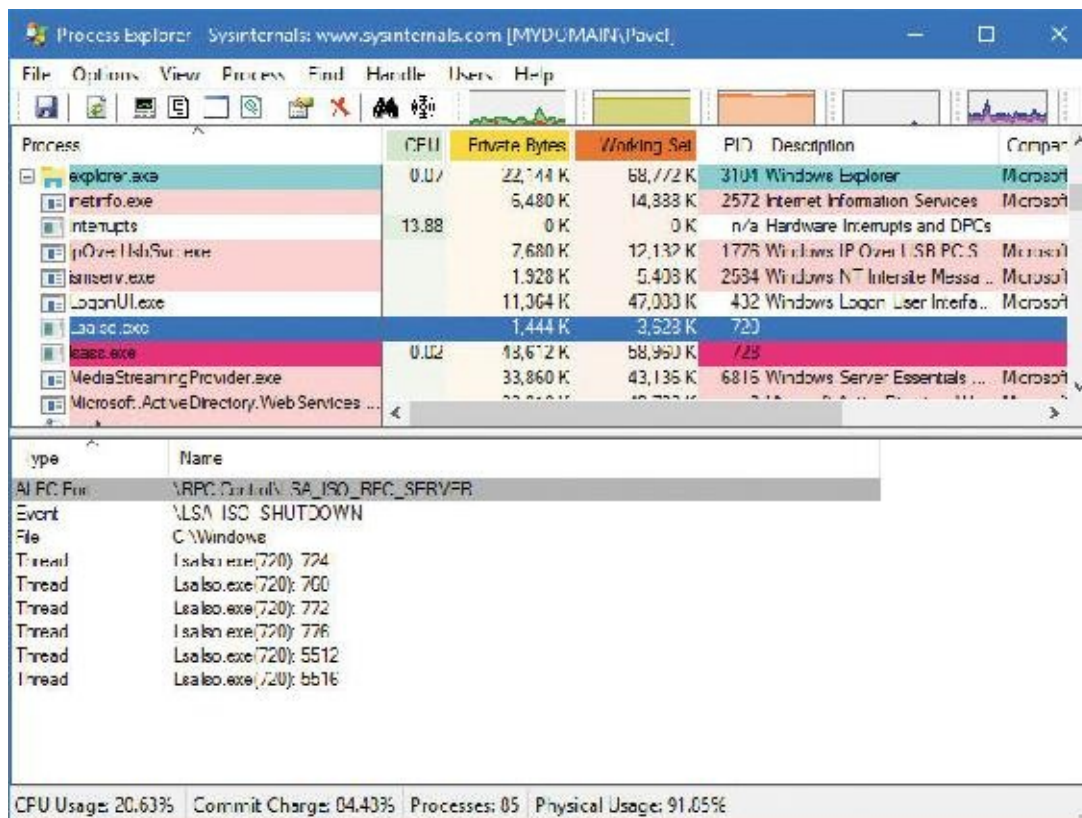


FIGURE 7-4 LsaIso.exe and its ALPC port.

To answer the second question, some understanding of cryptographic protocols and challenge/response models is required. If you're already familiar with some of the basic concepts of SSL/TLS technology and its use in Internet communications to prevent man-in-the-middle (MitM) attacks, you can think of the KDC and isolated LSA protocol in a similar way. Although Lsass sits in the middle as a proxy would, it only sees encrypted traffic between the KDC and isolated LSA, without the ability to understand its contents. Because isolated LSA establishes a local "session key," which only lives in VTL 1, and then uses a secure protocol to send this session key encrypted with yet another key, which only the KDC has, the KDC can then respond with the TGT and its key after encrypting it with the isolated LSA session key. Therefore, Lsass sees an encrypted message to the KDC (which it can't decrypt) and an encrypted message from the KDC (which it can't decrypt).

This model can even be used to protect legacy NTLM authentication, which is based on a challenge/response model. For example, when a user logs in with a plaintext credential, LSA sends it to isolated LSA, which then encrypts it with its session key and returns the encrypted credential to Lsass. Later, when an NTLM challenge/response is required, Lsass sends the NTLM challenge and the previously encrypted credentials to isolated LSA. At this point, only isolated LSA has the encryption key, so it decrypts the credentials and generates an NTLM response based on the challenge.

Note, however, that four possible attacks exist in this model:

- If the machine is already physically compromised, the plaintext password can be intercepted either as it is inputted or as it is sent to isolated LSA (if Lsass is already compromised). Using Windows Hello can mitigate against this.

- As mentioned, NTLM does not have anti-replay properties. Therefore, if the NTLM response is captured, it can be replayed for the same challenge. Alternatively, if the attacker can compromise Lsass after logon, it can capture the encrypted credential and force isolated LSA to generate new NTLM responses for arbitrary NTLM challenges. This attack, however, only works until reboot, because isolated LSA generates a new session key at that point.
- In the case of Kerberos logon, the NTOWF (which is not encrypted) can be intercepted and then reused, just like in a standard pass-the-hash attack. Again, however, this requires an already compromised machine (or physical network interception).
- The user, with physical access, may be able to disable Credential Guard. In this situation, the legacy authentication model is used (a so-called “downgrade attack”), and older attack models can now be employed.

UEFI lock

Because disabling Credential Guard (which is ultimately nothing more than a registry setting) is trivial for an attacker, Secure Boot and UEFI can be leveraged to prevent a non-physically present administrator (such as malware with admin rights) from disabling Credential Guard. This is done by enabling Credential Guard with UEFI Lock. In this mode, an EFI runtime variable is written to firmware memory and a reboot is required. At the reboot, the Windows boot loader, which still operates in EFI Boot Services mode, will write an EFI boot variable (which has the property of not being readable or writeable once EFI Boot Services mode is exited) to record the fact that Credential Guard is enabled. Additionally, a Boot Configuration Database (BCD) option will be recorded.

When the kernel boots, it will automatically rewrite the required Credential Guard registry key in the presence of the BCD option and/or UEFI runtime variable. If the BCD option is deleted by an attacker, BitLocker (if enabled) and TPM-based remote attestation (if enabled) will detect the change and require physical input of the admin’s recovery key before booting, which will then restore the BCD option based on the UEFI runtime variable. If the UEFI runtime variable is deleted, the Windows boot loader will restore it based on the UEFI boot variable. As such, without special code to delete the UEFI boot variable—which can only be done in EFI Boot Services mode—there is no way to disable Credential Guard in UEFI lock mode.

The only such code that exists is in a special Microsoft binary called SecComp.efi. This must be downloaded by the administrator, who must then either boot the computer from an alternate EFI-based device and manually execute it (which will require the BitLocker recovery key as well as physical access) or modify the BCD (which will require the BitLocker recovery key). At the reboot, SecComp.efi will require user confirmation while in UEFI mode (which can only be done by a physical user).

Authentication policies and armored Kerberos

Using a security model of “secure, unless already compromised before logon or by a physical administrator” is definitely an improvement over the traditional non-Credential Guard–based security model. However, some enterprises and organizations may want an even stronger security guarantee: that even a compromised machine cannot be used to fake or replay a user’s credentials, and that if a user’s credentials *have* been compromised, they cannot be used outside of specific systems. By leveraging a Server 2016 feature called Authentication Policies, and armored Kerberos, Credential Guard can operate in this heightened security mode.

In this mode, the VTL 1 Secure Kernel will collect, using the TPM (a file on disk can also be used, but makes the security moot), a special machine ID key. This key is then used to generate a machine TGT key during the initial domain join operation as the machine is provisioned (obviously, it is important to ensure

the machine is in a trusted state during provisioning), and this TGT key is sent to the KDC. Once configured, when the user logs in with his or her credential, it is combined with the machine's credential (which only isolated LSA has access to), which forms a proof-of-origin key. The KDC will then reply with the NTOWF and user TGT and its key after encrypting it with the proof-of-origin key. In this mode, two security guarantees are provided:

- **The user is authenticating from a known machine** If the user, or an attacker, has the original credentials, and attempts to use them on a different machine, its TPM-based machine credential will be different.
- **The NTLM response/user ticket is coming from isolated LSA and has not been manually generated from Lsass** This guarantees that Credential Guard is enabled on the machine, even if the physical user can disable it in some way.

Unfortunately, once again, if the machine is compromised in such a way that the proof-of-origin-encrypted KDC response that contains the user TGT and its key is intercepted, it can be stored and used to request session key-encrypted service tickets from isolated LSA. This can then be sent to a file server (for example) to access it until a reboot is issued to wipe the session key. As such, on a system with Credential Guard, it is recommended to reboot each time a user logs off. Otherwise, an attacker may be able to issue valid tickets even after the user is no longer present.

Future improvements

As discussed in [Chapter 2](#) and [Chapter 3](#), the Secure Kernel in VTL 1 is currently undergoing improvements to add support for specialized classes of PCI and USB hardware, which can exclusively be communicated with only through the hypervisor and VTL 1 code using the Secure Device Framework (SDF). Combined with BioIso.exe and FsIso.exe, which are new Trustlets to securely obtain biometric data and video frames (from a webcam), a VTL 0 kernel mode-based component cannot intercept the contents of a Windows Hello authentication attempt (which we've classified as safe compared to a user's plaintext password, but still technically capturable through custom driver-based interception). Once released, Windows Hello credentials will be guaranteed at the hardware level to not ever be available to VTL 0. In this mode, Lsass will not need to be involved in a Windows Hello authentication. Isolated LSA will obtain the credentials directly from the isolated biometrics or isolated frame service.



Note

The Secure Driver Framework (SDF) is the WDF-equivalent for VTL 1 drivers. This framework is not currently public, but is shared with Microsoft partners only for creating VTL 1 drivers.

Device Guard

While Credential Guard is concerned with safeguarding the user's credentials, Device Guard has a completely different goal: protecting the user's machine itself from different kinds of software- and hardware-based attacks. Device Guard leverages the Windows Code Integrity services, such as Kernel-Mode Code Signing (KMCS) and User-Mode Code Integrity (UMCI), and strengthens them through HyperVisor Code Integrity (HVCI). (See Chapter 8 in Part 2 for more information on Code Integrity.)

Additionally, Device Guard is fully configurable, thanks to Custom Code Integrity (CCI) and signing policies that are protected by Secure Boot and defined by the enterprise administrator. These policies, which are explained in Chapter 8, allow the enforcement of inclusion/exclusion lists that are based on cryptographically sound information (such as certificate signers or SHA-2 hashes) instead of file paths or

file names as with AppLocker's policies. (See the section "[AppLocker](#)" later in this chapter for more on AppLocker.)

Therefore, while we won't describe here the different ways in which Code Integrity policies can be defined and customized, we will show how Device Guard enforces whatever these policies may be set to, through the following guarantees:

- **If kernel-mode code signing is enforced, only signed code can load, regardless of the kernel itself being compromised** This is because the kernel-loading process will notify the Secure Kernel in VTL 1 whenever it loads a driver, and only successfully load it once HVCI has validated its signature.
- **If kernel-mode code signing is enforced, signed code cannot be modified once loaded, even by the kernel itself** This is because the executable code pages will be marked as read-only through the hypervisor's Second Level Address Translation (SLAT) mechanism, which is further explained in Chapter 8 in Part 2.
- **If kernel-mode code signing is enforced, dynamically allocated code is prohibited (a tautology of the first two bullets)** This is because the kernel does not have the ability to allocate executable entries in the SLAT page table entries, even though the kernel's page tables themselves may mark such code as executable.
- **If kernel-mode code signing is enforced, UEFI runtime code cannot be modified, even by other UEFI runtime code or by the kernel itself** Additionally, Secure Boot should already have validated that this code was signed at the time it was loaded. (Device Guard relies on this assumption.) Furthermore, UEFI runtime data cannot be made executable. This is done by reading all the UEFI runtime code and data, enforcing the correct permissions, and duplicating them in the SLAT page table entries, which are protected in VTL 1.
- **If kernel-mode code signing is enforced, only kernel-mode (ring 0) signed code can execute** This may once again sound like a tautology of the first three bullets, but consider signed ring 3 code. Such code is valid from UMCI's perspective and has been authorized as executable code in the SLAT page table entries. The Secure Kernel relies on the Mode-Based Execution Control (MBEC) feature, if present in hardware, which enhances the SLAT with a user/kernel executable bit, or the hypervisor's software emulation of this feature, called Restricted User Mode (RUM).
- **If user-mode code signing is enforced, only signed user-mode images can be loaded** This means all executable processes must be signed (.exe) files as well as the libraries they load (.dll).
- **If user-mode code signing is enforced, the kernel does not allow user-mode applications to make existing executable code pages writable** Obviously, it is impossible for user-mode code to allocate executable memory or to modify existing memory without asking the kernel permission. As such, the kernel can apply its usual enforcement rules. But even in the case of a compromised kernel, the SLAT ensures that no user-mode pages will be executable without the Secure Kernel's knowledge and approval, and that such executable pages can never be writeable.
- **If user-mode code signing is enforced, and hard code guarantees are requested by the signing policy, dynamically allocated code is prohibited** This is an important distinction from the kernel scenarios. By default, signed user-mode code is allowed to allocate additional executable memory to support JIT scenarios unless a special enhanced key usage (EKU) is present in the application's certificate, which serves as a dynamic code generation entitlement. At present, NGEN.EXE (.NET Native Image Generation) has this EKU, which allows IL-only .NET executables to function even in this mode.

- **If user-mode PowerShell constrained language mode is enforced, all PowerShell scripts that use dynamic types, reflection, or other language features that allow the execution of arbitrary code and/or marshalling to Windows/.NET API functions must also be signed** This prevents possibly malicious PowerShell scripts from escaping constrained mode.

SLAT page table entries are protected in VTL 1 and contain the “ground truth” for what permissions a given page of memory can have. By withholding the executable bit as needed, and/or withholding the writable bit from existing executable pages (a security model known as W^X , pronounced *double-you xor ex*), Device Guard moves all code-signing enforcement into VTL 1 (in a library called SKCI.DLL, or Secure Kernel Code Integrity).

Additionally, even if not configured explicitly on the machine, Device Guard operates in a third mode if Credential Guard is enabled by enforcing that all Trustlets have a specific Microsoft signature with a certificate that includes the Isolated User Mode ECU. Otherwise, an attacker with ring 0 privileges could attack the regular KMCS mechanism and load a malicious Trustlet to attack the isolated LSA component. Furthermore, all user-mode code-signing enforcements are active for the Trustlet, which executes in hard code guarantees mode.

Finally, as a performance optimization, it is important to understand that the HVCI mechanism will not reauthenticate every single page when the system resumes from hibernation (S4 sleep state). In some cases, the certificate data may not even be available. Even if this were the case, the SLAT data must be reconstructed, which means that the SLAT page table entries are stored in the hibernation file itself. As such, the hypervisor needs to trust the hibernation file has not been modified in any way. This is done by encrypting the hibernation file with a local machine key that is stored in the TPM. Unfortunately, without a TPM present, this key must be stored in a UEFI runtime variable, which allows a local attacker to decrypt the hibernation file, modify it, and re-encrypt it.

Protecting objects

Object protection and access logging are the essence of discretionary access control and auditing. The objects that can be protected on Windows include files, devices, mailslots, pipes (named and anonymous), jobs, processes, threads, events, keyed events, event pairs, mutexes, semaphores, shared memory sections, I/O completion ports, LPC ports, waitable timers, access tokens, volumes, window stations, desktops, network shares, services, registry keys, printers, Active Directory objects, and so on—theoretically, anything managed by the executive object manager. In practice, objects that are not exposed to user mode (such as driver objects) are usually not protected. Kernel-mode code is trusted and usually uses interfaces to the object manager that do not perform access checking. Because system resources that are exported to user mode (and hence require security validation) are implemented as objects in kernel mode, the Windows object manager plays a key role in enforcing object security.

You can view object protection with the WinObj Sysinternals tool (for named objects), shown in [Figure 7-5](#). [Figure 7-6](#) shows the Security property page of a section object in the user’s session. Although files are the resources most commonly associated with object protection, Windows uses the same security model and mechanism for executive objects as it does for files in the file system. As far as access controls are concerned, executive objects differ from files only in the access methods supported by each type of object.

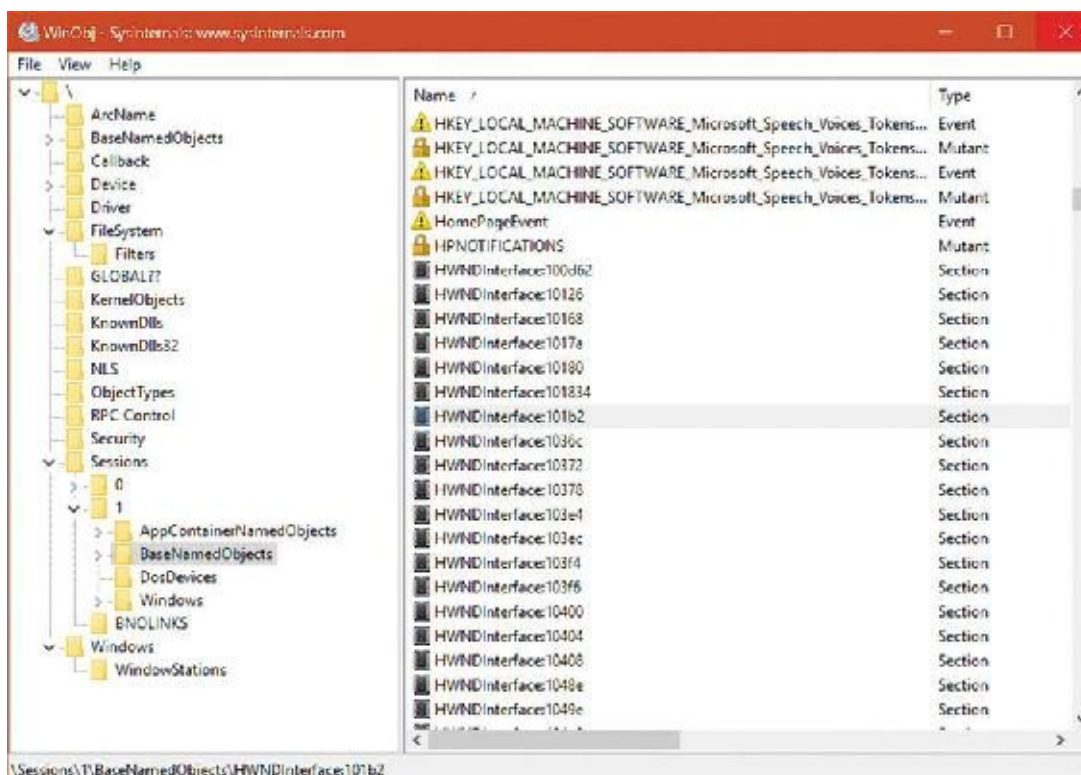


FIGURE 7-5 WinObj with a section object selected.

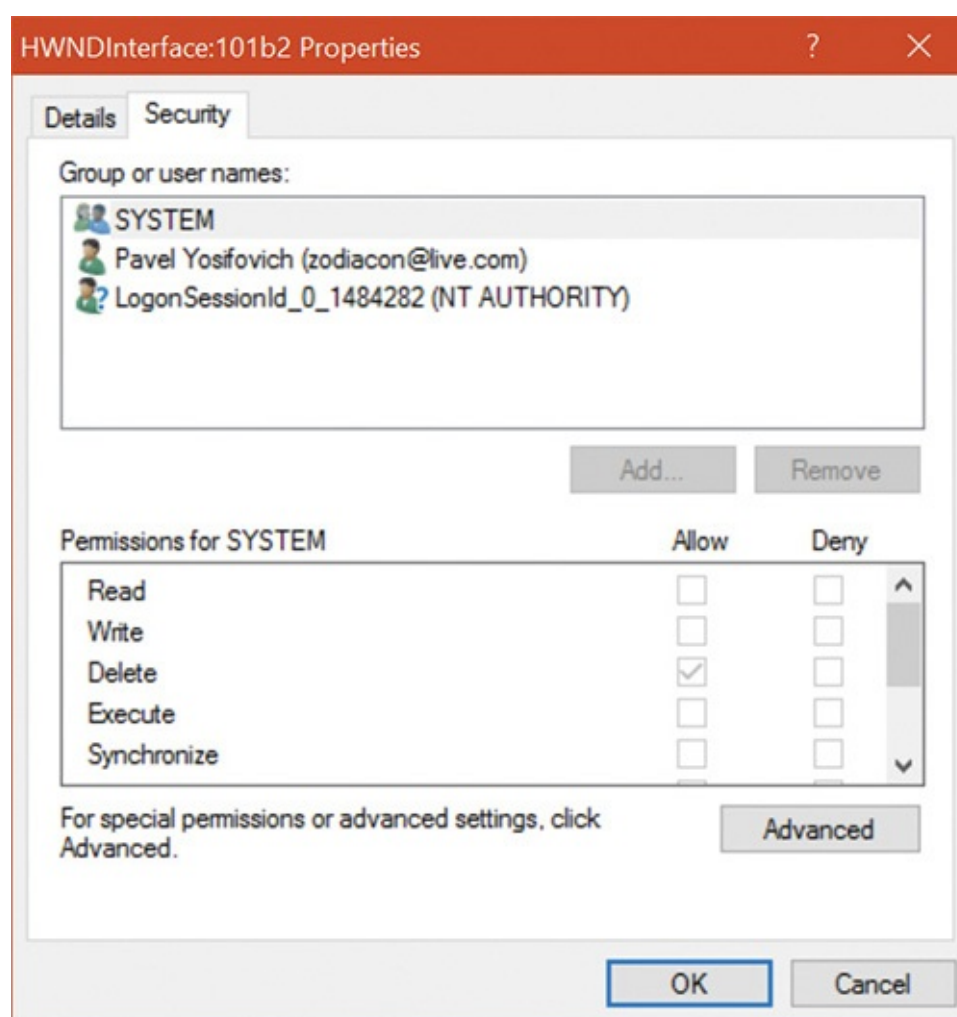


FIGURE 7-6 An executive object and its security descriptor, viewed by WinObj.

What is shown in [Figure 7-6](#) is actually the object’s discretionary access control list (DACL). We will describe DACLs in detail in the section “[Security descriptors and access control.](#)”

You can use Process Explorer to view the security properties of objects by double-clicking a handle in the lower pane view (when configured to show handles). This has the added benefit of displaying objects that are unnamed. The Property page shown is the same in both tools, as the page itself is provided by Windows.

To control who can manipulate an object, the security system must first be sure of each user's identity. This need to guarantee the user's identity is the reason that Windows requires authenticated logon before accessing any system resources. When a process requests a handle to an object, the object manager and the security system use the caller's security identification and the object's security descriptor to determine whether the caller should be assigned a handle that grants the process access to the object it desires.

As discussed later in this chapter, a thread can assume a different security context than that of its process. This mechanism is called *impersonation*. When a thread is impersonating, security validation mechanisms use the thread's security context instead of that of the thread's process. When a thread isn't impersonating, security validation falls back on using the security context of the thread's owning process. It's important to keep in mind that all the threads in a process share the same handle table, so when a thread opens an object—even if it's impersonating—all the threads of the process have access to the object.

Sometimes, validating the identity of a user isn't enough for the system to grant access to a resource that should be accessible by the account. Logically, one can think of a clear distinction between a service running under the Alice account and an unknown application that Alice downloaded while browsing the Internet. Windows achieves this kind of intra-user isolation with the Windows integrity mechanism, which implements integrity levels. The Windows integrity mechanism is used by User Account Control (UAC) elevations, User Interface Privilege Isolation (UIPI) and AppContainers, all described later in this chapter.

Access checks

The Windows security model requires that a thread specify up front, at the time that it opens an object, what types of actions it wants to perform on the object. The object manager calls the SRM to perform access checks based on a thread's desired access. If the access is granted, a handle is assigned to the thread's process with which the thread (or other threads in the process) can perform further operations on the object.

One event that causes the object manager to perform security access validation is when a thread opens an existing object using a name. When an object is opened by name, the object manager performs a lookup of the specified object in the object manager namespace. If the object isn't located in a secondary namespace, such as the configuration manager's registry namespace or a file system driver's file system namespace, the object manager calls the internal function `ObpCreateHandle` once it locates the object. As its name implies, `ObpCreateHandle` creates an entry in the process handle table that becomes associated with the object. `ObpCreateHandle` first calls `ObpGrantAccess` to see if the thread has permission to access the object. If so, `ObpCreateHandle` calls the executive function `ExCreateHandle` to create the entry in the process handle table. `ObpGrantAccess` calls `ObCheckObjectAccess` to initiate the security access check.

`ObpGrantAccess` passes to `ObCheckObjectAccess` the security credentials of the thread opening the object, the types of access to the object that the thread is requesting (read, write, delete, and so forth, including object-specific operations), and a pointer to the object. `ObCheckObjectAccess` first locks the object's security descriptor and the security context of the thread. The object security lock prevents another thread in the system from changing the object's security while the access check is in

progress. The lock on the thread's security context prevents another thread (from that process or a different process) from altering the security identity of the thread while security validation is in progress. `ObCheckObjectAccess` then calls the object's security method to obtain the security settings of the object. (See Chapter 8 in Part 2 for a description of object methods.) The call to the security method might invoke a function in a different executive component. However, many executive objects rely on the system's default security management support.

When an executive component defining an object doesn't want to override the SRM's default security policy, it marks the object type as having default security. Whenever the SRM calls an object's security method, it first checks to see whether the object has default security. An object with default security stores its security information in its header, and its security method is `SeDefaultObjectMethod`. An object that doesn't rely on default security must manage its own security information and supply a specific security method. Objects that rely on default security include mutexes, events, and semaphores. A file object is an example of an object that overrides default security. The I/O manager, which defines the file object type, has the file system driver on which a file resides manage (or choose not to implement) the security for its files. Thus, when the system queries the security on a file object that represents a file on an NTFS volume, the I/O manager file object security method retrieves the file's security using the NTFS file system driver. Note, however, that `ObCheckObjectAccess` isn't executed when files are opened because they reside in secondary namespaces. The system invokes a file object's security method only when a thread explicitly queries or sets the security on a file (with the Windows `SetFileSecurity` or `GetFileSecurity` functions, for example).

After obtaining an object's security information, `ObCheckObjectAccess` invokes the SRM function `SeAccessCheck`. `SeAccessCheck` is one of the functions at the heart of the Windows security model. Among the input parameters `SeAccessCheck` accepts are the object's security information, the security identity of the thread as captured by `ObCheckObjectAccess`, and the access that the thread is requesting. `SeAccessCheck` returns `true` or `false`, depending on whether the thread is granted the access it requested to the object.

Here is an example: Suppose a thread wants to know when a specific process exits (or terminates in some way). It needs to get a handle to the target process by calling the `OpenProcess` API, passing in two important arguments: the unique process ID (let's assume it's known or has been obtained in some way) and an access mask indicating the operations that the thread wants to perform using the returned handle. Lazy developers may just pass `PROCESS_ALL_ACCESS` for the access mask, specifying they want all possible access rights for the process. One of the following two results would occur:

- If the calling thread can be granted all the permissions, it would get back a valid handle and then could call `WaitForSingleObject` to wait for the process to exit. However, another thread in the process, perhaps with fewer privileges, can use the same handle to do other operations with the process, such as terminate it prematurely with `TerminateProcess`, because the handle allows all possible operations on the process.
- The call can fail if the calling thread does not have sufficient privileges to be granted all possible access and the result is an invalid handle, meaning no access to the process. This is unfortunate, because the thread just needed to ask for the `SYNCHRONIZE` access mask. That has a much better chance of succeeding than asking for `PROCESS_ALL_ACCESS`.

The simple conclusion here is that a thread should request the exact access it requires—no more, no less.

Another event that causes the object manager to execute access validation is when a process references

an object using an existing handle. Such references often occur indirectly, as when a process calls on a Windows API to manipulate an object and passes an object handle. For example, a thread opening a file can request read permission to the file. If the thread has permission to access the object in this way, as dictated by its security context and the security settings of the file, the object manager creates a handle—representing the file—in the handle table of the thread’s process. The types of accesses the threads in the process are granted through the handle are stored with the handle by the object manager.

Subsequently, the thread could attempt to write to the file using the `WriteFile` Windows function, passing the file’s handle as a parameter. The system service `NtWriteFile`, which `WriteFile` calls via `Ntdll.dll`, uses the object manager function `ObReferenceObjectByHandle` (documented in the WDK) to obtain a pointer to the file object from the handle. `ObReferenceObjectByHandle` accepts the access that the caller wants from the object as a parameter. After finding the handle entry in the process handle table, `ObReferenceObjectByHandle` compares the access being requested with the access granted at the time the file was opened. In this example, `ObReferenceObjectByHandle` will indicate that the write operation should fail because the caller didn’t obtain write access when the file was opened.

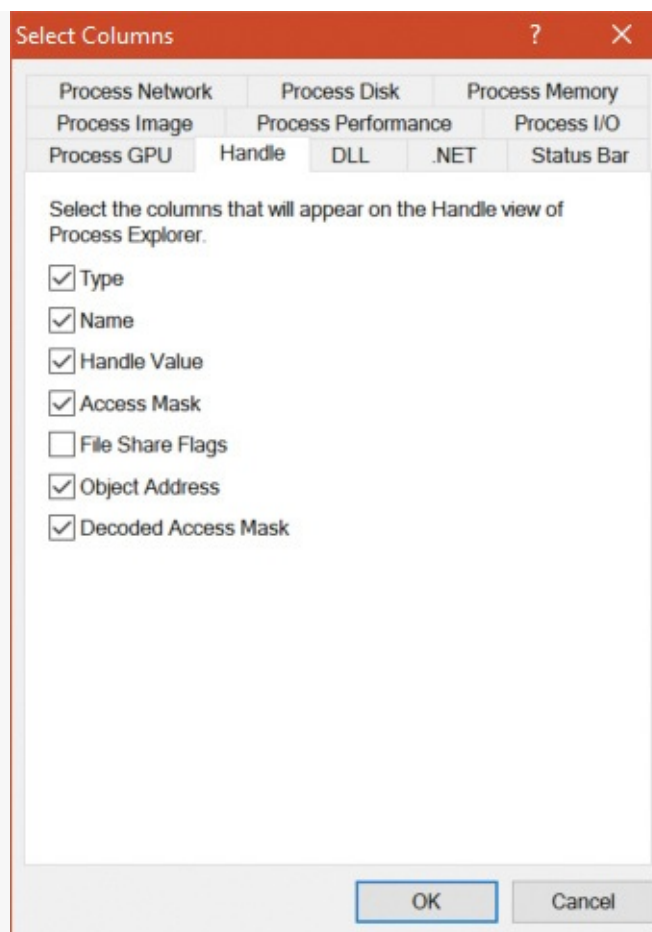
The Windows security functions also enable Windows applications to define their own private objects and to call on the services of the SRM (through the AuthZ user-mode APIs, described later) to enforce the Windows security model on those objects. Many kernel-mode functions that the object manager and other executive components use to protect their own objects are exported as Windows user-mode APIs. The user-mode equivalent of `SeAccessCheck` is the AuthZ API `AccessCheck`. Windows applications can therefore leverage the flexibility of the security model and transparently integrate with the authentication and administrative interfaces that are present in Windows.

The essence of the SRM’s security model is an equation that takes three inputs: the security identity of a thread, the access that the thread wants to an object, and the security settings of the object. The output is either yes or no and indicates whether the security model grants the thread the access it desires. The following sections describe the inputs in more detail and then document the model’s access-validation algorithm.

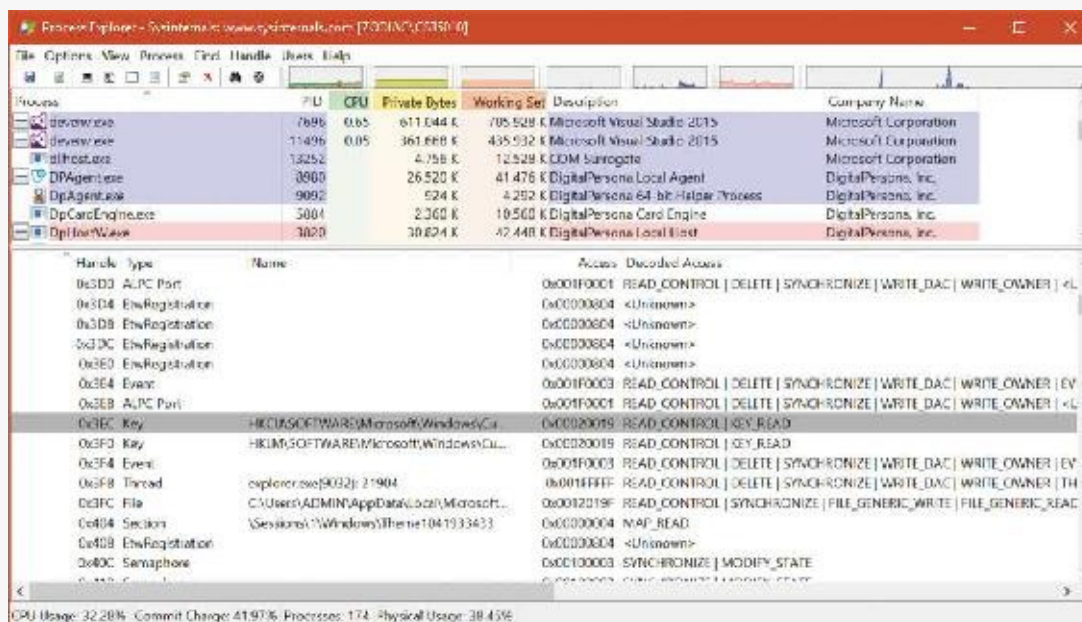
EXPERIMENT: Viewing handle access masks

Process Explorer can show the access mask associated with open handles. Follow these steps:

1. Open Process Explorer.
2. Open the **View** menu, choose **Lower Pane View**, and select **Handles** to configure the lower pane to show handles.
3. Right-click the column headers of the lower pane and choose **Select Columns** to open the dialog box shown here:



4. Select the **Access Mask** and **Decoded Access Mask** check boxes (the latter is available in version 16.10 and later) and click **OK**.
5. Select **Explorer.exe** from the process list and look at the lower pane handles. Each handle has an access mask, indicating the access granted using this handle. To help with interpreting the bits of the access mask, the decoded access mask column shows a textual representation of the access masks for many types of objects:



Notice there are generic access rights (for example, **READ_CONTROL** and **SYNCHRONIZE**) and specific ones (for example, **KEY_READ** and **MODIFY_STATE**). Most of the specific ones are shortened versions of the actual defines in the windows headers (for example, **MODIFY_STATE** instead of **EVENT_MODIFY_STATE**, **TERMINATE** instead of **PROCESS_TERMINATE**).

Security identifiers

Instead of using names (which might or might not be unique) to identify entities that perform actions in a system, Windows uses security identifiers (SIDs). Users have SIDs, as do local and domain groups, local computers, domains, domain members, and services. A SID is a variable-length numeric value that consists of a SID structure revision number, a 48-bit identifier authority value, and a variable number of 32-bit subauthority or relative identifier (RID) values. The authority value identifies the agent that issued the SID, and this agent is typically a Windows local system or a domain. Subauthority values identify trustees relative to the issuing authority, and RIDs are simply a way for Windows to create unique SIDs based on a common base SID. Because SIDs are long and Windows takes care to generate truly random values within each SID, it is virtually impossible for Windows to issue the same SID twice on machines or domains anywhere in the world.

When displayed textually, each SID carries an S prefix, and its various components are separated with hyphens like so:

S-1-5-21-1463437245-1224812800-863842198-1128

In this SID, the revision number is 1, the identifier authority value is 5 (the Windows security authority), and four subauthority values plus one RID (1128) make up the remainder of the SID. This SID is a domain SID, but a local computer on the domain would have a SID with the same revision number, identifier authority value, and number of subauthority values.

When you install Windows, the Windows Setup program issues the computer a machine SID. Windows assigns SIDs to local accounts on the computer. Each local-account SID is based on the source computer's SID and has a RID at the end. RIDs for user accounts and groups start at 1000 and increase in increments of 1 for each new user or group. Similarly, Domain Controller Promote (Dcpromo.exe), the utility used to create a new Windows domain, reuses the computer SID of the computer being promoted to domain controller as the domain SID and re-creates a new SID for the computer if it is ever demoted. Windows issues to new domain accounts SIDs that are based on the domain SID and have an appended RID (again starting at 1000 and increasing in increments of 1 for each new user or group). A RID of 1028 indicates that the SID is the twenty-ninth SID the domain issued.

Windows issues SIDs that consist of a computer or domain SID with a predefined RID to many predefined accounts and groups. For example, the RID for the Administrator account is 500, and the RID for the guest account is 501. A computer's local Administrator account, for example, has the computer SID as its base with the RID of 500 appended to it:

S-1-5-21-13124455-12541255-61235125-500

Windows also defines a number of built-in local and domain SIDs to represent well-known groups. For example, a SID that identifies any and all accounts (except anonymous users) is the Everyone SID: S-1-1-0. Another example of a group that a SID can represent is the Network group, which is the group that represents users who have logged on to a machine from the network. The Network group SID is S-1-5-2. [Table 7-2](#), reproduced here from the Windows SDK documentation, shows some basic well-known SIDs, their numeric values, and their use. Unlike users' SIDs, these SIDs are predefined constants, and have the same values on every Windows system and domain in the world. Thus, a file that is accessible by members of the Everyone group on the system where it was created is also accessible to Everyone on any other system or domain to which the hard drive where it resides happens to be moved. Users on those systems must, of course, authenticate to an account on those systems before becoming members of the Everyone group.

SID	Name	Use
5-1-0-0	Nobody	Used when the SID is unknown
5-1-1-0	Everyone	A group that includes all users except anonymous users
5-1-2-0	Local	Users who log on to terminals locally (physically) connected to the system
5-1-3-0	Creator Owner ID	A security identifier to be replaced by the security identifier of the user who created a new object (used in inheritable ACEs)
5-1-3-1	Creator Group ID	Identifies a security identifier to be replaced by the Primary group SID of the user who created a new object (used in inheritable ACEs)
5-1-5-18	Local System account	Used by services
5-1-5-19	Local Service account	Used by services
5-1-5-20	Network Service account	Used by services

TABLE 7-2 A few well-known SIDs



Note

See Microsoft Knowledge Base article 243330 for a list of defined SIDs at <http://support.microsoft.com/kb/243330>.

Finally, Winlogon creates a unique logon SID for each interactive logon session. A typical use of a logon SID is in an access control entry (ACE) that allows access for the duration of a client's logon session. For example, a Windows service can use the `LogonUser` function to start a new logon session. The `LogonUser` function returns an access token from which the service can extract the logon SID. The service can then use the SID in an ACE (described in the section "[Security descriptors and access control](#)" later in this chapter) that allows the client's logon session to access the interactive window station and desktop. The SID for a logon session is S-1-5-5-X-Y, where the X and Y are randomly generated.

EXPERIMENT: Using PsGetSid and Process Explorer to view SIDs

You can easily see the SID representation for any account you're using by running the `PsGetSid` utility from Sysinternals. `PsGetSid`'s options allow you to translate machine and user account names to their corresponding SIDs and vice versa.

If you run `PsGetSid` with no options, it prints the SID assigned to the local computer. Because the Administrator account always has a RID of 500, you can determine the name assigned to the account (in cases where a system administrator has renamed the account for security reasons) simply by passing the machine SID appended with -500 as `PsGetSid`'s command-line argument.

To obtain the SID of a domain account, enter the user name with the domain as a prefix:

```
c:\>psgetsid redmond\johndoe
```

You can determine the SID of a domain by specifying the domain's name as the argument to `PsGetSid`:

```
c:\>psgetsid Redmond
```

Finally, by examining the RID of your own account, you know at least a number of security accounts (equal to the number resulting from subtracting 999 from your RID) have been created in your domain or on your local machine (depending on whether you are using a domain or local machine account). You

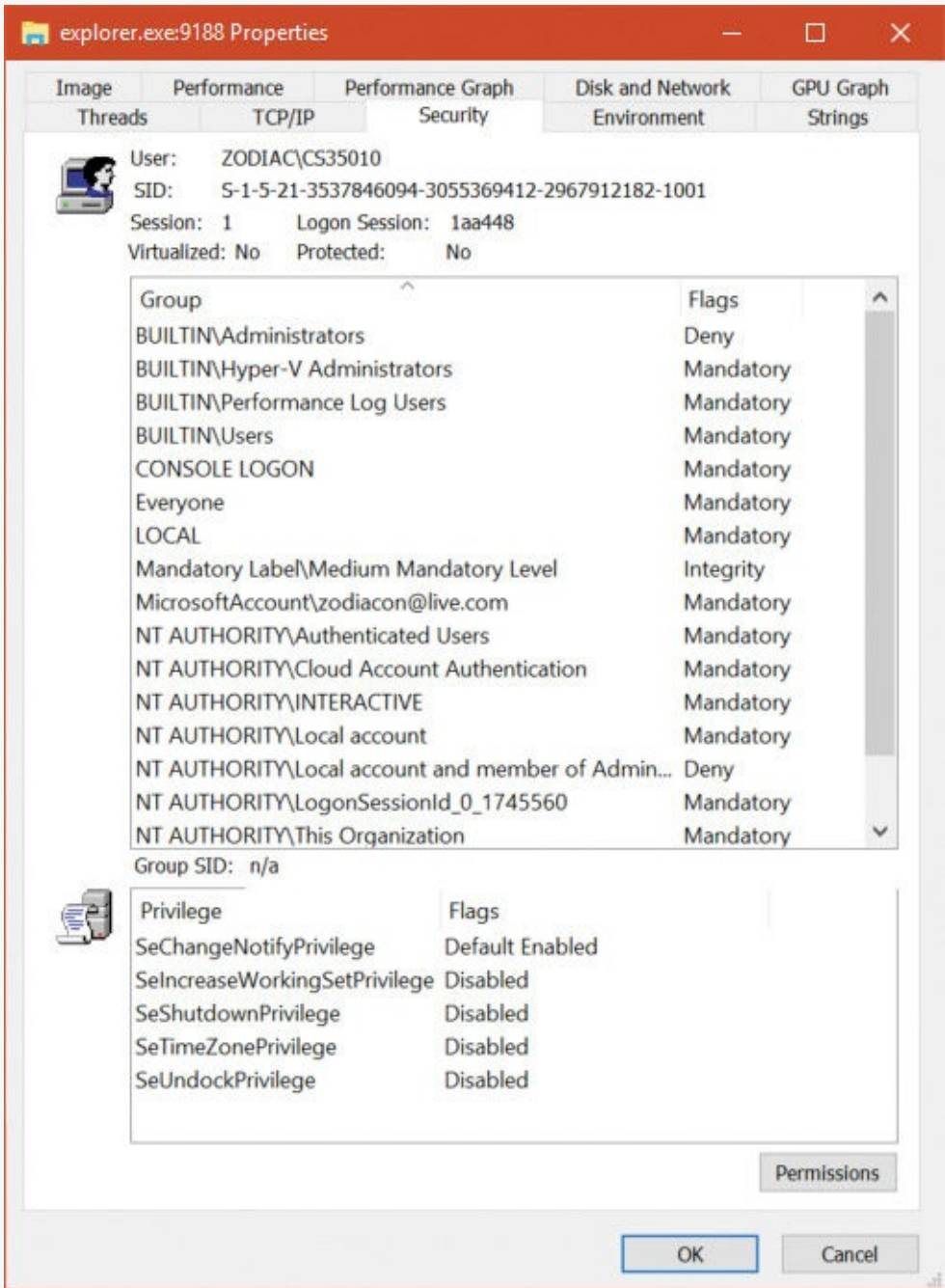
can determine what accounts have been assigned RIDs by passing a SID with the RID you want to query to PsGetSid. If PsGetSid reports that no mapping between the SID and an account name was possible and the RID is lower than that of your account, you know that the account assigned the RID has been deleted.

For example, to find out the name of the account assigned the 28th RID, pass the domain SID appended with -1027 to PsGetSid:

[Click here to view code image](#)

```
c:\>psgetsid S-1-5-21-1787744166-3910675280-2727264193-1027
Account for S-1-5-21-1787744166-3910675280-2727264193-1027:
User: redmond\johndoe
```

Process Explorer can also show you information on account and group SIDs on your system through its Security tab. This tab shows you information such as who owns this process and which groups the account is a member of. To view this information, simply double-click any process (for example, Explorer.exe) in the Process list and then click the **Security** tab. You should see something similar to the following:



The information displayed in the User field contains the friendly name of the account owning this process, while the SID field contains the actual SID value. The Group list includes information on all the groups that this account is a member of (groups are described later in this chapter).

Integrity levels

As mentioned, integrity levels can override discretionary access to differentiate a process and objects running as and owned by the same user, offering the ability to isolate code and data within a user account. The mechanism of Mandatory Integrity Control (MIC) allows the SRM to have more detailed information about the nature of the caller by associating it with an integrity level. It also provides information on the trust required to access the object by defining an integrity level for it.

The integrity level of a token can be obtained with the `GetTokenInformation` API with the `Token-IntegrityLevel` enumeration value. These integrity levels are specified by a SID. Although integrity levels can be arbitrary values, the system uses six primary levels to separate privilege levels, as described in [Table 7-3](#).

SID	Name (Level)	Use
S-1-16-0x0	Untrusted (0)	Used by processes started by the Anonymous group. It blocks most write access.
S-1-16-0x1000	Low (1)	Used by AppContainer processes (UWP) and Protected Mode Internet Explorer. It blocks write access to most objects (such as files and registry keys) on the system.
S-1-16-0x2000	Medium (2)	Used by normal applications being launched while UAC is enabled.
S-1-16-0x3000	High (3)	Used by administrative applications launched through elevation when UAC is enabled, or normal applications if UAC is disabled and the user is an administrator.
S-1-16-0x4000	System (4)	Used by services and other system-level processes (such as Wininit, Winlogon, Smss, and so on).
S-1-16-0x5000	Protected (5)	Currently unused by default. Can be set by kernel-mode caller only.

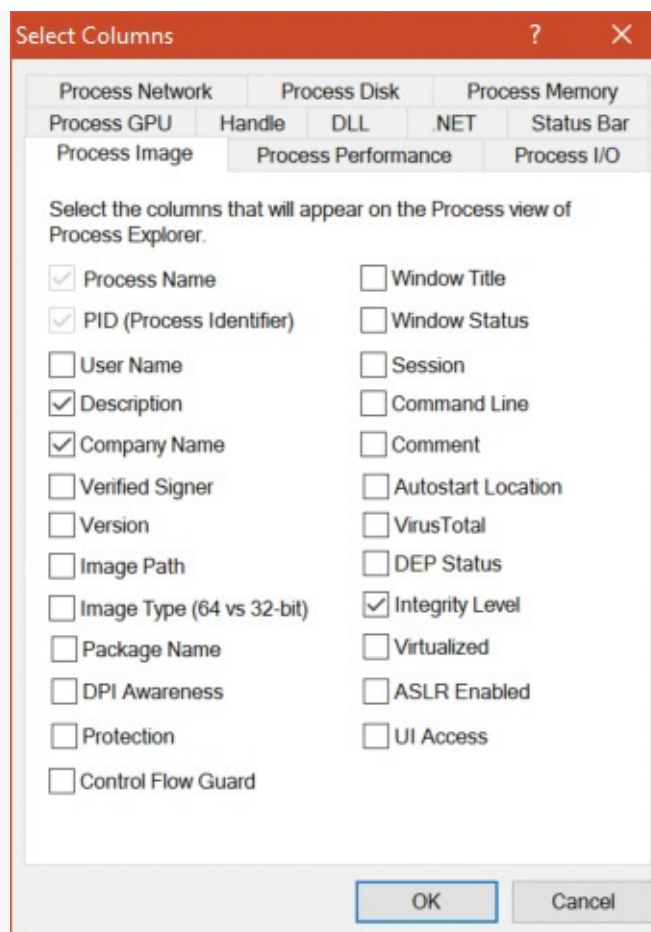
TABLE 7-3 Integrity level SIDs

Another, seemingly additional, integrity level is called *AppContainer*, used by UWP apps. Although seemingly another level, it’s in fact equal to Low. UWP process tokens have another attribute that indicates they are running inside an AppContainer (described in the “[AppContainers](#)” section). This information is available with the `GetTokenInformation` API with the `TokenIsAppContainer` enumeration value.

EXPERIMENT: Looking at the integrity level of processes

You can use Process Explorer to quickly display the integrity level for the processes on your system. The following steps demonstrate this functionality.

1. Launch Microsoft Edge browser and Calc.exe (Windows 10).
2. Open an elevated command prompt window.
3. Open Notepad normally (without elevating it).
4. Open an elevated Process Explorer, right-click any column header in the Process list, and then click **Select Columns**.
5. Select the **Process Image** tab and select the **Integrity Level** check box. The dialog box should look similar to the one shown here:



6. Process Explorer shows you the integrity level of the processes on your system. You should see the Notepad process at medium, the Edge (MicrosoftEdge.exe) process at AppContainer, and the elevated command prompt at High. Also note that the services and system processes are running at an even higher integrity level, System.

Process Explorer - Sysinternals: www.sysinternals.com [ZOD\AC\CS35010]

File Options View Process Find Users Help

Process	PID	CPU	Integrity	Private Bytes	Working Set	Descripti...
lsass.exe	496	< 0.01	System	9,164 K	20,884 K	Local Seci...
lync.exe	13928	0.04	Medium	183,243 K	142,044 K	Skype for...
macmnsvc.exe	3808	0.05	System	15,116 K	25,572 K	McAfee A...
macompatsvc.exe	7216	0.01	System	6,920 K	19,348 K	MA Com...
masvc.exe	3784	0.04	System	5,444 K	18,604 K	McAfee A...
mcshield.exe	6864	< 0.01	System	16,116 K	188,288 K	McAfee C...
mctray.exe	14372	0.11	Medium	8,555 K	1,356 K	McTray Aj...
mDNSResponder.exe	3484		System	2,048 K	7,168 K	Bonjour S...
Memory Compression	4568		System	156 K	18,224 K	
mfeain.exe	4860	< 0.01	System	4,336 K	3,152 K	VSCore A...
MfeEpeHost.exe	3800	0.02	System	29,884 K	32,904 K	McAfee E...
mfevtp.exe	3052		System	3,350 K	8,716 K	McAfee P...
mfevtp.exe	3824		System	1,544 K	6,596 K	McAfee P...
mfevtp.exe	6372		System	5,358 K	11,048 K	McAfee P...
Microsoft.Alm.Shared.Remoting.Remo...	11868		Medium	26,350 K	39,900 K	Microsoft...
Microsoft.VsHub.Server.HttpHost.exe	12120	0.01	Medium	202,704 K	202,616 K	Microsoft...
MicrosoftEdge.exe	17216	< 0.01	AppContainer	36,900 K	73,772 K	Microsoft...
MicrosoftEdgeCP.exe	16836	0.01	AppContainer	66,854 K	96,636 K	Microsoft...
mqsvc.exe	4208		System	5,696 K	13,492 K	Message...
notepad.exe	1184	< 0.01	Medium	2,372 K	14,228 K	Notepad
NvBackend.exe	10724	< 0.01	Medium	2,452 K	10,032 K	NVIDIA B...
nvtray.exe	10640		Medium	3,572 K	13,716 K	NYIDIA S...
nvusvc.exe	1952		System	2,492 K	10,136 K	NVIDIA D...
nvwm64.exe	1960		System	1,444 K	6,836 K	NVIDIA V...
nvwm64.exe	3756	< 0.01	System	5,428 K	15,232 K	NVIDIA V...
nvcdsync.exe	1900	< 0.01	System	10,836 K	26,724 K	NVIDIA U...
OfficeClickToRun.exe	3560	< 0.01	System	40,148 K	56,256 K	Microsoft...
OneDrive.exe	13888	0.06	Medium	219,172 K	242,160 K	Microsoft...
OUTLOOK.EXE	14236	0.03	Medium	167,308 K	212,160 K	Microsoft...
PerfWatson2.exe	15180	0.01	Medium	59,984 K	74,372 K	PerfWate...
PerfWatson2.exe	14836	0.01	High	41,044 K	55,548 K	PerfWate...
PresentationFontCache.exe	1540		System	23,980 K	24,220 K	Presentati...
PrivacyIconClient.exe	<					

CPU Usage: 13.74% Commit Charge: 35.87% Processes: 175 Physical Usage: 31.94%

Every process has an integrity level that is represented in its token and propagated according to the following rules:

- A process normally inherits the integrity level of its parent (which means an elevated command prompt will spawn other elevated processes).
- If the file object for the executable image to which the child process belongs has an integrity level and the parent process's integrity level is medium or higher, the child process will inherit the lower of the two.
- A parent process can create a child process with an explicit integrity level lower than its own. To do this, it uses `DuplicateTokenEx` to duplicate its own access token, it uses `SetTokenInformation` to change the integrity level in the new token to the desired level, and then it calls `CreateProcessAsUser` with that new token.

[Table 7-3](#) lists the integrity level associated with processes, but what about objects? Objects also have an integrity level stored as part of their security descriptor, in a structure that is called the *mandatory label*.

To support migrating from previous versions of Windows (whose registry keys and files would not include integrity-level information), and to make it simpler for application developers, all objects have an implicit integrity level to avoid having to manually specify one. This implicit integrity level is medium, meaning that the mandatory policy (described shortly) on the object will be performed on tokens accessing this object with an integrity level lower than medium.

When a process creates an object without specifying an integrity level, the system checks the integrity level in the token. For tokens with a level of medium or higher, the implicit integrity level of the object remains medium. However, when a token contains an integrity level lower than medium, the object is created with an explicit integrity level that matches the level in the token.

Objects that are created by high- or system-integrity-level processes have a medium integrity level themselves so that users can disable and enable UAC. If object integrity levels always inherited their creator’s integrity level, the applications of an administrator who disables UAC and subsequently re-enables it could fail because the administrator would not be able to modify any registry settings or files created when running at the high integrity level. Objects can also have an explicit integrity level that is set by the system or by the creator of the object. For example, processes, threads, tokens, and jobs are given an explicit integrity level by the kernel when it creates them. The reason for assigning an integrity level to these objects is to prevent a process for the same user, but one running at a lower integrity level, from accessing these objects and modifying their content or behavior (for example, DLL injection or code modification).

Apart from an integrity level, objects also have a mandatory policy, which defines the actual level of protection that’s applied based on the integrity-level check. Three types are possible, shown in [Table 7-4](#). The integrity level and the mandatory policy are stored together in the same ACE.

Policy	Present on, by Default	Description
No-Write-Up	Implicit on all objects	Used to restrict write access coming from a lower integrity level process to the object.
No-Read-Up	Only on process objects	Used to restrict read access coming from a lower integrity level process to the object. Specific use on process objects protects against information leakage by blocking address space reads from an external process.
No-Execute-Up	Only on binaries implementing COM classes	Used to restrict execute access coming from a lower integrity level process to the object. Specific use on COM classes is to restrict launch-activation permissions on a COM class.

TABLE 7-4 Object mandatory policies

EXPERIMENT: Looking at the integrity level of objects

You can use the AccessChk tool from Sysinternals to display the integrity level of objects on the system, such as files, processes, and registry keys. Here's an experiment showing the purpose of the LocalLow directory in Windows:

1. Browse to C:\Users\<UserName>\ in a command prompt window, where <username> is your user name.
2. Try running AccessChk on the AppData folder, as follows:

[Click here to view code image](#)

```
C:\Users\UserName> accesschk -v appdata
```

3. Note the differences between the Local and LocalLow subfolders in your output, similar to that shown here:

[Click here to view code image](#)

```
C:\Users\UserName\AppData\Local
Medium Mandatory Level (Default) [No-Write-Up]
[...]
C:\Users\UserName\AppData\LocalLow
Low Mandatory Level [No-Write-Up]
[...]
C:\Users\UserName\AppData\Roaming
Medium Mandatory Level (Default) [No-Write-Up]
[...]
```

4. Notice that the LocalLow directory has an integrity level set to Low, while the Local and Roaming directories have integrity levels of Medium (default). The default means the system is using an implicit integrity level.
5. Pass the -e flag to AccessChk so it displays only explicit integrity levels. If you run the tool on the AppData folder again, you'll notice only the LocalLow information is displayed.

The -o (object), -k (registry key), and -p (process) flags allow you to specify something other than a file or directory.

Tokens

The SRM uses an object called a *token* (or *access token*) to identify the security context of a process or thread. A security context consists of information that describes the account, groups, and privileges associated with the process or thread. Tokens also include information such as the session ID, the integrity level, and the UAC virtualization state. (We'll describe both privileges and UAC's virtualization mechanism later in this chapter.)

During the logon process (described later in this chapter), Lsass creates an initial token to represent the user logging on. It then determines whether the user logging on is a member of a powerful group or possesses a powerful privilege. The groups checked for in this step are as follows:

- Built-In Administrators
- Certificate Administrators
- Domain Administrators

- Enterprise Administrators
- Policy Administrators
- Schema Administrators
- Domain Controllers
- Enterprise Read-Only Domain Controllers
- Read-Only Domain Controllers
- Account Operators
- Backup Operators
- Cryptographic Operators
- Network Configuration Operators
- Print Operators
- System Operators
- RAS Servers
- Power Users
- Pre-Windows 2000 Compatible Access

Many of the groups listed are used only on domain-joined systems and don't give users local administrative rights directly. Instead, they allow users to modify domain-wide settings.

The privileges checked for are as follows:

- SeBackupPrivilege
- SeCreateTokenPrivilege
- SeDebugPrivilege
- SeImpersonatePrivilege
- SeLabelPrivilege
- SeLoadDriverPrivilege
- SeRestorePrivilege
- SeTakeOwnershipPrivilege
- SeTcbPrivilege

These privileges are described in detail in the section “Privileges,” later in this chapter.

If one or more of these groups or privileges are present, Lsass creates a restricted token for the user (also called a *filtered admin token*) and creates a logon session for both. The standard user token is attached to the initial process or processes that Winlogon starts (by default, Userinit.exe).



Note

If UAC has been disabled, administrators run with a token that includes their administrator group memberships and privileges.

Because child processes inherit a copy of the token of their creators by default, all processes in the user's session run under the same token. You can also generate a token by using the Windows LogonUser function. You can then use this token to create a process that runs within the security context

of the user logged on through the LogonUser function by passing the token to the Windows CreateProcessAsUser function. The CreateProcessWithLogonW function combines these into a single call, which is how the Runas command launches processes under alternative tokens.

Tokens vary in size because different user accounts have different sets of privileges and associated group accounts. However, all tokens contain the same types of information. The most important contents of a token are represented in [Figure 7-7](#).

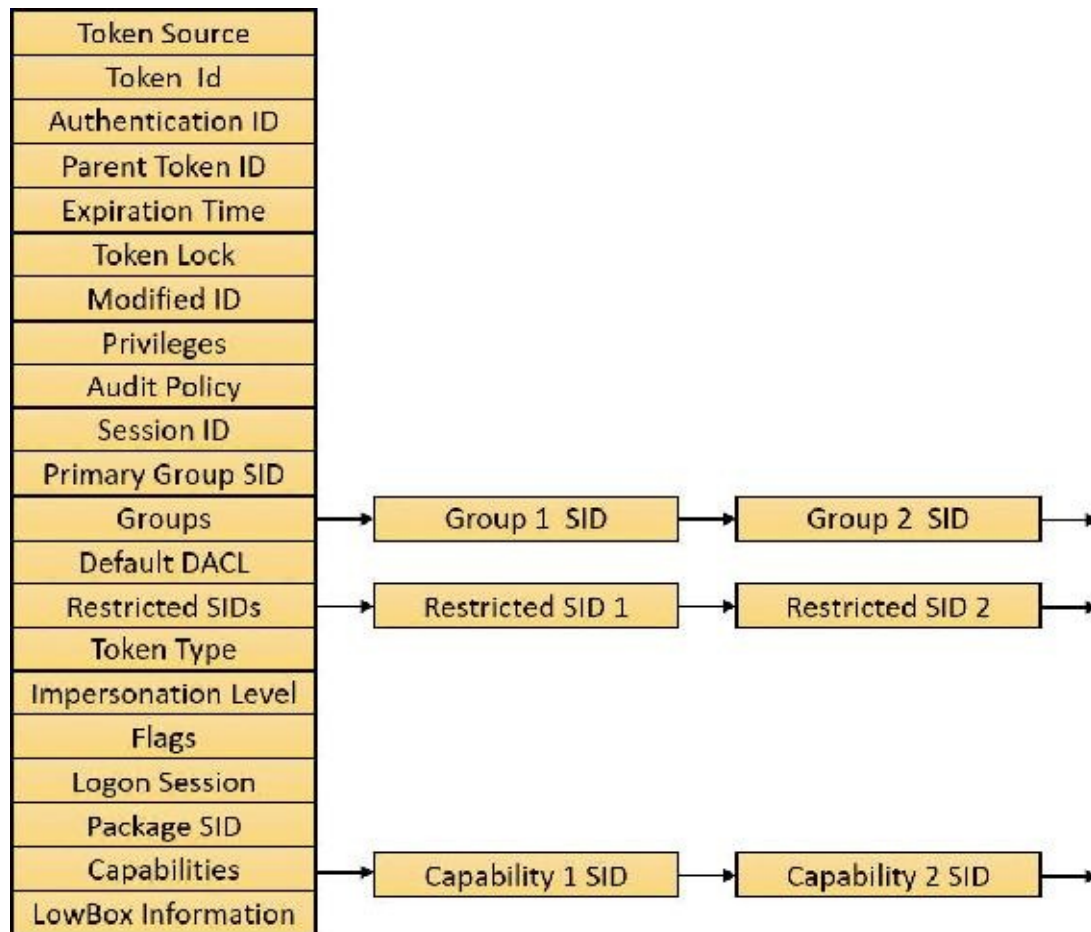


FIGURE 7-7 Access tokens.

The security mechanisms in Windows use two components to determine what objects can be accessed and what secure operations can be performed. One component comprises the token's user account SID and group SID fields. The SRM uses SIDs to determine whether a process or thread can obtain requested access to a securable object, such as an NTFS file.

The group SIDs in a token indicate which groups a user's account is a member of. For example, a server application can disable specific groups to restrict a token's credentials when the server application is performing actions requested by a client. Disabling a group produces nearly the same effect as if the group wasn't present in the token. (It results in a deny-only group, described in the section "[Restricted tokens](#)." Disabled SIDs are used as part of security access checks, described in the section "[Determining access](#)" later in the chapter.) Group SIDs can also include a special SID that contains the integrity level of the process or thread. The SRM uses another field in the token, which describes the mandatory integrity policy, to perform the mandatory integrity check described later in the chapter.

The second component in a token that determines what the token's thread or process can do is the privilege array. A token's privilege array is a list of rights associated with the token. An example of a privilege is the right of the process or thread associated with the token to shut down the computer. Privileges are described in more detail later in this chapter.

A token's default primary group field and default discretionary access control list (DACL) field are security attributes that Windows applies to objects that a process or thread creates when it uses the token. By including security information in tokens, Windows makes it convenient for a process or thread to create objects with standard security attributes because the process or thread doesn't need to request discrete security information for every object it creates.

Each token's type distinguishes a primary token (a token that identifies the security context of a process) from an impersonation token (a type of token that threads use to temporarily adopt a different security context, usually of another user). Impersonation tokens carry an impersonation level that signifies what type of impersonation is active in the token. (Impersonation is described later in this chapter.)

A token also includes the mandatory policy for the process or thread, which defines how MIC will behave when processing this token. There are two policies:

- **TOKEN_MANDATORY_NO_WRITE_UP** Enabled by default, this sets the No-Write-Up policy on this token, specifying that the process or thread will not be able to access objects with a higher integrity level for write access.
- **TOKEN_MANDATORY_NEW_PROCESS_MIN** Also enabled by default, this specifies that the SRM should look at the integrity level of the executable image when launching a child process and compute the minimum integrity level of the parent process and the file object's integrity level as the child's integrity level.

Token flags include parameters that determine the behavior of certain UAC and UIPI mechanisms, such as virtualization and user interface access. Those mechanisms will be described later in this chapter.

Each token can also contain attributes that are assigned by the Application Identification service (part of AppLocker) when AppLocker rules have been defined. AppLocker and its use of attributes in the access token are described later in this chapter.

A token for a UWP process includes information on the AppContainer hosting the process. First, it stores a package SID, identifying the UWP package the process originated from. The significance of this SID will be described in the "[AppContainers](#)" section later in this chapter. Second, UWP processes need to request capabilities for operations that require the user's consent. Examples of capabilities include network access, using the phone capabilities of the device (if any), accessing the camera on the device, and more. Each such capability is represented with a SID, stored as part of the token. (Capabilities will be discussed further in the "[AppContainers](#)" section.)

The remaining fields in a token serve informational purposes. The token source field contains a short textual description of the entity that created the token. Programs that want to know where a token originated use the token source to distinguish among sources such as the Windows Session Manager, a network file server, or the remote procedure call (RPC) server. The token identifier is a locally unique identifier (LUID) that the SRM assigns to the token when it creates the token. The Windows executive maintains the executive LUID, a monotonically increasing counter it uses to assign a unique numeric identifier to each token. A LUID is guaranteed to be unique only until the system is shut down.

The token authentication ID is another kind of LUID. A token's creator assigns the token's authentication ID when calling the `LsaLogonUser` function. If the creator doesn't specify a LUID, Lsass obtains the LUID from the executive LUID. Lsass copies the authentication ID for all tokens descended from an initial logon token. A program can obtain a token's authentication ID to see whether the token belongs to the same logon session as other tokens the program has examined.

The executive LUID refreshes the modified ID every time a token's characteristics are modified. An application can test the modified ID to discover changes in a security context since the context's last use.

Tokens contain an expiration time field that can be used by applications performing their own security to reject a token after a specified amount of time. However, Windows itself does not enforce the expiration time of tokens.



Note

To guarantee system security, the fields in a token are immutable (because they are located in kernel memory). Except for fields that can be modified through a specific system call designed to modify certain token attributes (assuming the caller has the appropriate access rights to the token object), data such as privileges and SIDs in a token can never be modified from user mode.

EXPERIMENT: Viewing access tokens

The kernel debugger `dt _TOKEN` command displays the format of an internal token object. Although this structure differs from the user-mode token structure returned by Windows API security functions, the fields are similar. For further information on tokens, see the description in the Windows SDK documentation.

Here's a token structure on Windows 10:

[Click here to view code image](#)

```
lkd> dt nt!_token
+0x000 TokenSource      : _TOKEN_SOURCE
+0x010 TokenId          : _LUID
+0x018 AuthenticationId : _LUID
+0x020 ParentTokenId   : _LUID
+0x028 ExpirationTime   : _LARGE_INTEGER
+0x030 TokenLock        : Ptr64 _ERESOURCE
+0x038 ModifiedId       : _LUID
+0x040 Privileges       : _SEP_TOKEN_PRIVILEGES
+0x058 AuditPolicy      : _SEP_AUDIT_POLICY
+0x078 SessionId        : Uint4B
+0x07c UserAndGroupCount : Uint4B
+0x080 RestrictedSidCount : Uint4B
+0x084 VariableLength   : Uint4B
+0x088 DynamicCharged   : Uint4B
+0x08c DynamicAvailable : Uint4B
+0x090 DefaultOwnerIndex : Uint4B
+0x098 UserAndGroups     : Ptr64 _SID_AND_ATTRIBUTES
+0x0a0 RestrictedSids    : Ptr64 _SID_AND_ATTRIBUTES
+0x0a8 PrimaryGroup      : Ptr64 Void
+0x0b0 DynamicPart       : Ptr64 Uint4B
+0x0b8 DefaultDacl       : Ptr64 _ACL
+0x0c0 TokenType         : _TOKEN_TYPE
+0x0c4 ImpersonationLevel : _SECURITY_IMPERSONATION_LEVEL
+0x0c8 TokenFlags        : Uint4B
+0x0cc TokenInUse        : UChar
```

```

+0x0d0 IntegrityLevelIndex : Uint4B
+0x0d4 MandatoryPolicy : Uint4B
+0x0d8 LogonSession : Ptr64 _SEP_LOGON_SESSION_REFERENCES
+0x0e0 OriginatingLogonSession : _LUID
+0x0e8 SidHash : _SID_AND_ATTRIBUTES_HASH
+0x1f8 RestrictedSidHash : _SID_AND_ATTRIBUTES_HASH
+0x308 pSecurityAttributes : Ptr64
_AUTHZBASEP_SECURITY_ATTRIBUTES_INFORMATION
+0x310 Package : Ptr64 Void
+0x318 Capabilities : Ptr64 _SID_AND_ATTRIBUTES
+0x320 CapabilityCount : Uint4B
+0x328 CapabilitiesHash : _SID_AND_ATTRIBUTES_HASH
+0x438 LowboxNumberEntry : Ptr64 _SEP_LOWBOX_NUMBER_ENTRY
+0x440 LowboxHandlesEntry : Ptr64 _SEP_LOWBOX_HANDLES_ENTRY
+0x448 pClaimAttributes : Ptr64
_AUTHZBASEP_CLAIM_ATTRIBUTES_COLLECTION
+0x450 TrustLevelSid : Ptr64 Void
+0x458 TrustLinkedToken : Ptr64 _TOKEN
+0x460 IntegrityLevelSidValue : Ptr64 Void
+0x468 TokenSidValues : Ptr64 _SEP_SID_VALUES_BLOCK
+0x470 IndexEntry : Ptr64 _SEP_LUID_TO_INDEX_MAP_ENTRY
+0x478 DiagnosticInfo : Ptr64 _SEP_TOKEN_DIAG_TRACK_ENTRY
+0x480 SessionObject : Ptr64 Void
+0x488 VariablePart : Uint8B

```

You can examine the token for a process with the `!token` command. You'll find the address of the token in the output of the `!process` command. Here's an example for an explorer.exe process:

[Click here to view code image](#)

```

lkd> !process 0 1 explorer.exe
PROCESS fffffe18304dfd780
    SessionId: 1 Cid: 23e4 Peb: 00c2a000 ParentCid: 2264
    DirBase: 2aa0f6000 ObjectTable: fffffcd82c72fcd80 HandleCount:
<Data Not
Accessible>
    Image: explorer.exe
    VadRoot fffffe18303655840 Vads 705 Clone 0 Private 12264.
    Modified 376410. Locked 18.
    DeviceMap fffffcd82c39bc0d0
    Token ffffcd82c72fc060
    ...

PROCESS fffffe1830670a080
    SessionId: 1 Cid: 27b8 Peb: 00950000 ParentCid: 035c
    DirBase: 2cba97000 ObjectTable: fffffcd82c7ccc500 HandleCount:
<Data Not
Accessible>

```

Image: explorer.exe
VadRoot fffffe183064e9f60 Vads 1991 Clone 0 Private 19576.
Modified 87095. Locked 0.
DeviceMap fffffcd82c39bc0d0
Token fffffcd82c7cd9060
...

lkd> !token **ffffcd82c72fc060**

_TOKEN 0xffffcd82c72fc060

TS Session ID: 0x1

User: S-1-5-21-3537846094-3055369412-2967912182-1001

User Groups:

00 S-1-16-8192
Attributes - GroupIntegrity GroupIntegrityEnabled

01 S-1-1-0
Attributes - Mandatory Default Enabled

02 S-1-5-114
Attributes - DenyOnly

03 S-1-5-21-3537846094-3055369412-2967912182-1004
Attributes - Mandatory Default Enabled

04 S-1-5-32-544
Attributes - DenyOnly

05 S-1-5-32-578
Attributes - Mandatory Default Enabled

06 S-1-5-32-559
Attributes - Mandatory Default Enabled

07 S-1-5-32-545
Attributes - Mandatory Default Enabled

08 S-1-5-4
Attributes - Mandatory Default Enabled

09 S-1-2-1
Attributes - Mandatory Default Enabled

10 S-1-5-11
Attributes - Mandatory Default Enabled

11 S-1-5-15
Attributes - Mandatory Default Enabled

12 S-1-11-96-3623454863-58364-18864-2661722203-1597581903-
1225312835-2511459453-
1556397606-2735945305-1404291241
Attributes - Mandatory Default Enabled

13 S-1-5-113
Attributes - Mandatory Default Enabled

14 S-1-5-5-0-1745560
Attributes - Mandatory Default Enabled LogonId

15 S-1-2-0
Attributes - Mandatory Default Enabled

16 S-1-5-64-36

```

Attributes - Mandatory Default Enabled
Primary Group: S-1-5-21-3537846094-3055369412-2967912182-1001
Privs:
  19 0x0000000013 SeShutdownPrivilege           Attributes -
  23 0x0000000017 SeChangeNotifyPrivilege        Attributes -
Enabled Default
  25 0x0000000019 SeUndockPrivilege              Attributes -
  33 0x0000000021 SeIncreaseWorkingSetPrivilege  Attributes -
  34 0x0000000022 SeTimeZonePrivilege            Attributes -
Authentication ID: (0,1aa448)
Impersonation Level: Anonymous
TokenType: Primary
Source: User32 TokenFlags: 0x2a00 ( Token in use )
Token ID: 1be803 ParentToken ID: 1aa44b
Modified ID: (0, 43d9289)
RestrictedSidCount: 0 RestrictedSids: 0x000000000000000000
OriginatingLogonSession: 3e7
PackageSid: (null)
CapabilityCount: 0 Capabilities: 0x000000000000000000
LowboxNumberEntry: 0x000000000000000000
Security Attributes:
Unable to get the offset of
nt!_AUTHZBASEP_SECURITY_ATTRIBUTE.ListLink
Process Token TrustLevelSid: (null)

```

Notice that there is no package SID for Explorer, since it's not running inside an AppContainer.

Run calc.exe under Windows 10, which spawns calculator.exe (now a UWP app), and examine its token:

[Click here to view code image](#)

```

lkd> !process 0 1 calculator.exe
PROCESS fffffe18309e874c0
  SessionId: 1 Cid: 3c18 Peb: cd0182c000 ParentCid: 035c
  DirBase: 7a15e4000 ObjectTable: fffffcd82ec9a37c0 HandleCount:
<Data Not
Accessible>
  Image: Calculator.exe
  VadRoot fffffe1831cf197c0 Vads 181 Clone 0 Private 3800. Modified
3746. Locked 503.
  DeviceMap fffffcd82c39bc0d0
  Token fffffcd82e26168f0
...

lkd> !token fffffcd82e26168f0
_TOKEN 0xffffcd82e26168f0
TS Session ID: 0x1
User: S-1-5-21-3537846094-3055369412-2967912182-1001

```

User Groups:

00 S-1-16-4096
Attributes - GroupIntegrity GroupIntegrityEnabled

01 S-1-1-0
Attributes - Mandatory Default Enabled

02 S-1-5-114
Attributes - DenyOnly

03 S-1-5-21-3537846094-3055369412-2967912182-1004
Attributes - Mandatory Default Enabled

04 S-1-5-32-544
Attributes - DenyOnly

05 S-1-5-32-578
Attributes - Mandatory Default Enabled

06 S-1-5-32-559
Attributes - Mandatory Default Enabled

07 S-1-5-32-545
Attributes - Mandatory Default Enabled

08 S-1-5-4
Attributes - Mandatory Default Enabled

09 S-1-2-1
Attributes - Mandatory Default Enabled

10 S-1-5-11
Attributes - Mandatory Default Enabled

11 S-1-5-15
Attributes - Mandatory Default Enabled

12 S-1-11-96-3623454863-58364-18864-2661722203-1597581903-
1225312835-2511459453-
1556397606-2735945305-1404291241
Attributes - Mandatory Default Enabled

13 S-1-5-113
Attributes - Mandatory Default Enabled

14 S-1-5-5-0-1745560
Attributes - Mandatory Default Enabled LogonId

15 S-1-2-0
Attributes - Mandatory Default Enabled

16 S-1-5-64-36
Attributes - Mandatory Default Enabled

Primary Group: S-1-5-21-3537846094-3055369412-2967912182-1001

Privs:

19	0x0000000013	SeShutdownPrivilege	Attributes -
23	0x0000000017	SeChangeNotifyPrivilege	Attributes -
Enabled Default			
25	0x0000000019	SeUndockPrivilege	Attributes -
33	0x0000000021	SeIncreaseWorkingSetPrivilege	Attributes -
34	0x0000000022	SeTimeZonePrivilege	Attributes -

Authentication ID: (0,1aa448)

Impersonation Level: Anonymous

```

TokenType: Primary
Source: User32 TokenFlags: 0x4a00 ( Token in use )
Token ID: 4ddb8c0 ParentToken ID: 1aa44b
Modified ID: (0, 4ddb8b2)
RestrictedSidCount: 0 RestrictedSids: 0x0000000000000000
OriginatingLogonSession: 3e7
PackageSid: S-1-15-2-466767348-3739614953-2700836392-1801644223-4227750657-1087833535-2488631167
CapabilityCount: 1 Capabilities: 0xfffffcd82e1bfccd0
Capabilities:
  00 S-1-15-3-466767348-3739614953-2700836392-1801644223-4227750657-1087833535-2488631167
    Attributes - Enabled
LowboxNumberEntry: 0xfffffcd82fa2c1670
LowboxNumber: 5
Security Attributes:
Unable to get the offset of
nt!_AUTHZBASEP_SECURITY_ATTRIBUTE.ListLink
Process Token TrustLevelSid: (null)

```

You can see there is one capability required by Calculator (which is in fact equal to its AppContainer SID RID, as described in the section “[AppContainers](#)” later in this chapter). Looking at the token of the Cortana process (searchui.exe) shows the following capabilities:

[Click here to view code image](#)

```

lkd> !process 0 1 searchui.exe
PROCESS fffffe1831307d080
  SessionId: 1 Cid: 29d8 Peb: fb407ec000 ParentCid: 035c
  DeepFreeze
  DirBase: 38b635000 ObjectTable: fffffcd830059e580 HandleCount:
<Data Not
Accessible>
  Image: SearchUI.exe
  VadRoot fffffe1831fe89130 Vads 420 Clone 0 Private 11029.
  Modified 2031. Locked 0.
  DeviceMap fffffcd82c39bc0d0
  Token fffffcd82d97d18f0
  ...

```

```

lkd> !token fffffcd82d97d18f0
_TOKEN 0xfffffcd82d97d18f0
TS Session ID: 0x1
User: S-1-5-21-3537846094-3055369412-2967912182-1001
User Groups:
...
Primary Group: S-1-5-21-3537846094-3055369412-2967912182-1001

```

Privs:

19	0x0000000013	SeShutdownPrivilege	Attributes -
23	0x0000000017	SeChangeNotifyPrivilege	Attributes -

Enabled Default

25	0x0000000019	SeUndockPrivilege	Attributes -
33	0x0000000021	SeIncreaseWorkingSetPrivilege	Attributes -
34	0x0000000022	SeTimeZonePrivilege	Attributes -

Authentication ID: (0,1aa448)
Impersonation Level: Anonymous
TokenType: Primary
Source: User32 TokenFlags: 0x4a00 (Token in use)
Token ID: 4483430 ParentToken ID: 1aa44b
Modified ID: (0, 4481b11)
RestrictedSidCount: 0 RestrictedSids: 0x0000000000000000
OriginatingLogonSession: 3e7
PackageSid: S-1-15-2-1861897761-1695161497-2927542615-642690995-327840285-2659745135-2630312742
CapabilityCount: 32 Capabilities: 0xffffcd82f78149b0
Capabilities:

00	S-1-15-3-1024-1216833578-114521899-3977640588-1343180512-2505059295-473916851-3379430393-3088591068	Attributes - Enabled
01	S-1-15-3-1024-3299255270-1847605585-2201808924-710406709-3613095291-873286183-3101090833-2655911836	Attributes - Enabled
02	S-1-15-3-1024-34359262-2669769421-2130994847-3068338639-3284271446-2009814230-2411358368-814686995	Attributes - Enabled
03	S-1-15-3-1	Attributes - Enabled

...

29	S-1-15-3-3633849274-1266774400-1199443125-2736873758	Attributes - Enabled
30	S-1-15-3-2569730672-1095266119-53537203-1209375796	Attributes - Enabled
31	S-1-15-3-2452736844-1257488215-2818397580-3305426111	Attributes - Enabled

LowboxNumberEntry: 0xffffcd82c7539110
LowboxNumber: 2
Security Attributes:
Unable to get the offset of
nt!_AUTHZBASEP_SECURITY_ATTRIBUTE.ListLink
Process Token TrustLevelSid: (null)

There are 32 capabilities required by Cortana. This simply indicates the process is richer in features that need to be accepted by end users and validated by the system.

You can indirectly view token contents with Process Explorer's Security tab in the process Properties dialog box. The dialog box shows the groups and privileges included in the token of the process you examine.

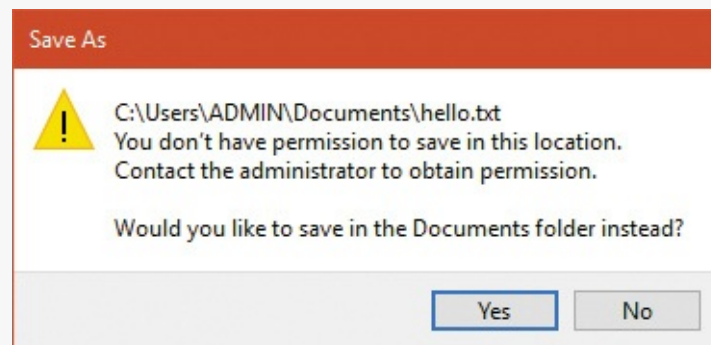
EXPERIMENT: Launching a program at low integrity level

When you elevate a program, either by using the Run as Administrator option or because the program is requesting it, the program is explicitly launched at high integrity level. However, it is also possible to launch a program at low integrity level by using PSEXEC from Sysinternals:

1. Launch Notepad at low integrity level by using the following command:

```
c:\psexec -l notepad.exe
```

2. Try opening a file (such as one of the XML files) in the %SystemRoot%\System32 directory. Notice that you can browse the directory and open any file contained within it.
3. In Notepad, open the **File** menu and choose **New**.
4. Enter some text in the window and try saving it in the %SystemRoot%\System32 directory. Notepad displays a dialog box indicating a lack of permissions and suggests saving the file in the Documents folder.



5. Accept Notepad's suggestion. You will get the same message box again, and repeatedly for each attempt.
6. Now try saving the file in the LocalLow directory of your user profile, shown in an experiment earlier in the chapter.

In the previous experiment, saving a file in the LocalLow directory worked because Notepad was running with low integrity level, and only the LocalLow directory also had low integrity level. All the other directories where you tried to save the file had an implicit medium integrity level. (You can verify this with AccessChk.) However, reading from the %SystemRoot%\System32 directory, as well as opening files within it, did work, even though the directory and its file also have an implicit medium integrity level.

Impersonation

Impersonation is a powerful feature Windows uses frequently in its security model. Windows also uses impersonation in its client/server programming model. For example, a server application can provide access to resources such as files, printers, and databases. Clients wanting to access a resource send a request to the server. When the server receives the request, it must ensure that the client has permission to perform the desired operations on the resource. For example, if a user on a remote machine tries to delete a file on an NTFS share, the server exporting the share must determine whether the user is allowed to delete the file. The obvious way to determine whether a user has permission is for the server to query the user's account and group SIDs and scan the security attributes on the file. This approach is tedious to program, prone to errors, and wouldn't permit new security features to be supported transparently. Thus, Windows provides impersonation services to simplify the server's job.

Impersonation lets a server notify the SRM that the server is temporarily adopting the security profile of a client making a resource request. The server can then access resources on behalf of the client, and the SRM carries out the access validation, but it does so based on the impersonated client security context. Usually, a server has access to more resources than a client does and loses some of its security credentials during impersonation. However, the reverse can be true: The server can gain security credentials during impersonation.

A server impersonates a client only within the thread that makes the impersonation request. Thread-control data structures contain an optional entry for an impersonation token. However, a thread's primary token, which represents the thread's real security credentials, is always accessible in the process's control structure.

Windows makes impersonation available through several mechanisms. For example, if a server communicates with a client through a named pipe, the server can use the `ImpersonateNamedPipeClient` Windows API function to tell the SRM that it wants to impersonate the user on the other end of the pipe. If the server is communicating with the client through Dynamic Data Exchange (DDE) or RPC, it can make similar impersonation requests using `DdeImpersonateClient` and `RpcImpersonateClient`. A thread can create an impersonation token that's simply a copy of its process token with the `ImpersonateSelf` function. The thread can then alter its impersonation token, perhaps to disable SIDs or privileges. A Security Support Provider Interface (SSPI) package can impersonate its clients with `ImpersonateSecurityContext`. SSPIs implement a network authentication protocol such as LAN Manager version 2 or Kerberos. Other interfaces such as COM expose impersonation through APIs of their own, such as `CoImpersonateClient`.

After the server thread finishes its task, it reverts to its primary security context. These forms of impersonation are convenient for carrying out specific actions at the request of a client and for ensuring that object accesses are audited correctly. (For example, the audit that is generated gives the identity of the impersonated client rather than that of the server process.) The disadvantage to these forms of impersonation is that they can't execute an entire program in the context of a client. In addition, an impersonation token can't access files or printers on network shares unless it is a delegation-level impersonation (described shortly) and has sufficient credentials to authenticate to the remote machine, or the file or printer share supports null sessions. (A null session is one that results from an anonymous logon.)

If an entire application must execute in a client's security context or must access network resources without using impersonation, the client must be logged on to the system. The `LogonUser` Windows API function enables this action. `LogonUser` takes an account name, a password, a domain or computer

name, a logon type (such as interactive, batch, or service), and a logon provider as input, and it returns a primary token. A server thread can adopt the token as an impersonation token, or the server can start a program that has the client's credentials as its primary token. From a security standpoint, a process created using the token returned from an interactive logon via `LogonUser`, such as with the `CreateProcessAsUser` API, looks like a program a user starts by logging on to the machine interactively. The disadvantage to this approach is that a server must obtain the user's account name and password. If the server transmits this information across the network, the server must encrypt it securely so that a malicious user snooping network traffic can't capture it.

To prevent the misuse of impersonation, Windows doesn't let servers perform impersonation without a client's consent. A client process can limit the level of impersonation that a server process can perform by specifying a security quality of service (SQOS) when connecting to the server. For instance, when opening a named pipe, a process can specify `SECURITY_ANONYMOUS`, `SECURITY_IDENTIFICATION`, `SECURITY_IMPERSONATION`, or `SECURITY_DELEGATION` as flags for the Windows `CreateFile` function. These same options apply to other impersonation-related functions listed earlier. Each level lets a server perform different types of operations with respect to the client's security context:

- **SecurityAnonymous** This is the most restrictive level of impersonation. The server can't impersonate or identify the client.
- **SecurityIdentification** This lets the server obtain the identity (the SIDs) of the client and the client's privileges, but the server can't impersonate the client.
- **SecurityImpersonation** This lets the server identify and impersonate the client on the local system.
- **SecurityDelegation** This is the most permissive level of impersonation. It lets the server impersonate the client on local and remote systems.

Other interfaces such as RPC use different constants with similar meanings (for example, `RPC_C_IMP_LEVEL_IMPERSONATE`).

If the client doesn't set an impersonation level, Windows chooses the `SecurityImpersonation` level by default. The `CreateFile` function also accepts `SECURITY_EFFECTIVE_ONLY` and `SECURITY_CONTEXT_TRACKING` as modifiers for the impersonation setting:

- **SECURITY_EFFECTIVE_ONLY** This prevents a server from enabling or disabling a client's privileges or groups while the server is impersonating.
- **SECURITY_CONTEXT_TRACKING** This specifies that any changes a client makes to its security context are reflected in a server that is impersonating it. If this option isn't specified, the server adopts the context of the client at the time of the impersonation and doesn't receive any changes. This option is honored only when the client and server processes are on the same system.

To prevent spoofing scenarios in which a low-integrity process could create a user interface that captured user credentials and then used `LogonUser` to obtain that user's token, a special integrity policy applies to impersonation scenarios: a thread cannot impersonate a token of higher integrity than its own. For example, a low-integrity application cannot spoof a dialog box that queries administrative credentials and then attempt to launch a process at a higher privilege level. The integrity-mechanism policy for impersonation access tokens is that the integrity level of the access token that is returned by `LsaLogonUser` must be no higher than the integrity level of the calling process.

Restricted tokens

A restricted token is created from a primary or impersonation token using the `CreateRestrictedToken` function. The restricted token is a copy of the token it's derived from, with the following possible modifications:

- Privileges can be removed from the token's privilege array.
- SIDs in the token can be marked as deny-only. These SIDs remove access to any resources for which the SID's access is denied by using a matching access-denied ACE that would otherwise be overridden by an ACE granting access to a group containing the SID earlier in the security descriptor.
- SIDs in the token can be marked as restricted. These SIDs are subject to a second pass of the access-check algorithm, which will parse only the restricted SIDs in the token. The results of both the first pass and the second pass must grant access to the resource or no access is granted to the object.

Restricted tokens are useful when an application wants to impersonate a client at a reduced security level, primarily for safety reasons when running untrusted code. For example, the restricted token can have the shutdown-system privilege removed from it to prevent code executed in the restricted token's security context from rebooting the system.

Filtered admin token

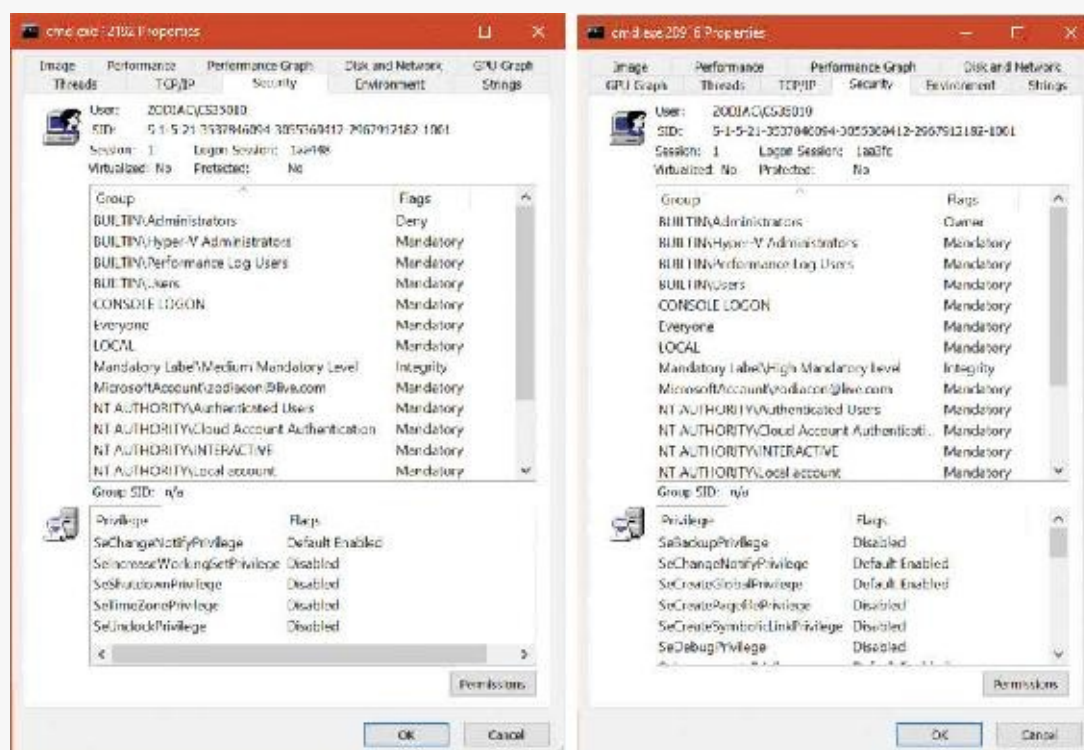
As you saw earlier, restricted tokens are also used by UAC to create the filtered admin token that all user applications will inherit. A filtered admin token has the following characteristics:

- The integrity level is set to medium.
- The administrator and administrator-like SIDs mentioned previously are marked as deny-only to prevent a security hole if the group were to be removed altogether. For example, if a file had an access control list (ACL) that denied the Administrators group all access but granted some access to another group the user belongs to, the user would be granted access if the Administrators group was absent from the token, which would give the standard user version of the user's identity more access than the user's administrator identity.
- All privileges are stripped except Change Notify, Shutdown, Undock, Increase Working Set, and Time Zone.

EXPERIMENT: Looking at filtered admin tokens

You can make Explorer launch a process with either the standard user token or the administrator token by following these steps on a machine with UAC enabled:

1. Log on to an account that's a member of the Administrators group.
2. Open the **Start** menu, type **command**, right-click the **Command Prompt** option that appears, and choose **Run as Administrator** to run an elevated command prompt.
3. Run a new instance of cmd.exe, but this time do it normally (that is, not elevated).
4. Run Process Explorer elevated, open the Properties dialog boxes for the two command prompt processes, and click the **Security** tabs. Note that the standard user token contains a deny-only SID and a medium mandatory label, and that it has only a couple of privileges. The properties on the right in the following screenshot are from a command prompt running with an administrator token, and the properties on the left are from one running with the filtered administrator token:



Virtual service accounts

Windows provides a specialized type of account known as a *virtual service account* (or simply *virtual account*) to improve the security isolation and access control of Windows services with minimal administrative effort. (See Chapter 9 in Part 2 for more information on Windows services.) Without this mechanism, Windows services must run under one of the accounts defined by Windows for its built-in services (such as Local Service or Network Service) or under a regular domain account. The accounts such as Local Service are shared by many existing services and so offer limited granularity for privilege and access control; furthermore, they cannot be managed across the domain. Domain accounts require periodic password changes for security, and the availability of services during a password-change cycle might be affected. Furthermore, for best isolation, each service should run under its own account, but with ordinary accounts this multiplies the management effort.

With virtual service accounts, each service runs under its own account with its own security ID. The name of the account is always NT SERVICE\ followed by the internal name of the service. Virtual service

accounts can appear in access control lists and can be associated with privileges via Group Policy like any other account name. They cannot, however, be created or deleted through the usual account-management tools, nor assigned to groups.

Windows automatically sets and periodically changes the password of the virtual service account. Similar to the Local System and Other Service Accounts, there is a password, but the password is unknown to the system administrators.

EXPERIMENT: Using virtual service accounts

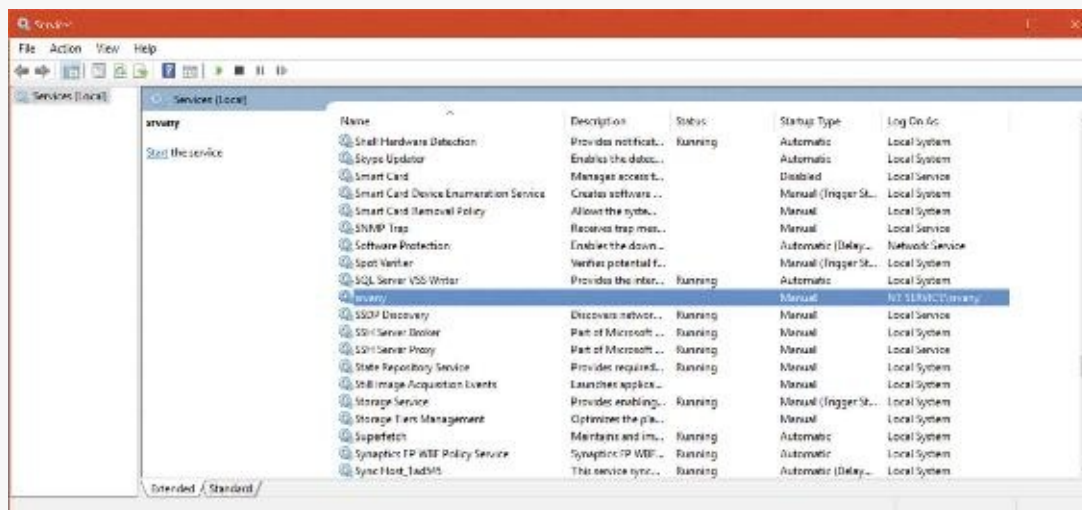
You can create a service that runs under a virtual service account by using the Service Control (Sc.exe) tool. Follow these steps:

1. In an Administrator command prompt, type the **create** command in the Sc.exe command-line tool to create a service and a virtual account in which it will run. This example uses the **srvany** service from the Windows 2003 resource kit, which you can download here: <https://www.microsoft.com/en-us/download/details.aspx?id=17657>.

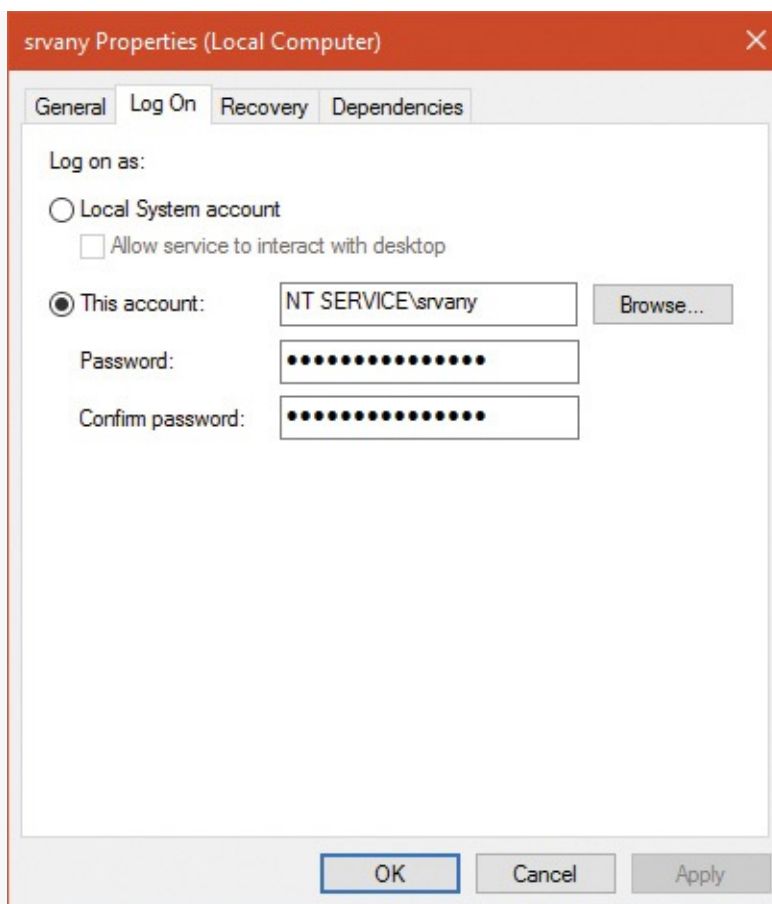
[Click here to view code image](#)

```
C:\Windows\system32>sc create srvany obj= "NT SERVICE\srvany"
binPath=
"c:\temp\srvany.exe"
[SC] CreateService SUCCESS
```

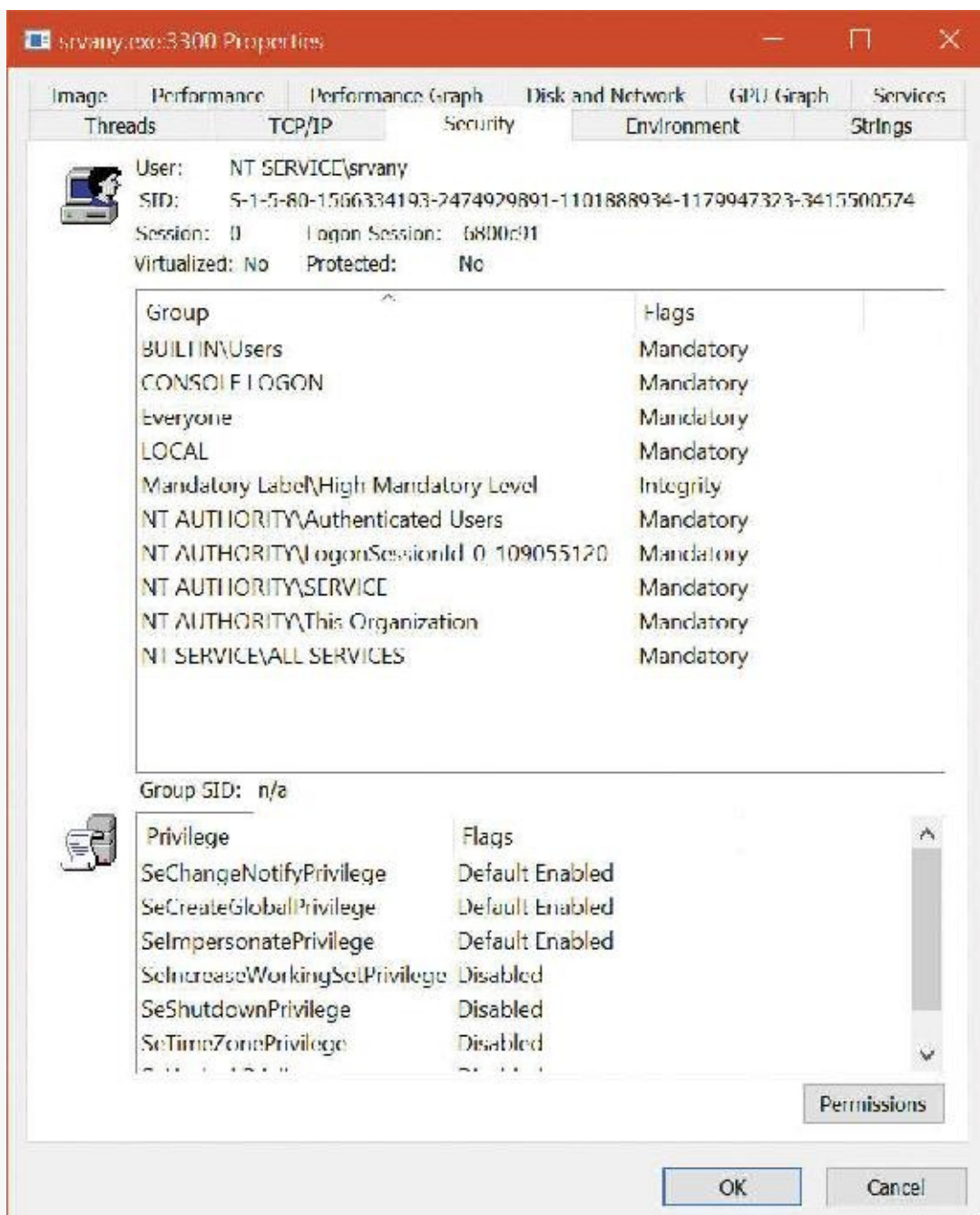
2. The previous command created the service (in the registry and in the service controller manager's internal list) and created the virtual service account. Now run the Services MMC snap-in (services.msc), select the new service, and open its Properties dialog box.



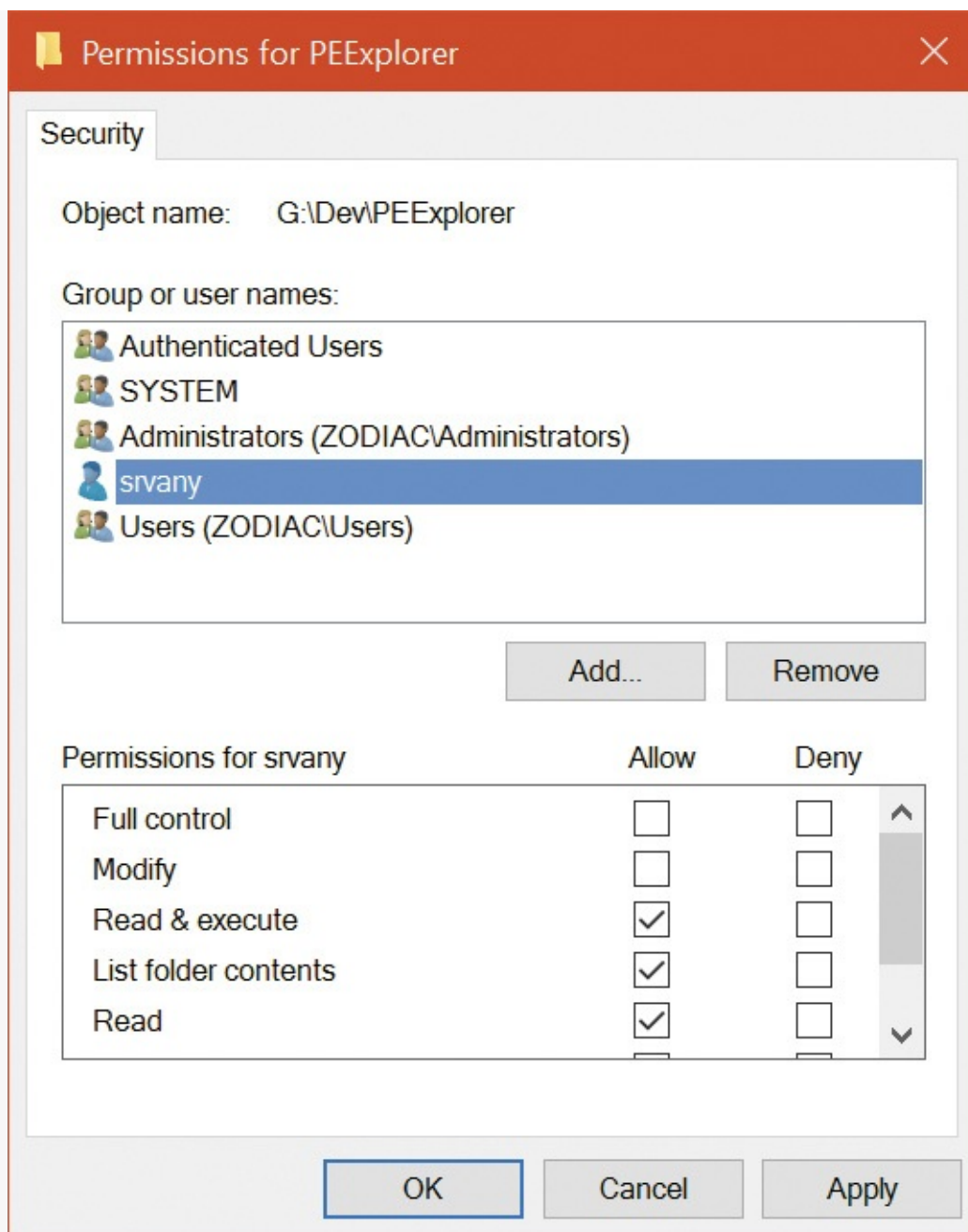
3. Click the **Log On** tab.



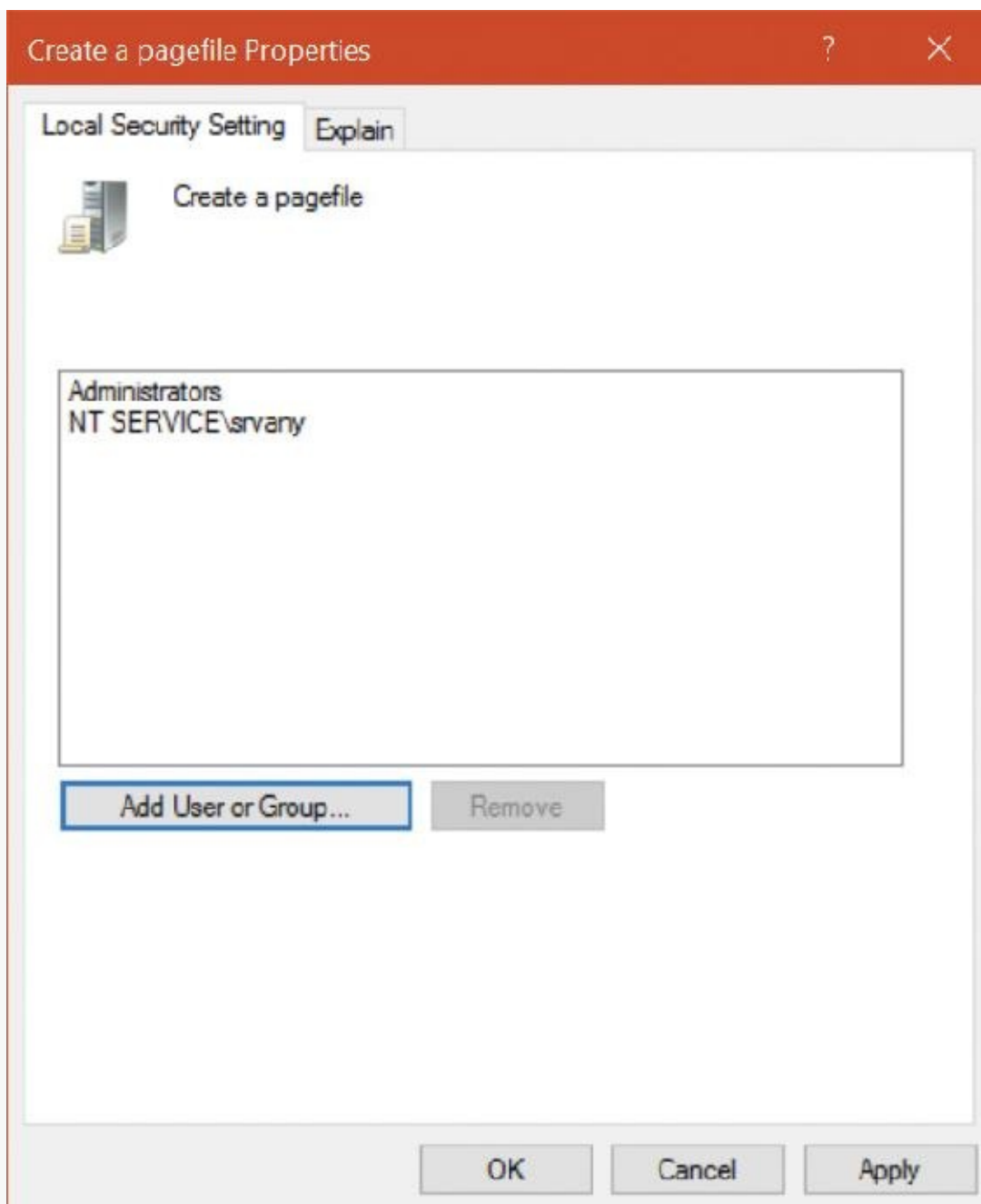
4. You can use the service's Properties dialog box to create a virtual service account for an existing service. To do so, change the account name to **NT SERVICE\servicename** in the This Account field and clear both password fields. Note, however, that existing services might not run correctly under a virtual service account because the account might not have access to files or other resources needed by the service.
5. If you run Process Explorer and view the Security tab in the Properties dialog box for a service that uses a virtual account, you can observe the virtual account name and its security ID (SID). To try this, in the Properties dialog box of the **srvany** service, enter the command-line arguments **notepad.exe**. (**srvany** can be used to turn normal executables into services, so it must accept some executable on the command line.) Then click the **Start** button to start the service.



6. The virtual service account can appear in an access control entry for any object (such as a file) the service needs to access. If you click the **Security** tab in a file's Properties dialog box and create an ACL that references the virtual service account, you will find that the account name you typed (for example, NT SERVICE\srvary) is changed to simply the service name (srvary) by the Check Names function, and it appears in the access control list in this shortened form.

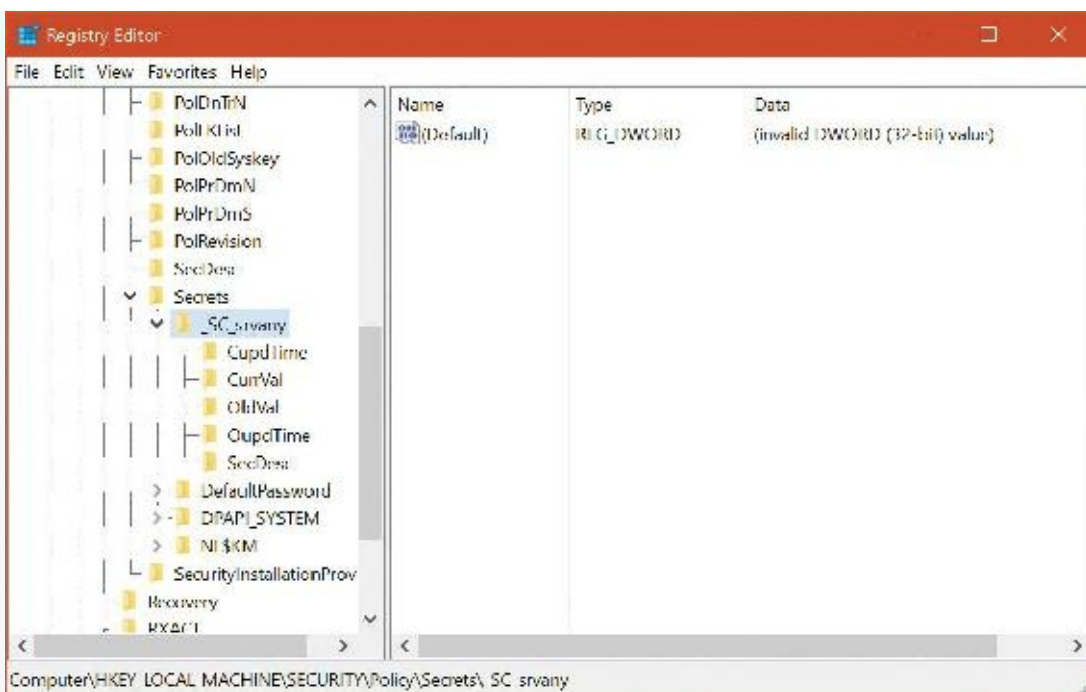


7. The virtual service account can be granted permissions (or user rights) via Group Policy. In this example, the virtual account for the **srvary** service has been granted the right to create a pagefile (using the Local Security Policy editor, secpol.msc).



8. You won't see the virtual service account in user-administration tools like `lusrmgr.msc` because it is not stored in the SAM registry hive. However, if you examine the registry within the context of the built-in System account (as described previously), you will see evidence of the account in the `HKLM\Security\Policy\Secrets` key:

```
C:\>psexec -s -i -d regedit.exe
```



Security descriptors and access control

Tokens, which identify a user's credentials, are only part of the object security equation. Another part of the equation is the security information associated with an object, which specifies who can perform what actions on the object. The data structure for this information is called a *security descriptor*. A security descriptor consists of the following attributes:

- **Revision number** This is the version of the SRM security model used to create the descriptor.
- **Flags** These are optional modifiers that define the behavior or characteristics of the descriptor. These flags are listed in [Table 7-5](#) (most are documented in the Windows SDK).
- **Owner SID** This is the owner's SID.
- **Group SID** This is the SID of the primary group for the object (used only by the POSIX subsystem, now unused since POSIX is no longer supported).
- **Discretionary access control list (DACL)** This specifies who has what access to the object.
- **System access control list (SACL)** This specifies which operations by which users should be logged in the security audit log and the explicit integrity level of an object.

Flag	Meaning
SE_OWNER_DEFAULTED	This indicates a security descriptor with a default owner security identifier (SID). Use this bit to find all the objects that have default owner permissions set.
SE_GROUP_DEFAULTED	This indicates a security descriptor with a default group SID. Use this bit to find all the objects that have default group permissions set.
SE_DACL_PRESENT	This indicates a security descriptor that has a DACL. If this flag is not set, or if this flag is set and the DACL is NULL, the security descriptor allows full access to everyone.
SE_DACL_DEFAULTED	This indicates a security descriptor with a default DACL. For example, if an object creator does not specify a DACL, the object receives the default DACL from the access token of the creator. This flag can affect how the system treats the DACL with respect to access control entry (ACE) inheritance. The system ignores this flag if the SE_DACL_PRESENT flag is not set.
SE_SACL_PRESENT	This indicates a security descriptor that has a system access control list (SACL).
SE_SACL_DEFAULTED	This indicates a security descriptor with a default SACL. For example, if an object creator does not specify a SACL, the object receives the default SACL from the access token of the creator. This flag can affect how the system treats the SACL with respect to ACE inheritance. The system ignores this flag if the SE_SACL_PRESENT flag is not set.
SE_DACL_UNTRUSTED	This indicates that the ACL pointed to by the DACL of the security descriptor was provided by an untrusted source. If this flag is set and a compound ACE is encountered, the system will substitute known valid SIDs for the server SIDs in the ACEs.
SE_SERVER_SECURITY	This requests that the provider for the object protected by the security descriptor be a server ACL based on the input ACL, regardless of its source (explicit or defaulting). This is done by replacing all the GRANT ACEs with compound ACEs granting the current server access. This flag is meaningful only if the subject is impersonating.
SE_DACL_AUTO_INHERIT_REQ	This requests that the provider for the object protected by the security descriptor automatically propagate the DACL to existing child objects. If the provider supports automatic inheritance, the DACL is propagated to any existing child objects, and the SE_DACL_AUTO_INHERITED bit in the security descriptor of the parent and child objects is set.
SE_SACL_AUTO_INHERIT_REQ	This requests that the provider for the object protected by the security descriptor automatically propagate the SACL to existing child objects. If the provider supports automatic inheritance, the SACL is propagated to any existing child objects, and the SE_SACL_AUTO_INHERITED bit in the security descriptors of the parent object and child objects is set.
SE_DACL_AUTO_INHERITED	This indicates a security descriptor in which the DACL is set up to support automatic propagation of inheritable ACEs to existing child objects. The system sets this bit when it performs the automatic inheritance algorithm for the object and its existing child objects.
SE_SACL_AUTO_INHERITED	This indicates a security descriptor in which the SACL is set up to support automatic propagation of inheritable ACEs to existing child objects. The system sets this bit when it performs the automatic inheritance algorithm for the object and its existing child objects.
SE_DACL_PROTECTED	This prevents the DACL of a security descriptor from being modified by inheritable ACEs.
SE_SACL_PROTECTED	This prevents the SACL of a security descriptor from being modified by inheritable ACEs.
SE_RM_CONTROL_VALID	This indicates that the resource control manager bits in the security descriptor are valid. The resource control manager bits are 8 bits in the security descriptor structure that contains information specific to the resource manager accessing the structure.
SE_SELF_RELATIVE	This indicates a security descriptor in self-relative format, with all the security information in a contiguous block of memory. If this flag is not set, the security descriptor is in absolute format.

TABLE 7-5 Security descriptor flags

Security descriptors (SDs) can be retrieved programmatically by using various functions, such as `GetSecurityInfo`, `GetKernelObjectSecurity`, `GetFileSecurity`, `GetNamedSecurityInfo`, and other more esoteric functions. After retrieval, the SD can be

manipulated and then the relevant `Set` function called to make the change. Furthermore, a security descriptor can be constructed using a string in a language called Security Descriptor Definition Language (SDDL), which is capable of representing a security descriptor using a compact string. This string can be converted to a true SD by calling `ConvertStringSecurityDescriptorToSecurityDescriptor`. As you might expect, the converse function exists as well (`ConvertSecurityDescriptorToStringSecurityDescriptor`). See the Windows SDK for a detailed description of the SDDL.

An access control list (ACL) is made up of a header and zero or more access control entry (ACE) structures. There are two types of ACLs: DACLs and SACLs. In a DACL, each ACE contains a SID and an access mask (and a set of flags, explained shortly), which typically specifies the access rights (read, write, delete, and so forth) that are granted or denied to the holder of the SID. There are nine types of ACEs that can appear in a DACL: access allowed, access denied, allowed object, denied object, allowed callback, denied callback, allowed object callback, denied-object callback, and conditional claims. As you would expect, the access-allowed ACE grants access to a user, and the access-denied ACE denies the access rights specified in the access mask. The callback ACEs are used by applications that make use of the AuthZ API (described later) to register a callback that AuthZ will call when it performs an access check involving this ACE.

The difference between allowed object and access allowed, and between denied object and access denied, is that the object types are used only within Active Directory. ACEs of these types have a globally unique identifier (GUID) field that indicates that the ACE applies only to particular objects or subobjects (those that have GUID identifiers). (A GUID is a 128-bit identifier guaranteed to be universally unique.) In addition, another optional GUID indicates what type of child object will inherit the ACE when a child is created within an Active Directory container that has the ACE applied to it. The conditional claims ACE is stored in a `*-callback` type ACE structure and is described in the section on the AuthZ APIs.

The accumulation of access rights granted by individual ACEs forms the set of access rights granted by an ACL. If no DACL is present (a null DACL) in a security descriptor, everyone has full access to the object. If the DACL is empty (that is, it has zero ACEs), no user has access to the object.

The ACEs used in DACLs also have a set of flags that control and specify characteristics of the ACE related to inheritance. Some object namespaces have containers and objects. A container can hold other container objects and leaf objects, which are its child objects. Examples of containers are directories in the file system namespace and keys in the registry namespace. Certain flags in an ACE control how the ACE propagates to child objects of the container associated with the ACE. [Table 7-6](#), reproduced in part from the Windows SDK, lists the inheritance rules for ACE flags.

Flag	Inheritance Rule
CONTAINER_INHERIT_ACE	Child objects that are containers, such as directories, inherit the ACE as an effective ACE. The inherited ACE is inheritable unless the NO_PROPAGATE_INHERIT_ACE bit flag is also set.
INHERIT_ONLY_ACE	This flag indicates an inherit-only ACE that doesn't control access to the object it's attached to. If this flag is not set, the ACE controls access to the object to which it is attached.
INHERITED_ACE	This flag indicates that the ACE was inherited. The system sets this bit when it propagates an inheritable ACE to a child object.
NO_PROPAGATE_INHERIT_ACE	If the ACE is inherited by a child object, the system clears the OBJECT_INHERIT_ACE and CONTAINER_INHERIT_ACE flags in the inherited ACE. This action prevents the ACE from being inherited by subsequent generations of objects.
OBJECT_INHERIT_ACE	Non-container child objects inherit the ACE as an effective ACE. For child objects that are containers, the ACE is inherited as an inherit-only ACE unless the NO_PROPAGATE_INHERIT_ACE bit flag is also set.

TABLE 7-6 Inheritance rules for ACE flags

A SACL contains two types of ACEs: system audit ACEs and system audit-object ACEs. These ACEs specify which operations performed on the object by specific users or groups should be audited. Audit information is stored in the system audit log. Both successful and unsuccessful attempts can be audited. Like their DACL object-specific ACE cousins, system audit-object ACEs specify a GUID indicating the types of objects or sub-objects that the ACE applies to and an optional GUID that controls propagation of the ACE to particular child object types. If a SACL is null, no auditing takes place on the object. (Security auditing is described later in this chapter.) The inheritance flags that apply to DACL ACEs also apply to system audit and system audit-object ACEs.

[Figure 7-8](#) is a simplified picture of a file object and its DACL. As shown, the first ACE allows USER1 to read the file. The second ACE denies members of the group TEAM1 write access to the file. The third ACE grants all other users (Everyone) execute access.

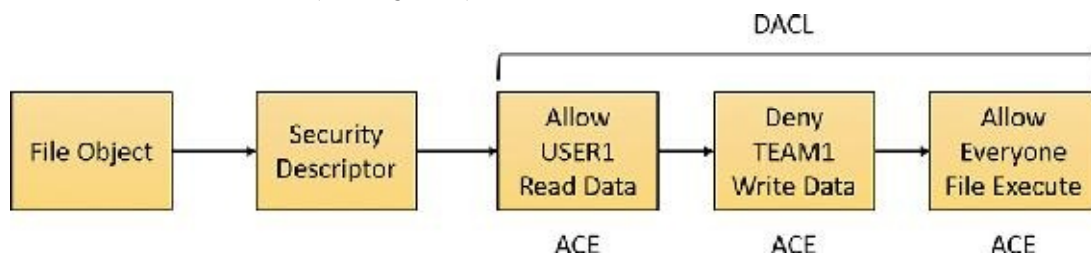


FIGURE 7-8 Discretionary access control list (DACL).

EXPERIMENT: Viewing a security descriptor

Most executive subsystems rely on the object manager's default security functionality to manage security descriptors for their objects. The object manager's default security functions use the security descriptor pointer to store security descriptors for such objects. For example, the process manager uses default security, so the object manager stores process and thread security descriptors in the object headers of process and thread objects, respectively. The security descriptor pointer of events, mutexes, and semaphores also store their security descriptors. You can use live kernel debugging to view the security descriptors of these objects once you locate their object header, as outlined in the following steps. (Note that both Process Explorer and AccessChk can also show security descriptors for processes.)

1. Start local kernel debugging.
2. Type **!process 0 0 explorer.exe** to obtain process information about Explorer:

[Click here to view code image](#)

```
lkd> !process 0 0 explorer.exe
PROCESS fffffe18304dfd780
    SessionId: 1 Cid: 23e4 Peb: 00c2a000 ParentCid: 2264
    DirBase: 2aa0f6000 ObjectTable:
ffffcd82c72fcd80 HandleCount:
<Data Not Accessible>
    Image: explorer.exe
```

```
PROCESS fffffe1830670a080
    SessionId: 1 Cid: 27b8 Peb: 00950000 ParentCid: 035c
    DirBase: 2cba97000 ObjectTable:
ffffcd82c7ccc500 HandleCount:
<Data Not Accessible>
    Image: explorer.exe
```

3. If more than one instance of explorer is listed, choose one. (It doesn't matter which.) Type **!object** with the address of the EPROCESS in the output of the previous command as the argument to show the object data structure:

[Click here to view code image](#)

```
lkd> !object fffffe18304dfd780
Object: fffffe18304dfd780 Type: (fffffe182f7496690) Process
    ObjectHeader: fffffe18304dfd750 (new version)
    HandleCount: 15 PointerCount: 504639
```

4. Type **dt _OBJECT_HEADER** and the address of the object header field from the previous command's output to show the object header data structure, including the security descriptor pointer value:

[Click here to view code image](#)

```
lkd> dt nt!_object_header fffffe18304dfd750
+0x000 PointerCount      : 0n504448
+0x008 HandleCount      : 0n15
+0x008 NextToFree       : 0x00000000'0000000f Void
+0x010 Lock             : _EX_PUSH_LOCK
+0x018 TypeIndex        : 0xe5 ''
+0x019 TraceFlags       : 0 ''
+0x019 DbgRefTrace      : 0y0
+0x019 DbgTracePermanent : 0y0
+0x01a InfoMask         : 0x88 ''
+0x01b Flags            : 0 ''
+0x01b NewObject        : 0y0
+0x01b KernelObject     : 0y0
+0x01b KernelOnlyAccess : 0y0
+0x01b ExclusiveObject  : 0y0
+0x01b PermanentObject  : 0y0
+0x01b DefaultSecurityQuota : 0y0
+0x01b SingleHandleEntry : 0y0
```

```

+0x01b DeletedInline      : 0y0
+0x01c Reserved           : 0x30003100
+0x020 ObjectCreateInfo   : 0xfffffe183'09e84ac0
_OBJECT_CREATE_INFORMATION
+0x020 QuotaBlockCharged   : 0xfffffe183'09e84ac0 Void
+0x028 SecurityDescriptor : 0xffffcd82'cd0e97ed Void
+0x030 Body                : _QUAD

```

5. Finally, use the debugger's !sd command to dump the security descriptor. The security descriptor pointer in the object header uses some of the low-order bits as flags, and these must be zeroed before following the pointer. On 32-bit systems there are three flag bits, so use & -8 with the security descriptor address displayed in the object header structure, as follows. On 64-bit systems there are four flag bits, so you use & -10 instead.

[Click here to view code image](#)

```

lkd> !sd 0xffffcd82'cd0e97ed & -10
->Revision: 0x1
->Sbz1      : 0x0
->Control    : 0x8814
               SE_DACL_PRESENT
               SE_SACL_PRESENT
               SE_SACL_AUTO_INHERITED
               SE_SELF_RELATIVE
->Owner      : S-1-5-21-3537846094-3055369412-2967912182-1001
->Group      : S-1-5-21-3537846094-3055369412-2967912182-1001
->Dacl       :
->Dacl       : ->AclRevision: 0x2
->Dacl       : ->Sbz1        : 0x0
->Dacl       : ->AclSize     : 0x5c
->Dacl       : ->AceCount    : 0x3
->Dacl       : ->Sbz2        : 0x0
->Dacl       : ->Ace[0]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl       : ->Ace[0]: ->AceFlags: 0x0
->Dacl       : ->Ace[0]: ->AceSize: 0x24
->Dacl       : ->Ace[0]: ->Mask : 0x001ffffff
->Dacl       : ->Ace[0]: ->SID: S-1-5-21-3537846094-3055369412-
2967912182-1001

->Dacl       : ->Ace[1]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl       : ->Ace[1]: ->AceFlags: 0x0
->Dacl       : ->Ace[1]: ->AceSize: 0x14
->Dacl       : ->Ace[1]: ->Mask : 0x001ffffff
->Dacl       : ->Ace[1]: ->SID: S-1-5-18

->Dacl       : ->Ace[2]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl       : ->Ace[2]: ->AceFlags: 0x0
->Dacl       : ->Ace[2]: ->AceSize: 0x1c

```

```
->Dacl      : ->Ace[2]: ->Mask : 0x00121411
->Dacl      : ->Ace[2]: ->SID: S-1-5-5-0-1745560
```

```
->Sacl      :
->Sacl      : ->AclRevision: 0x2
->Sacl      : ->Sbz1       : 0x0
->Sacl      : ->AclSize    : 0x1c
->Sacl      : ->AceCount   : 0x1
->Sacl      : ->Sbz2       : 0x0
->Sacl      : ->Ace[0]: ->AceType: SYSTEM_MANDATORY_LABEL_ACE_TYPE
->Sacl      : ->Ace[0]: ->AceFlags: 0x0
->Sacl      : ->Ace[0]: ->AceSize: 0x14
->Sacl      : ->Ace[0]: ->Mask : 0x00000003
->Sacl      : ->Ace[0]: ->SID: S-1-16-8192
```

The security descriptor contains three access-allowed ACEs: one for the current user (S-1-5-21-3537846094-3055369412-2967912182-1001), one for the System account (S-1-5-18), and the last for the Logon SID (S-1-5-5-0-1745560). The system access control list has one entry (S-1-16-8192) labeling the process as medium integrity level.

ACL assignment

To determine which DACL to assign to a new object, the security system uses the first applicable rule of the following four assignment rules:

1. If a caller explicitly provides a security descriptor when creating the object, the security system applies it to the object. If the object has a name and resides in a container object (for example, a named event object in the \BaseNamedObjects object manager namespace directory), the system merges any inheritable ACEs (ACEs that might propagate from the object's container) into the DACL unless the security descriptor has the SE_DACL_PROTECTED flag set, which prevents inheritance.
2. If a caller doesn't supply a security descriptor and the object has a name, the security system looks at the security descriptor in the container in which the new object name is stored. Some of the object directory's ACEs might be marked as inheritable, meaning they should be applied to new objects created in the object directory. If any of these inheritable ACEs are present, the security system forms them into an ACL, which it attaches to the new object. (Separate flags indicate ACEs that should be inherited only by container objects rather than by objects that aren't containers.)
3. If no security descriptor is specified and the object doesn't inherit any ACEs, the security system retrieves the default DACL from the caller's access token and applies it to the new object. Several subsystems on Windows have hard-coded DACLs that they assign on object creation (for example, services, LSA, and SAM objects).
4. If there is no specified descriptor, no inherited ACEs, and no default DACL, the system creates the object with no DACL, which allows everyone (all users and groups) full access to the object. This rule is the same as the third rule, in which a token contains a null default DACL.

The rules the system uses when assigning a SACL to a new object are similar to those used for DACL

assignment, with some exceptions:

- Inherited system audit ACEs don't propagate to objects with security descriptors marked with the `SE_SACL_PROTECTED` flag (similar to the `SE_DACL_PROTECTED` flag, which protects DACLs).
- If there are no specified security audit ACEs and there is no inherited SACL, no SACL is applied to the object. This behavior is different from that used to apply default DACLs because tokens don't have a default SACL.

When a new security descriptor containing inheritable ACEs is applied to a container, the system automatically propagates the inheritable ACEs to the security descriptors of child objects. (Note that a security descriptor's DACL doesn't accept inherited DACL ACEs if its `SE_DACL_PROTECTED` flag is enabled, and its SACL doesn't inherit SACL ACEs if the descriptor has the `SE_SACL_PROTECTED` flag set.) The order in which inheritable ACEs are merged with an existing child object's security descriptor is such that any ACEs that were explicitly applied to the ACL are kept ahead of ACEs that the object inherits. The system uses the following rules for propagating inheritable ACEs:

- If a child object with no DACL inherits an ACE, the result is a child object with a DACL containing only the inherited ACE.
- If a child object with an empty DACL inherits an ACE, the result is a child object with a DACL containing only the inherited ACE.
- For objects in Active Directory only, if an inheritable ACE is removed from a parent object, automatic inheritance removes any copies of the ACE inherited by child objects.
- For objects in Active Directory only, if automatic inheritance results in the removal of all ACEs from a child object's DACL, the child object has an empty DACL rather than no DACL.

As you'll soon discover, the order of ACEs in an ACL is an important aspect of the Windows security model.



Note

Inheritance is generally not directly supported by the object stores, such as file systems, the registry, or Active Directory. Windows APIs that support inheritance, including `SetEntriesInAcl`, do so by invoking appropriate functions within the security inheritance support DLL (`%SystemRoot%\System32\Ntmaria.dll`) that know how to traverse those object stores.

Trust ACEs

The advent of protected processes and Protected Processes Light (PPL, discussed in [Chapter 3](#)) created a need for such a process to make objects as accessible by protected processes only. This is important to protect certain resources such as the `KnownDlls` registry key from tampering, even by admin-level code. Such ACEs are specified with well-known SIDs that provide the protection level and signer that is required to obtain access. [Table 7-7](#) shows the SIDs and their level and meaning.

SID	Protection Level	Protection Signer
1-19-512-0	Protected Light	None
1-19-512-4096	Protected Light	Windows
1-19-512-8192	Protected Light	WinTcb
1-19-1024-0	Protected	None
1-19-1024-4096	Protected	Windows
1-19-1024-8192	Protected	WinTcb

TABLE 7-7 Trust SIDs

A *trust SID* is part of a token object that exists for tokens attached to protected or PPL processes. The higher the SID number, the more powerful the token is. (remember that Protected is higher than Protected Light).

EXPERIMENT: Viewing trust SIDs

In this experiment, you'll look at trust SIDs in tokens of protected processes.

Start local kernel debugging.

List Csrss.exe processes with basic information:

[Click here to view code image](#)

```
lkd> !process 0 1 csrss.exe
PROCESS fffff8188e50b5780
    SessionId: 0  Cid: 0358    Peb: b3a9f5e000  ParentCid: 02ec
    DirBase: 1273a3000  ObjectTable:
fffffb0d829e2040  HandleCount:
<Data Not Accessible>
    Image: csrss.exe
    VadRoot fffff8188e6ccc8e0  Vads 159 Clone 0 Private 324.
Modified 4470.
Locked 0.
    DeviceMap fffffb0d70c15620
    Token                                     fffffb0d829e7060
    ...
PROCESS fffff8188e7a92080
    SessionId: 1  Cid: 03d4    Peb: d5b0de4000  ParentCid: 03bc
    DirBase: 162d93000  ObjectTable:
fffffb0d8362d7c0  HandleCount:
<Data Not Accessible>Modified 462372.  Locked 0.
    DeviceMap fffffb0d70c15620
    Token                                     fffffb0d8362d060
    ...
```

Select one of the tokens and show its details:

[Click here to view code image](#)

```
lkd> !token fffffb0d829e7060
_TOKEN 0xfffffb0d829e7060
TS Session ID: 0
User: S-1-5-18
...
Process Token TrustLevelSid: S-1-19-512-8192
```

That's a PPL with a WinTcb signer.

Determining access

Two methods are used for determining access to an object:

- The mandatory integrity check, which determines whether the integrity level of the caller is high enough to access the resource, based on the resource's own integrity level and its mandatory policy.
- The discretionary access check, which determines the access that a specific user account has to an object.

When a process tries to open an object, the integrity check takes place before the standard Windows DACL check in the kernel’s SeAccessCheck function because it is faster to execute and can quickly eliminate the need to perform the full discretionary access check. Given the default integrity policies in its access token (TOKEN_MANDATORY_NO_WRITE_UP and TOKEN_MANDATORY_NEW_PROCESS_MIN, described previously), a process can open an object for write access if its integrity level is equal to or higher than the object’s integrity level and the DACL also grants the process the accesses it desires. For example, a low-integrity-level process cannot open a medium-integrity-level process for write access, even if the DACL grants the process write access.

With the default integrity policies, processes can open any object—with the exception of process, thread, and token objects—for read access as long as the object’s DACL grants them read access. That means a process running at low integrity level can open any files accessible to the user account in which it’s running. Protected Mode Internet Explorer uses integrity levels to help prevent malware that infects it from modifying user account settings, but it does not stop malware from reading the user’s documents.

Recall that process, thread, and token objects are exceptions because their integrity policy also includes No-Read-Up. That means a process integrity level must be equal to or higher than the integrity level of the process or thread it wants to open, and the DACL must grant it the access it wants for an attempt to open it to succeed. Assuming the DACLs allow the desired access, [Table 7-8](#) shows the types of access that processes running at various integrity levels have to other processes and objects.

Accessing Process	Access to Objects	Access to Other Processes
High integrity level	Read/write to all objects with integrity level of High or lower Read access to objects with integrity level of System	Read/write access to all processes with High or lower integrity level No read/write access to processes with System integrity level
Medium integrity level	Read/write to all objects with integrity level of Medium or Low Read access to objects with integrity level of High or System	Read/write access to all processes with Medium or Low integrity level No read/write access to processes with High or System integrity level
Low integrity level	Read/write to all objects with integrity level of Low Read access to objects with integrity level of Medium or higher	Read/write access to all processes with Low integrity level No read/write access to processes with Medium or higher integrity level

TABLE 7-8 Accessing objects and processes based on integrity level



Note

The read access to a process described in this section means full read access, such as reading the contents of the process address space. No-Read-Up does not prevent opening a higher-integrity-level process from a lower one for a more limited access, such as `PROCESS_QUERY_LIMITED_INFORMATION`, which provides only basic information about the process.

User Interface Privilege Isolation

The Windows messaging subsystem also honors integrity levels to implement User Interface Privilege Isolation (UIPI). The subsystem does this by preventing a process from sending window messages to the windows owned by a process having a higher integrity level, with the following informational messages being exceptions:

- WM_NULL
- WM_MOVE
- WM_SIZE
- WM_GETTEXT
- WM_GETTEXTLENGTH
- WM_GETHOTKEY
- WM_GETICON
- WM_RENDERFORMAT
- WM_DRAWCLIPBOARD
- WM_CHANGECHAIN
- WM_THEMECHANGED

This use of integrity levels prevents standard user processes from driving input into the windows of elevated processes or from performing a shatter attack (such as sending the process malformed messages that trigger internal buffer overflows, which can lead to the execution of code at the elevated process's privilege level). UIPI also blocks window hooks (`SetWindowsHookEx` API) from affecting the windows of higher-integrity-level processes so that a standard user process can't log the keystrokes the user types into an administrative application, for example. Journal hooks are also blocked in the same way to prevent lower-integrity-level processes from monitoring the behavior of higher-integrity-level processes.

Processes (running with medium or higher integrity level only) can choose to allow additional messages to pass the guard by calling the `ChangeWindowMessageFilterEx` API. This function is typically used to add messages required by custom controls to communicate outside native common controls in Windows. An older API, `ChangeWindowMessageFilter`, performs a similar function, but it is per-process rather than per-window. With `ChangeWindowMessageFilter`, it is possible for two custom controls inside the same process to be using the same internal window messages, which could lead to one control's potentially malicious window message to be allowed through, simply because it happens to be a query-only message for the other custom control.

Because accessibility applications such as the On-Screen Keyboard (`Osk.exe`) are subject to UIPI's restrictions (which would require the accessibility application to be executed for each kind of visible integrity-level process on the desktop), these processes can enable UI access. This flag can be present in the manifest file of the image and will run the process at a slightly higher integrity level than medium (between `0x2000` and `0x3000`) if launched from a standard user account, or at high integrity level if launched from an Administrator account. Note that in the second case, an elevation request won't actually be displayed. For a process to set this flag, its image must also be signed and in one of several secure locations, including `%SystemRoot%` and `%ProgramFiles%`.

After the integrity check is complete, and assuming the mandatory policy allows access to the object

based on the caller's integrity, one of two algorithms is used for the discretionary check to an object, which will determine the outcome of the access check:

- Determine the maximum access allowed to the object, a form of which is exported to user mode using the AuthZ API (described in the section “[The AuthZ API](#)” later in this chapter) or the older `GetEffectiveRightsFromAcl` function. This is also used when a program specifies a desired access of `MAXIMUM_ALLOWED`, which is what the legacy APIs that don't have a desired access parameter use.
- Determine whether a specific desired access is allowed, which can be done with the Windows `AccessCheck` function or the `AccessCheckByType` function.

The first algorithm examines the entries in the DACL as follows:

1. If the object has no DACL (a null DACL), the object has no protection and the security system grants all access, unless the access is from an AppContainer process (discussed in the “[AppContainers](#)” section later in this chapter), which means access is denied.
2. If the caller has the take-ownership privilege, the security system grants write-owner access before examining the DACL. (Take-ownership privilege and write-owner access are explained in a moment.)
3. If the caller is the owner of the object, the system looks for an `OWNER_RIGHTS` SID and uses that SID as the SID for the next steps. Otherwise, read-control and write-DACL access rights are granted.
4. For each access-denied ACE that contains a SID that matches one in the caller's access token, the ACE's access mask is removed from the granted-access mask.
5. For each access-allowed ACE that contains a SID that matches one in the caller's access token, the ACE's access mask is added to the granted-access mask being computed, unless that access has already been denied.

When all the entries in the DACL have been examined, the computed granted-access mask is returned to the caller as the maximum allowed access to the object. This mask represents the total set of access types that the caller will be able to successfully request when opening the object.

The preceding description applies only to the kernel-mode form of the algorithm. The Windows version implemented by `GetEffectiveRightsFromAcl` differs in that it doesn't perform step 2, and it considers a single user or group SID rather than an access token.

Owner Rights

Because owners of an object can normally override the security of an object by always being granted read-control and write-DACL rights, a specialized method of controlling this behavior is exposed by Windows: the Owner Rights SID.

The Owner Rights SID exists for two main reasons:

- **To improve service hardening in the operating system** Whenever a service creates an object at run time, the Owner SID associated with that object is the account the service is running in (such as local system or local service) and not the actual service SID. This means that any other service in the same account would have access to the object by being an owner. The Owner Rights SID prevents that unwanted behavior.
- **To allow more flexibility for specific usage scenarios** For example, suppose an administrator wants to allow users to create files and folders but not to modify the ACLs on those objects. (Users could inadvertently or maliciously grant access to those files or folders to unwanted accounts.) By using an inheritable Owner Rights SID, the users can be prevented from editing or even viewing the ACL on the objects they create. A second usage scenario relates to group changes. Suppose an employee has been part of some confidential or sensitive group, has created several files while a member of that group, and has now been removed from the group for business reasons. Because that employee is still a user, he could continue accessing the sensitive files.

The second algorithm is used to determine whether a specific access request can be granted based on the caller's access token. Each open function in the Windows API that deals with securable objects has a parameter that specifies the desired access mask, which is the last component of the security equation. To determine whether the caller has access, the following steps are performed:

1. If the object has no DACL (a null DACL), the object has no protection and the security system grants the desired access.
2. If the caller has the take-ownership privilege, the security system grants write-owner access if requested and then examines the DACL. However, if write-owner access was the only access requested by a caller with take-ownership privilege, the security system grants that access and never examines the DACL.
3. If the caller is the owner of the object, the system looks for an `OWNER_RIGHTS` SID and uses that SID as the SID for the next steps. Otherwise, read-control and write-DACL access rights are granted. If these rights were the only access rights that the caller requested, access is granted without examining the DACL.
4. Each ACE in the DACL is examined from first to last. An ACE is processed if one of the following conditions is satisfied:
 - The ACE is an access-deny ACE, and the SID in the ACE matches an enabled SID (SIDs can be enabled or disabled) or a deny-only SID in the caller's access token.
 - The ACE is an access-allowed ACE, and the SID in the ACE matches an enabled SID in the caller's token that isn't of type deny-only.
 - It is the second pass through the descriptor for restricted-SID checks, and the SID in the ACE matches a restricted SID in the caller's access token.
 - The ACE isn't marked as inherit-only.

5. If it is an access-allowed ACE, the rights in the access mask in the ACE that were requested are granted. If all the requested access rights have been granted, the access check succeeds. If it is an access-denied ACE and any of the requested access rights are in the denied-access rights, access is denied to the object.
6. If the end of the DACL is reached and some of the requested access rights still haven't been granted, access is denied.
7. If all accesses are granted but the caller's access token has at least one restricted SID, the system rescans the DACL's ACEs looking for ACEs with access-mask matches for the accesses the user is requesting and a match of the ACE's SID with any of the caller's restricted SIDs. Only if both scans of the DACL grant the requested access rights is the user granted access to the object.

The behavior of both access-validation algorithms depends on the relative ordering of allow and deny ACEs. Consider an object with only two ACEs: one that specifies that a certain user is allowed full access to an object and one that denies the user access. If the allow ACE precedes the deny ACE, the user can obtain full access to the object, but if the order is reversed, the user cannot gain any access to the object.

Several Windows functions, such as `SetSecurityInfo` and `SetNamedSecurityInfo`, apply ACEs in the preferred order of explicit deny ACEs preceding explicit allow ACEs. For example, the security editor dialog boxes with which you edit permissions on NTFS files and registry keys use these functions. `SetSecurityInfo` and `SetNamedSecurityInfo` also apply ACE inheritance rules to the security descriptor on which they are applied.

[Figure 7-9](#) shows an example of access validation demonstrating the importance of ACE ordering. In the example, access is denied to a user wanting to open a file even though an ACE in the object's DACL grants the access. This is because the ACE denying the user access (by virtue of the user's membership in the Writers group) precedes the ACE granting access.

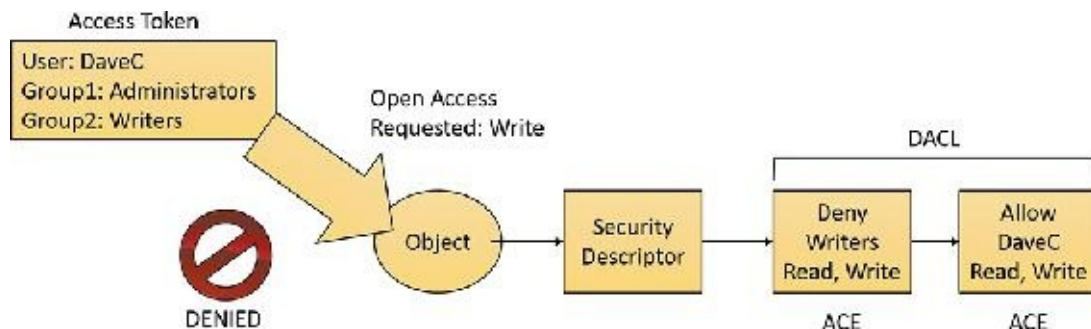


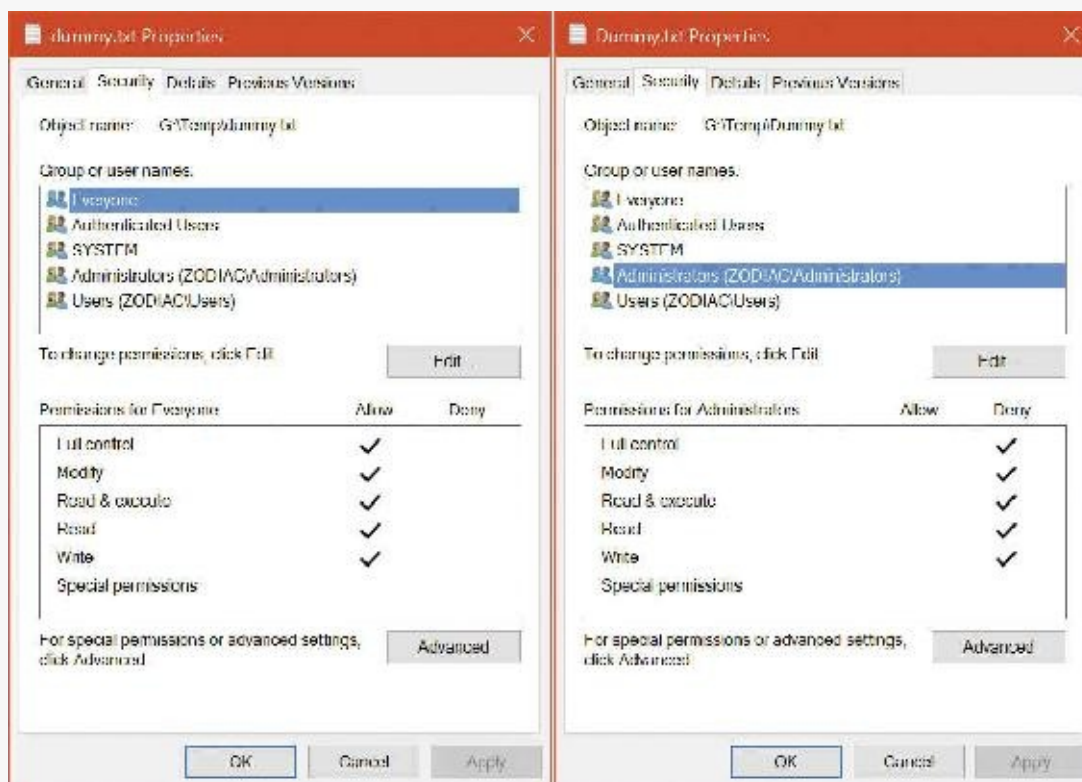
FIGURE 7-9 Access-validation example.

As stated, because it wouldn't be efficient for the security system to process the DACL every time a process uses a handle, the SRM makes this access check only when a handle is opened, not each time the handle is used. Thus, once a process successfully opens a handle, the security system can't revoke the access rights that have been granted, even if the object's DACL changes. Also keep in mind that because kernel-mode code uses pointers rather than handles to access objects, the access check isn't performed when the operating system uses objects. In other words, the Windows executive trusts itself (and all loaded drivers) in a security sense.

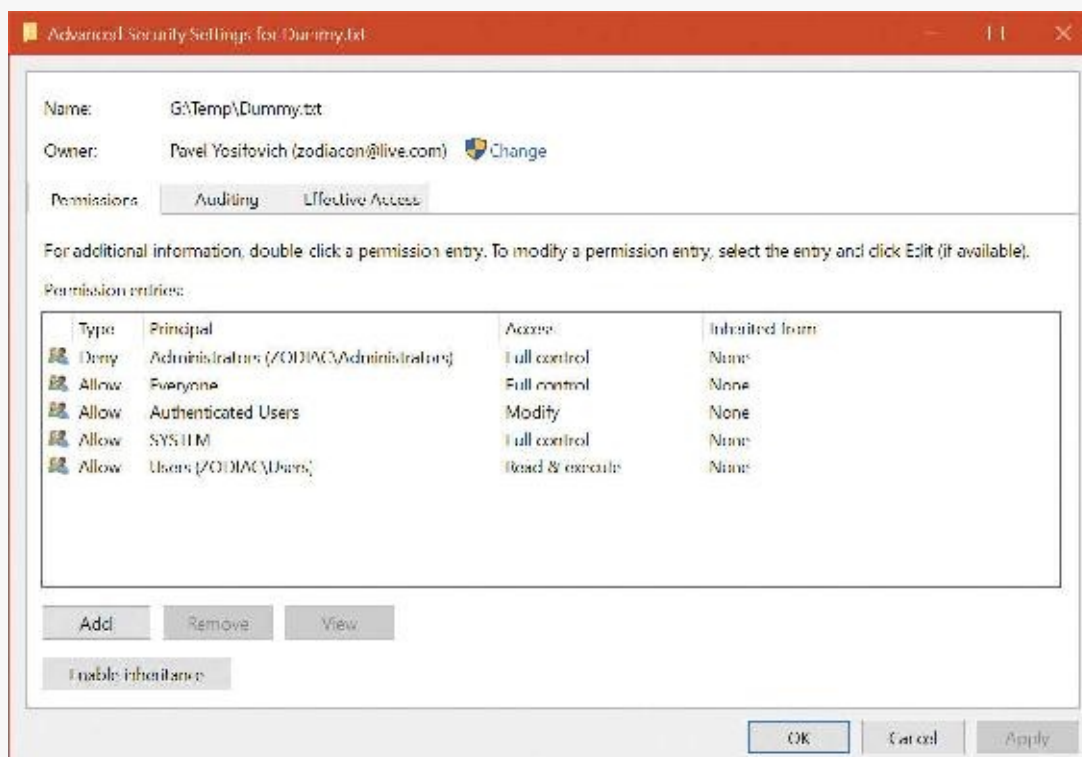
The fact that an object's owner is always granted write-DACL access to an object means that users can never be prevented from accessing the objects they own. If, for some reason, an object had an empty DACL (no access), the owner would still be able to open the object with write-DACL access and then apply a new DACL with the desired access permissions.

A warning regarding the GUI security editors

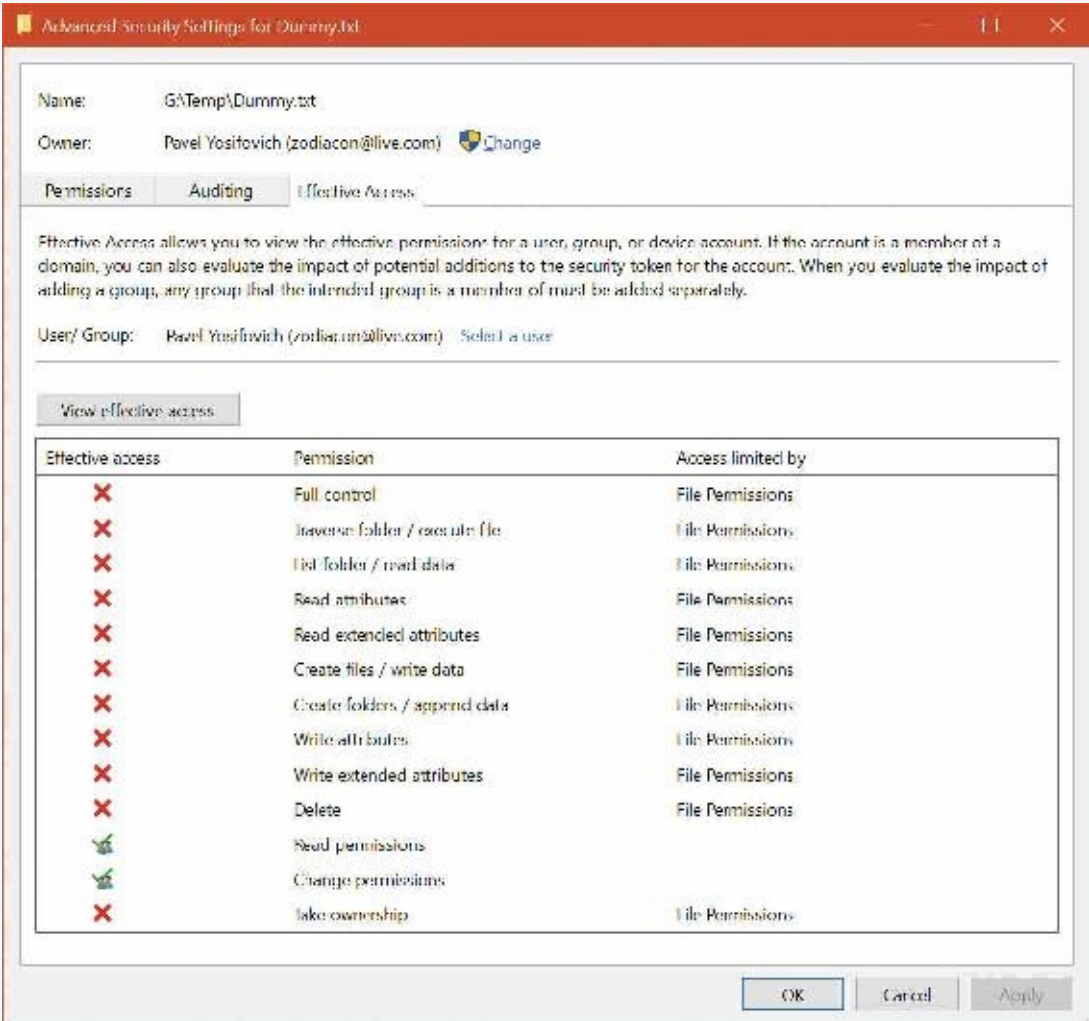
When you use the GUI permissions editors to modify security settings on a file, a registry, or an Active Directory object, or on another securable object, the main security dialog box shows you a potentially misleading view of the security that's applied to the object. If you allow Full Control to the Everyone group and deny the Administrator group Full Control, the list might lead you to believe that the Everyone group access-allowed ACE precedes the Administrator deny ACE because that's the order in which they appear. However, as we've said, the editors place deny ACEs before allow ACEs when they apply the ACL to the object.



The Permissions tab of the Advanced Security Settings dialog box shows the order of ACEs in the DACL. However, even this dialog box can be confusing because a complex DACL can have deny ACEs for various accesses followed by allow ACEs for other access types.



The only definitive way to know what access a particular user or group will have to an object (other than having that user or a member of the group try to access the object) is to use the **Effective Access** tab of the dialog box that is displayed when you click the **Advanced** button in the Properties dialog box. Enter the name of the user or group you want to check and the dialog box shows you what permissions they are allowed for the object.



Dynamic Access Control

The discretionary access control mechanism discussed in previous sections has existed since the first Windows NT version and is useful in many scenarios. There are scenarios, however, where this scheme is not flexible enough. For example, consider a requirement that users accessing a shared file should be allowed to do so if they are using a computer in the workplace, but should not be allowed if accessing the file from their computer at home. There is no way to specify such a condition using an ACE.

Windows 8 and Server 2012 introduced Dynamic Access Control (DAC), a flexible mechanism that can be used to define rules based on custom attributes defined in Active Directory. DAC does not replace the existing mechanism, but adds to it. This means that for an operation to be allowed, both DAC and the classic DACL must grant the permission. [Figure 7-10](#) shows the main aspects of Dynamic Access Control.

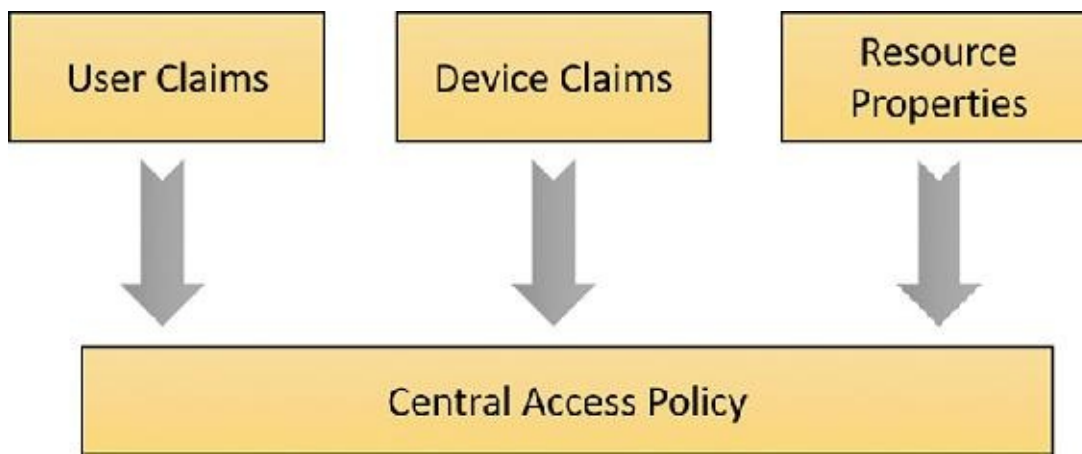


FIGURE 7-10 Dynamic Access Control components.

A claim is any piece of information about a user, device (computer in a domain), or resource (generic attribute) that has been published by a domain controller. Examples of valid claims are a user's title and department classification of a file. Any combination of claims can be used in expressions for building rules. These rules collectively become the central access policy.

DAC configuration is done in Active Directory and pushed through policy. The Kerberos tickets protocol has been enhanced to support authenticated transport of user and device claims (known as *Kerberos armoring*).

The AuthZ API

The AuthZ Windows API provides authorization functions and implements the same security model as the security reference monitor (SRM), but it implements the model totally in user mode in the %SystemRoot%\System32\Authz.dll library. This gives applications that want to protect their own private objects, such as database tables, the ability to leverage the Windows security model without incurring the cost of user mode-to-kernel mode transitions that they would make if they relied on the SRM.

The AuthZ API uses standard security descriptor data structures, SIDs, and privileges. Instead of using tokens to represent clients, AuthZ uses AUTHZ_CLIENT_CONTEXT. AuthZ includes user-mode equivalents of all access-check and Windows security functions—for example, AuthZAccessCheck is the AuthZ version of the AccessCheck Windows API that uses the SeAccessCheck SRM function.

Another advantage available to applications that use AuthZ is that they can direct AuthZ to cache the results of security checks to improve subsequent checks that use the same client context and security descriptor. AuthZ is fully documented in the Windows SDK.

This type of access checking, using a SID and security group membership in a static, controlled environment, is known as *Identity-Based Access Control (IBAC)*, and it requires that the security system know the identity of every possible accessor when the DACL is placed in an object's security descriptor.

Windows includes support for *Claims Based Access Control (CBAC)*, where access is granted not based upon the accessor's identity or group membership, but upon arbitrary attributes assigned to the accessor and stored in the accessor's access token. Attributes are supplied by an attribute provider, such as AppLocker. The CBAC mechanism provides many benefits, including the ability to create a DACL for a user whose identity is not yet known or dynamically calculated user attributes. The CBAC ACE (also known as a conditional ACE) is stored in a *-callback ACE structure, which is essentially private to AuthZ and is ignored by the system SeAccessCheck API. The kernel-mode routine SeSrpAccessCheck does not understand conditional ACEs, so only applications calling the AuthZ

APIs can make use of CBAC. The only system component that makes use of CBAC is AppLocker, for setting attributes such as path or publisher. Third-party applications can make use of CBAC by taking advantage of the CBAC AuthZ APIs.

Using CBAC security checks allows powerful management policies, such as the following:

- Run only applications approved by the corporate IT department.
- Allow only approved applications to access your Microsoft Outlook contacts or calendar.
- Allow only people in a particular building on a specific floor to access printers on that floor.
- Allow access to an intranet website only to full-time employees (as opposed to contractors).

Attributes can be referenced in what is known as a *conditional ACE*, where the presence, absence, or value of one or more attributes is checked. An attribute name can contain any alphanumeric Unicode characters, as well as the following characters: colon (:), forward slash (/), and underscore (_). The value of an attribute can be one of the following: 64-bit integer, Unicode string, byte string, or array.

Conditional ACEs

The format of SDDL strings has been expanded to support ACEs with conditional expressions. The new format of an SDDL string is this:

AceType;AceFlags;Rights;ObjectGuid;InheritObjectGuid; AccountSid; (ConditionalExpression).

The AceType for a conditional ACE is either XA (for SDDL_CALLBACK_ACCESS_ALLOWED) or XD (for SDDL_CALLBACK_ACCESS_DENIED). Note that ACEs with conditional expressions are used for claims-type authorization (specifically, the AuthZ APIs and AppLocker) and are not recognized by the object manager or file systems.

A conditional expression can include any of the elements shown in [Table 7-9](#).

Expression Element	Description
AttributeName	Tests whether the specified attribute has a non-zero value.
exists AttributeName	Tests whether the specified attribute exists in the client context.
AttributeName Operator Value	Returns the result of the specified operation. The following operators are defined for use in conditional expressions to test the values of attributes. All these are binary operators (as opposed to unary) and are used in the form AttributeName Operator Value. The operators are Contains any_of , --, !=, <, <=, >, >=.
ConditionalExpression ConditionalExpression	Tests whether either of the specified conditional expressions is true.
ConditionalExpression && ConditionalExpression	Tests whether both of the specified conditional expressions are true.
!(ConditionalExpression)	The inverse of a conditional expression.
Member of [SidArray]	Tests whether the SID AND_ATTRIBUTES array of the client context contains all the security identifiers (SIDs) in the comma-separated list specified by SidArray.

TABLE 7-9 Acceptable Elements for a Conditional Expression

A conditional ACE can contain any number of conditions. It is ignored if the resultant evaluation of the condition is false or applied if the result is true. A conditional ACE can be added to an object using the AddConditionalAce API and checked using the AuthzAccessCheck API.

A conditional ACE could specify that access to certain data records within a program should be granted only to a user who meets the following criteria (for example):

- Holds the Role attribute, with a value of Architect, Program Manager, or

- Development Lead, and the Division attribute with a value of Windows
- Whose ManagementChain attribute contains the value John Smith
- Whose CommissionType attribute is Officer and whose PayGrade attribute is greater than 6 (that is, the rank of General Officer in the US military)

Windows does not include tools to view or edit conditional ACEs.

Account rights and privileges

Many operations performed by processes as they execute cannot be authorized through object access protection because they do not involve interaction with a particular object. For example, the ability to bypass security checks when opening files for backup is an attribute of an account, not of a particular object. Windows uses both privileges and account rights to allow a system administrator to control what accounts can perform security-related operations.

A *privilege* is the right of an account to perform a particular system-related operation, such as shutting down the computer or changing the system time. An *account right* grants or denies the account to which it's assigned the ability to perform a particular type of logon, such as a local logon or interactive logon, to a computer.

A system administrator assigns privileges to groups and accounts using tools such as the Active Directory Users and Groups MMC snap-in for domain accounts or the Local Security Policy editor (%SystemRoot%\System32\secpol.msc). [Figure 7-11](#) shows the User Rights Assignment configuration in the Local Security Policy editor, which displays the complete list of privileges and account rights available on Windows. Note that the tool makes no distinction between privileges and account rights. However, you can differentiate between them: Any user right that does not contain the words *log on* is an account privilege.

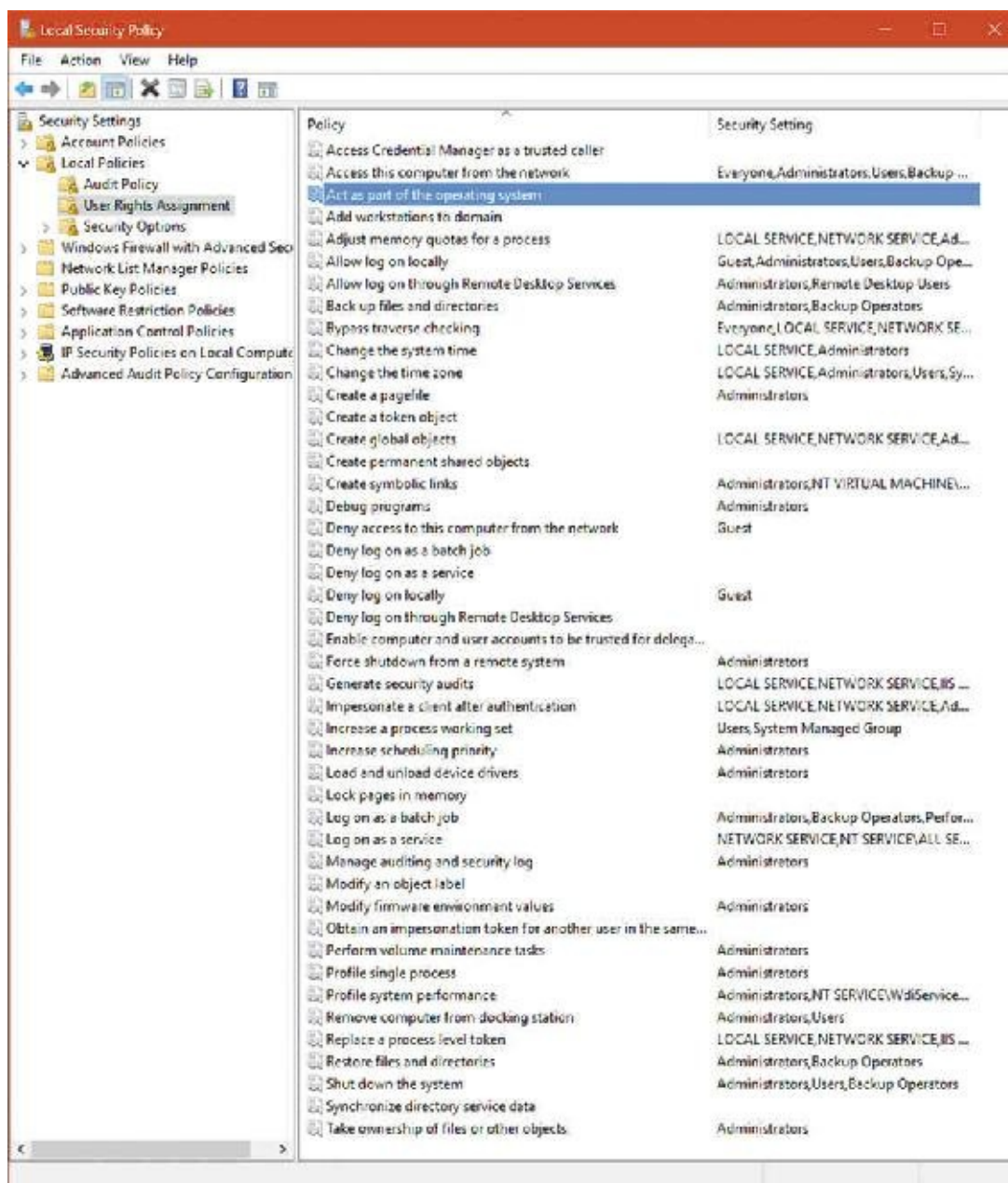


FIGURE 7-11 Local Security Policy editor user rights assignment.

Account rights

Account rights are not enforced by the SRM, nor are they stored in tokens. The function responsible for logon is `LsaLogonUser`. `Winlogon`, for example, calls the `LogonUser` API when a user logs on interactively to a computer, and `LogonUser` calls `LsaLogonUser`. `LogonUser` takes a parameter that indicates the type of logon being performed, which includes interactive, network, batch, service, and Terminal Server client.

In response to logon requests, the Local Security Authority (LSA) retrieves account rights assigned to a user from the LSA policy database at the time that a user attempts to log on to the system. The LSA checks the logon type against the account rights assigned to the user account logging on and denies the logon if the account does not have the right that permits the logon type or it has the right that denies the logon type.

[Table 7-10](#) lists the user rights defined by Windows.

User Right	Role
Deny logon locally, allow logon locally	Used for interactive logons that originate on the local machine
Deny logon over the network, allow logon over the network	Used for logons that originate from a remote machine
Deny logon through Terminal Services, allow logon through Terminal Services	Used for logons through a Terminal Server client
Deny logon as a service, allow logon as a service	Used by the service control manager when starting a service in a particular user account
Deny logon as a batch job, allow logon as a batch job	Used when performing a logon of type batch

TABLE 7-10 Account rights

Windows applications can add and remove user rights from an account by using the `LsaAddAccountRights` and `LsaRemoveAccountRights` functions, and they can determine what rights are assigned to an account with `LsaEnumerateAccountRights`.

Privileges

The number of privileges defined by the operating system has grown over time. Unlike user rights, which are enforced in one place by the LSA, different privileges are defined by different components and enforced by those components. For example, the debug privilege, which allows a process to bypass security checks when opening a handle to another process with the `OpenProcess` Windows API, is checked for by the process manager.

[Table 7-11](#) is a full list of privileges and describes how and when system components check for them. Each privilege has a macro defined in the SDK headers, in the form `SE_privilege_NAME`, where *privilege* is a privilege constant—for example, `SE_DEBUG_NAME` for the debug privilege. These macros are defined as strings that start with `Se` and end with `Privilege`, as in `SeDebugPrivilege`. This may seem to indicate that privileges are identified by strings, but in fact they are identified by LUIDs, which naturally are unique for the current boot. Every access to a privilege needs to lookup the correct LUID by calling the `LookupPrivilegeValue` function. Note, however, that `Ntdll` and kernel code can identify privileges with integer constants directly without going through a LUID.

Privilege	User Right	Privilege Usage
<code>SeAssignPrimaryTokenPrivilege</code>	Replace a process-level token	Checked for by various components, such as <code>NTSetInformationJobObject</code> , that set a process's token.
<code>SeAuditPrivilege</code>	Generate security audits	Required to generate events for the Security event log with the <code>ReportEvent</code> API.
<code>SeBackupPrivilege</code>	Back up files and directories	Causes NTFS to grant the following access to any file or directory, regardless of the security descriptor that's present: <code>READ_CONTROL</code> , <code>ACCESS_SYSTEM_SECURITY</code> , <code>FILE_GENERIC_READ</code> , and <code>FILE_TRAVERSE</code> . Note that when opening a file for backup, the caller must specify the <code>FILE_FLAG_BACKUP_SEMANTICS</code> flag. Also allows corresponding access to registry keys when using <code>RegSaveKey</code> .
<code>SeChangeNotifyPrivilege</code>	Bypass traverse checking	Used by NTFS to avoid checking permissions on intermediate directories of a multilevel directory lookup. Also used by file systems when applications register for notification of changes to the file system structure.

SeCreateGlobalPrivilege	Create global objects	Required for a process to create section and symbolic link objects in the directories of the object manager namespace that are assigned to a different session than the caller.
SeCreatePagefilePrivilege	Create a pagefile	Checked for by NtCreatePageFile, which is the function used to create a new paging file.
SeCreatePermanentPrivilege	Create permanent shared objects	Checked for by the object manager when creating a permanent object (one that doesn't get deallocated when there are no more references to it).
SeCreateSymbolicLinkPrivilege	Create symbolic links	Checked for by NTFS when creating symbolic links on the file system with the CreateSymbolicLink API.
SeCreateTokenPrivilege	Create a token object	NtCreateToken, the function that creates a token object, checks for this privilege.
SeDebugPrivilege	Debug programs	If the caller has this privilege enabled, the process manager allows access to any process or thread using NtOpenProcess or NtOpenThread, regardless of the process's or thread's security descriptor (except for protected processes).
SeEnableDelegationPrivilege	Enable computer and user accounts to be trusted for delegation	Used by Active Directory services to delegate authenticated credentials.
SeImpersonatePrivilege	Impersonate a client after authentication	The process manager checks for this when a thread wants to use a token for impersonation and the token represents a different user than that of the thread's process token.
SeIncreaseBasePriorityPrivilege	Increase scheduling priority	Checked for by the process manager and is required to raise the priority of a process.
SeIncreaseQuotaPrivilege	Adjust memory quotas for a process	Enforced when changing a process's working set thresholds, a process's paged and nonpaged pool quotas, and a process's CPU rate quota.
SeIncreaseWorkingSetPrivilege	Increase a process working set	Required to call SetProcessWorkingSetSize to increase the minimum working set. This indirectly allows the process to lock up to the minimum working set of memory using VirtualLock.
SeLoadDriverPrivilege	Load and unload device drivers	Checked for by the NtLoadDriver and NtUnloadDriver driver functions.
SeLockMemoryPrivilege	Lock pages in memory	Checked for by NtLockVirtualMemory, the kernel implementation of VirtualLock.
SeMachineAccountPrivilege	Add workstations to the domain	Checked for by the Security Account Manager on a domain controller when creating a machine account in a domain.
SeManageVolumePrivilege	Perform volume maintenance tasks	Enforced by file system drivers during a volume open operation, which is required to perform disk checking and defragmenting activities.
SeProfileSingleProcessPrivilege	Profile single process	Checked by Superfetch and the prefetcher when requesting information for an individual process through the NTQuerySystemInformation API.

SeRelabelPrivilege	Modify an object label	Checked for by the SRM when raising the integrity level of an object owned by another user, or when attempting to raise the integrity level of an object higher than that of the caller's token.
SeRemoteShutdownPrivilege	Force shutdown from a remote system	Winlogon checks that remote callers of the <code>InitiateSystemShutdown</code> function have this privilege.
SeRestorePrivilege	Restore files and directories	This privilege causes NTFS to grant the following access to any file or directory, regardless of the security descriptor that's present: <code>WRITE_DAC</code> , <code>WRITE_OWNER</code> , <code>ACCESS_SYSTEM_SECURITY</code> , <code>FILE_GENERIC_WRITE</code> , <code>FILE_ADD_FILE</code> , <code>FILE_ADD_SUBDIRECTORY</code> , and <code>DELETE</code> . Note that when opening a file for restore, the caller must specify the <code>FILE_FLAG_BACKUP_SEMANTICS</code> flag. Allows corresponding access to registry keys when using <code>RegSaveKey</code> .
SeSecurityPrivilege	Manage auditing and security log	Required to access the SACL of a security descriptor and to read and clear the security event log.
SeShutdownPrivilege	Shut down the system	Checked for by <code>NtShutdownSystem</code> and <code>NtRaiseLastError</code> , which presents a system error dialog box on the interactive console.
SeSyncAgentPrivilege	Synchronize directory service data	Required to use the LDAP directory synchronization services. It allows the holder to read all objects and properties in the directory, regardless of the protection on the objects and properties.
SeSystemEnvironmentPrivilege	Modify firmware environment variables	Required by <code>NtSetSystemEnvironmentValue</code> and <code>NtQuerySystemEnvironmentValue</code> to modify and read firmware environment variables using the hardware abstraction layer (HAL).
SeSystemProfilePrivilege	Profile system performance	Checked for by <code>NtCreateProfile</code> , the function used to perform profiling of the system. This is used by the <code>Kernprof</code> tool, for example.
SeSystemTimePrivilege	Change the system time	Required to change the time or date.
SeTakeOwnershipPrivilege	Take ownership of files and other objects	Required to take ownership of an object without being granted discretionary access.
SeTcbPrivilege	Act as part of the operating system	Checked for by the SRM when the session ID is set in a token, by the Plug and Play manager for Plug and Play event creation and management, by <code>BroadcastSystemMessageEx</code> when called with <code>BSM_ALLODESKTOPS</code> , by <code>LsaRegisterLogonProcess</code> , and when specifying an application as a VDM with <code>NtSetInformationProcess</code> .
SeTimeZonePrivilege	Change the time zone	Required to change the time zone.
SeTrustedCredManAccessPrivilege	Access Credential Manager as a trusted caller	Checked by the Credential Manager to verify that it should trust the caller with credential information that can be queried in plaintext. It is granted only to Winlogon by default.
SeUndockPrivilege	Remove computer from a docking station	Checked for by the user-mode Plug and Play manager when either a computer undock is initiated or a device eject request is made.
SeUnsolicitedInputPrivilege	Receive unsolicited data from a terminal device	This privilege isn't currently used by Windows.

TABLE 7-11 Privileges

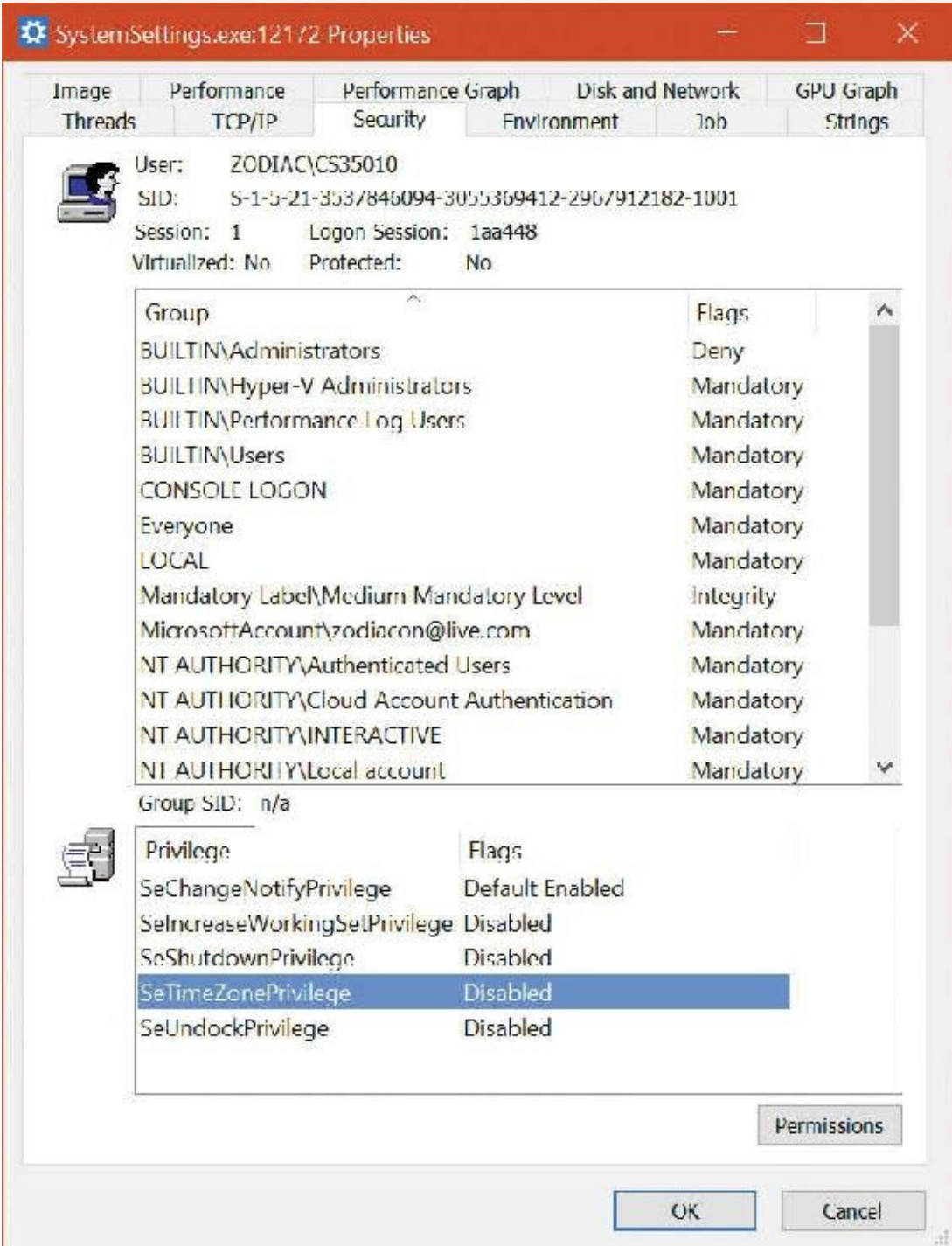
When a component wants to check a token to see whether a privilege is present, it uses the `Privilege-Check` or `LsaEnumerateAccountRights` APIs if running in user mode and `SeSinglePrivilegeCheck` or `SePrivilegeCheck` if running in kernel mode. The privilege-related APIs are not account-right aware, but the account-right APIs are privilege-aware.

Unlike account rights, privileges can be enabled and disabled. For a privilege check to succeed, the privilege must be in the specified token and it must be enabled. The idea behind this scheme is that privileges should be enabled only when their use is required so that a process cannot inadvertently perform a privileged security operation. Enabling or disabling privileges can be done with the

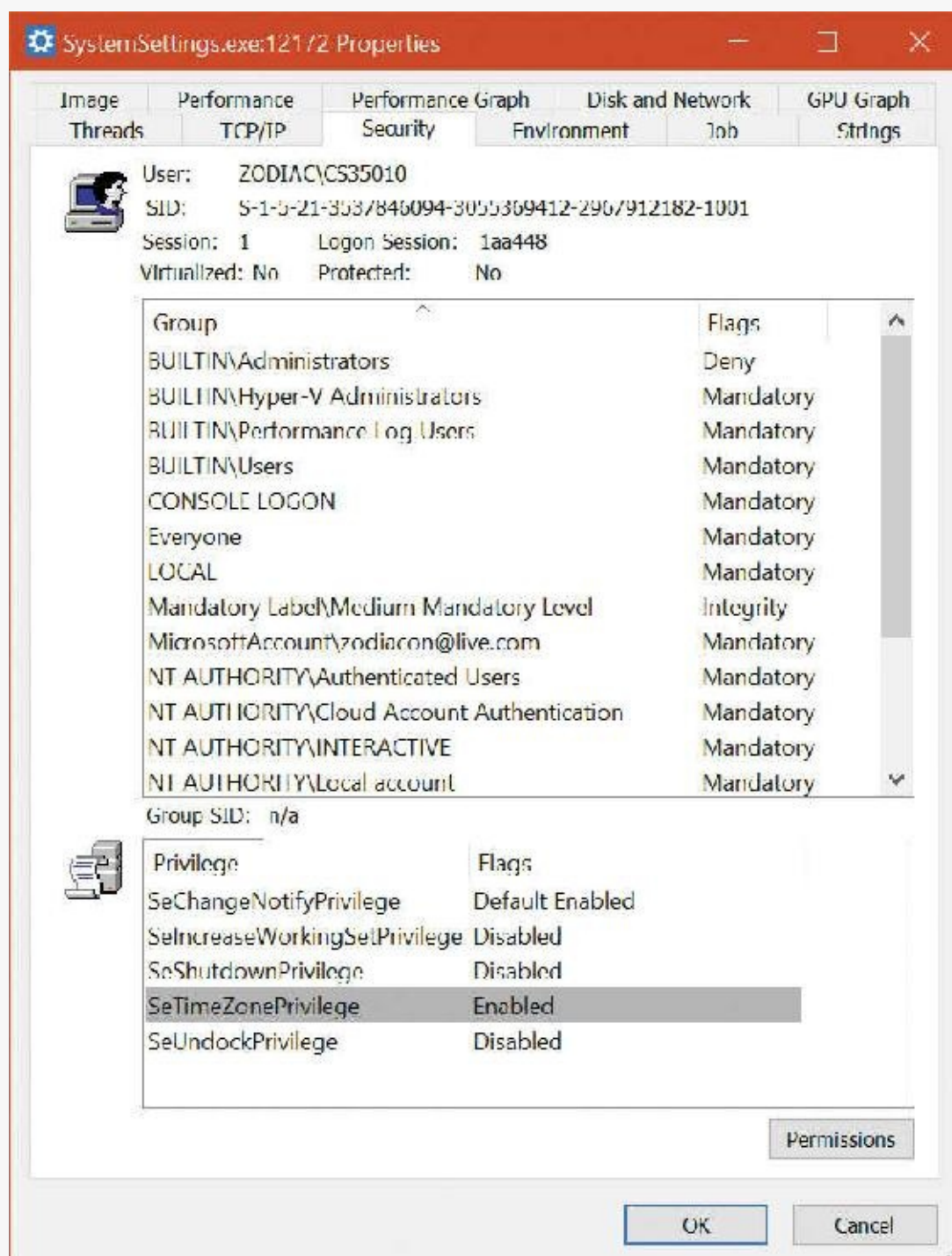
EXPERIMENT: Seeing a privilege get enabled

By following these steps, you can see that the Date and Time Control Panel applet enables the `SeTimeZonePrivilege` privilege in response to you using its interface to change the time zone of the computer (Windows 10):

- 1. Run Process Explorer elevated.
- 2. Right-click the clock in the system tray in the taskbar and choose **Adjust Date/Time**.
Alternatively, open the Settings app and search for **time** to open the Date and Time settings page.
- 3. Right-click the **SystemSettings.exe** process in Process Explorer and choose **Properties**.
Then click the **Security** tab in the Properties dialog box. You should see that the `SeTimeZonePrivilege` privilege is disabled.



4. Change the time zone, close the Properties dialog box, and then open it again. On the Security tab, you should now see that the SeTimeZonePrivilege privilege is enabled:



EXPERIMENT: The Bypass Traverse Checking privilege

If you are a systems administrator, you must be aware of the Bypass Traverse Checking privilege (internally called `SeNotifyPrivilege`) and its implications. This experiment demonstrates that not understanding its behavior can lead to improperly applied security.

1. Create a folder and, within that folder, a new text file with some sample text.
2. Navigate in Explorer to the new file, open its Properties dialog box, and click the **Security** tab.
3. Click the **Advanced** button.
4. Deselect the **Inheritance** check box.
5. Select **Copy** when you are prompted as to whether you want to remove or copy inherited permissions.
6. Modify the security of the new folder so that your account does not have any access to the folder. To do so, select your account and check all the **Deny** boxes in the permissions list.
7. Run Notepad. Then open the **File** menu, choose **Open**, and browse to the new directory in the dialog box that appears. You should be denied access to the directory.
8. In the File Name field of the Open dialog box, type the full path of the new file. The file should open.

If your account does not have the Bypass Traverse Checking privilege, NTFS performs an access check on each directory of the path to a file when you try to open a file, which results in you being denied access to the file in this example.

Super privileges

Several privileges are so powerful that a user to which they are assigned is effectively a “super user” who has full control over a computer. These privileges can be used in an infinite number of ways to gain unauthorized access to otherwise off-limit resources and to perform unauthorized operations. However, we’ll focus on using the privilege to execute code that grants the user privileges not assigned to the user, with the knowledge that this capability can be leveraged to perform any operation on the local machine that the user desires.

This section lists the privileges and discusses some of the ways they can be exploited. Other privileges, such as Lock Pages in Physical Memory (`SeLockMemoryPrivilege`), can be exploited for denial-of-service attacks on a system, but these are not discussed. Note that on systems with UAC enabled, these privileges will be granted only to applications running at high integrity level or higher, even if the account possesses them:

- **Debug programs (`SeDebugPrivilege`)** A user with this privilege can open any process on the system (except for a protected process) without regard to the security descriptor present on the process. For example, the user could implement a program that opens the `Lsass` process, copy executable code into its address space, and then inject a thread with the `CreateRemoteThread` Windows API to execute the injected code in a more-privileged security context. The code could grant the user additional privileges and group memberships.
- **Take ownership (`SeTakeOwnershipPrivilege`)** This privilege allows a holder to take ownership of any securable object (even protected processes and threads) by writing his own SID into the owner field of the object’s security descriptor. Recall that an owner is always granted

permission to read and modify the DACL of the security descriptor, so a process with this privilege could modify the DACL to grant itself full access to the object and then close and reopen the object with full access. This would allow the owner to see sensitive data and to even replace system files that execute as part of normal system operation, such as Lsass, with his own programs that grant a user elevated privileges.

- **Restore files and directories (SeRestorePrivilege)** A user assigned this privilege can replace any file on the system with her own. She could exploit this power by replacing system files as described in the preceding paragraph.
- **Load and unload device drivers (SeLoadDriverPrivilege)** A malicious user could use this privilege to load a device driver into the system. Device drivers are considered trusted parts of the operating system that can execute within it with System account credentials, so a driver could launch privileged programs that assign the user other rights.
- **Create a token object (SeCreateTokenPrivilege)** This privilege can be used in the obvious way to generate tokens that represent arbitrary user accounts with arbitrary group membership and privilege assignment.
- **Act as part of operating system (SeTcbPrivilege)** LsaRegisterLogonProcess, the function a process calls to establish a trusted connection to Lsass, checks for this privilege. A malicious user with this privilege can establish a trusted-Lsass connection and then execute LsaLogonUser, a function used to create new logon sessions. LsaLogonUser requires a valid user name and password and accepts an optional list of SIDs that it adds to the initial token created for a new logon session. The user could therefore use her own user name and password to create a new logon session that includes the SIDs of more privileged groups or users in the resulting token.



Note

The use of an elevated privilege does not extend past the machine boundary to the network because any interaction with another computer requires authentication with a domain controller and validation of domain passwords. Domain passwords are not stored on a computer either in plaintext or encrypted form, so they are not accessible to malicious code.

Access tokens of processes and threads

[Figure 7-12](#) brings together the concepts covered so far in this chapter by illustrating the basic process and thread security structures. In the figure, notice that the process object and the thread objects have ACLs, as do the access token objects themselves. Also in this figure, thread 2 and thread 3 each have an impersonation token, whereas thread 1 uses the default process access token.

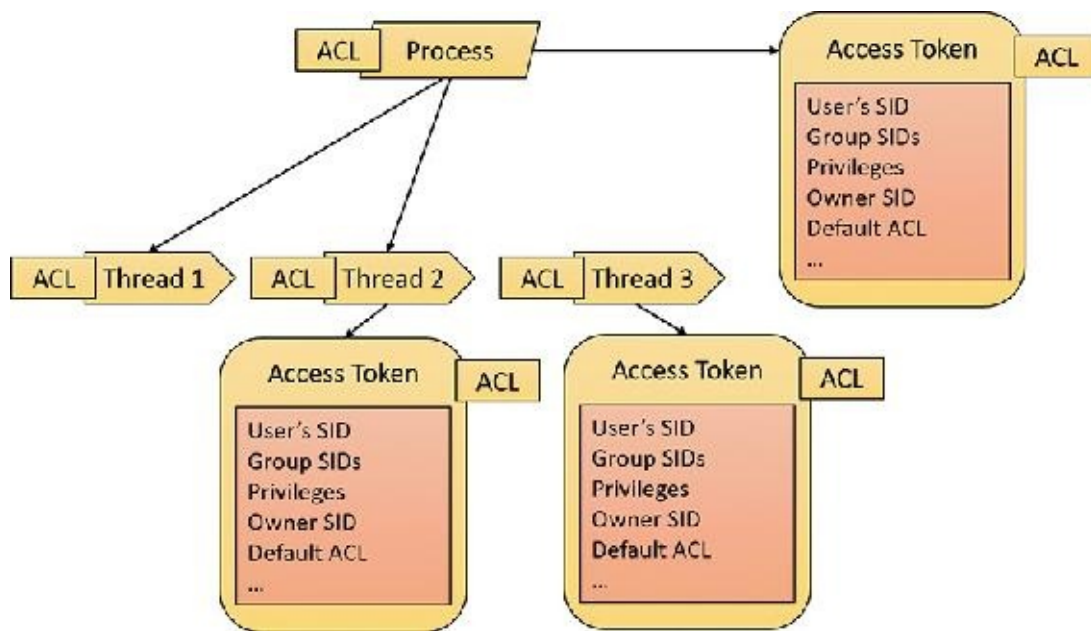


FIGURE 7-12 Process and thread security structures.

Security auditing

The object manager can generate audit events as a result of an access check, and Windows functions available to user applications can generate them directly. Kernel-mode code is always allowed to generate an audit event. Two privileges, `SeSecurityPrivilege` and `SeAuditPrivilege`, relate to auditing. A process must have the `SeSecurityPrivilege` privilege to manage the security event log and to view or set an object's SACL. Processes that call audit system services, however, must have the `SeAuditPrivilege` privilege to successfully generate an audit record.

The audit policy of the local system controls the decision to audit a particular type of security event. The audit policy, also called the Local Security Policy, is one part of the security policy Lsass maintains on the local system. It is configured with the Local Security Policy editor as shown in [Figure 7-13](#). The audit policy configuration (both the basic settings under Local Policies and the Advanced Audit Policy Configuration) is stored in the registry as a bitmapped value in the `HKEY_LOCAL_MACHINE\SECURITY\Policy\PolAdtEv` key.

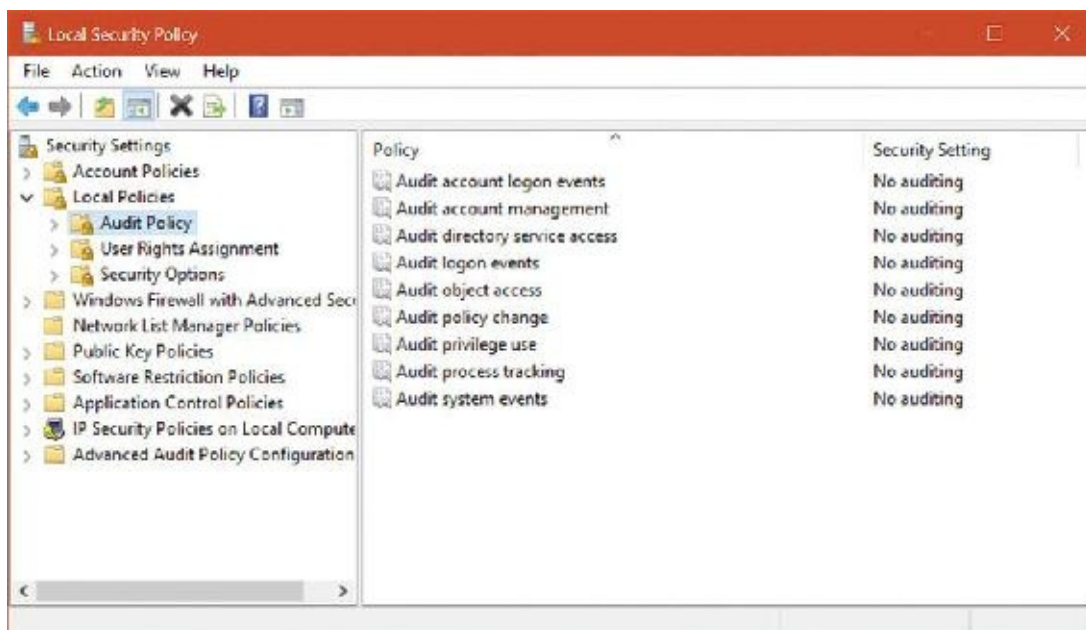


FIGURE 7-13 Local Security Policy editor audit policy configuration.

Lsass sends messages to the SRM to inform it of the auditing policy at system-initialization time and when the policy changes. Lsass is responsible for receiving audit records generated based on the audit events from the SRM, editing the records, and sending them to the event logger. Lsass (instead of the SRM) sends these records because it adds pertinent details, such as the information needed to more completely identify the process that is being audited.

The SRM sends audit records via its ALPC connection to Lsass. The event logger then writes the audit record to the security event log. In addition to audit records the SRM passes, both Lsass and the SAM generate audit records that Lsass sends directly to the event logger, and the AuthZ APIs allow for applications to generate application-defined audits. [Figure 7-14](#) depicts this overall flow.

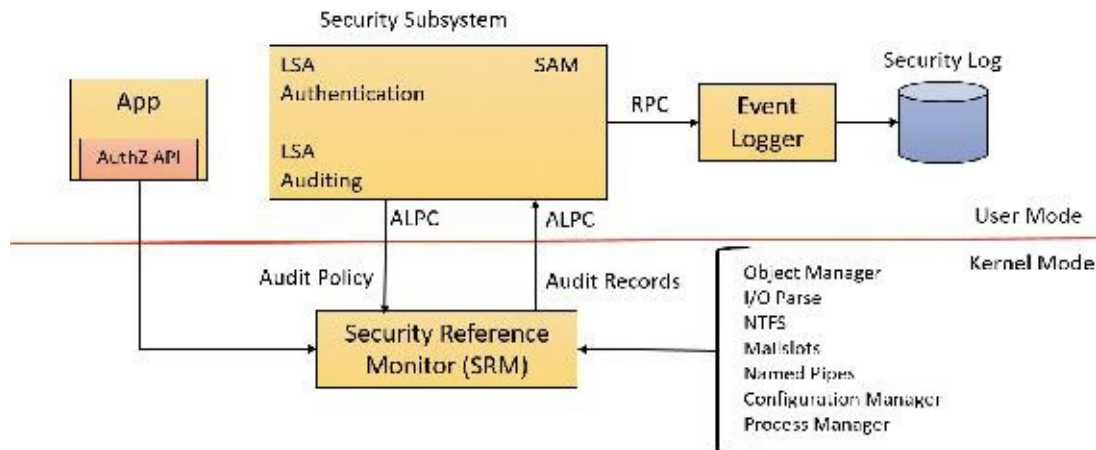


FIGURE 7-14 Flow of security audit records.

Audit records are put on a queue to be sent to the LSA as they are received. They are not submitted in batches. The audit records are moved from the SRM to the security subsystem in one of two ways. If the audit record is small (less than the maximum ALPC message size), it is sent as an ALPC message. The audit records are copied from the address space of the SRM to the address space of the Lsass process. If the audit record is large, the SRM uses shared memory to make the message available to Lsass and simply passes a pointer in an ALPC message.

Object access auditing

An important use of the auditing mechanism in many environments is to maintain a log of accesses to secured objects—in particular, files. To do this, the Audit object access policy must be enabled, and there must be audit ACEs in system access control lists that enable auditing for the objects in question.

When an accessor attempts to open a handle to an object, the SRM first determines whether the attempt is allowed or denied. If object access auditing is enabled, the SRM then scans the system ACL of the object. There are two types of audit ACEs: access allowed and access denied. An audit ACE must match any of the security IDs held by the accessor, it must match any of the access methods requested, and its type (access allowed or access denied) must match the result of the access check to generate an object access audit record.

Object access audit records include not just the fact of access allowed or denied, but also the reason for the success or failure. This “reason for access” reporting generally takes the form of an access control entry, specified in Security Descriptor Definition Language (SDDL), in the audit record. This allows for a diagnosis of scenarios in which an object to which you believe access should be denied is being permitted, or vice versa, by identifying the specific access control entry that caused the attempted access to succeed or fail.

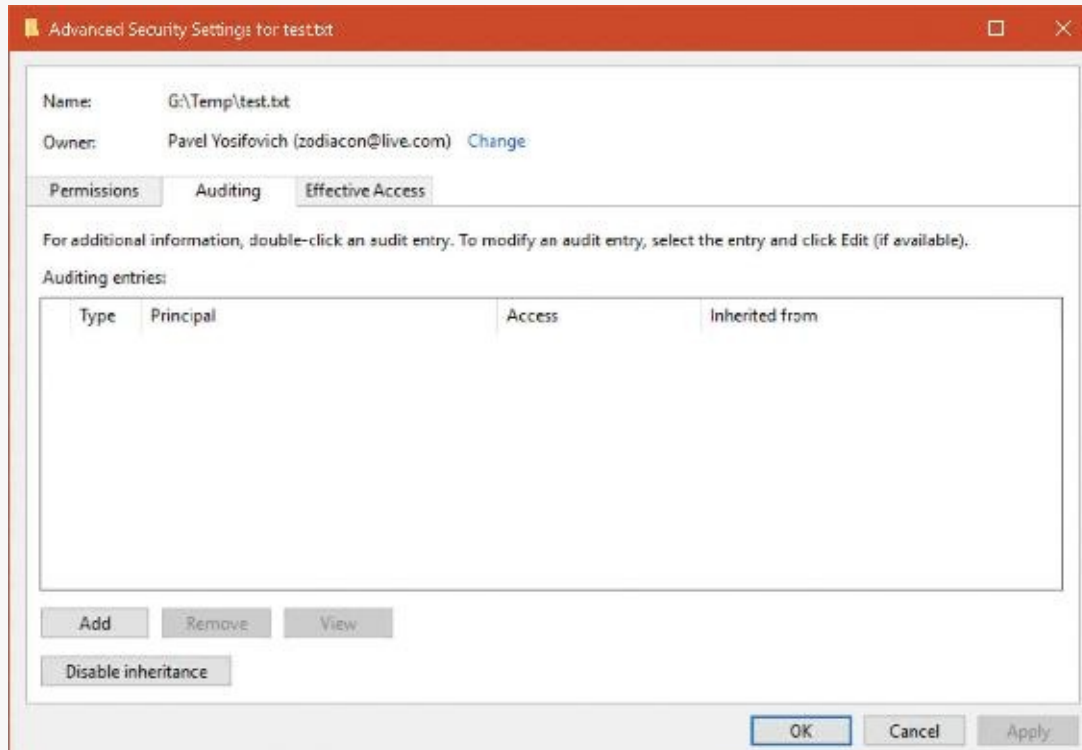
As was shown in [Figure 7-13](#), object access auditing is disabled by default (as are all other auditing

policies).

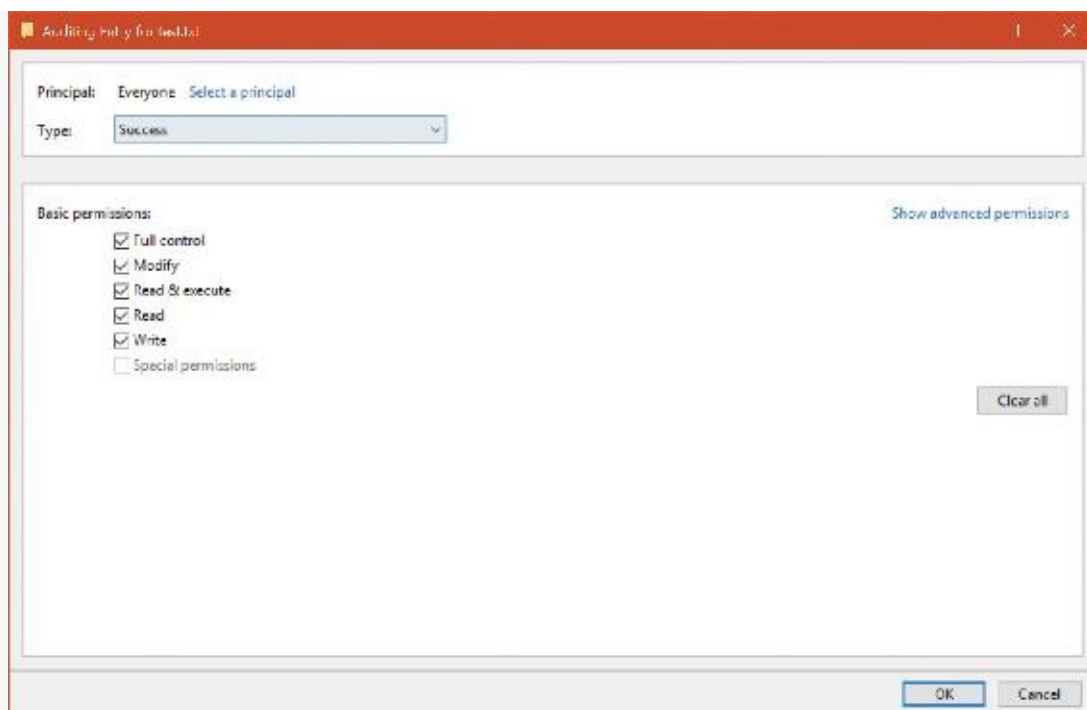
EXPERIMENT: Object access auditing

You can observe object access auditing by following these steps:

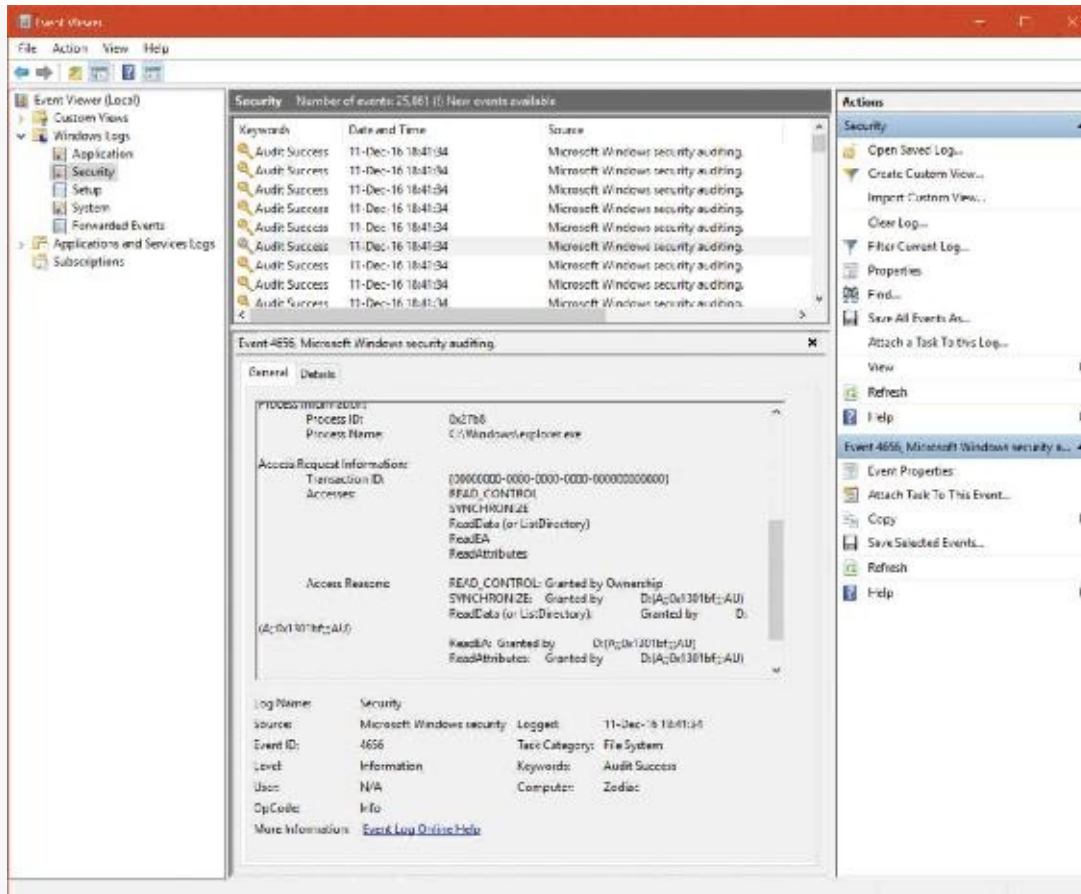
1. In Explorer, navigate to a file to which you would normally have access (such as a text file), open its Properties dialog box, click the **Security** tab, and then select the **Advanced** settings.
2. Click the **Auditing** tab and click through the administrative privileges warning. The resulting dialog box allows you to add auditing of access control entries to the file's system access control list.



3. Click the **Add** button and choose **Select a Principal**.
4. In the resulting Select User or Group dialog box, enter your own user name or a group to which you belong, such as **Everyone**. Click **Check Names** and then click **OK**. This presents a dialog box for creating an Audit access control entry for this user or group for this file.



5. Click **OK** three times to close the file Properties dialog box.
6. In Explorer, double-click the file to open it with its associated program (for example, Notepad for a text file).
7. Click the **Start** menu, type **event**, and choose **Event Viewer**.
8. Navigate to the Security log. Note that there is no entry for access to the file. This is because the audit policy for object access is not yet configured.
9. In the Local Security Policy editor, navigate to **Local Policies** and choose **Audit Policy**.
10. Double-click **Audit Object Access** and click **Success** to enable auditing of successful access to files.
11. In Event Viewer, click **Action** (from the menu) and **Refresh**. Note that the changes to audit policy resulted in audit records.
12. In Explorer, double-click the file to open it again.
13. In Event Viewer, click **Action** and **Refresh**. Note that several file access audit records are now present.
14. Find one of the file access audit records for event ID 4656. This shows up as “a handle to an object was requested.” (You can use the Find option to search for the file name you opened.)
15. Scroll down in the text box to find the Access Reasons section. The following example shows that two access methods, `READ_CONTROL`, `SYNCHRONIZE`, and `ReadAttributes`, `ReadEA` (extended attributes), and `ReadData` were requested. `READ_CONTROL` was granted because the accessor was the owner of the file. The others were granted because of the indicated access control entry.



Global audit policy

In addition to object-access ACEs on individual objects, a global audit policy can be defined for the system that enables object-access auditing for all file-system objects, all registry keys, or for both. A security auditor can therefore be certain that the desired auditing will be performed, without having to set or examine SACLs on all the individual objects of interest.

An administrator can set or query the global audit policy via the `AuditPol` command with the `/resourceSACL` option. This can also be done programmatically by calling the `AuditSetGlobalSacl` and `AuditQueryGlobalSacl` APIs. As with changes to objects' SACLs, changing these global SACLs requires `SeSecurityPrivilege`.

EXPERIMENT: Setting global audit policy

You can use the `AuditPol` command to enable global audit policy.

1. If you didn't already do so in the previous experiment, open the Local Security Policy editor, navigate to the Audit Policy settings (refer to [Figure 7-13](#)), double-click **Audit Object Access**, and enable auditing for both success and failure. On most systems, SACLs specifying object access auditing are uncommon, so few if any object-access audit records will be produced at this point.
2. In an elevated command prompt window, enter the following command. This will produce a summary of the commands for setting and querying global audit policy.

```
C:\> auditpol /resourceSACL
```

3. In the same elevated command prompt window, enter the following commands. On a typical system, each of these commands will report that no global SACL exists for the respective resource type. (Note that the `File` and `Key` keywords are case-sensitive.)

[Click here to view code image](#)

```
C:\> auditpol /resourceSACL /type:File /view  
C:\> auditpol /resourceSACL /type:Key /view
```

4. In the same elevated command prompt window, enter the following command. This will set a global audit policy such that all attempts to open files for write access (`FW`) by the indicated user will result in audit records, whether the open attempts succeed or fail. The user name can be a specific user name on the system, a group such as `Everyone`, a domain-qualified user name such as `domainname\username`, or a `SID`.

[Click here to view code image](#)

```
C:\> auditpol /resourceSACL /set /type:File /user:yourusername  
/success  
/failure /access:FW
```

5. While running under the user name indicated, use Explorer or other tools to open a file. Then look at the security log in the system event log to find the audit records.
6. At the end of the experiment, use the `auditpol` command to remove the global SACL you created in step 4, as follows:

[Click here to view code image](#)

```
C:\> auditpol /resourceSACL /remove /type:File  
/user:yourusername
```

The global audit policy is stored in the registry as a pair of system access control lists in `HKLM\SECURITY\Policy\GlobalSaclNameFile` and `HKLM\SECURITY\Policy\GlobalSaclNameKey`. You can examine these keys by running `Regedit.exe` under the System account, as described in the “[Security system components](#)” section earlier in this chapter. These keys will not exist until the corresponding global SACLs have been set at least once.

The global audit policy cannot be overridden by SACLs on objects, but object-specific SACLs can allow for additional auditing. For example, global audit policy could require auditing of read access by

all users to all files, but SACLs on individual files could add auditing of write access to those files by specific users or by more specific user groups.

Global audit policy can also be configured via the Local Security Policy editor in the Advanced Audit Policy settings, described in the next section.

Advanced Audit Policy settings

In addition to the Audit Policy settings described previously, the Local Security Policy editor offers a much more fine-grained set of audit controls under the Advanced Audit Policy Configuration heading, shown in [Figure 7-15](#).

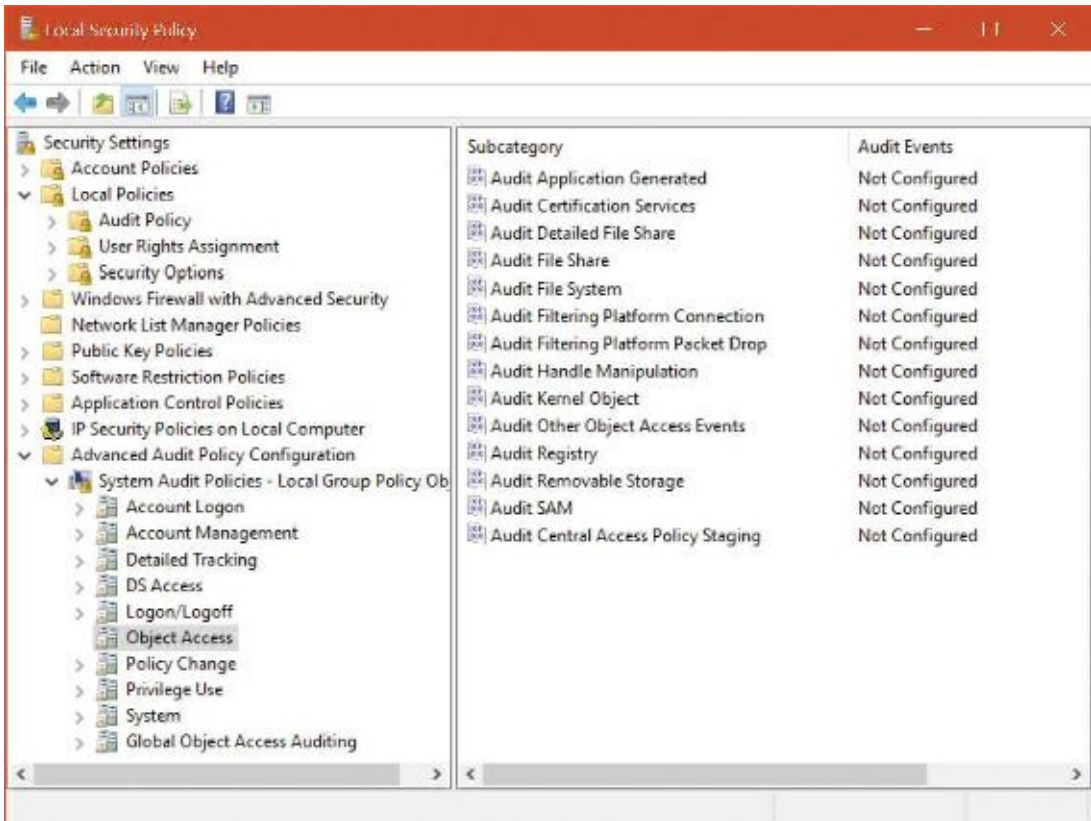


FIGURE 7-15 The Local Security Policy editor’s Advanced Audit Policy Configuration settings.

Each of the nine audit policy settings under Local Policies (refer to [Figure 7-13](#)) maps to a group of settings here that provide more detailed control. For example, while the Audit Object Access settings under Local Policies allow access to all objects to be audited, the settings here allow auditing of access to various types of objects to be controlled individually. Enabling one of the audit policy settings under Local Policies implicitly enables all the corresponding advanced audit policy events, but if finer control over the contents of the audit log is desired, the advanced settings can be set individually. The standard settings then become a product of the advanced settings. However, this is not visible in the Local Security Policy editor. Attempts to specify audit settings by using both the basic and the advanced options can cause unexpected results.

You can use the Global Object Access Auditing option under Advanced Audit Policy Configuration to configure the global SACLs described in the previous section, using a graphical interface identical to that seen in Explorer or the Registry editor for security descriptors in the file system or the registry.

AppContainers

Windows 8 introduced a new security sandbox called an *AppContainer*. Although it was created primarily to host UWP processes, AppContainers can actually be used for “normal” processes as well (although there is no built-in tool to do that). This section will mostly cover the attributes of packaged AppContainers, which is the term that refers to AppContainers associated with UWP processes and their resulting .Appx format. A complete treatment of UWP apps is beyond the scope of this chapter. You can find more information in [Chapter 3](#) of this book, and in Chapters 8 and 9 in Part 2. Here we’ll concentrate on the security aspects of AppContainers and their typical usage as hosts of UWP apps.



Note

The term *Universal Windows Platform (UWP) app* is the latest used to describe processes that host the Windows Runtime. Older names include *immersive app*, *modern app*, *metro app*, and sometimes simply *Windows app*. The *Universal* part indicates the ability of such apps to be deployed and run on various Windows 10 editions and form factors, from IoT core, to mobile, to desktop, to Xbox, to HoloLens. However, they are essentially the same as the ones first introduced in Windows 8. Therefore, the concept of AppContainers discussed in this section is relevant to Windows 8 and later versions of Windows. Note that Universal Application Platform (UAP) is sometimes used instead of UWP; it’s the same thing.



Note

The original codename for AppContainer was LowBox. You may see this term come up in many of the API names and data structures throughout this section. They refer to the same concept.

Overview of UWP apps

The mobile device revolution established new ways of obtaining and running software. Mobile devices normally get their applications from a central store, with automatic installation and updates, all with little user intervention. Once a user selects an app from the store, she can see the permissions the app requires to function correctly. These permissions are called *capabilities* and are declared as part of the package when it’s submitted to the store. This way, the user can decide whether these capabilities are acceptable.

[Figure 7-16](#) shows an example of a capabilities list for a UWP game (*Minecraft*, Windows 10 beta edition). The game requires internet access as a client and as a server and access to the local home or work network. Once the user downloads the game, she implicitly agrees the game may exercise these capabilities. Conversely, the user can be confident that the game uses *only* those capabilities. That is, there is no way the game could use other unapproved capabilities, such as accessing the camera on the device.

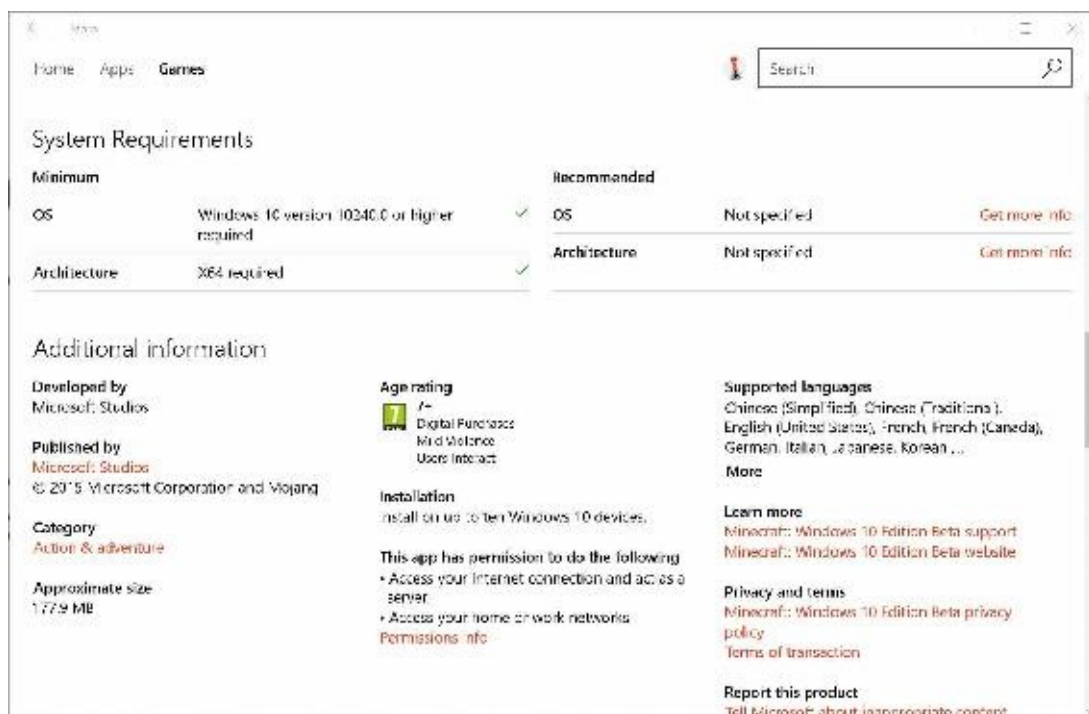


FIGURE 7-16 Part of an app’s page in the store, showing capabilities, among other things.

To get a sense of the differences between UWP apps and desktop (classic) apps at a high level, consult [Table 7-12](#). From a developer’s perspective, the Windows platform can be seen as shown in [Figure 7-17](#).

	UWP App	Desktop (Classic) App
Device Support	Runs on all Windows device families	Runs on PCs only
APIs	Can access WinRT, subset of COM, and subset of Win32 APIs	Can access COM, Win32, and subset of WinRT APIs
Identity	Strong app identity (static and dynamic)	Raw EXEs and processes
Information	Declarative APPX manifest	Opaque binaries
Installation	Self-contained APPX package	Loose files or MSI
App Data	Isolated per-user/per-app storage (local and roaming)	Shared user profile
Lifecycle	Participates in app resource management and PLM	Process-level lifecycle
Instancing	Single instance only	Any number of instances

TABLE 7-12 High-level comparison of UWP and desktop apps

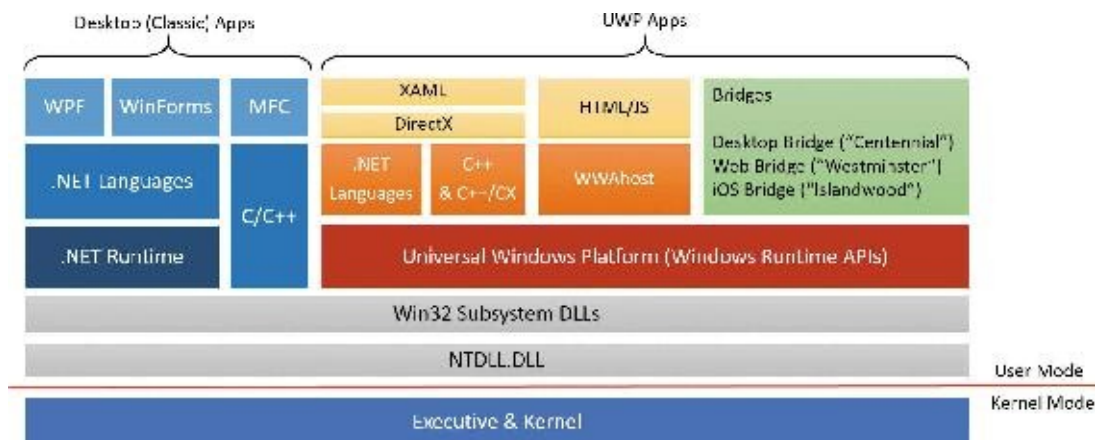


FIGURE 7-17 The Windows platform landscape.

A few items in [Figure 7-17](#) are worth elaborating on:

- UWP apps can produce normal executables, just like desktop apps. Wwahost.exe (%SystemRoot%\

System32\wwahost.exe) is used to host HTML/JavaScript-based UWP apps, as those produce a DLL, not an executable.

- The UWP is implemented by the Windows Runtime APIs, which are based on an enhanced version of COM. Language projections are provided for C++ (through proprietary language extensions known as C++/CX), .NET languages, and JavaScript. These projections make it relatively easy to access WinRT types, methods, properties, and events from developers' familiar environments.
- Several bridging technologies are available, which can transform other types of applications into UWP. See the MSDN documentation for more information on utilizing these technologies.
- The Windows Runtime is layered on top of the Windows subsystem DLLs, just like the .NET Framework. It has no kernel components and is not part of a different subsystem because it still leverages the same Win32 APIs that the system offers. However, some policies are implemented in the kernel, as well as the general support for AppContainers.
- The Windows Runtime APIs are implemented in DLLs residing in the %SystemRoot%\System32 directory, with names in the form *Windows.Xxx.Yyy...Dll*, where the file name usually indicates the Windows Runtime API namespace implemented. For example, *Windows.Globalization.Dll* implements the classes residing in the *Windows.Globalization* namespace. (See the MSDN documentation for the complete WinRT API reference.)

The AppContainer

We've seen the steps required to create processes back in [Chapter 3](#); we've also seen some of the extra steps required to create UWP processes. The initiation of creation is performed by the DCOMLaunch service, because UWP packages support a set of protocols, one of which is the Launch protocol. The resulting process gets to run inside an AppContainer. Here are several characteristics of packaged processes running inside an AppContainer:

- The process token integrity level is set to Low, which automatically restricts access to many objects and limits access to certain APIs or functionality for the process, as discussed earlier in this chapter.
- UWP processes are always created inside a job (one job per UWP app). This job manages the UWP process and any background processes that execute on its behalf (through nested jobs). The jobs allow the Process State Manager (PSM) to suspend or resume the app or background processing in a single stroke.
- The token for UWP processes has an AppContainer SID, which represents a distinct identity based on the SHA-2 hash of the UWP package name. As you'll see, this SID is used by the system and other applications to explicitly allow access to files and other kernel objects. This SID is part of the APPLICATION PACKAGE AUTHORITY instead of the NT AUTHORITY you've mostly seen so far in this chapter. Thus, it begins with S-1-15-2 in its string format, corresponding to SECURITY_APP_PACKAGE_BASE_RID (15) and SECURITY_APP_PACKAGE_BASE_RID (2). Because a SHA-2 hash is 32 bytes, there are a total of eight RIDs (recall that a RID is the size of a 4-byte ULONG) in the remainder of the SID.
- The token may contain a set of capabilities, each represented with a SID. These capabilities are declared in the application manifest and shown on the app's page in the store. Stored in the capability section of the manifest, they are converted to SID format using rules we'll see shortly, and belong to the same SID authority as in the previous bullet, but using the well-known SECURITY_CAPABILITY_BASE_RID (3) instead. Various components in the Windows Runtime, user-mode device-access classes, and kernel can look for capabilities to allow or deny certain operations.
- The token may only contain the following privileges: SeChangeNotifyPrivilege, SeIncrease-WorkingSetPrivilege, SeShutdownPrivilege, SeTimeZonePrivilege, and SeUndockPrivilege. These are the default set of privileges associated with standard user accounts. Additionally, the AppContainerPrivilegesEnabledExt function part of the ms-win-ntos-ksecurity API Set contract extension can be present on certain devices to further restrict which privileges are enabled by default.
- The token will contain up to four security attributes (see the section on attribute-based access control earlier in this chapter) that identify this token as being associated with a UWP packaged application. These attributes are added by the DCOMLaunch service as indicated earlier, which is responsible for the activation of UWP applications. They are as follows:
 - **WIN://PKG** This identifies this token as belonging to a UWP packaged application. It contains an integer value with the application's origin as well as some flags. See [Table 7-13](#) and [Table 7-14](#) for these values.

Origin	Meaning
Unknown (0)	The package origin is unknown.
Unsigned (1)	The package is unsigned.
Inbox (2)	The package is associated with a built-in (inbox) Windows application.
Store (3)	The package is associated with a UWP application downloaded from the store. This origin is validated by checking if the DACL of the file associated with the main UWP application's executable contains a trust ACE.
Developer Unsigned (4)	The package is associated with an unsigned developer key.
Developer Signed (5)	The package is associated with a signed developer key.
Line-of-Business (6)	The package is associated with a side-loaded line-of-business (LOB) application.

TABLE 7-13 Package origins

Flag	Meaning
PSM_ACTIVATION_TOKEN_PACKAGED_APPLICATION (0x1)	This indicates that the AppContainer UWP application is stored in AppX packaged format. This is the default.
PSM_ACTIVATION_TOKEN_SHARED_ENTITY (0x2)	This indicates that this token is being used for multiple executables all part of the same AppX packaged UWP application.
PSM_ACTIVATION_TOKEN_FULL_TRUST (0x4)	This indicates that this AppContainer token is being used to host a Project Centennial (Windows Bridge for Desktop) converted Win32 application.
PSM_ACTIVATION_TOKEN_NATIVE_SERVICE (0x8)	This indicates that this AppContainer token is being used to host a packaged service created by the Service Control Manager (SCM)'s Resource Manager. See Chapter 9 in Part 2 for more information on services.
PSM_ACTIVATION_TOKEN_DEVELOPMENT_APP (0x10)	This indicates that this is an internal development application. Not used on retail systems.
BREAKAWAY_INHIBITED (0x20)	The package cannot create a process that is not itself packaged as well. This is set by using the <code>PROCESS_THREAD_ATTRIBUTE_DESKTOP_APP_POOLCY</code> process-creation attribute. (See Chapter 3 for more information.)

TABLE 7-14 Package flags

- **WIN://SYSAPPID** This contains the application identifiers (called *package monikers* or *string names*) as an array of Unicode string values.
- **WIN://PKGHOSTID** This identifies the UWP package host ID for packages that have an explicit host through an integer value.
- **WIN://BGKD** This is only used for background hosts (such as the generic background task host BackgroundTaskHost.exe) that can store packaged UWP services running as COM providers. The attribute's name stands for *background* and contains an integer value that stores its explicit host ID.

The `TOKEN_LOWBOX (0x4000)` flag will be set in the token's `Flags` member, which can be queried with various Windows and kernel APIs (such as `GetTokenInformation`). This allows components to identity and operate differently under the presence of an AppContainer token.

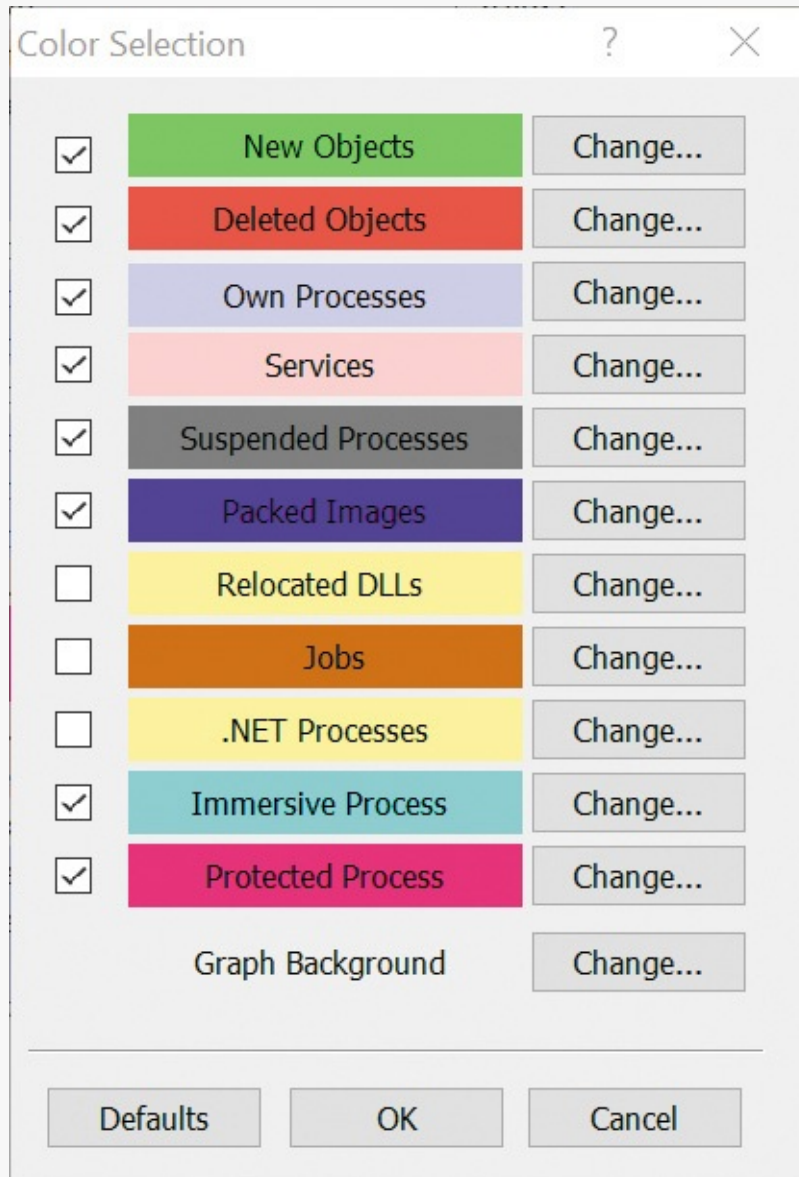


Note

A second type of AppContainer exists: a child AppContainer. This is used when a UWP AppContainer (or parent AppContainer) wishes to create its own nested AppContainer to further lock down the security of the application. Instead of eight RIDs, a child AppContainer has four additional RIDs (the first eight match the parents') to uniquely identify it.

EXPERIMENT: Viewing UWP process information

There are several ways to look at UWP processes, some more obvious than others. Process Explorer can highlight processes that use the Windows Runtime in color (cyan by default). To see this in action, open Process Explorer, open the **Options** menu, and choose **Configure Colors**. Then make sure the **Immersive Processes** check box is selected.



Immersive process is the original term used to describe WinRT (now UWP) apps in Windows 8. (They were mostly full screen, and therefore “immersive.”) This distinction is available by calling the `IsImmersiveProcess` API.

Run `Calc.exe` and switch to Process Explorer. You should see several processes highlighted in cyan, including `Calculator.exe`. Now minimize the Calculator app and notice that the cyan highlight has turned

gray. This is because Calculator has been suspended. Restore Calculator’s window, and it’s back to cyan.

You should have similar experiences with other apps—for example, Cortana (SearchUI.exe). Click or tap the **Cortana** icon on the taskbar and then close it. You should see the gray to cyan and back to gray transition. Or, click or tap the **Start** button. ShellExperienceHost.exe highlights in a similar fashion.

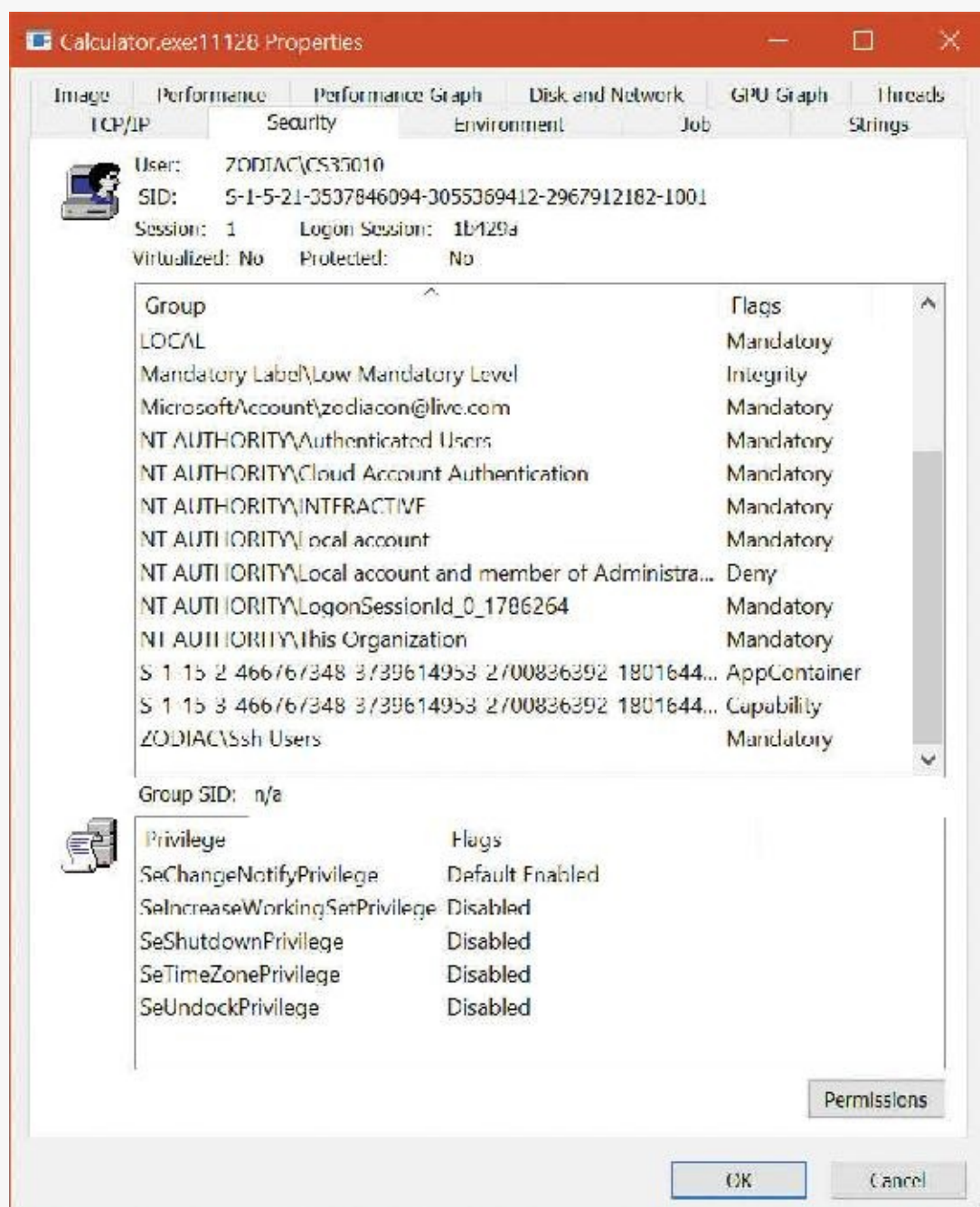
The presence of some cyan-highlighted processes might surprise you, such as Explorer.exe, TaskMgr.Exe, and RuntimeBroker.exe. These are not really apps, but use Windows Runtime APIs, and so are classified as immersive. (The role of RuntimeBroker will be discussed shortly.)

Finally, make sure the Integrity column is visible in Process Explorer and sort by that column. You’ll find processes such as Calculator.exe and SearchUI.exe with AppContainer integrity level. Notice that Explorer and TaskMgr are not there, clearly showing they are not UWP processes, and so live under different rules.

Process	PID	CPU	Integrity	Private Bytes	Working Set	Description
Interrupts	n/a	0.74		0 K	0 K	Hardware Interrupt
Calculator.exe	11128		AppContainer	13,420 K	36,712 K	
Microsoft.Msn.Weather.exe	3040	Suspended	AppContainer	45,600 K	15,764 K	Weather
SearchUI.exe	3728	Suspended	AppContainer	43,508 K	102,564 K	Search and Cortana
ShellExperienceHost.exe	1872	Suspended	AppContainer	79,872 K	130,636 K	Windows Shell Exp
SkypeHost.exe	9624	Suspended	AppContainer	33,084 K	26,844 K	Microsoft Skype Pi
System Idle Process	0	92.57		0 K	4 K	
Twitter.Windows.exe	2084	Suspended	AppContainer	33,348 K	8,576 K	Twitter.Windows
Video.UI.exe	8772	Suspended	AppContainer	19,900 K	37,444 K	Video Application
firefox.exe	11836	0.31	Low	571,804 K	615,372 K	Firefox
ApptlExe.exe	1060		Medium	1,044 K	5,960 K	Alps Pointing-device
...	...	...	...	...	...	...

EXPERIMENT: Viewing an AppContainer token

You can look at the properties of an AppContainer hosted process with several tools. In Process Explorer, the Security tab shows the capabilities associated with the token. Here's the Security tab for Calculator.exe:



Notice two interesting pieces of information: the AppContainer SID, shown in the Flags column as AppContainer, and a single capability, right underneath the AppContainer SID. Except for the base RID (SECURITY_APP_PACKAGE_BASE_RID versus SECURITY_CAPABILITY_BASE_RID), the remaining eight RIDs are identical, and both refer to the package name in SHA-2 format as discussed. This shows you that there will always be one implicit capability, the capability of being the package itself, which really means Calculator requires no capabilities at all. The upcoming capabilities section covers a much more complex example.

EXPERIMENT: Viewing AppContainer token attributes

You can obtain similar information on the command line by using the AccessChk Sysinternals tool

while also adding a full list of all of the token's attributes. For example, running AccessChk with the -p -f switches followed by the process ID for SearchUI.exe, which hosts Cortana, shows the following:

[Click here to view code image](#)

```
C:\>accesschk -p -f 3728
```

```
Accesschk v6.10 - Reports effective permissions for securable objects
```

```
Copyright (C) 2006-2016 Mark Russinovich
```

```
Sysinternals - www.sysinternals.com
```

```
[7416] SearchUI.exe
```

```
  RW DESKTOP-DD6KTPM\aigne
```

```
  RW NT AUTHORITY\SYSTEM
```

```
  RW Package\S-1-15-2-1861897761-1695161497-2927542615-642690995-327840285-2659745135-2630312742
```

```
Token security:
```

```
  RW DESKTOP-DD6KTPM\aigne
```

```
  RW NT AUTHORITY\SYSTEM
```

```
  RW DESKTOP-DD6KTPM\aigne-S-1-5-5-0-459087
```

```
  RW Package\S-1-15-2-1861897761-1695161497-2927542615-642690995-327840285-2659745135-2630312742
```

```
  R BUILTIN\Administrators
```

```
Token contents:
```

```
  User:
```

```
    DESKTOP-DD6KTPM\aigne
```

```
  AppContainer:
```

```
    Package
```

```
\S-1-15-2-1861897761-1695161497-2927542615-642690995-327840285-2659745135-2630312742
```

```
  Groups:
```

```
    Mandatory Label\Low Mandatory Level
```

```
    INTEGRITY
```

```
    Everyone
```

```
    MANDATORY
```

```
    NT AUTHORITY\Local account and member of Administrators group
```

```
DENY
```

```
...
```

```
  Security Attributes:
```

```
WIN://PKGHOSTID
```

```
  TOKEN_SECURITY_ATTRIBUTE_TYPE_UINT64
```

```
  [0] 1794402976530433
```

```
WIN://SYSAPPID
```

```
  TOKEN_SECURITY_ATTRIBUTE_TYPE_STRING
```

```
  [0]
```

```
Microsoft.Windows.Cortana_1.8.3.14986_neutral_neutral_cw5n1h2txyewy
```

```
  [1] CortanaUI
```

```
  [2] Microsoft.Windows.Cortana_cw5n1h2txyewy
```

```
WIN://PKG
    TOKEN_SECURITY_ATTRIBUTE_TYPE_UINT64
    [0] 131073
TSA://ProcUnique
    [TOKEN_SECURITY_ATTRIBUTE_NON_INHERITABLE]
    [TOKEN_SECURITY_ATTRIBUTE_COMPARE_IGNORE]
    TOKEN_SECURITY_ATTRIBUTE_TYPE_UINT64
    [0] 204
    [1] 24566825
```

First is the package host ID, converted to hex: 0x66000000000001. Because all package host IDs begin with 0x66, this means Cortana is using the first available host identifier: 1. Next are the system application IDs, which contain three strings: the strong package moniker, the friendly application name, and the simplified package name. Finally, you have the package claim, which is 0x20001 in hex. Based on the [Table 7-13](#) and [Table 7-14](#) fields you saw, this indicates an origin of Inbox (2) and flags set to PSM_ACTIVATION_TOKEN_PACKAGED_APPLICATION, confirming that Cortana is part of an AppX package.

AppContainer security environment

One of the biggest side-effects caused by the presence of an AppContainer SID and related flags is that the access check algorithm you saw in the “[Access checks](#)” section earlier in this chapter is modified to essentially ignore all regular user and group SIDs that the token may contain, essentially treating them as deny-only SIDs. This means that even though Calculator may be launched by a user John Doe belonging to the Users and Everyone groups, it will fail any access checks that grant access to John Doe’s SID, the Users group SID, or the Everyone group SID. In fact, the only SIDs that are checked during the discretionary access check algorithm will be that of the AppContainer SID, followed by the capability access check algorithm, which will look at any capability SIDs part of the token.

Taking things even further than merely treating the discretionary SIDs as deny-only, AppContainer tokens effect one further critical security change to the access check algorithm: a NULL DACL, typically treated as an allow-anyone situation due to the lack of any information (recall that this is different from an empty DACL, which is a deny-everyone situation due to explicit allow rules), is ignored and treated as a deny situation. To make matters simple, the only types of securable objects that an AppContainer can access are those that explicitly have an allow ACE for its AppContainer SID or for one of its capabilities. Even unsecured (NULL DACL) objects are out of the game.

This situation causes compatibility problems. Without access to even the most basic file system, registry, and object manager resources, how can an application even function? Windows takes this into account by preparing a custom execution environment, or “jail” if you will, specifically for each AppContainer. These jails are as follows:



Note

So far we've implied that each UWP packaged application corresponds to one AppContainer token. However, this doesn't necessarily imply that only a single executable file can be associated with an AppContainer. UWP packages can contain multiple executable files, which all belong to the same AppContainer. This allows them to share the same SID and capabilities and exchange data between each other, such as a micro-service back-end executable and a foreground front-end executable.

- The AppContainer SID's string representation is used to create a subdirectory in the object manager's namespace under `\Sessions\x\AppDataContainerNamedObjects`. This becomes the private directory of named kernel objects. This specific subdirectory object is then ACLed with the AppContainer SID associated with the AppContainer that has an allow-all access mask. This is in contrast to desktop apps, which all use the `\Sessions\x\BaseNamedObjects` subdirectory (within the same session `x`). We'll discuss the implications of that shortly, as well as the requirement for the token to now store handles.
- The token will contain a LowBox number, which is a unique identifier into an array of LowBox Number Entry structures that the kernel stores in the `g_SessionLowboxArray` global variable. Each of these maps to a `SEP_LOWBOX_NUMBER_ENTRY` structure that, most importantly, contains an atom table unique to this AppContainer, because the Windows Subsystem Kernel Mode Driver (`Win32k.sys`) does not allow AppContainers access to the global atom table.
- The file system contains a directory in `%LOCALAPPDATA%` called `Packages`. Inside it are the package monikers (the string version of the AppContainer SID—that is, the package name) of all the installed UWP applications. Each of these application directories contains application-specific directories, such as `TempState`, `RoamingState`, `Settings`, `LocalCache`, and others, which are all ACLed with the specific AppContainer SID corresponding to the application, set to an allow-all access mask.
- Within the `Settings` directory is a `Settings.dat` file, which is a registry hive file that is loaded as an application hive. (You will learn more about application hives in Chapter 9 in Part 2.) The hive acts as the local registry for the application, where WinRT APIs store the various persistent state of the application. Once again, the ACL on the registry keys explicitly grants allow-all access to the associated AppContainer SID.

These four jails allow AppContainers to securely, and locally, store their file system, registry, and atom table without requiring access to sensitive user and system areas on the system. That being said, what about the ability to access, at least in read-only mode, critical system files (such as `Ntdll.dll` and `Kernel32.dll`) or registry keys (such as the ones these libraries will need), or even named objects (such as the `\RPC Control\DNSResolver` ALPC port used for DNS lookups)? It would not make sense, on each UWP application or uninstallation, to re-ACL entire directories, registry keys, and object namespaces to add or remove various SIDs.

To solve this problem, the security subsystem understands a specific group SID called `ALL APPLICATION PACKAGES`, which automatically binds itself to any AppContainer token. Many critical system locations, such as `%SystemRoot%\System32` and `HKLM\Software\Microsoft\Windows\CurrentVersion`, will have this SID as part of their DACL, typically with a read or read-and-execute access mask. Certain objects in the object manager namespace will have

this as well, such as the DNSResolver ALPC port in the \RPC Control object manager directory. Other examples include certain COM objects, which grant the execute right. Although not officially documented, third-party developers, as they create non-UWP applications, can also allow interactions with UWP applications by also applying this SID to their own resources.

Unfortunately, because UWP applications can technically load almost any Win32 DLL as part of their WinRT needs (because WinRT is built on top of Win32, as you saw), and because it's hard to predict what an individual UWP application might need, many system resources have the **ALL APPLICATION PACKAGES** SID associated with their DACL as a precaution. This now means there is no way for a UWP developer, for example, to prevent DNS lookups from their application. This greater-than-needed access is also helpful for exploit writers, which could leverage it to escape from the AppContainer sandbox. Newer versions of Windows 10, starting with version 1607 (Anniversary Update), contain an additional element of security to combat this risk: Restricted AppContainers.

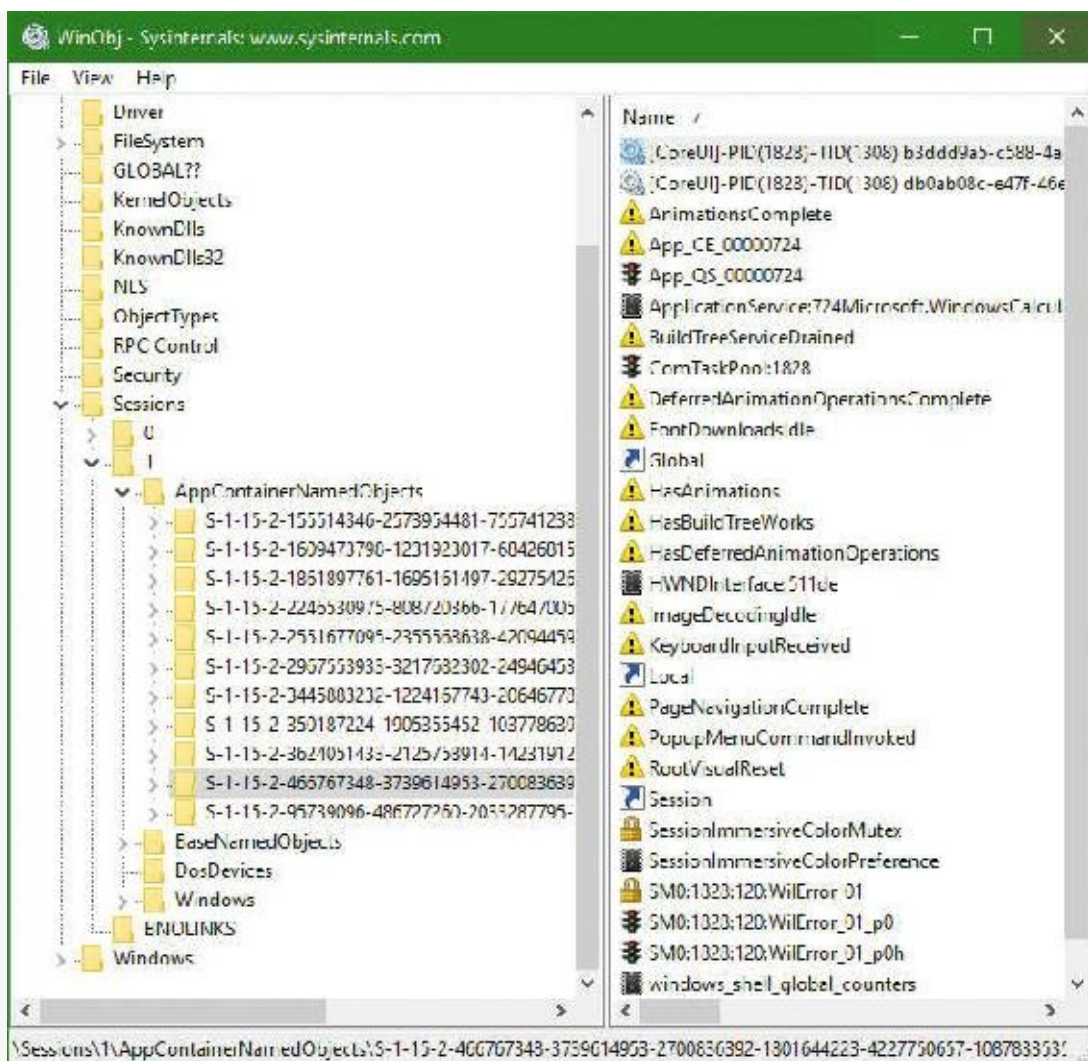
By using the **PROC_THREAD_ATTRIBUTE_ALL_APPLICATION_PACKAGES_POLICY** process attribute and setting it to **PROCESS_CREATION_ALL_APPLICATION_PACKAGES_OPT_OUT** during process creation (see [Chapter 3](#) for more information on process attributes), the token will not be associated with any ACEs that specify the **ALL APPLICATION PACKAGES** SID, cutting off access to many system resources that would otherwise be accessible. Such tokens can be identified by the presence of a fourth token attribute named **WIN: //NOALLAPPPKG** with an integer value set to 1.

Of course, this takes us back to the same problem: How would such an application even be able to load Ntdll.dll, which is key to any process initialization? Windows 10 version 1607 introduces a new group, called **ALL RESTRICTED APPLICATION PACKAGES**, which takes care of this problem. For example, the System32 directory now also contains this SID, also set to allow read and execute permissions, because loading DLLs in this directory is key even to the most sandboxed process. However, the DNSResolver ALPC port does not, so such an AppContainer would lose access to DNS.

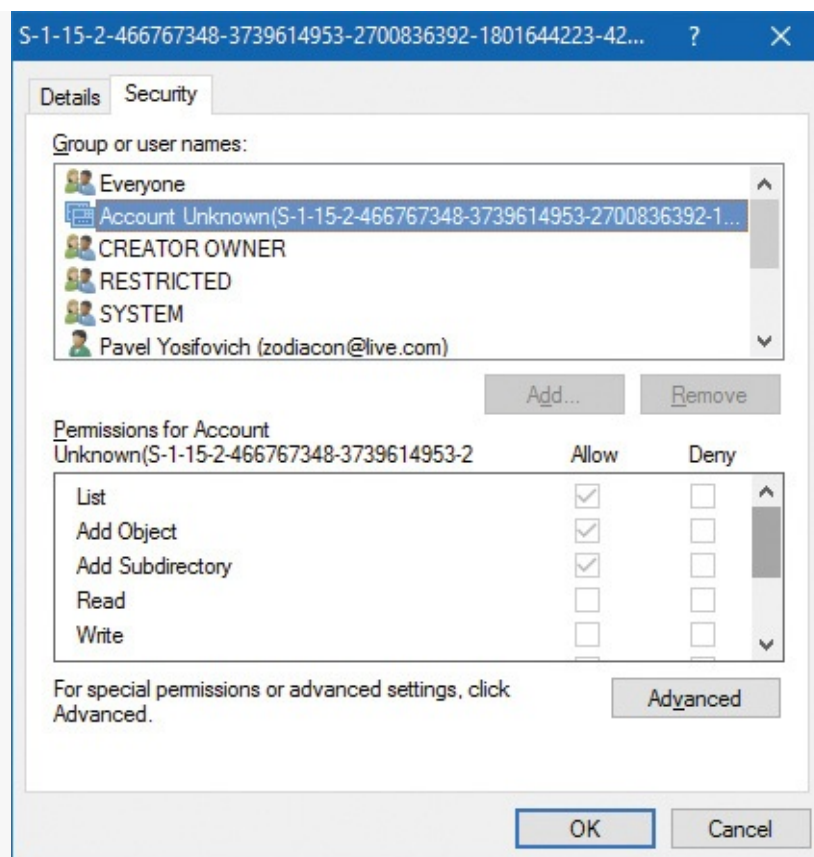
EXPERIMENT: Viewing AppContainer security attributes

In this experiment, we'll look at the security attributes of some of the directories mentioned in the previous section.

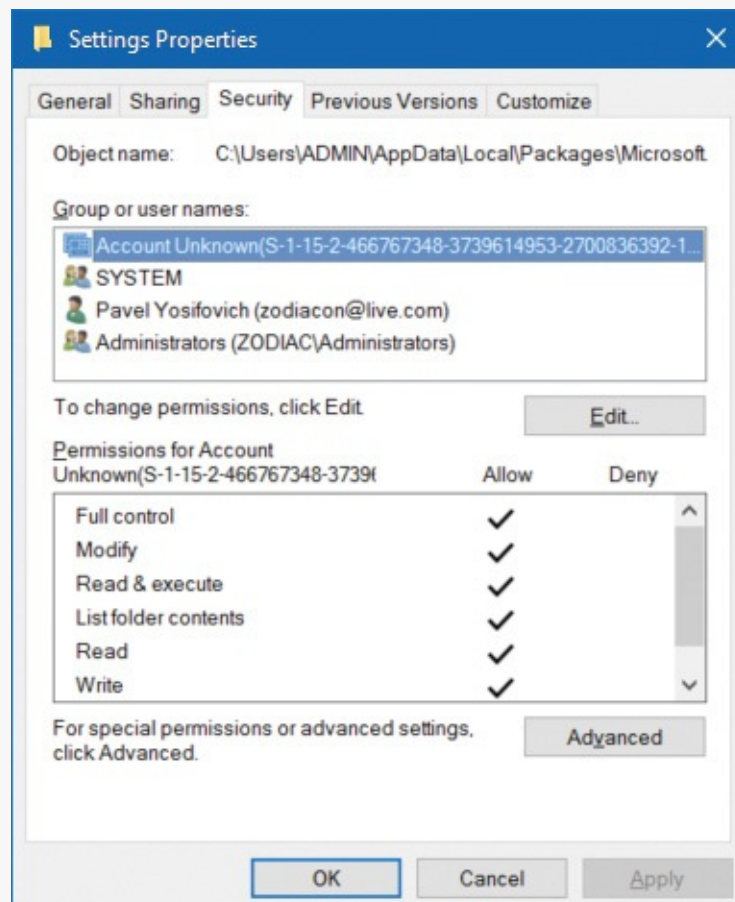
1. Make sure Calculator is running.
2. Open WinObj elevated from Sysinternals and navigate to the object directory corresponding to Calculator's AppContainer SID. (You saw it in a previous experiment.)



3. Right-click the directory, select **Properties**, and click the **Security** tab. You should see something like the following screenshot. Calculator's AppContainer SID has permission to list, add object, and add subdirectory (among others scrolled out of view), which simply means Calculator can create kernel objects under this directory.

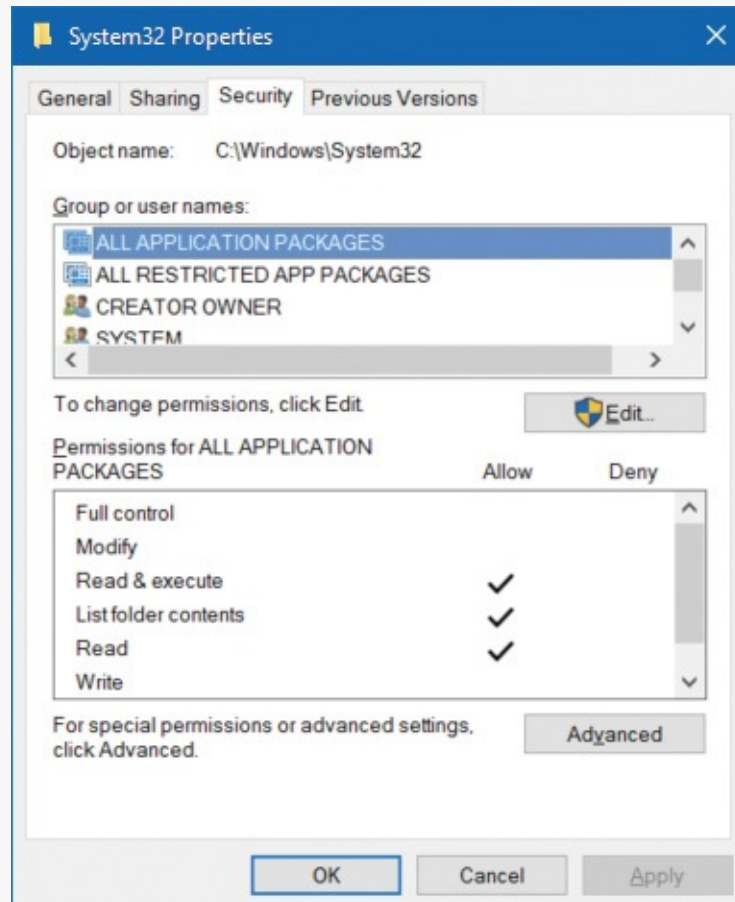


4. Open Calculator's local folder by navigating to %LOCALAPPDATA%\Packages\Microsoft.WindowsCalculator_8wekyb3d8bbwe. Then right-click the **Settings** subdirectory, select **Properties**, and click the **Security** tab. You should see Calculator's AppContainer SID having full permissions for the folder:



5. In Explorer, open the %SystemRoot% directory (for example, C:\Windows), right-click the **System32** directory, select **Properties**, and click the **Security** tab. You should see the read

and execute permissions for all application packages and all restricted application packages (if using Windows 10 version 1607 or later):



As an alternative, you can use the AccessChk Sysinternals command-line tool to view the same information.

EXPERIMENT: Viewing the AppContainer atom table

An atom table is a hash table of integers to strings that's used by the windowing system for various identification purposes, such as Window Class registration (`RegisterClassEx`) and custom Windows messages. The AppContainer private atom table can be viewed with the kernel debugger:

1. Run Calculator, open WinDbg, and start local kernel debugging.
2. Find the Calculator process:

[Click here to view code image](#)

```
lkd> !process 0 1 calculator.exe
PROCESS fffff828cc9ed1080
    SessionId: 1  Cid: 4bd8    Peb: d040bbc000  ParentCid: 03a4
    DeepFreeze
    DirBase: 5fccaa000  ObjectTable:
fffff950ad9fa2800  HandleCount:
<Data Not Accessible>
    Image: Calculator.exe
    VadRoot fffff828cd2c9b6a0  Vads 168  Clone 0  Private 2938.
    Modified 3332.
    Locked 0.
```

DeviceMap ffff950aad2cd2f0
Token

ffff950adb313060

...

3. Use the token value with the following expressions:

[Click here to view code image](#)

```
lkd> r? @$t1 = @$t0->NumberOfBuckets
lkd> r? @$t0 = (nt!_RTL_ATOM_TABLE*)
((nt!_token*)0xffff950adb313060) -
>LowboxNumberEntry->AtomTable
lkd> .for (r @$t3 = 0; @$t3 < @$t1; r @$t3 = @$t3 + 1) { ??
(wchar_t*)@$t0-
>Buckets[$t3]->Name }
wchar_t * 0xffff950a'ac39b78a
"Protocols"
wchar_t * 0xffff950a'ac17b7aa
"Topics"
wchar_t * 0xffff950a'b2fd282a
"TaskbarDPI_Deskband"
wchar_t * 0xffff950a'b3e2b47a
"Static"
wchar_t * 0xffff950a'b3c9458a
"SysTreeView32"
wchar_t * 0xffff950a'ac34143a
"UxSubclassInfo"
wchar_t * 0xffff950a'ac5520fa
"StdShowItem"
wchar_t * 0xffff950a'abc6762a
"SysSetRedraw"
wchar_t * 0xffff950a'b4a5340a
"UIA_WindowVisibilityOverridden"
wchar_t * 0xffff950a'ab2c536a
"True"
...
wchar_t * 0xffff950a'b492c3ea
"tooltips_class"
wchar_t * 0xffff950a'ac23f46a
"Save"
wchar_t * 0xffff950a'ac29568a
"MSDraw"
wchar_t * 0xffff950a'ac54f32a
"StdNewDocument"
wchar_t * 0xffff950a'b546127a
"{FB2E3E59-B442-4B5B-9128-2319BF8DE3B0}"
wchar_t * 0xffff950a'ac2e6f4a
"Status"
wchar_t * 0xffff950a'ad9426da
```

```
"ThemePropScrollBarCtl"
wchar_t * 0xffff950a'b3edf5ba
"Edit"
wchar_t * 0xffff950a'ab02e32a
"System"
wchar_t * 0xffff950a'b3e6c53a
"MDIClient"
wchar_t * 0xffff950a'ac17a6ca
"StdDocumentName"
wchar_t * 0xffff950a'ac6cbeea
"StdExit"
wchar_t * 0xffff950a'b033c70a
"{C56C5799-4BB3-7FAE-7FAD-4DB2F6A53EFF}"
wchar_t * 0xffff950a'ab0360fa
"MicrosoftTabletPenServiceProperty"
wchar_t * 0xffff950a'ac2f8fea
"OLESystem"
```

AppContainer capabilities

As you've just seen, UWP applications have very restricted access rights. So how, for example, is the Microsoft Edge application able to parse the local file system and open PDF files in the user's Documents folder? Similarly, how can the Music application play MP3 files from the Music directory? Whether done directly through kernel access checks or by brokers (which you'll see in the next section), the key lies in capability SIDs. Let's see where these come from, how they are created, and when they are used.

First, UWP developers begin by creating an application manifest that specifies many details of their application, such as the package name, logo, resources, supported devices, and more. One of the key elements for capability management is the list of capabilities in the manifest. For example, let's take a look at Cortana's application manifest, located in
%SystemRoot%\SystemApps\Microsoft.Windows.Cortana_cw5n1h2txywey\AppxManifest.xml:

[Click here to view code image](#)

```
<Capabilities>
  <wincap:Capability Name="packageContents"/>
  <!-- Needed for resolving MRT strings -->
  <wincap:Capability Name="cortanaSettings"/>
  <wincap:Capability Name="cloudStore"/>
  <wincap:Capability Name="visualElementsSystem"/>
  <wincap:Capability Name="perceptionSystem"/>
  <Capability Name="internetClient"/>
  <Capability Name="internetClientServer"/>
  <Capability Name="privateNetworkClientServer"/>
  <uap:Capability Name="enterpriseAuthentication"/>
  <uap:Capability Name="musicLibrary"/>
  <uap:Capability Name="phoneCall"/>
  <uap:Capability Name="picturesLibrary"/>
```

```

    <uap:Capability Name="sharedUserCertificates"/>
    <rescap:Capability Name="locationHistory"/>
    <rescap:Capability Name="userDataSystem"/>
    <rescap:Capability Name="contactsSystem"/>
    <rescap:Capability Name="phoneCallHistorySystem"/>
    <rescap:Capability Name="appointmentsSystem"/>
    <rescap:Capability Name="chatSystem"/>
    <rescap:Capability Name="smsSend"/>
    <rescap:Capability Name="emailSystem"/>
    <rescap:Capability Name="packageQuery"/>
    <rescap:Capability Name="slapiQueryLicenseValue"/>
    <rescap:Capability Name="secondaryAuthenticationFactor"/>
    <DeviceCapability Name="microphone"/>
    <DeviceCapability Name="location"/>
    <DeviceCapability Name="wiFiControl"/>
</Capabilities>

```

You'll see many types of entries in this list. For example, the `Capability` entries contain the well-known SIDs associated with the original capability set that was implemented in Windows 8. These begin with `SECURITY_CAPABILITY_`—for example, `SECURITY_CAPABILITY_INTERNET_CLIENT`, which is part of the capability RID under the `APPLICATION PACKAGE AUTHORITY`. This gives us a SID of S-1-15-3-1 in string format.

Other entries are prefixed with `uap`, `rescap`, and `wincap`. One of these (`rescap`) refers to restricted capabilities. These are capabilities that require special onboarding from Microsoft and custom approvals before being allowed on the store. In Cortana's case, these include capabilities such as accessing SMS text messages, emails, contacts, location, and user data. Windows capabilities, on the other hand, refer to capabilities that are reserved for Windows and system applications. No store application can use these. Finally, UAP capabilities refer to standard capabilities that anyone can request on the store. (Recall that UAP is the older name for UWP.)

Unlike the first set of capabilities, which map to hard-coded RIDs, these capabilities are implemented in a different fashion. This ensures a list of well-known RIDs doesn't have to be constantly maintained. Instead, with this mode, capabilities can be fully custom and updated on the fly. To do this, they simply take the capability string, convert it to full upper-case format, and take a SHA-2 hash of the resulting string, much like AppContainer package SIDs are the SHA-2 hash of the package moniker. Again, since SHA-2 hashes are 32 bytes, this results in 8 RIDs for each capability, following the well-known `SECURITY_CAPABILITY_BASE_RID` (3).

Finally, you'll notice a few `DeviceCapability` entries. These refer to device classes that the UWP application will need to access, and can be identified either through well-known strings such as the ones you see above or directly by a GUID that identifies the device class. Rather than using one of the two methods of SID creation already described, this one uses yet a third! For these types of capabilities, the GUID is converted into a binary format and then mapped out into four RIDs (because a GUID is 16 bytes). On the other hand, if a well-known name was specified instead, it must first be converted to a GUID. This is done by looking at the `HKLM\Software\Microsoft\Windows\CurrentVersion\DeviceAccess\CapabilityMappings` registry key, which contains a list of registry keys associated with device capabilities and a list of GUIDs that map to these capabilities. The GUIDs are then converted to a SID as you've just seen.



Note

For an up-to-date list of supported capabilities, see <https://msdn.microsoft.com/en-us/windows/uwp/packaging/app-capability-declarations>.

As part of encoding all of these capabilities into the token, two additional rules are applied:

- As you may have seen in the earlier experiment, each AppContainer token contains its own package SID encoded as a capability. This can be used by the capability system to specifically lock down access to a particular app through a common security check instead of obtaining and validating the package SID separately.
- Each capability is re-encoded as a group SID through the use of the `SECURITY_CAPABILITY_APP_RID (1024)` RID as an additional sub-authority preceding the regular eight-capability hash RIDs.

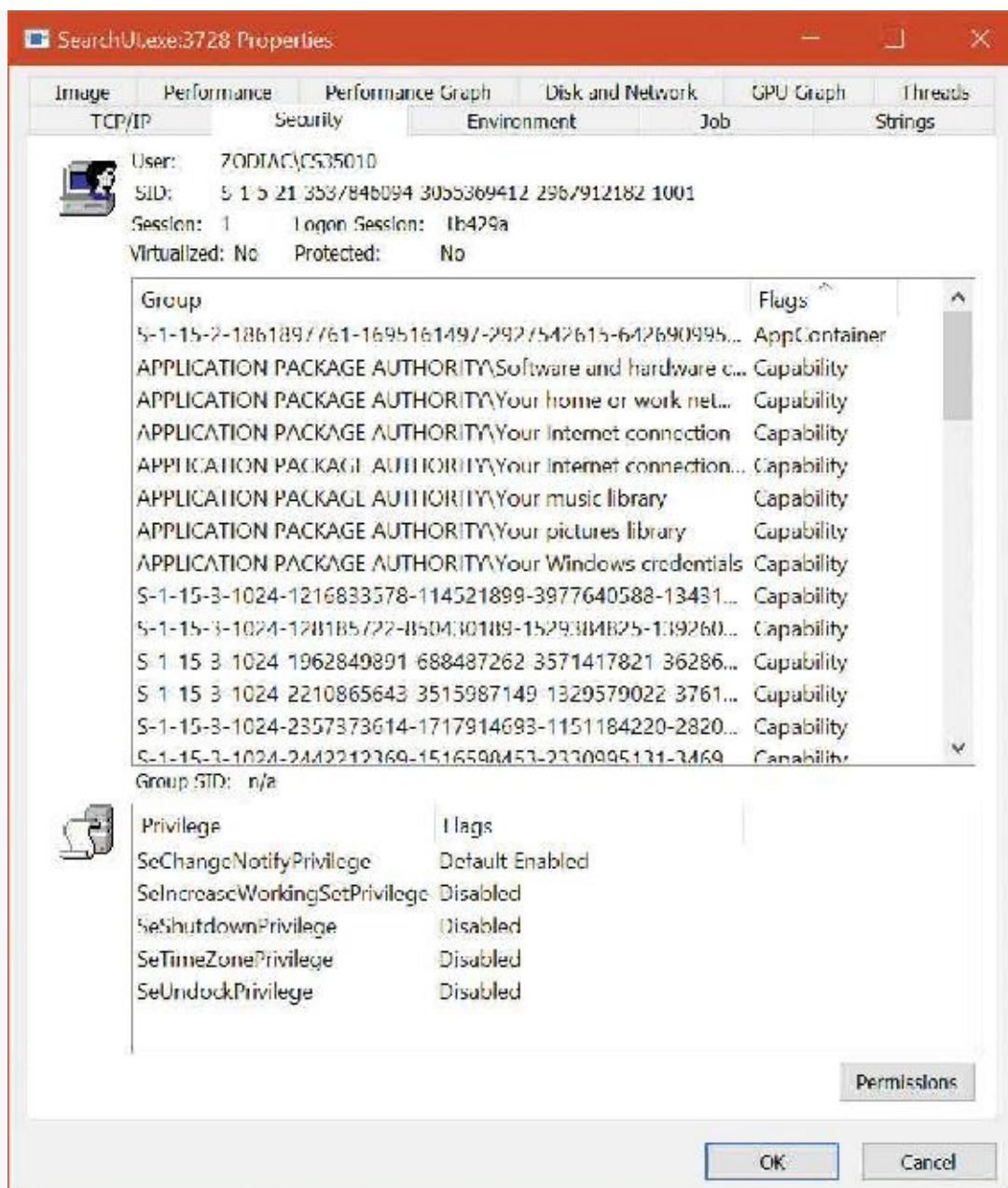
After the capabilities are encoded into the token, various components of the system will read them to determine whether an operation being performed by an AppContainer should be permitted. You'll note most of the APIs are undocumented, as communication and interoperability with UWP applications is not officially supported and best left to broker services, inbox drivers, or kernel components. For example, the kernel and drivers can use the `RtlCapabilityCheck` API to authenticate access to certain hardware interfaces or APIs.

As an example, the Power Manager checks for the `ID_CAP_SCREENOFF` capability before allowing a request to shut off the screen from an AppContainer. The Bluetooth port driver checks for the `bluetoothDiagnostics` capability, while the application identity driver checks for Enterprise Data Protection (EDP) support through the `enterpriseDataPolicy` capability. In user mode, the documented `CheckTokenCapability` API can be used, although it must know the capability SID instead of providing the name (the undocumented `RtlDeriveCapabilitySidFromName` can generate this, however). Another option is the undocumented `CapabilityCheck` API, which does accept a string.

Finally, many RPC services leverage the `RpcClientCapabilityCheck` API, which is a helper function that takes care of retrieving the token and requires only the capability string. This function is very commonly used by many of the WinRT-enlightened services and brokers, which utilize RPC to communicate with UWP client applications.

EXPERIMENT: Viewing AppContainer capabilities

To clearly demonstrate all these various capability combinations and their population in the token, let's look at the capabilities for a complex app such as Cortana. You've already seen its manifest, so you can use that output to compare with the UI. First, looking at the Security tab for `SearchUI.exe` shows the following (sorted by the Flags column):



Clearly, Cortana has obtained many capabilities—all the ones in its manifest. Some are those that were originally in Windows 8 and are known to functions like `ISWellKnownSid`, for which Process Explorer shows a friendly name. Other capabilities are just shown using their SID, as they represent either hashes or GUIDs, as discussed.

To get the details of the package from which the UWP process was created, you can use the `UwPList` tool provided with the downloadable resources for this book. It can show all immersive processes on the system or a single process based on its ID:

[Click here to view code image](#)

```
C:\WindowsInternals>UwPList.exe 3728
```

```
List UWP Processes - version 1.1 (C)2016 by Pavel Yosifovich
```

```
Building capabilities map... done.
```

```
Process ID: 3728
```

```
-----
```

```
Image name:
```

```
C:\Windows\SystemApps\Microsoft.Windows.Cortana_cw5n1h2txyewy\SearchU.exe
```

Package name: Microsoft.Windows.Cortana
Publisher: CN=Microsoft Windows, O=Microsoft Corporation, L=Redmond, S=Washington, C=US
Published ID: cw5n1h2txyewy
Architecture: Neutral
Version: 1.7.0.14393
AppContainer SID: S-1-15-2-1861897761-1695161497-2927542615-642690995-327840285-2659745135-2630312742
Lowbox Number: 3
Capabilities: 32
cortanaSettings (S-1-15-3-1024-1216833578-114521899-3977640588-1343180512-2505059295-473916851-3379430393-3088591068) (ENABLED)
visualElementsSystem (S-1-15-3-1024-3299255270-1847605585-2201808924-710406709-3613095291-873286183-3101090833-2655911836) (ENABLED)
perceptionSystem (S-1-15-3-1024-34359262-2669769421-2130994847-3068338639-3284271446-2009814230-2411358368-814686995) (ENABLED)
internetClient (S-1-15-3-1) (ENABLED)
internetClientServer (S-1-15-3-2) (ENABLED)
privateNetworkClientServer (S-1-15-3-3) (ENABLED)
enterpriseAuthentication (S-1-15-3-8) (ENABLED)
musicLibrary (S-1-15-3-6) (ENABLED)
phoneCall (S-1-15-3-1024-383293015-3350740429-1839969850-1819881064-1569454686-4198502490-78857879-1413643331) (ENABLED)
picturesLibrary (S-1-15-3-4) (ENABLED)
sharedUserCertificates (S-1-15-3-9) (ENABLED)
locationHistory (S-1-15-3-1024-3029335854-3332959268-2610968494-1944663922-1108717379-267808753-1292335239-2860040626) (ENABLED)
userDataSystem (S-1-15-3-1024-3324773698-3647103388-1207114580-2173246572-4287945184-2279574858-157813651-603457015) (ENABLED)
contactsSystem (S-1-15-3-1024-2897291008-3029319760-3330334796-465641623-3782203132-742823505-3649274736-3650177846) (ENABLED)
phoneCallHistorySystem (S-1-15-3-1024-2442212369-1516598453-2330995131-3469896071-605735848-2536580394-3691267241-2105387825) (ENABLED)
appointmentsSystem (S-1-15-3-1024-2643354558-482754284-283940418-2629559125-2595130947-547758827-818480453-1102480765) (ENABLED)
chatSystem (S-1-15-3-1024-2210865643-3515987149-1329579022-3761842879-3142652231-

```
371911945-4180581417-4284864962) (ENABLED)
smsSend (S-1-15-3-1024-128185722-850430189-1529384825-139260854-
329499951-
1660931883-3499805589-3019957964) (ENABLED)
emailSystem (S-1-15-3-1024-2357373614-1717914693-1151184220-
2820539834-3900626439-
4045196508-2174624583-3459390060) (ENABLED)
packageQuery (S-1-15-3-1024-1962849891-688487262-3571417821-
3628679630-802580238-
1922556387-206211640-3335523193) (ENABLED)
slapiQueryLicenseValue (S-1-15-3-1024-3578703928-3742718786-7859573-
1930844942-
2949799617-2910175080-1780299064-4145191454) (ENABLED)
S-1-15-3-1861897761-1695161497-2927542615-642690995-327840285-
2659745135-2630312742
(ENABLED)
S-1-15-3-787448254-1207972858-3558633622-1059886964 (ENABLED)
S-1-15-3-3215430884-1339816292-89257616-1145831019 (ENABLED)
S-1-15-3-3071617654-1314403908-1117750160-3581451107 (ENABLED)
S-1-15-3-593192589-1214558892-284007604-3553228420 (ENABLED)
S-1-15-3-3870101518-1154309966-1696731070-4111764952 (ENABLED)
S-1-15-3-2105443330-1210154068-4021178019-2481794518 (ENABLED)
S-1-15-3-2345035983-1170044712-735049875-2883010875 (ENABLED)
S-1-15-3-3633849274-1266774400-1199443125-2736873758 (ENABLED)
S-1-15-3-2569730672-1095266119-53537203-1209375796 (ENABLED)
S-1-15-3-2452736844-1257488215-2818397580-3305426111 (ENABLED)
```

The output shows the package full name, executable directory, AppContainer SID, publisher information, version, and list of capabilities. Also shown is the LowBox number, which is just a local index of the app.

Lastly, you can inspect these properties in the kernel debugger with the `!token` command.

Some UWP apps are called *trusted*, and although they use the Windows Runtime platform like other UWP apps, they do not run inside an AppContainer, and have an integrity level higher than Low. The canonical example is the System Settings app (`%SystemRoot%\ImmersiveControlPanel\SystemSettings.exe`); this seems reasonable, as the Settings app must be able to make changes to the system that would be impossible to do from an AppContainer-hosted process. If you look at its token, you will see the same three attributes—`PKG`, `SYSAPPID`, and `PKGHOSTID`—which confirm that it's still a packaged application, even without the AppContainer token present.

AppContainer and object namespace

Desktop applications can easily share kernel objects by name. For example, suppose process A creates an event object by calling `CreateEvent(Ex)` with the name `MyEvent`. It gets back a handle it can later use to manipulate the event. Process B running in the same session can call `CreateEvent(Ex)` or `OpenEvent` with the same name, `MyEvent`, and (assuming it has appropriate permissions, which is usually the case if running under the same session) get back another handle to the same underlying event object. Now if process A calls `SetEvent` on the event object while process B was blocked in a call to `WaitForSingleObject` on its event handle, process B's waiting thread would be released because it's the same event object. This sharing works because named objects are created in the object manager directory `\Sessions\x\BaseNamedObjects`, as shown in [Figure 7-18](#) with the WinObj Sysinternals tool.

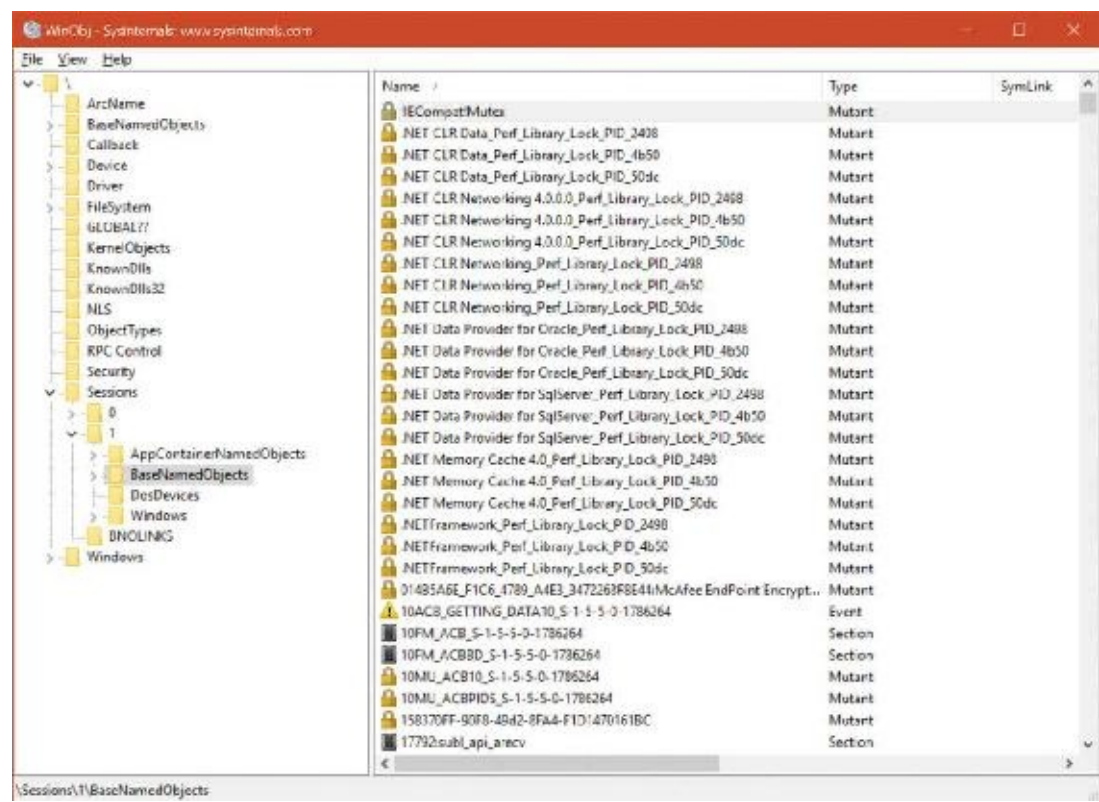


FIGURE 7-18 Object manager directory for named objects.

Furthermore, desktop apps can share objects between sessions by using a name prefixed with `Global\`. This creates the object in the session 0 object directory located in `\BaseNamedObjects` (refer to [Figure 7-18](#)).

AppContainer-based processes have their root object namespace under `\Sessions\x\AppDataContainerNamedObjects\<AppContainerSID>`. Since every AppContainer has a different AppContainer SID, there is no way two UWP apps can share kernel objects. The ability to create a named kernel object in the session 0 object namespace is not allowed for AppContainer processes. [Figure 7-19](#) shows the object manager's directory for the Windows UWP Calculator app.

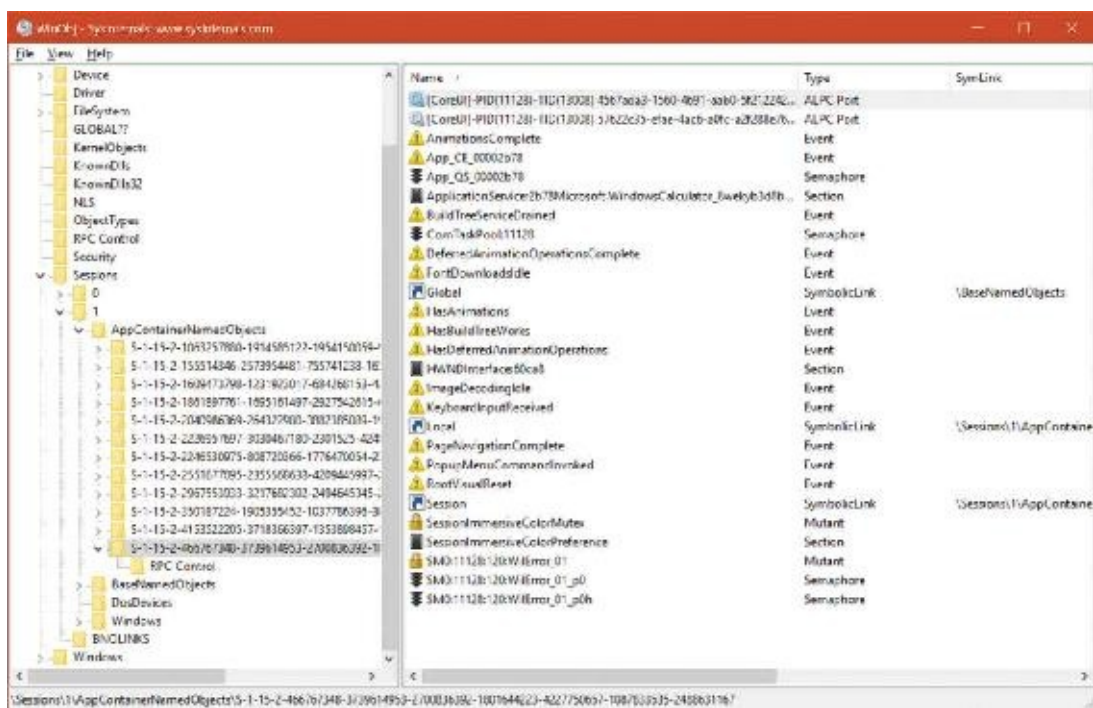


FIGURE 7-19 Object manager directory for Calculator.

UWP apps that want to share data can do so using well-defined contracts, managed by the Windows Runtime. (See the MSDN documentation for more information.)

Sharing kernel objects between desktop apps and UWP apps is possible, and often done by broker services. For example, when requesting access to a file in the Documents folder (and getting the right capability validated) from the file picker broker, the UWP app will receive a file handle that it can use for reads and writes directly, without the cost of marshalling requests back and forth. This is achieved by having the broker duplicate the file handle it obtained directly in the handle table of the UWP application. (More information on handle duplication appears in Chapter 8 in Part 2.) To simplify things even further, the ALPC subsystem (also described in Chapter 8) allows the automatic transfer of handles in this way through ALPC handle attributes, and the Remote Procedure Call (RPC) services that use ALPC as their underlying protocol can use this functionality as part of their interfaces. Marshallable handles in the IDL file will automatically be transferred in this way through the ALPC subsystem.

Outside of official broker RPC services, a desktop app can create a named (or even unnamed) object normally, and then use the `DuplicateHandle` function to inject a handle to the same object into the UWP process manually. This works because desktop apps typically run with medium integrity level and there's nothing preventing them from duplicating handles into UWP processes—only the other way around.



Note

Communication between a desktop app and a UWP is not usually required because a store app cannot have a desktop app companion, and cannot rely on such an app to exist on the device. The capability to inject handles into a UWP app may be needed in specialized cases such as using the desktop bridge (Centennial) to convert a desktop app to a UWP app and communicate with another desktop app that is known to exist.

AppContainer handles

In a typical Win32 application, the presence of the session-local and global BaseNamedObjects directory is guaranteed by the Windows subsystem, as it creates this on boot and session creation. Unfortunately, the AppContainerBaseNamedObjects directory is actually created by the launch application itself. In the case of UWP activation, this is the trusted DComLaunch service, but recall that not all AppContainers are necessarily tied to UWP. They can also be manually created through the right process-creation attributes. (See [Chapter 3](#) for more information on which ones to use.) In this case, it's possible for an untrusted application to have created the object directory (and required symbolic links within it), which would result in the ability for this application to close the handles from underneath the AppContainer application. Even without malicious intent, the original launching application might exit, cleaning up its handles and destroying the AppContainer-specific object directory. To avoid this situation, AppContainer tokens have the ability to store an array of handles that are guaranteed to exist throughout the lifetime of any application using the token. These handles are initially passed in when the AppContainer token is being created (through `NtCreateLowBoxToken`) and are duplicated as kernel handles.

Similar to the per-AppContainer atom table, a special `SEP_CACHED_HANDLES_ENTRY` structure is used, this time based on a hash table that's stored in the logon session structure for this user. (See the "[Logon](#)" section later in this chapter for more information on logon sessions.) This structure contains an array of kernel handles that have been duplicated during the creation of the AppContainer token. They will be closed either when this token is destroyed (because the application is exiting) or when the user logs off (which will result in tearing down the logon session).

EXPERIMENT: Viewing token stored handles

To view token stored handles, follow these steps:

1. Run Calculator and launch local kernel debugging.
2. Search for the calculator process:

[Click here to view code image](#)

```
lkd> !process 0 1 calculator.exe
PROCESS fffff828cc9ed1080
    SessionId: 1  Cid: 4bd8    Peb: d040bbc000  ParentCid: 03a4
DeepFreeze
    DirBase: 5fccaa000  ObjectTable:
ffff950ad9fa2800  HandleCount:
<Data Not Accessible>
    Image: Calculator.exe
    VadRoot fffff828cd2c9b6a0 Vads 168 Clone 0 Private 2938.
Modified 3332.
Locked 0.
    DeviceMap fffff950aad2cd2f0
    Token fffff950adb313060
    ElapsedTime 1 Day 08:01:47.018
    UserTime 00:00:00.015
    KernelTime 00:00:00.031
    QuotaPoolUsage[PagedPool] 465880
    QuotaPoolUsage[NonPagedPool] 23288
    Working Set Sizes (now,min,max) (7434, 50, 345) (29736KB,
```

```

200KB, 1380KB)
    PeakWorkingSetSize          11097
    VirtualSize                 303 Mb
    PeakVirtualSize             314 Mb
    PageFaultCount              21281
    MemoryPriority               BACKGROUND
    BasePriority                 8
    CommitCharge                4925
    Job                         fffff828cd4914060

```

3. Dump the token using the dt command. (Remember to mask the lower 3 or 4 bits if they are not zero.)

[Click here to view code image](#)

```

lkd> dt nt!_token fffff950adb313060
+0x000 TokenSource           : _TOKEN_SOURCE
+0x010 TokenId               : _LUID
+0x018 AuthenticationId     : _LUID
+0x020 ParentTokenId        : _LUID
...
+0x0c8 TokenFlags           : 0x4a00
+0x0cc TokenInUse           : 0x1 ''
+0x0d0 IntegrityLevelIndex  : 1
+0x0d4 MandatoryPolicy      : 1
+0x0d8 LogonSession         : 0xfffff950a'b4bb35c0
_SEP_LOGON_SESSION_REFERENCES
+0x0e0 OriginatingLogonSession : _LUID
+0x0e8 SidHash              : _SID_AND_ATTRIBUTES_HASH
+0x1f8 RestrictedSidHash    : _SID_AND_ATTRIBUTES_HASH
+0x308 pSecurityAttributes  : 0xfffff950a'e4ff57f0
_AUTHZBASEP_SECURITY_ATTRIBUTES_INFORMATION
+0x310 Package              : 0xfffff950a'e00ed6d0 Void
+0x318 Capabilities         : 0xfffff950a'e8e8fbc0
_SID_AND_ATTRIBUTES
+0x320 CapabilityCount      : 1
+0x328 CapabilitiesHash     : _SID_AND_ATTRIBUTES_HASH
+0x438 LowboxNumberEntry    : 0xfffff950a'b3fd55d0
_SEP_LOWBOX_NUMBER_ENTRY
+0x440 LowboxHandlesEntry   : 0xfffff950a'e6ff91d0
_SEP_LOWBOX_HANDLES_ENTRY
+0x448 pClaimAttributes     : (null)
...

```

4. Dump the LowboxHandlesEntry member:

[Click here to view code image](#)

```

lkd> dt nt!_sep_lowbox_handles_entry 0xfffff950a'e6ff91d0

```

```

+0x000 HashEntry          : _RTL_DYNAMIC_HASH_TABLE_ENTRY
+0x018 ReferenceCount     : 0n10
+0x020 PackageSid        : 0xfffff950a'e6ff9208 Void
+0x028 HandleCount       : 6
+0x030 Handles           : 0xfffff950a'e91d8490 ->
0xfffffffff'800023cc Void

```

5. There are six handles. Let's dump their values:

[Click here to view code image](#)

```

lkd> dq 0xfffff950ae91d8490 L6
fffff950a'e91d8490  ffffffff'800023cc  ffffffff'80001e80
fffff950a'e91d84a0  ffffffff'80004214  ffffffff'8000425c
fffff950a'e91d84b0  ffffffff'800028c8  ffffffff'80001834

```

6. You can see that these handles are kernel handles—that is, handle values starting with 0xfffffffff (64 bit). Now you can use the !handle command to look at individual handles. Here are two examples from the six handles above:

[Click here to view code image](#)

```

lkd> !handle ffffffff'80001e80

```

```

PROCESS fffff828cd71b3600

```

```

    SessionId: 1  Cid: 27c4    Peb: 3fdfb2f000  ParentCid: 2324

```

```

    DirBase: 80bb85000  ObjectTable:

```

```

fffff950addabf7c0  HandleCount:

```

```

<Data Not Accessible>

```

```

    Image: windbg.exe

```

Kernel handle Error reading handle count.

```

80001e80: Object: fffff950ada206ea0  GrantedAccess: 0000000f
(Protected)

```

```

(Inherit) (Audit) Entry: fffff950ab5406a00

```

```

Object: fffff950ada206ea0  Type: (ffff828cb66b33b0) Directory

```

```

    ObjectHeader: fffff950ada206e70 (new version)

```

```

        HandleCount: 1  PointerCount: 32770

```

```

        Directory Object: fffff950ad9a62950  Name: RPC Control

```

Hash	Address	Type	Name
----	-----	----	----

23	ffff828cb6ce6950	ALPC Port	
----	------------------	-----------	--

```

OLE376512B99BCCA5DE4208534E7732

```

```

lkd> !handle ffffffff'800028c8

```

```

PROCESS fffff828cd71b3600

```

```

    SessionId: 1  Cid: 27c4    Peb: 3fdfb2f000  ParentCid: 2324

```

```

    DirBase: 80bb85000  ObjectTable:

```

```
ffff950addabf7c0  HandleCount: <Data
Not Accessible>
    Image: windbg.exe
```

Kernel handle Error reading handle count.

```
800028c8: Object: ffff950ae7a8fa70  GrantedAccess: 000f0001
(Audit) Entry:
ffff950acc426320
Object: ffff950ae7a8fa70  Type: (ffff828cb66296f0) SymbolicLink
    ObjectHeader: ffff950ae7a8fa40 (new version)
        HandleCount: 1  PointerCount: 32769
        Directory Object: ffff950ad9a62950  Name: Session
        Flags: 00000000 ( Local )
        Target String is '\Sessions\1\AppDataContainerNamedObjects
\S-1-15-2-466767348-3739614953-2700836392-1801644223-4227750657
-1087833535-2488631167'
```

Finally, because the ability to restrict named objects to a particular object directory namespace is a valuable security tool for sandboxing named object access, the upcoming (at the time of this writing) Windows 10 Creators Update includes an additional token capability called *BNO isolation* (where *BNO* refers to *BaseNamedObjects*). Using the same `SEP_CACHE_HANDLES_ENTRY` structure, a new field, `BnoIsolationHandlesEntry`, is added to the `TOKEN` structure, with the type set to `SepCachedHandlesEntryBnoIsolation` instead of `SepCachedHandlesEntryLowbox`. To use this feature, a special process attribute must be used (see [Chapter 3](#) for more information), which contains an isolation prefix and a list of handles. At this point, the same `LowBox` mechanism is used, but instead of an `AppContainer` SID object directory, a directory with the prefix indicated in the attribute is used.

Brokers

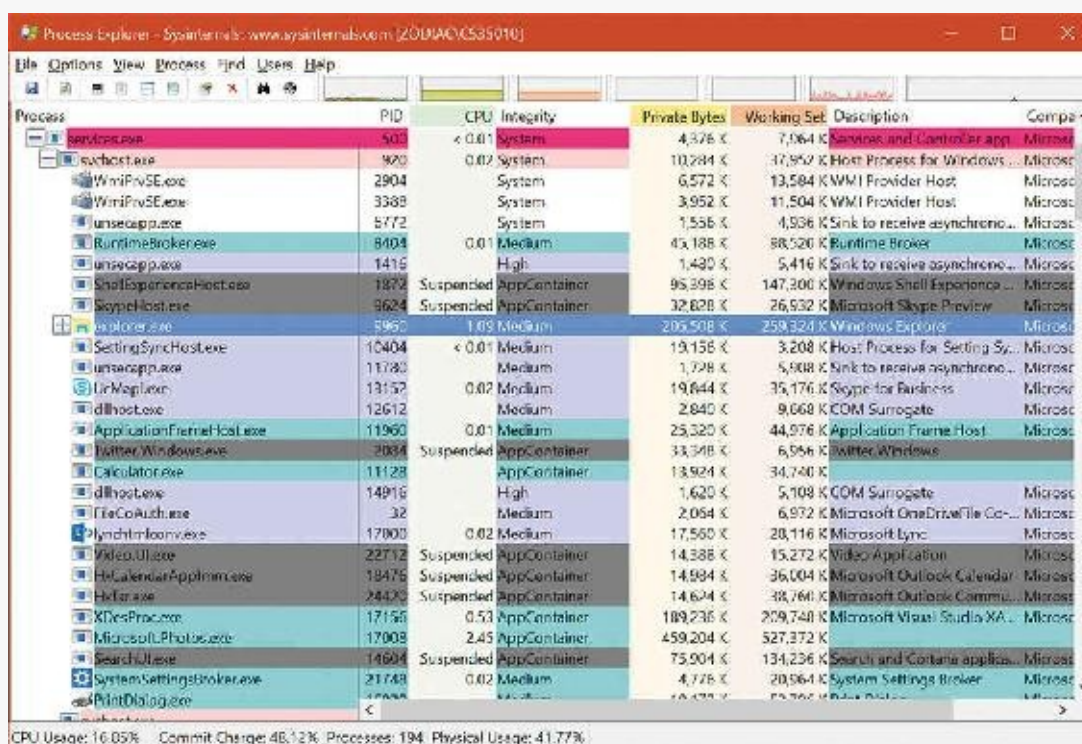
Because `AppContainer` processes have almost no permissions except for those implicitly granted with capabilities, some common operations cannot be performed directly by the `AppContainer` and require help. (There are no capabilities for these, as these are too low level to be visible to users in the store, and difficult to manage.) Some examples include selecting files using the common File Open dialog box or printing with a Print dialog box. For these and other similar operations, Windows provides helper processes, called *brokers*, managed by the system broker process, `RuntimeBroker.exe`.

An `AppContainer` process that requires any of these services communicates with the Runtime Broker through a secure `ALPC` channel and Runtime Broker initiates the creation of the requested broker process. Examples are `%SystemRoot%\PrintDialog\PrintDialog.exe` and `%SystemRoot%\System32\PickerHost.exe`.

EXPERIMENT: Brokers

The following steps show how broker processes are launched and terminated:

1. Click the **Start** button, type **Photos**, and select the **Photos** option to run the built-in Windows 10 Photos application.
2. Open Process Explorer, switch the process list to a tree view, and locate the Microsoft.Photos.exe process. Place both windows side by side.
3. In the Photos app, select a picture file, and click **Print** in the top ellipsis menu or right-click the picture and choose **Print** from the menu that appears. The Print dialog box should open, and Process Explorer should show the newly created broker (PrintDialog.exe). Notice they are all children of the same Svchost process. (All UWP processes are launched by the DCOMLaunch service hosted inside that process.)



4. Close the Print dialog box. The PrintDialog.exe process should exit.

Logon

Interactive logon (as opposed to network logon) occurs through the interaction of the following:

- The logon process (Winlogon.exe)
- The logon user interface process (LogonUI.exe) and its credential providers
- Lsass.exe
- One or more authentication packages
- SAM or Active Directory

Authentication packages are DLLs that perform authentication checks. Kerberos is the Windows authentication package for interactive logon to a domain. MSV1_0 is the Windows authentication package for interactive logon to a local computer, for domain logons to trusted pre-Windows 2000 domains, and for times when no domain controller is accessible.

Winlogon is a trusted process responsible for managing security-related user interactions. It

coordinates logon, starts the user's first process at logon, and handles logoff. It also manages various other operations relevant to security, including launching LogonUI for entering passwords at logon, changing passwords, and locking and unlocking the workstation. The Winlogon process must ensure that operations relevant to security aren't visible to any other active processes. For example, Winlogon guarantees that an untrusted process can't get control of the desktop during one of these operations and thus gain access to the password.

Winlogon relies on the credential providers installed on the system to obtain a user's account name or password. Credential providers are COM objects located inside DLLs. The default providers are `authui.dll`, `SmartcardCredentialProvider.dll`, and `FaceCredentialProvider.dll`, which support password, smartcard PIN, and face-recognition authentication, respectively. Allowing other credential providers to be installed enables Windows to use different user-identification mechanisms. For example, a third party might supply a credential provider that uses a thumbprint-recognition device to identify users and extract their passwords from an encrypted database. Credential providers are listed in `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Authentication\Credential Providers`, where each subkey identifies a credential provider class by its COM CLSID. (The CLSID itself must be registered at `HKCR\CLSID` like any other COM class.) You can use the `CPLIST.exe` tool provided with the downloadable resources for this book to list the credential providers with their CLSID, friendly name, and implementation DLL.

To protect Winlogon's address space from bugs in credential providers that might cause the Winlogon process to crash (which, in turn, will result in a system crash, because Winlogon is considered a critical system process), a separate process, `LogonUI.exe`, is used to actually load the credential providers and display the Windows logon interface to users. This process is started on demand whenever Winlogon needs to present a user interface to the user, and it exits after the action has finished. It also allows Winlogon to simply restart a new LogonUI process should it crash for any reason.

Winlogon is the only process that intercepts logon requests from the keyboard. These are sent through an RPC message from `Win32k.sys`. Winlogon immediately launches the LogonUI application to display the user interface for logon. After obtaining a user name and password from credential providers, Winlogon calls `Lsass` to authenticate the user attempting to log on. If the user is authenticated, the logon process activates a logon shell on behalf of that user. The interaction between the components involved in logon is illustrated in [Figure 7-20](#).

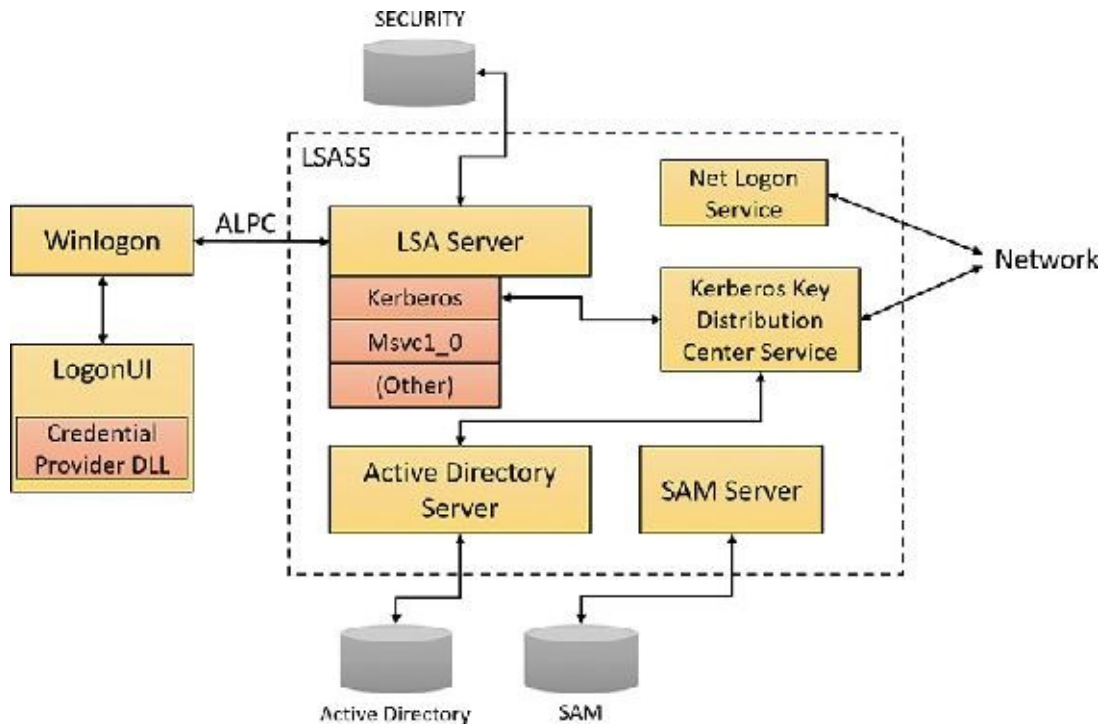


FIGURE 7-20 Components involved in login.

In addition to supporting alternative credential providers, LogonUI can load additional network provider DLLs that need to perform secondary authentication. This capability allows multiple network providers to gather identification and authentication information all at one time during normal login. A user logging on to a Windows system might simultaneously be authenticated on a Linux server. That user would then be able to access resources of the UNIX server from the Windows machine without requiring additional authentication. Such a capability is known as one form of single sign-on.

Winlogon initialization

During system initialization, before any user applications are active, Winlogon performs the following steps to ensure that it controls the workstation once the system is ready for user interaction:

1. It creates and opens an interactive window station (for example, \Sessions\1\Windows\WindowStations\WinSta0 in the object manager namespace) to represent the keyboard, mouse, and monitor. Winlogon creates a security descriptor for the station that has one and only one ACE containing only the system SID. This unique security descriptor ensures that no other process can access the workstation unless explicitly allowed by Winlogon.
2. It creates and opens two desktops: an application desktop (\Sessions\1\Windows\WinSta0\Default, also known as the interactive desktop) and a Winlogon desktop (\Sessions\1\Windows\WinSta0\Winlogon, also known as the Secure Desktop). The security on the Winlogon desktop is created so that only Winlogon can access that desktop. The other desktop allows both Winlogon and users to access it. This arrangement means that any time the Winlogon desktop is active, no other process has access to any active code or data associated with the desktop. Windows uses this feature to protect the secure operations that involve passwords and locking and unlocking the desktop.
3. Before anyone logs on to a computer, the visible desktop is Winlogon's. After a user logs on, pressing the SAS sequence (by default, Ctrl+Alt+Del) switches the desktop from Default to Winlogon and launches LogonUI. (This explains why all the windows on your interactive desktop seem to disappear when you press Ctrl+Alt+Del, and then return when you dismiss the Windows

- Security dialog box.) Thus, the SAS always brings up a Secure Desktop controlled by Winlogon.
4. It establishes an ALPC connection with Lsass. This connection will be used for exchanging information during logon, logoff, and password operations, and is made by calling `LsaRegisterLogonProcess`.
 5. It registers the Winlogon RPC message server, which listens for SAS, logoff, and workstation lock notifications from Win32k. This measure prevents Trojan horse programs from gaining control of the screen when the SAS is entered.



Note

The Wininit process performs steps similar to steps 1 and 2 to allow legacy interactive services running on session 0 to display windows, but it does not perform any other steps because session 0 is not available for user logon.

How SAS is implemented

The SAS is secure because no application can intercept the Ctrl+Alt+Del keystroke combination or prevent Winlogon from receiving it. Win32k.sys reserves the Ctrl+Alt+Del key combination so that whenever the Windows input system (implemented in the raw input thread in Win32k) sees the combination, it sends an RPC message to Winlogon's message server, which listens for such notifications. The keystrokes that map to a registered hot key are not sent to any process other than the one that registered it, and only the thread that registered a hot key can unregister it, so a Trojan horse application cannot deregister Winlogon's ownership of the SAS.

A Windows function, `SetWindowsHookEx`, enables an application to install a hook procedure that's invoked every time a keystroke is pressed, even before hot keys are processed, and allows the hook to squash keystrokes. However, the Windows hot key processing code contains a special case for Ctrl+Alt+Del that disables hooks so that the keystroke sequence can't be intercepted. In addition, if the interactive desktop is locked, only hot keys owned by Winlogon are processed.

Once the Winlogon desktop is created during initialization, it becomes the active desktop. When the Winlogon desktop is active, it is always locked. Winlogon unlocks its desktop only to switch to the application desktop or the screen-saver desktop. (Only the Winlogon process can lock or unlock a desktop.)

User logon steps

Logon begins when a user presses the SAS (Ctrl+Alt+Del). After the SAS is pressed, Winlogon starts LogonUI, which calls the credential providers to obtain a user name and password. Winlogon also creates a unique local logon SID for this user, which it assigns to this instance of the desktop (keyboard, screen, and mouse). Winlogon passes this SID to Lsass as part of the `LsaLogonUser` call. If the user is successfully logged on, this SID will be included in the logon process token—a step that protects access to the desktop. For example, another logon to the same account but on a different system will be unable to write to the first machine's desktop because this second logon won't be in the first logon's desktop token.

When the user name and password have been entered, Winlogon retrieves a handle to a package by calling the Lsass function `LsaLookupAuthenticationPackage`. Authentication packages are listed in the registry under `HKLM\SYSTEM\CurrentControlSet\Control\Lsa`. Winlogon passes logon

information to the authentication package via `LsaLogonUser`. Once a package authenticates a user, Winlogon continues the logon process for that user. If none of the authentication packages indicates a successful logon, the logon process is aborted.

Windows uses two standard authentication packages for interactive username/password-based logons:

- **MSV1_0** The default authentication package on a stand-alone Windows system is MSV1_0 (`Msv1_0.dll`), an authentication package that implements LAN Manager 2 protocol. Lsass also uses MSV1_0 on domain-member computers to authenticate to pre-Windows 2000 domains and computers that can't locate a domain controller for authentication. (Computers that are disconnected from the network fall into this latter category.)
- **Kerberos** The Kerberos authentication package, `Kerberos.dll`, is used on computers that are members of Windows domains. The Windows Kerberos package, with the cooperation of Kerberos services running on a domain controller, supports the Kerberos protocol. This protocol is based on Internet RFC 1510. (Visit the Internet Engineering Task Force [IETF] website at <http://www.ietf.org> for detailed information on the Kerberos standard.)

MSV1_0

The MSV1_0 authentication package takes the user name and a hashed version of the password and sends a request to the local SAM to retrieve the account information, which includes the hashed password, the groups to which the user belongs, and any account restrictions. MSV1_0 first checks the account restrictions, such as hours or type of accesses allowed. If the user can't log on because of the restrictions in the SAM database, the logon call fails and MSV1_0 returns a failure status to the LSA.

MSV1_0 then compares the hashed password and user name to that obtained from the SAM. In the case of a cached domain logon, MSV1_0 accesses the cached information by using Lsass functions that store and retrieve "secrets" from the LSA database (the SECURITY hive of the registry). If the information matches, MSV1_0 generates a LUID for the logon session and creates the logon session by calling Lsass, associating this unique identifier with the session and passing the information needed to ultimately create an access token for the user. (Recall that an access token includes the user's SID, group SIDs, and assigned privileges.)



Note

MSV1_0 does not cache a user's entire password hash in the registry because that would enable someone with physical access to the system to easily compromise a user's domain account and gain access to encrypted files and to network resources the user is authorized to access. Instead, it caches half of the hash. The cached half-hash is sufficient to verify that a user's password is correct, but it isn't sufficient to gain access to EFS keys and to authenticate as the user on a domain because these actions require the full hash.

If MSV1_0 needs to authenticate using a remote system, as when a user logs on to a trusted pre-Windows 2000 domain, MSV1_0 uses the Netlogon service to communicate with an instance of Netlogon on the remote system. Netlogon on the remote system interacts with the MSV1_0 authentication package on that system, passing back authentication results to the system on which the logon is being performed.

Kerberos

The basic control flow for Kerberos authentication is the same as the flow for MSV1_0. However, in most cases, domain logons are performed from member workstations or servers rather than on a domain controller, so the authentication package must communicate across the network as part of the authentication process. The package does so by communicating via the Kerberos TCP/IP port (port 88) with the Kerberos service on a domain controller. The Kerberos Key Distribution Center service (Kdcsvc.dll), which implements the Kerberos authentication protocol, runs in the Lsass process on domain controllers.

After validating hashed user-name and password information with Active Directory's user account objects (using the Active Directory server Ntdsa.dll), Kdcsvc returns domain credentials to Lsass, which returns the result of the authentication and the user's domain logon credentials (if the logon was successful) across the network to the system where the logon is taking place.



Note

This description of Kerberos authentication is highly simplified, but it highlights the roles of the various components involved. Although the Kerberos authentication protocol plays a key role in distributed domain security in Windows, its details are outside the scope of this book.

After a logon has been authenticated, Lsass looks in the local policy database for the user's allowed access, including interactive, network, batch, or service process. If the requested logon doesn't match the allowed access, the logon attempt will be terminated. Lsass deletes the newly created logon session by cleaning up any of its data structures and then returns failure to Winlogon, which in turn displays an appropriate message to the user. If the requested access is allowed, Lsass adds the appropriate additional security IDs (such as Everyone, Interactive, and the like). It then checks its policy database for any granted privileges for all the SIDs for this user and adds these privileges to the user's access token.

When Lsass has accumulated all the necessary information, it calls the executive to create the access token. The executive creates a primary access token for an interactive or service logon and an impersonation token for a network logon. After the access token is successfully created, Lsass duplicates the token, creating a handle that can be passed to Winlogon, and closes its own handle. If necessary, the logon operation is audited. At this point, Lsass returns success to Winlogon along with a handle to the access token, the LUID for the logon session, and the profile information, if any, that the authentication package returned.

EXPERIMENT: Listing active logon sessions

As long as at least one token exists with a given logon session LUID, Windows considers the logon session to be active. You can use the LogonSessions tool from Sysinternals, which uses the LsaEnumerateLogonSessions function (documented in the Windows SDK) to list the active logon sessions:

[Click here to view code image](#)

```
C:\WINDOWS\system32>logonsessions
```

```
LogonSessions v1.4 - Lists logon session information  
Copyright (C) 2004-2016 Mark Russinovich
```

- [0] Logon session 00000000:000003e7:
User name: WORKGROUP\ZODIAC\$
Auth package: NTLM
Logon type: (none)
Session: 0
Sid: S-1-5-18
Logon time: 09-Dec-16 15:22:31
Logon server:
DNS Domain:
UPN:
- [1] Logon session 00000000:0000cdce:
User name:
Auth package: NTLM
Logon type: (none)
Session: 0
Sid: (none)
Logon time: 09-Dec-16 15:22:31
Logon server:
DNS Domain:
UPN:
- [2] Logon session 00000000:000003e4:
User name: WORKGROUP\ZODIAC\$
Auth package: Negotiate
Logon type: Service
Session: 0
Sid: S-1-5-20
Logon time: 09-Dec-16 15:22:31
Logon server:
DNS Domain:
UPN:
- [3] Logon session 00000000:00016239:
User name: Window Manager\DWM-1
Auth package: Negotiate
Logon type: Interactive
Session: 1
Sid: S-1-5-90-0-1
Logon time: 09-Dec-16 15:22:32
Logon server:
DNS Domain:
UPN:
- [4] Logon session 00000000:00016265:

User name: Window Manager\DWM-1
Auth package: Negotiate
Logon type: Interactive
Session: 1
Sid: S-1-5-90-0-1
Logon time: 09-Dec-16 15:22:32
Logon server:
DNS Domain:
UPN:

[5] Logon session 00000000:000003e5:
User name: NT AUTHORITY\LOCAL SERVICE
Auth package: Negotiate
Logon type: Service
Session: 0
Sid: S-1-5-19
Logon time: 09-Dec-16 15:22:32
Logon server:
DNS Domain:
UPN:

...

[8] Logon session 00000000:0005c203:
User name: NT VIRTUAL MACHINE\AC9081B6-1E96-4BC8-8B3B-C609D4F85F7D
Auth package: Negotiate
Logon type: Service
Session: 0
Sid: S-1-5-83-1-2895151542-1271406230-163986315-2103441620
Logon time: 09-Dec-16 15:22:35
Logon server:
DNS Domain:
UPN:

[9] Logon session 00000000:0005d524:
User name: NT VIRTUAL MACHINE\B37F4A3A-21EF-422D-8B37-AB6B0A016ED8
Auth package: Negotiate
Logon type: Service
Session: 0
Sid: S-1-5-83-1-3011463738-1110254063-1806382987-3631087882
Logon time: 09-Dec-16 15:22:35
Logon server:
DNS Domain:
UPN:

```

...
[12] Logon session 00000000:0429ab2c:
    User name:      IIS APPPOOL\DefaultAppPool
    Auth package:   Negotiate
    Logon type:     Service
    Session:        0
    Sid:            S-1-5-82-3006700770-424185619-1745488364-
794895919-4004696415
    Logon time:     09-Dec-16 22:33:03
    Logon server:
    DNS Domain:
    UPN:

```

Information reported for a session includes the SID and name of the user associated with the session, as well as the session's authentication package and logon time. Note that the Negotiate authentication package, seen in logon sessions 2 and 9 in the preceding output, will attempt to authenticate via Kerberos or NTLM, depending on which is most appropriate for the authentication request.

The LUID for a session is displayed on the `Logon Session` line of each session block. Using the `Handle.exe` utility (also from Sysinternals), you can find the tokens that represent a particular logon session. For example, to find the tokens for logon session 8 in the output just shown, you could enter this command:

[Click here to view code image](#)

```
C:\WINDOWS\system32>handle -a 5c203
```

```

Nthandle v4.1 - Handle viewer
Copyright (C) 1997-2016 Mark Russinovich
Sysinternals - www.sysinternals.com

```

System	pid: 4	type: Directory	1274:
\Sessions\0\			
DosDevices\00000000-0005c203			
lsass.exe	pid: 496	type: Token	D7C: NT VIRTUAL
MACHINE\			
AC9081B6-1E96-4BC8-8B3B-C609D4F85F7D:5c203			
lsass.exe	pid: 496	type: Token	2350: NT VIRTUAL
MACHINE\			
AC9081B6-1E96-4BC8-8B3B-C609D4F85F7D:5c203			
lsass.exe	pid: 496	type: Token	2390: NT VIRTUAL
MACHINE\			
AC9081B6-1E96-4BC8-8B3B-C609D4F85F7D:5c203			
svchost.exe	pid: 900	type: Token	804: NT VIRTUAL
MACHINE\			
AC9081B6-1E96-4BC8-8B3B-C609D4F85F7D:5c203			
svchost.exe	pid: 1468	type: Token	10EC: NT VIRTUAL
MACHINE\			

```

AC9081B6-1E96-4BC8-8B3B-C609D4F85F7D:5c203
vmms.exe          pid: 4380    type: Token          A34: NT VIRTUAL
MACHINE\
AC9081B6-1E96-4BC8-8B3B-C609D4F85F7D:5c203
vmcompute.exe     pid: 6592    type: Token          200: NT VIRTUAL
MACHINE\
AC9081B6-1E96-4BC8-8B3B-C609D4F85F7D:5c203
vmwp.exe          pid: 7136    type: WindowStation  168:
\Windows\WindowStations\
Service-0x0-5c203$
vmwp.exe          pid: 7136    type: WindowStation  170:
\Windows\WindowStations\
Service-0x0-5c203$

```

Winlogon then looks in the registry at the value HKLM\SOFTWARE\Microsoft\Windows NT\Current Version\Winlogon\Userinit and creates a process to run whatever the value of that string is. (This value can be several EXEs separated by commas.) The default value is Userinit.exe, which loads the user profile and then creates a process to run whatever the value of HKCU\SOFTWARE\Microsoft\Windows NT\Current Version\Winlogon\Shell is, if that value exists. That value does not exist by default, however. If it doesn't exist, Userinit.exe does the same for HKLM\SOFTWARE\Microsoft\Windows NT\Current Version\Winlogon\Shell, which defaults to Explorer.exe. Userinit then exits (which is why Explorer.exe shows up as having no parent when examined in Process Explorer). For more information on the steps followed during the user logon process, see Chapter 11 in Part 2.

Assured authentication

A fundamental problem with password-based authentication is that passwords can be revealed or stolen and used by malicious third parties. Windows includes a mechanism that tracks the authentication strength of how a user authenticated with the system, which allows objects to be protected from access if a user did not authenticate securely. (Smartcard authentication is considered to be a stronger form of authentication than password authentication.)

On systems that are joined to a domain, the domain administrator can specify a mapping between an object identifier (OID) (a unique numeric string representing a specific object type) on a certificate used for authenticating a user (such as on a smartcard or hardware security token) and a SID that is placed into the user's access token when the user successfully authenticates with the system. An ACE in a DACL on an object can specify such a SID be part of a user's token in order for the user to gain access to the object. Technically, this is known as a *group claim*. In other words, the user is claiming membership in a particular group, which is allowed certain access rights on specific objects, with the claim based upon the authentication mechanism. This feature is not enabled by default, and it must be configured by the domain administrator in a domain with certificate-based authentication.

Assured authentication builds on existing Windows security features in a way that provides a great deal of flexibility to IT administrators and anyone concerned with enterprise IT security. The enterprise decides which OIDs to embed in the certificates it uses for authenticating users and the mapping of particular OIDs to Active Directory universal groups (SIDs). A user's group membership can be used to identify whether a certificate was used during the logon operation. Different certificates can have different issuance policies and, thus, different levels of security, which can be used to protect highly sensitive objects (such as files or anything else that might have a security descriptor).

Authentication protocols (APs) retrieve OIDs from certificates during certificate-based authentication. These OIDs must be mapped to SIDs, which are in turn processed during group membership expansion, and placed in the access token. The mapping of OID to universal group is specified in Active Directory.

As an example, an organization might have several certificate-issuance policies named Contractor, Full Time Employee, and Senior Management, which map to the universal groups Contractor-Users, FTE-Users, and SM-Users, respectively. A user named Abby has a smartcard with a certificate issued using the Senior Management issuance policy. When she logs in using her smartcard, she receives an additional group membership (which is represented by a SID in her access token) indicating that she is a member of the SM-Users group. Permissions can be set on objects (using an ACL) such that only members of the FTE-Users or SM-Users group (identified by their SIDs within an ACE) are granted access. If Abby logs in using her smartcard, she can access those objects, but if she logs in with just her user name and password (without the smartcard), she cannot access those objects because she will not have either the FTE-Users or SM-Users group in her access token. A user named Toby who logs in with a smartcard that has a certificate issued using the Contractor issuance policy would not be able to access an object that has an ACE requiring FTE-Users or SM-Users group membership.

Windows Biometric Framework

Windows provides a standardized mechanism for supporting certain types of biometric devices, such as fingerprint scanners, used to enable user identification via a fingerprint swipe: the Windows Biometric Framework (WBF). Like many other such frameworks, the WBF was developed to isolate the various functions involved in supporting such devices, so as to minimize the code required to implement a new device.

The primary components of the WBF are shown in [Figure 7-21](#). Except as noted in the following list, all of these components are supplied by Windows:

- **The Windows Biometric Service (%SystemRoot%\System32\Wbiosrv.dll)** This provides the process-execution environment in which one or more biometric service providers can execute.
- **The Windows Biometric Driver Interface (WBDI)** This is a set of interface definitions (IRP major function codes, DeviceIoControl codes, and so forth) to which any driver for a biometric scanner device must conform if it is to be compatible with the Windows Biometric Service. WBDI drivers can be developed using any of the standard driver frameworks (UMDF, KMDF and WDM). However, UMDF is recommended to reduce code size and increase reliability. WBDI is described in the Windows Driver Kit documentation.
- **The Windows Biometric API** This allows existing Windows components such as Winlogon and LogonUI to access the biometric service. Third-party applications have access to the Windows Biometric API and can use the biometric scanner for functions other than logging in to Windows. An example of a function in this API is `WinBioEnumServiceProviders`. The Biometric API is exposed by `%SystemRoot%\System32\Winbio.dll`.
- **The fingerprint biometric service provider** This wraps the functions of biometric-type-specific adapters to present a common interface, independent of the type of biometric, to the Windows Biometric Service. In the future, additional types of biometrics, such as retinal scans or voiceprint analyzers, might be supported by additional biometric service providers. The biometric service provider in turn uses three adapters, which are user-mode DLLs:
 - **The sensor adapter** This exposes the data-capture functionality of the scanner. The sensor adapter usually uses Windows I/O calls to access the scanner hardware. Windows provides a sensor adapter that can be used with simple sensors, those for which a WBDI driver exists. For

more complex sensors, the sensor adapter is written by the sensor vendor.

- **The engine adapter** This exposes processing and comparison functionality specific to the scanner's raw data format and other features. The actual processing and comparison might be performed within the engine adapter DLL, or the DLL might communicate with some other module. The engine adapter is always provided by the sensor vendor.
 - **The storage adapter** This exposes a set of secure storage functions. These are used to store and retrieve templates against which scanned biometric data is matched by the engine adapter. Windows provides a storage adapter using Windows cryptography services and standard disk file storage. A sensor vendor might provide a different storage adapter.
- **The functional device driver for the actual biometric scanner device** This exposes the WBDI at its upper edge. It usually uses the services of a lower-level bus driver, such as the USB bus driver, to access the scanner device. This driver is always provided by the sensor vendor.

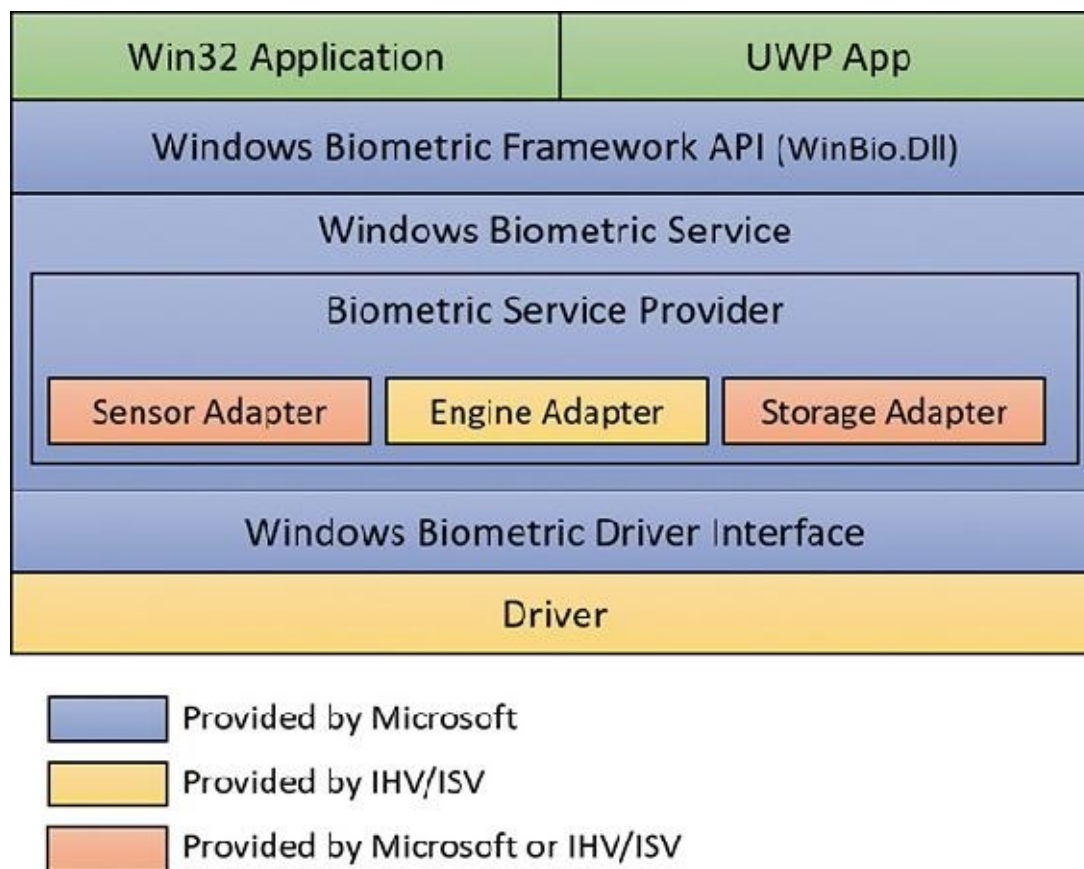


FIGURE 7-21 Windows Biometric Framework components and architecture.

A typical sequence of operations to support logging in via a fingerprint scan might be as follows:

1. After initialization, the sensor adapter receives from the service provider a request for capture data. The sensor adapter in turn sends a `DeviceIoControl` request with the `IOCTL_BIOMETRIC_CAPTURE_DATA` control code to the WBDI driver for the fingerprint scanner device.
2. The WBDI driver puts the scanner into capture mode and queues the `IOCTL_BIOMETRIC_CAPTURE_DATA` request until a fingerprint scan occurs.
3. A prospective user swipes a finger across the scanner. The WBDI driver receives notification of this, obtains the raw scan data from the sensor, and returns this data to the sensor driver in a buffer associated with the `IOCTL_BIOMETRIC_CAPTURE_DATA` request.
4. The sensor adapter provides the data to the fingerprint biometric service provider, which in turn

- passes the data to the engine adapter.
5. The engine adapter processes the raw data into a form compatible with its template storage.
 6. The fingerprint biometric service provider uses the storage adapter to obtain templates and corresponding security IDs from secure storage. It invokes the engine adapter to compare each template to the processed scan data. The engine adapter returns a status indicating whether it's a match or not a match.
 7. If a match is found, the Windows Biometric Service notifies Winlogon, via a credential provider DLL, of a successful login and passes it the security ID of the identified user. This notification is sent via an ALPC message, providing a path that cannot be spoofed.

Windows Hello

Windows Hello, introduced in Windows 10, provides new ways to authenticate users based on biometric information. With this technology, users can log in effortlessly just by showing themselves to the device's camera or swiping their finger.

At the time of this writing, Windows Hello supports three types of biometric identification:

- Fingerprint
- Face
- Iris

The security aspect of biometrics needs to be considered first. What is the likelihood of someone being identified as you? What is the likelihood of you *not* being identified as you? These questions are parameterized by two factors:

- **False accept rate (uniqueness)** This is the probability of another user having the same biometric data as you. Microsoft's algorithms make sure the likelihood is 1 in 100,000.
- **False reject rate (reliability)** This is the probability of you not being correctly recognized as you (for example, in abnormal lighting conditions for face or iris recognition). Microsoft's implementation makes sure there is less than 1 percent chance of this happening. If it does happen, the user can try again or use a PIN code instead.

Using a PIN code may seem less secure than using a full-blown password (the PIN can be as simple as a four-digit number). However, a PIN is more secure than a password for two main reasons:

- The PIN code is local to the device and is never transmitted across the network. This means that even if someone gets a hold of the PIN, they cannot use it to log in as the user from any other device. Passwords, on the other hand, travel to the domain controller. If someone gets hold of the password, they can log in from another machine into the domain.
- The PIN code is stored in the Trusted Platform Module (TPM)—a piece of hardware that also plays a part in Secure Boot (discussed in detail in Chapter 11 in Part 2)—so is difficult to access. In any case, it requires physical access to the device, raising the bar considerably for a potential security compromise.

Windows Hello is built upon the Windows Biometric Framework (WBF) (described in the previous section). Current laptop devices support fingerprint and face biometrics, while iris is only supported on the Microsoft Lumia 950 and 950 XL phones. (This will likely change and expand in future devices.) Note that face recognition requires an infrared (IR) camera as well as a normal (RGB) one, and is supported on devices such as the Microsoft Surface Pro 4 and the Surface Book.

User Account Control and virtualization

User Account Control (UAC) is meant to enable users to run with standard user rights as opposed to administrative rights. Without administrative rights, users cannot accidentally (or deliberately) modify system settings, malware can't normally alter system security settings or disable antivirus software, and users can't compromise the sensitive information of other users on shared computers. Running with standard user rights can thus mitigate the impact of malware and protect sensitive data on shared computers.

UAC had to address a couple of problems to make it practical for a user to run with a standard user account. First, because the Windows usage model has been one of assumed administrative rights, software developers assumed their programs would run with those rights and could therefore access and modify any file, registry key, or operating system setting. Second, users sometimes need administrative rights to perform such operations as installing software, changing the system time, and opening ports in the firewall.

The UAC solution to these problems is to run most applications with standard user rights, even though the user is logged in to an account with administrative rights. At the same time, UAC makes it possible for standard users to access administrative rights when they need them—whether for legacy applications that require them or for changing certain system settings. As described, UAC accomplishes this by creating a filtered admin token as well as the normal admin token when a user logs in to an administrative account. All processes created under the user's session will normally have the filtered admin token in effect so that applications that can run with standard user rights will do so. However, the administrative user can run a program or perform other functions that require full Administrator rights through UAC elevation.

Windows also allows certain tasks that were previously reserved for administrators to be performed by standard users, enhancing the usability of the standard user environment. For example, Group Policy settings exist that can enable standard users to install printers and other device drivers approved by IT administrators and to install ActiveX controls from administrator-approved sites.

Finally, when software developers test in the UAC environment, they are encouraged to develop applications that can run without administrative rights. Fundamentally, non-administrative programs should not need to run with administrator privileges; programs that often require administrator privileges are typically legacy programs using old APIs or techniques, and they should be updated.

Together, these changes obviate the need for users to run with administrative rights all the time.

File system and registry virtualization

Although some software legitimately requires administrative rights, many programs needlessly store user data in system-global locations. When an application executes, it can be running in different user accounts, and it should therefore store user-specific data in the per-user %AppData% directory and save per-user settings in the user's registry profile under HKEY_CURRENT_USER\Software. Standard user accounts don't have write access to the %ProgramFiles% directory or HKEY_LOCAL_MACHINE\Software, but because most Windows systems are single-user and most users have been administrators until UAC was implemented, applications that incorrectly saved user data and settings to these locations worked anyway.

Windows enables these legacy applications to run in standard user accounts through the help of file system and registry namespace virtualization. When an application modifies a system-global location in the file system or registry and that operation fails because access is denied, Windows redirects the operation to a per-user area. When the application reads from a system-global location, Windows first

checks for data in the per-user area and, if none is found, permits the read attempt from the global location.

Windows will always enable this type of virtualization unless:

- **The application is 64-bit** Because virtualization is purely an application-compatibility technology meant to help legacy applications, it is enabled only for 32-bit applications. The world of 64-bit applications is relatively new and developers should follow the development guidelines for creating standard user-compatible applications.
- **The application is already running with administrative rights** In this case, there is no need for any virtualization.
- **The operation came from a kernel-mode caller**
- **The operation is being performed while the caller is impersonating** For example, any operations not originating from a process classified as legacy according to this definition, including network file-sharing accesses, are not virtualized.
- **The executable image for the process has a UAC-compatible manifest** Specifying a requestedExecutionLevel setting, described in the next section.
- **The administrator does not have write access to the file or registry key** This exception exists to enforce backward compatibility because the legacy application would have failed before UAC was implemented even if the application was run with administrative rights.
- **Services are never virtualized**

You can see the virtualization status (the process virtualization status is stored as a flag in its token) of a process by adding the UAC Virtualization column to Task Manager's Details page, as shown in [Figure 7-22](#). Most Windows components—including the Desktop Window Manager (Dwm.exe), the Client Server Run-Time Subsystem (Csrss.exe), and Explorer—have virtualization disabled because they have a UAC-compatible manifest or are running with administrative rights and so do not allow virtualization. However, 32-bit Internet Explorer (iexplore.exe) has virtualization enabled because it can host multiple ActiveX controls and scripts and must assume that they were not written to operate correctly with standard user rights. Note that, if required, virtualization can be completely disabled for a system using a Local Security Policy setting.

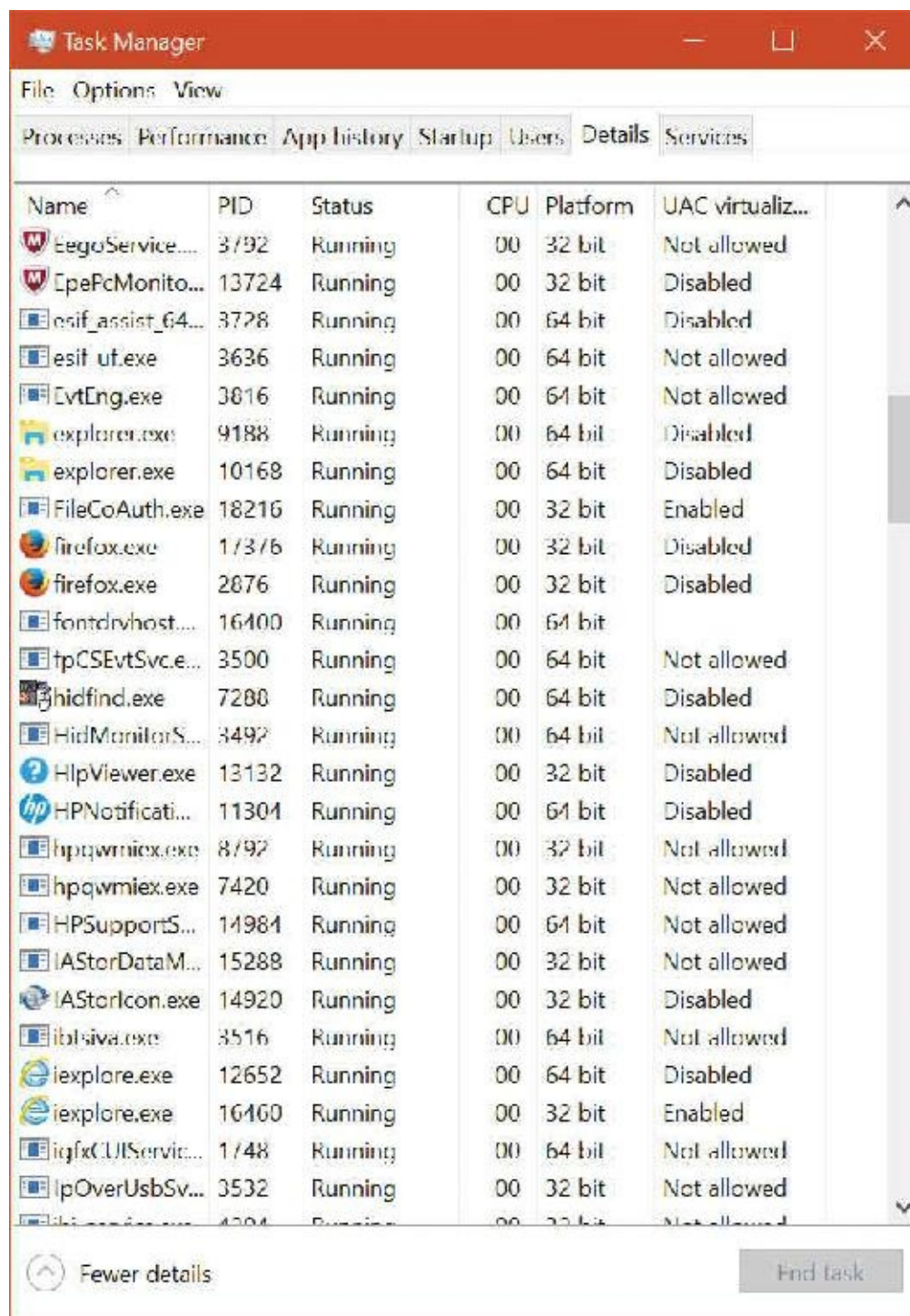


FIGURE 7-22 Using Task Manager to view virtualization status.

In addition to file system and registry virtualization, some applications require additional help to run correctly with standard user rights. For example, an application that tests the account in which it's running for membership in the Administrators group might otherwise work, but it won't run if it's not in that group. Windows defines a number of application-compatibility shims to enable such applications to work anyway. The shims most commonly applied to legacy applications for operation with standard user rights are shown in [Table 7-15](#).

Flag	Meaning
ElevateCreateProcess	This changes CreateProcess to handle ERROR_ELEVATION_REQUIRED errors by calling the application information service to prompt for elevation.
ForceAdminAccess	This spoofs queries of Administrator group membership.
VirtualizeDeleteFile	This spoofs successful deletion of global files and directories.
LocalMappedObjectL	This forces global section objects into the user's namespace.
VirtualizeIKernelFile	This redirects global registration of COM objects to a per-user location.
VirtualizeRegisterTypeLib	This converts per-machine type1ib registrations to per-user registrations.

TABLE 7-15 UAC virtualization shims

File virtualization

The file system locations that are virtualized for legacy processes are %ProgramFiles%, %ProgramData%, and %SystemRoot%, excluding some specific subdirectories. However, any file with an executable extension—including .exe, .bat, .scr, .vbs, and others—is excluded from virtualization. This means that programs that update themselves from a standard user account fail instead of creating private versions of their executables that aren't visible to an administrator running a global updater.



Note

To add extensions to the exception list, enter them in the HKLM\System\CurrentControlSet\Services\Luafv\Parameters\ExcludedExtensions registry key and reboot. Use a multistring type to delimit multiple extensions, and do not include a leading dot in the extension name.

Modifications to virtualized directories by legacy processes are redirected to the user's virtual root directory, %LocalAppData%\VirtualStore. The Local component of the path highlights the fact that virtualized files don't roam with the rest of the profile when the account has a roaming profile.

The UAC File Virtualization filter driver (%SystemRoot%\System32\Drivers\Luafv.sys) implements file system virtualization. Because this is a file system filter driver, it sees all local file system operations, but it implements functionality only for operations from legacy processes. As shown in [Figure 7-23](#), the filter driver changes the target file path for a legacy process that creates a file in a system-global location but does not for a non-virtualized process with standard user rights. Default permissions on the \Windows directory deny access to the application written with UAC support, but the legacy process acts as though the operation succeeds when it really created the file in a location fully accessible by the user.

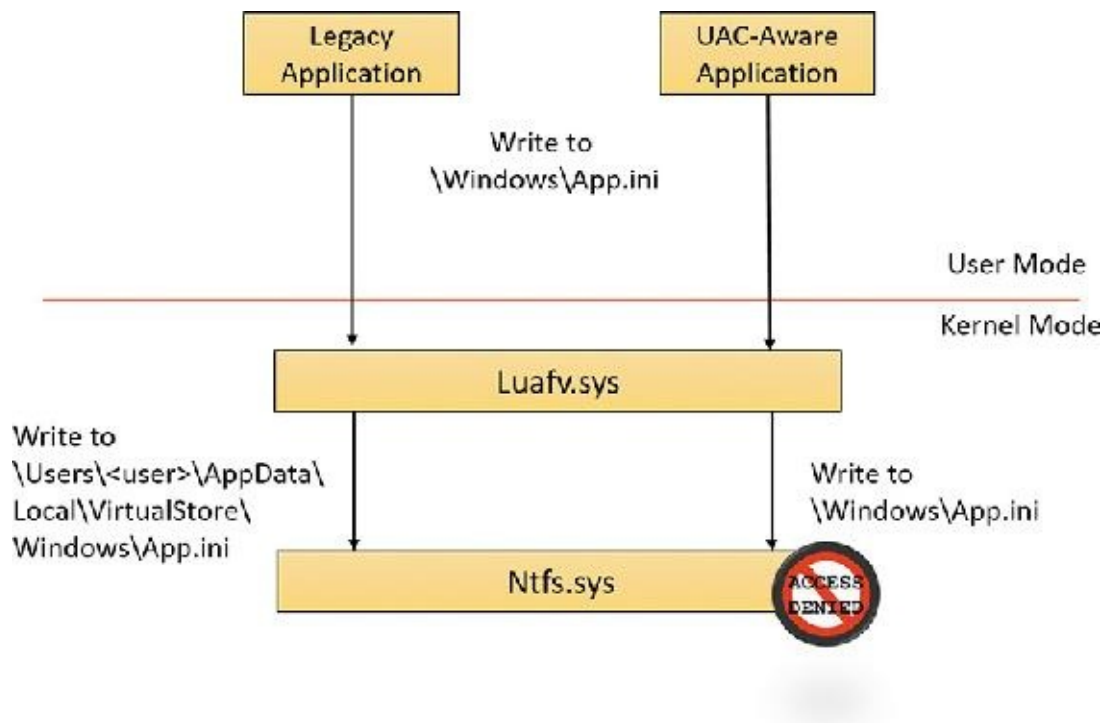


FIGURE 7-23 UAC File Virtualization filter driver operation.

EXPERIMENT: File virtualization behavior

In this experiment, you will enable and disable virtualization on the command prompt and see several behaviors to demonstrate UAC file virtualization:

1. Open a non-elevated command prompt (you must have UAC enabled for this to work) and enable virtualization for it. You can change the virtualization status of a process by right-clicking the process in the Task Manager Details tab and selecting **UAC Virtualization** from the shortcut menu that appears.

2. Navigate to the C:\Windows directory and use the following command to write a file:

```
echo hello-1 > test.txt
```

3. List the contents of the directory. You'll see that the file appears.

```
dir test.txt
```

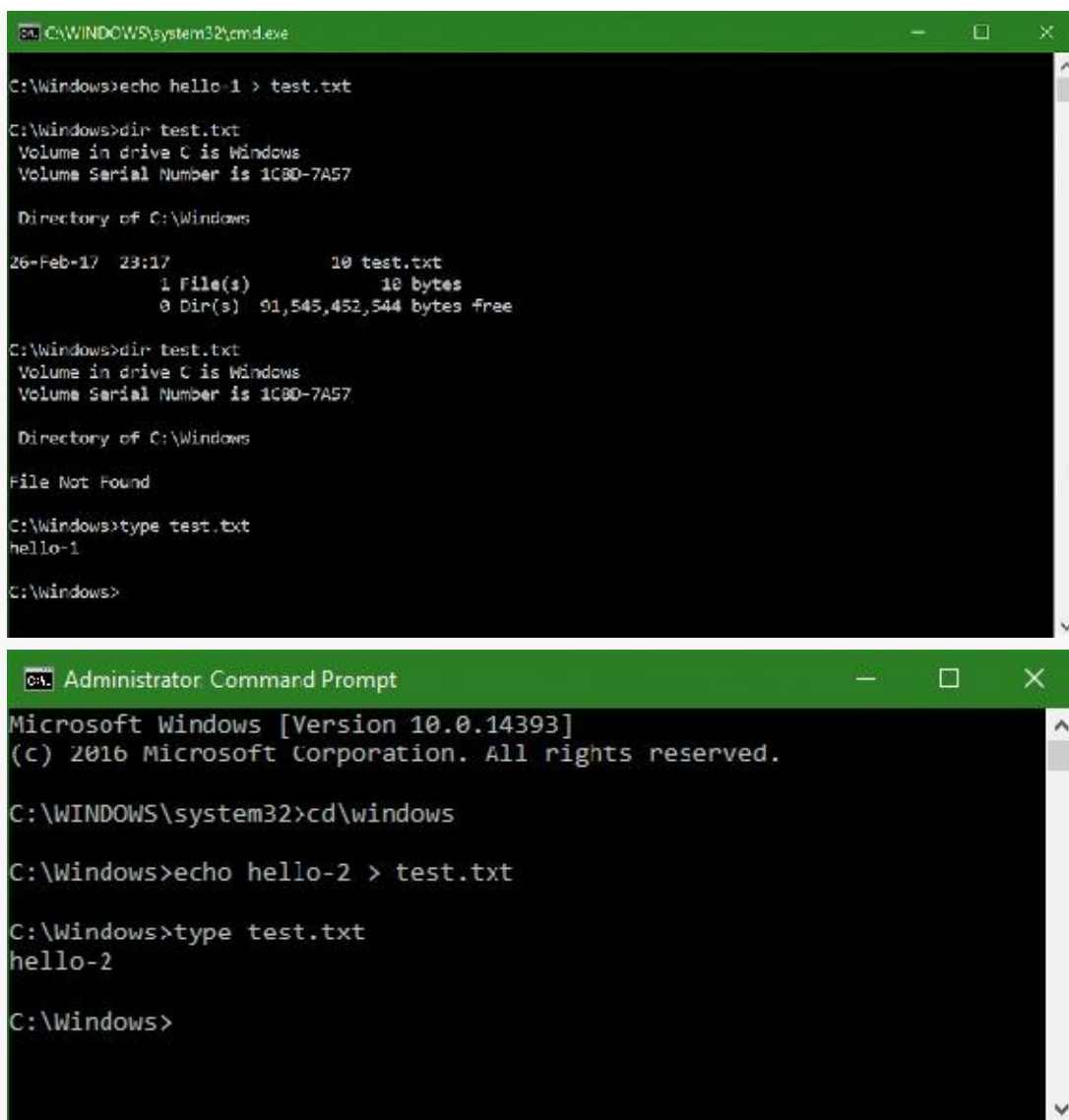
4. Disable virtualization by right-clicking the process in the Task Manager Details tab and deselecting **UAC Virtualization**. Then list the directory as in step 3. Notice that the file is gone. However, a directory listing of the VirtualStore directory will reveal the file:

[Click here to view code image](#)

```
dir %LOCALAPPDATA%\VirtualStore\Windows\test.txt
```

5. Enable virtualization again for this process.
6. To look at a more complex scenario, create a new command prompt window, but elevated this time. Then repeat steps 2 and 3 using the string hello-2.
7. Examine the text inside these files by using the following command in both command prompts. The screenshots that follow show the expected output.

```
type test.txt
```



```
C:\WINDOWS\system32\cmd.exe

C:\Windows>echo hello-1 > test.txt

C:\Windows>dir test.txt
Volume in drive C is Windows
Volume Serial Number is 1C8D-7A57

Directory of C:\Windows

26-Feb-17  23:17                10 test.txt
               1 File(s)              10 bytes
               0 Dir(s)  91,545,452,544 bytes free

C:\Windows>dir test.txt
Volume in drive C is Windows
Volume Serial Number is 1C8D-7A57

Directory of C:\Windows

File Not Found

C:\Windows>type test.txt
hello-1

C:\Windows>

C:\WINDOWS\system32\cmd.exe
Administrator: Command Prompt

Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>cd\windows

C:\Windows>echo hello-2 > test.txt

C:\Windows>type test.txt
hello-2

C:\Windows>
```

8. From your elevated command prompt, delete the test.txt file:
`del test.txt`
9. Repeat step 3 of the experiment in both windows. Notice that the elevated command prompt cannot find the file anymore, while the standard user command prompt shows the old contents of the file again. This demonstrates the failover mechanism described earlier: Read operations look in the per-user virtual store location first, but if the file doesn't exist, read access to the system location will be granted.

Registry virtualization

Registry virtualization is implemented slightly differently from file system virtualization. Virtualized registry keys include most of the HKEY_LOCAL_MACHINE\Software branch, but there are numerous exceptions, such as the following:

- HKLM\Software\Microsoft\Windows
- HKLM\Software\Microsoft\Windows NT
- HKLM\Software\Classes

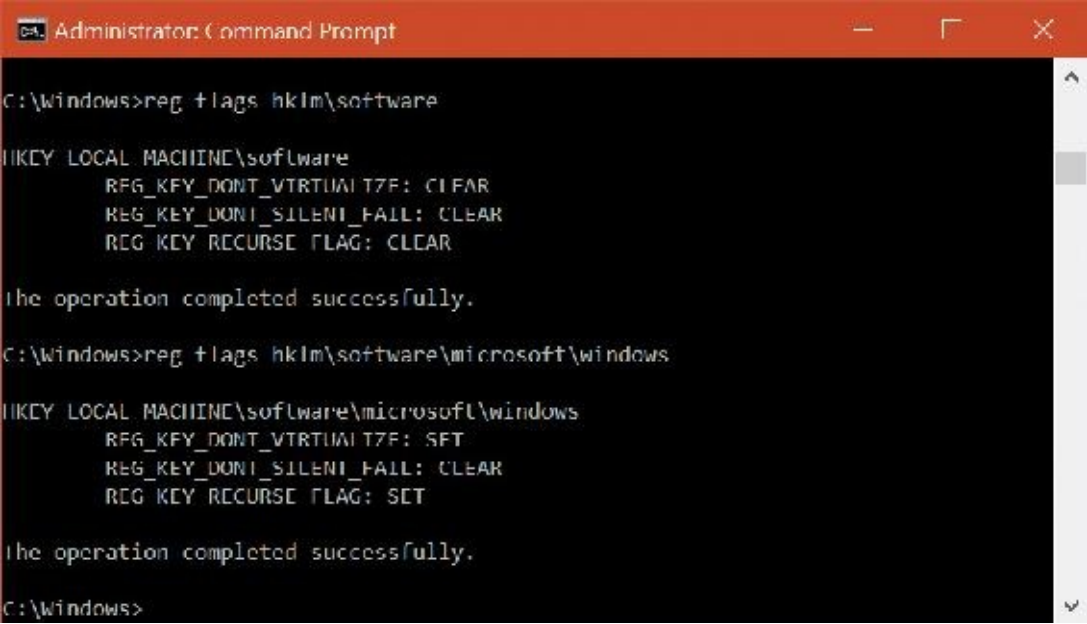
Only keys that are commonly modified by legacy applications, but that don't introduce compatibility or interoperability problems, are virtualized. Windows redirects modifications of virtualized keys by a legacy application to a user's registry virtual root at

HKKEY_CURRENT_USER\Software\Classes\VirtualStore. The key is located in the user’s Classes hive, %LocalAppData%\Microsoft\Windows\UsrClass.dat, which, like any other virtualized file data, does not roam with a roaming user profile. Instead of maintaining a fixed list of virtualized locations as Windows does for the file system, the virtualization status of a key is stored as a combination of flags, shown in [Table 7-16](#).

Flag	Meaning
REG_KEY_DONT_VIRTUALIZE	This specifies whether virtualization is enabled for this key. If the flag is set, virtualization is disabled.
REG_KEY_DONT_SILENT_FAIL	If the REG_KEY_DONT_VIRTUALIZE flag is set (virtualization is disabled), this key specifies that a legacy application that would be denied access performing an operation on the key is instead granted MAXIMUM_ALLOWED rights to the key (any access the account is granted) instead of the rights the application requested. If this flag is set, it implicitly disables virtualization as well.
REG_KEY_RECURSE_FLAG	This determines whether the virtualization flags will propagate to the child keys (subkeys) of this key.

TABLE 7-16 Registry virtualization flags

You can use the Reg.exe utility included in Windows, with the flags option, to display the current virtualization state for a key or to set it. In [Figure 7-24](#), note that the HKLM\Software key is fully virtualized, but the Windows subkey (and all its children) have only silent failure enabled.



```
Administrator: Command Prompt

C:\Windows>reg +flags hklm\software

HKEY_LOCAL_MACHINE\software
    REG_KEY_DONT_VIRTUALIZE: CLEAR
    REG_KEY_DONT_SILENT_FAIL: CLEAR
    REG_KEY_RECURSE_FLAG: CLEAR

The operation completed successfully.

C:\Windows>reg +flags hklm\software\microsoft\windows

HKEY_LOCAL_MACHINE\software\microsoft\windows
    REG_KEY_DONT_VIRTUALIZE: SET
    REG_KEY_DONT_SILENT_FAIL: CLEAR
    REG_KEY_RECURSE_FLAG: SET

The operation completed successfully.

C:\Windows>
```

FIGURE 7-24 UAC registry virtualization flags on the Software and Windows keys.

Unlike file virtualization, which uses a filter driver, registry virtualization is implemented in the configuration manager. (See Chapter 9 in Part 2 for more information on the registry and the configuration manager.) As with file system virtualization, a legacy process creating a subkey of a virtualized key is redirected to the user’s registry virtual root, but a UAC-compatible process is denied access by default permissions. This is shown in [Figure 7-25](#).

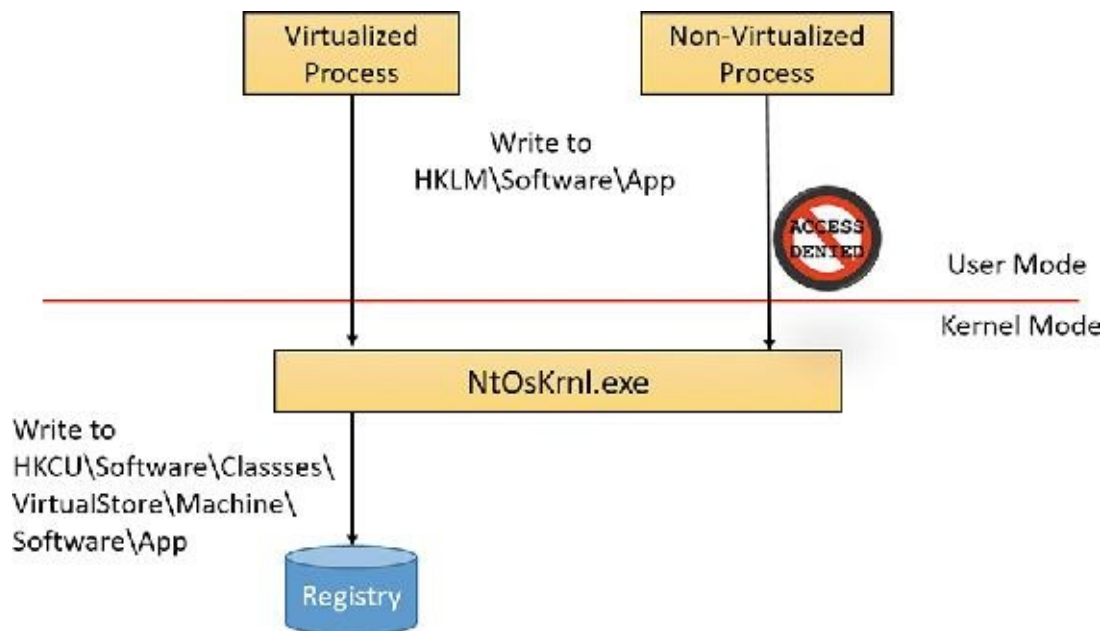


FIGURE 7-25 UAC registry virtualization operation.

Elevation

Even if users run only programs that are compatible with standard user rights, some operations still require administrative rights. For example, the vast majority of software installations require administrative rights to create directories and registry keys in system-global locations or to install services or device drivers. Modifying system-global Windows and application settings also requires administrative rights, as does the parental controls feature. It would be possible to perform most of these operations by switching to a dedicated administrator account, but the inconvenience of doing so would likely result in most users remaining in the administrator account to perform their daily tasks, most of which do not require administrative rights.

It's important to be aware that UAC elevations are conveniences and not security boundaries. A security boundary requires that security policy dictate what can pass through the boundary. User accounts are an example of a security boundary in Windows because one user can't access the data belonging to another user without having that user's permission.

Because elevations aren't security boundaries, there's no guarantee that malware running on a system with standard user rights can't compromise an elevated process to gain administrative rights. For example, elevation dialog boxes only identify the executable that will be elevated; they say nothing about what it will do when it executes.

Running with administrative rights

Windows includes enhanced "run as" functionality so that standard users can conveniently launch processes with administrative rights. This functionality requires giving applications a way to identify operations for which the system can obtain administrative rights on behalf of the application, as necessary (we'll say more on this topic shortly).

To enable users acting as system administrators to run with standard user rights but not have to enter user names and passwords every time they want to access administrative rights, Windows makes use of a mechanism called *Admin Approval Mode* (AAM). This feature creates two identities for the user at logon: one with standard user rights and another with administrative rights. Since every user on a Windows system is either a standard user or acting for the most part as a standard user in AAM, developers must assume that all Windows users are standard users, which will result in more programs working with

standard user rights without virtualization or shims.

Granting administrative rights to a process is called *elevation*. When elevation is performed by a standard user account (or by a user who is part of an administrative group but not the actual Administrators group), it's referred to as an *over-the-shoulder (OTS) elevation* because it requires the entry of credentials for an account that's a member of the Administrators group, something that's usually completed by a privileged user typing over the shoulder of a standard user. An elevation performed by an AAM user is called a *consent elevation* because the user simply has to approve the assignment of his administrative rights.

Stand-alone systems, which are typically home computers, and domain-joined systems treat AAM access by remote users differently because domain-connected computers can use domain administrative groups in their resource permissions. When a user accesses a stand-alone computer's file share, Windows requests the remote user's standard user identity. But on domain-joined systems, Windows honors all the user's domain group memberships by requesting the user's administrative identity. Executing an image that requests administrative rights causes the application information service (AIS, contained in %SystemRoot%\System32\Appinfo.dll), which runs inside a standard service host process (SvcHost.exe), to launch %SystemRoot%\System32\Consent.exe. Consent captures a bitmap of the screen, applies a fade effect to it, switches to a desktop that's accessible only to the local system account (the Secure Desktop), paints the bitmap as the background, and displays an elevation dialog box that contains information about the executable. Displaying this dialog box on a separate desktop prevents any application present in the user's account from modifying the appearance of the dialog box.

If an image is a Windows component digitally signed (by Microsoft or another entity), the dialog box displays a light blue stripe across the top, as shown at the left of [Figure 7-26](#) (the distinction between Microsoft signed images and other signers has been removed in Windows 10). If the image is unsigned, the stripe becomes yellow, and the prompt stresses the unknown origin of the image (see the right of [Figure 7-26](#)). The elevation dialog box shows the image's icon, description, and publisher for digitally signed images, but it shows only the file name and "Publisher: Unknown" for unsigned images. This difference makes it harder for malware to mimic the appearance of legitimate software. The Show More Details link at the bottom of the dialog box expands it to show the command line that will be passed to the executable if it launches.

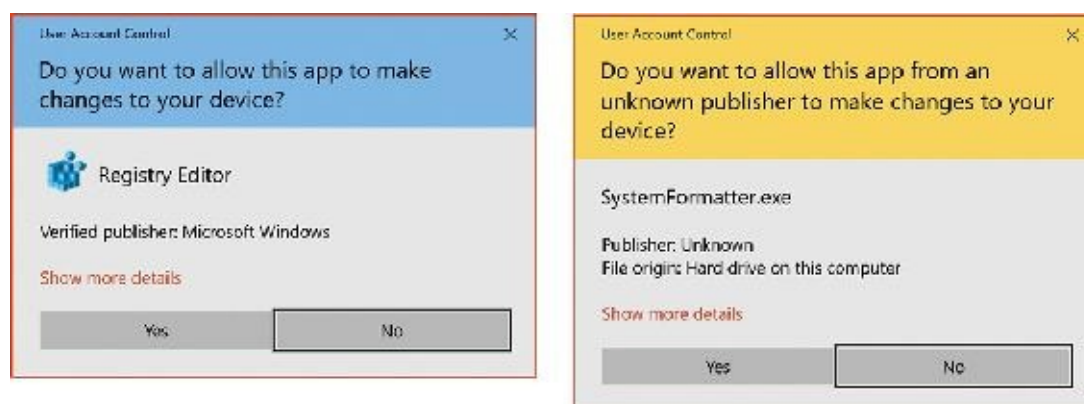


FIGURE 7-26 AAC UAC elevation dialog boxes based on image signature.

The OTS consent dialog box, shown in [Figure 7-27](#), is similar, but prompts for administrator credentials. It will list any accounts with administrator rights.

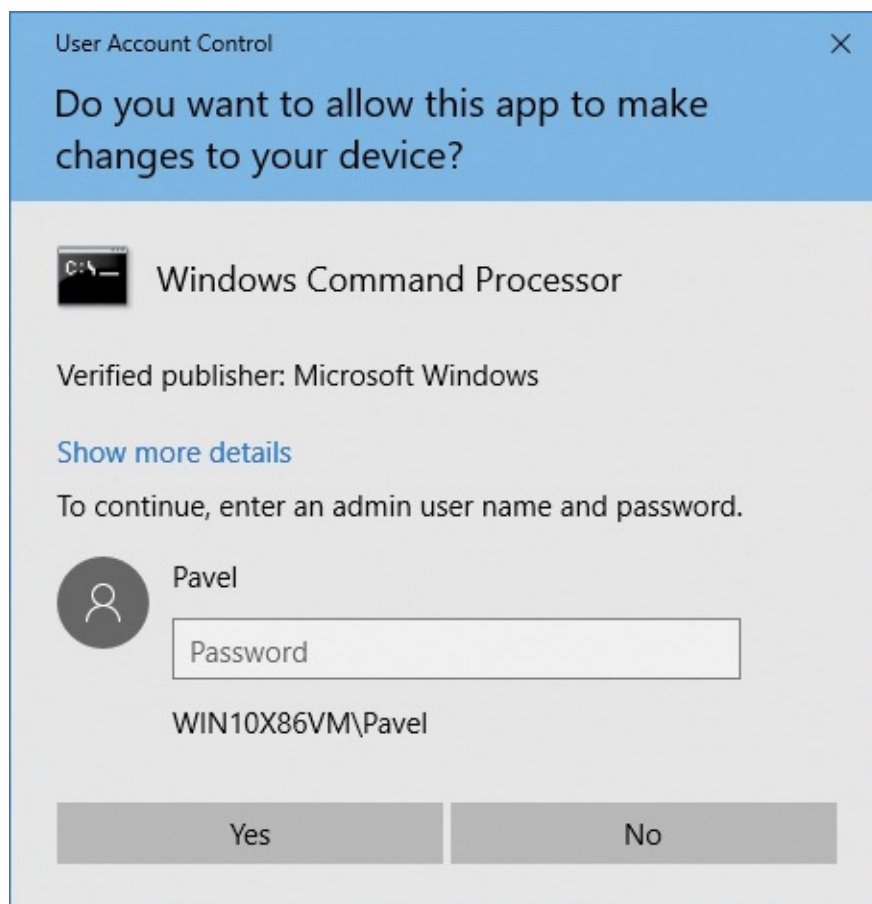


FIGURE 7-27 OTS consent dialog box.

If a user declines an elevation, Windows returns an access-denied error to the process that initiated the launch. When a user agrees to an elevation by either entering administrator credentials or clicking Yes, AIS calls `CreateProcessAsUser` to launch the process with the appropriate administrative identity. Although AIS is technically the parent of the elevated process, AIS uses new support in the `CreateProcessAsUser` API that sets the process's parent process ID to that of the process that originally launched it. That's why elevated processes don't appear as children of the AIS service-hosting process in tools such as Process Explorer that show process trees. [Figure 7-28](#) shows the operations involved in launching an elevated process from a standard user account.

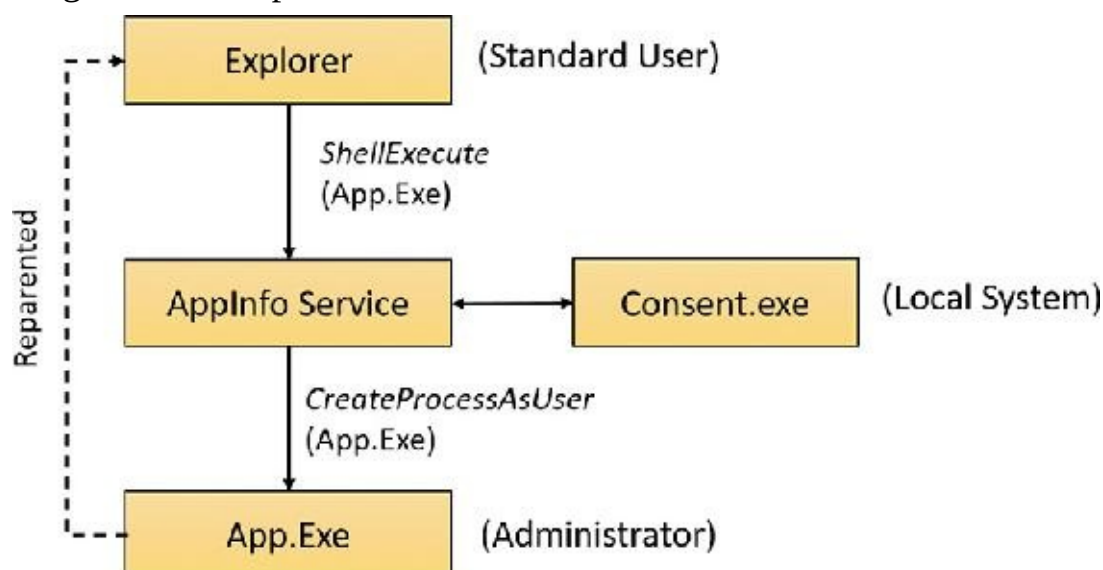


FIGURE 7-28 Launching an administrative application as a standard user.

Requesting administrative rights

There are a number of ways the system and applications identify a need for administrative rights. One that shows up in the Explorer user interface is the Run as Administrator context menu command and shortcut option. These items also include a blue and gold shield icon that should be placed next to any button or menu item that will result in an elevation of rights when it is selected. Choosing the Run as Administrator command causes Explorer to call the `ShellExecute` API with the `runas` verb.

The vast majority of installation programs require administrative rights, so the image loader, which initiates the launch of an executable, includes installer-detection code to identify likely legacy installers. Some of the heuristics it uses are as simple as detecting internal version information or whether the image has the words *setup*, *install*, or *update* in its file name. More sophisticated means of detection involve scanning for byte sequences in the executable that are common to third-party installation wrapper utilities. The image loader also calls the application-compatibility library to see if the target executable requires administrator rights. The library looks in the application-compatibility database to see whether the executable has the `RequireAdministrator` or `RunAsInvoker` compatibility flag associated with it.

The most common way for an executable to request administrative rights is for it to include a `requestedExecutionLevel` tag in its application manifest file. The element's `level` attribute can have one of the three values shown in [Table 7-17](#).

Elevation Level	Meaning	Usage
As invoker	No need for administrative rights; never ask for elevation.	Typical user applications that don't need administrative privileges—for example, Notepad.
Highest available	Request approval for highest rights available. If the user is logged on as a standard user, the process will be launched as invoker; otherwise, an AAM elevation prompt will appear, and the process will run with full administrative rights.	Applications that can function without full administrative rights but expect users to want full access if it's easily accessible. For example, the Registry Editor, Microsoft Management Console, and the Event Viewer use this level.
Require administrator	Always request administrative rights. An UAC elevation dialog box prompt will be shown for standard users; otherwise, AAM.	Applications that require administrative rights to work, such as the Firewall Settings Editor, which affects system-wide security.

TABLE 7-17 Requested elevation levels

The presence of the `trustInfo` element in a manifest (which you can see in the manifest dump of `eventvwr.exe`) denotes an executable that was written with support for UAC and the `requestedExecutionLevel` element nests within it. The `uiAccess` attribute is where accessibility applications can use the UIPI bypass functionality mentioned earlier.

[Click here to view code image](#)

```
C:\>sigcheck -m c:\Windows\System32\eventvwr.exe
...
<trustInfo xmlns="urn:schemas-microsoft-com:asm.v3">
  <security>
    <requestedPrivileges>
      <requestedExecutionLevel
        level="highestAvailable"
        uiAccess="false"
      />
    </requestedPrivileges>
  </security>
```

```

</trustInfo>
<asmv3:application>
  <asmv3:windowsSettings
xmlns="http://schemas.microsoft.com/SMI/2005/WindowsSettings">
    <autoElevate>true</autoElevate>
  </asmv3:windowsSettings>
</asmv3:application>
...

```

Auto-elevation

In the default configuration (see the next section for information on changing this), most Windows executables and control panel applets do not result in elevation prompts for administrative users, even if they need administrative rights to run. This is because of a mechanism called *auto-elevation*. Auto-elevation is intended to preclude administrative users from seeing elevation prompts for most of their work; the programs will automatically run under the user's full administrative token.

Auto-elevation has several requirements. One is that the executable in question must be considered as a Windows executable. This means it must be signed by the Windows publisher (not just by Microsoft; oddly, they are not the same—Windows-signed is considered more privileged than Microsoft-signed). It must also be in one of several directories considered secure: %SystemRoot%\System32 and most of its subdirectories, %Systemroot%\Ehome, and a small number of directories under %ProgramFiles% (for example, those containing Windows Defender and Windows Journal).

There are additional requirements, depending on the type of executable. EXE files other than Mmc.exe auto-elevate if they are requested via an `autoElevate` element in their manifest. The manifest shown earlier of eventvwr.exe in the previous section illustrates this.

Mmc.exe is treated as a special case because whether it should auto-elevate or not depends on which system management snap-ins it is to load. Mmc.exe is normally invoked with a command line specifying an MSC file, which in turn specifies which snap-ins are to be loaded. When Mmc.exe is run from a protected administrator account (one running with the limited administrator token), it asks Windows for administrative rights. Windows validates that Mmc.exe is a Windows executable and then checks the MSC. The MSC must also pass the tests for a Windows executable, and furthermore must be on an internal list of auto-elevate MSCs. This list includes nearly all MSC files in Windows.

Finally, COM (out-of-process server) classes can request administrative rights within their registry key. To do so requires a subkey named `Elevation` with a `DWORD` value named `Enabled`, having a value of 1. Both the COM class and its instantiating executable must meet the Windows executable requirements, although the executable need not have requested auto-elevation.

Controlling UAC behavior

UAC can be modified via the dialog box shown in [Figure 7-29](#). This dialog box is available under Change User Account Control Settings. [Figure 7-29](#) shows the control in its default position.

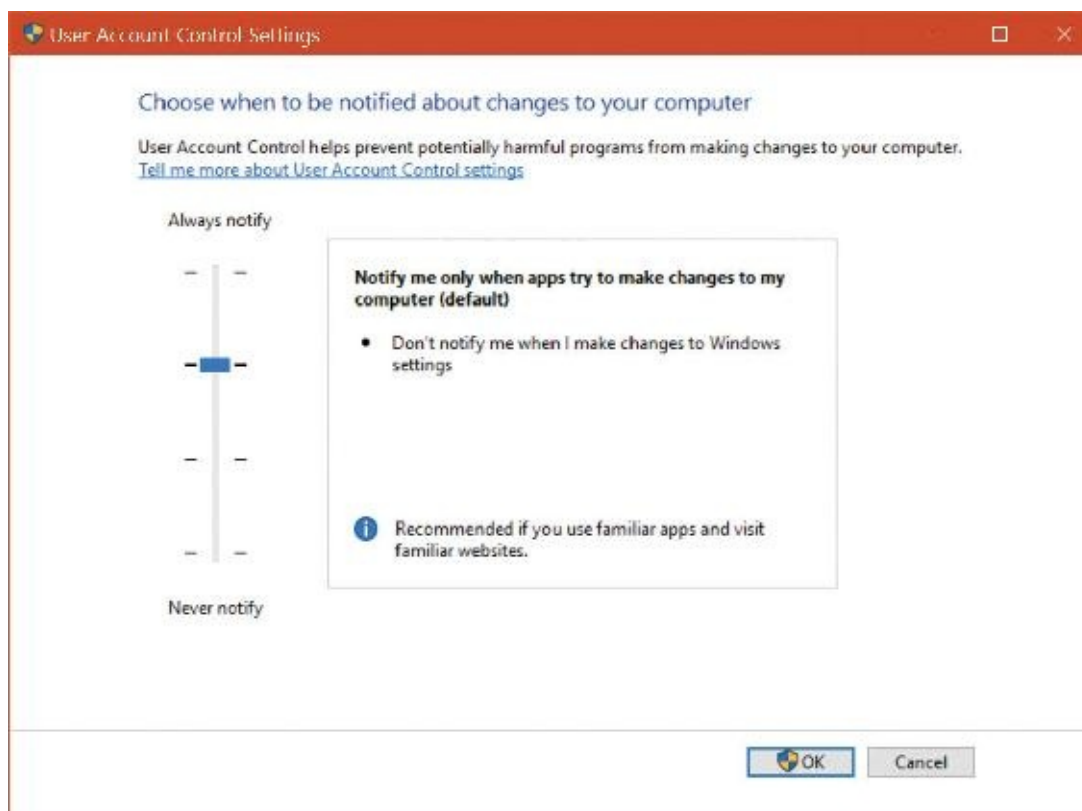


FIGURE 7-29 User Account Control Settings dialog box.

The four possible settings have the effects described in [Table 7-18](#).

Slider Position	When administrative user not running with administrative rights...		Remarks
	...attempts to change Windows settings (for example, use certain Control Panel applets)	...attempts to install software or run a program whose manifest calls for elevation, or uses Run as Administrator	
Highest position (Always Notify)	A UAC elevation prompt appears on the Secure Desktop.	A UAC elevation prompt appears on the Secure Desktop.	This was the Windows Vista behavior.
Second position	UAC elevation occurs automatically with no prompt or notification.	A UAC elevation prompt appears on the Secure Desktop.	Windows default setting.
Third position	UAC elevation occurs automatically with no prompt or notification.	A UAC elevation prompt appears on the user's normal desktop.	Not recommended.
Lowest position (Never Notify)	UAC is turned off for administrative users.	UAC is turned off for administrative users.	Not recommended.

TABLE 7-18 UAC options

The third position is not recommended because the UAC elevation prompt appears not on the Secure Desktop but on the normal user's desktop. This could allow a malicious program running in the same session to change the appearance of the prompt. It is intended for use only in systems where the video subsystem takes a long time to dim the desktop or is otherwise unsuitable for the usual UAC display.

The lowest position is strongly discouraged because it turns UAC off completely as far as administrative accounts are concerned. All processes run by a user with an administrative account will be run with the user's full administrative rights in effect; there is no filtered admin token. Registry and file system virtualization are disabled as well for these accounts, and the Protected mode of Internet Explorer is disabled. However, virtualization is still in effect for non-administrative accounts, and non-administrative accounts will still see an OTS elevation prompt when they attempt to change Windows settings, run a program that requires elevation, or use the Run as Administrator context menu option in Explorer.

The UAC setting is stored in four values in the registry under

HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System, as shown in [Table 7-19](#). ConsentPromptBehaviorAdmin controls the UAC elevation prompt for administrators running with a filtered admin token, and ConsentPromptBehaviorUser controls the UAC prompt for users other than administrators.

Slider Position	ConsentPromptBehaviorAdmin	ConsentPromptBehaviorUser	EnableLUA	PromptOnSecureDesktop
Highest position (Always Notify)	7 (display AAC UAC elevation prompt)	3 (display QTS UAC elevation prompt)	1 (enabled)	1 (enabled)
Second position	5 (display AAC UAC elevation prompt, except for changes to Windows settings)	3	1	1
Third position	5	3	1	0 (disabled; UAC prompt appears on user's normal desktop)
Lowest position (Never Notify)	0	3	0 (disabled; logins to administrative accounts do not create a restricted admin access token)	0

TABLE 7-19 UAC registry values

Exploit mitigations

Throughout this chapter, we’ve seen a number of technologies that help protect the user, guarantee the code-signing properties of executable code, and lock down access to resources through sandboxing. At the end of the day, however, all secure systems have failure points, all code has bugs, and attackers leverage increasingly complex attacks to exploit them. A security model in which all code is assumed to be bug-free, or in which a software developer assumes all bugs will eventually be found and fixed, is destined to fail. Additionally, many security features that provide code-execution “guarantees” do so at a cost of performance or compatibility, which may be unacceptable in such scenarios.

A much more successful approach is to identify the most common techniques used by attackers, as well as employ an internal “red team” (that is, an internal team attacking its own software) to discover new techniques before attackers do and to implement mitigations against such techniques. (These mitigations can be as simple as moving some data around or as complex as employing Control Flow Integrity [CFI] techniques.) Because vulnerabilities can number in the thousands in a complex code base such as Windows, but exploit techniques are limited, the idea is to make large classes of bugs very difficult (or in some cases, impossible) to exploit, without worrying about finding all the bugs.

Process-mitigation policies

While individual applications can implement various exploit mitigations on their own (such as Microsoft Edge, which leverages a mitigation called *MemGC* to avoid many classes of memory-corruption attacks), this section will cover mitigations that are provided by the operating system to all applications or to the system itself to reduce exploitable bug classes. [Table 7-20](#) describes all mitigations in the latest version of Windows 10 Creators Update, the type of bug class they mitigate against, and mechanisms to activate them.

Mitigation Name	Use Case	Enabling Mechanism
ASLR Bottom Up Randomization	This makes calls to VirtualAlloc subject to ASLR with 8-bit entropy, including stack base randomization.	This is set with the PROCESS_CREATION_MITIGATION_POLICY_BOTTOM_UP_ASLR_ALWAYS_ON process-creation attribute flag.
ASLR Force Relocate Images	This forces ASLR even on binaries that do not have the /DYNAMICBASE linker flag.	This is set with SetProcessMitigationPolicy or the PROCESS_CREATION_MITIGATION_POLICY_FORCE_RELOCATE_IMAGES_ALWAYS_ON process-creation flag.
High Entropy ASLR (HEASLR)	This significantly increases entropy of ASLR on 64-bit images, increasing bottom-up randomization to up to 1 TB of variance (that is, bottom-up allocations may start anywhere between 64 KB and 1 TB into the address space, giving 24 bits of entropy).	Must be set through /HIGHEENTROPYVA at link time or the PROCESS_CREATION_MITIGATION_POLICY_HIGH_ENTROPY_ASLR_ALWAYS_ON process-creation attribute flag.
ASLR Disallow Stripped Images	This blocks the load of any library without relocations (linked with the /FIXED flag) when combined with ASLR Force Relocate Images.	This is set with SetProcessMitigationPolicy or with the PROCESS_CREATION_MITIGATION_POLICY_FORCE_RELOCATE_IMAGES_ALWAYS_ON_REQUIRED process-creation flag.
DEP: Permanent	This prevents the process from disabling DEP on itself. Only relevant on x86. Only relevant on 32-bit applications (and/or under WoW64).	This is set with the SetProcessMitigationPolicy, process-creation attribute or with SetProcessDEPPolicy.
DEP: Disable ATL Thunk Emulation	This prevents legacy ATL library code from executing ATL thunks in the heap, even if a known compatibility issue. Only relevant on 32-bit applications (and/or under WoW64).	This is set with the SetProcessMitigationPolicy, process-creation attribute or with SetProcessDEPPolicy.
SEH Overwrite Protection (SEHOP)	This prevents structure exception handlers from being overwritten with incorrect ones, even if the image was not linked with Safe SEH (/SAFESEH). Only relevant on 32-bit applications (and/or under WoW64).	This can be set with SetProcessDEPPolicy or with the PROCESS_CREATION_MITIGATION_POLICY_SEHOP_DISABLE process-creation flag.
Raise Exception on Invalid Handle	This helps catch handle reuse (use-after-handle-close) attacks in which a process uses a handle that is no longer the handle it expected (for example: SetEvent on a mutex) by crashing the process instead of returning a failure that the process might ignore.	This is set with SetProcessMitigationPolicy or the PROCESS_CREATION_MITIGATION_POLICY_STRICT_HANDLE_CHECKS_ALWAYS_ON process-creation attribute flag.
Raise Exception on Invalid Handle Close	This helps catch handle reuse (double-handle-close) attacks in which a process is attempting to close a handle that has already been closed, suggesting that a different handle may potentially be used in other scenarios, in which an exploit would be successful, ultimately limiting its universal effectiveness.	Undocumented, and can only be set through an undocumented API.

Disallow Win32k System Calls	This disables all access to the Win32k kernel-mode subsystem driver, which implements the Window Manager (GUI) and Graphics Device Interface (GDI) and DirectX. No system calls to this component will be possible.	This is set with SetProcessMitigationPolicy or the PROCESS_CREATION_MITIGATION_POLICY_WIN32K_SYSTEM_CALL_DISABLE_ALWAYS_ON process-creation attribute flag.
Filter Win32k System Calls	This filters access to the Win32k kernel-mode subsystem driver only to certain APIs allowing simple GUI and DirectX access, mitigating many of the possible attacks, without completely disabling availability of the GUI/GDI services.	This is set through an internal process-creation attribute flag, which can define one out of three possible sets of Win32k filters that are enabled. However, because the filter sets are hard-coded, this mitigation is reserved for Microsoft internal usage.
Disable Extension Points	This prevents a process from loading an input method editor (IME), a Windows hook DLL (SetWindowsHookEx), an app initialization DLL (AppInitDlls value in the registry), or a Winsock layered service provider (LSP).	This is set with SetProcessMitigationPolicy or the PROCESS_CREATION_MITIGATION_POLICY_EXTENSION_POINT_DISABLE_ALWAYS_ON process-creation attribute flag.
Arbitrary Code Guard (CFG)	This prevents a process from allocating executable code or from changing the permission of existing executable code to make it writeable. It can be configured to allow a particular thread inside the process to request this capability or to allow a remote process from disabling this mitigation, which are not supported from a security point of view.	This is set with SetProcessMitigationPolicy or the PROCESS_CREATION_MITIGATION_POLICY_PROHIBIT_DYNAMIC_CODE_ALWAYS_ON and PROCESS_CREATION_MITIGATION_POLICY_PROHIBIT_DYNAMIC_CODE_ALWAYS_ON_ALLOW_OPT_OUT process-creation attribute flags.
Control Flow Guard (CFG)	This helps prevent memory corruption vulnerabilities from being used to hijack control flow by validating the target of any indirect CALL or JMP instruction against a list of valid expected target functions. Part of Control Flow Integrity (CFI) mechanisms described in the next section.	The image must be compiled with the /guard:cf option, and linked with the /guard:cf option. It can be set with the PROCESS_CREATION_MITIGATION_POLICY_CONTROL_FLOW_GUARD_ALWAYS_ON process-creation attribute flag in case the image does not support it, but CFG enforcement is still desirable for other images loading in the process.
CFG Export Suppression	This strengthens CFG by suppressing indirect calls to the exported API table of the image.	The image must be compiled with /guard:export suppress, and can also be configured through SetProcessMitigationPolicy or with the PROCESS_CREATION_MITIGATION_POLICY_CONTROL_FLOW_GUARD_EXPORT_SUPPRESSION process-creation attribute flag.
CFG Strict Mode	This prevents the loading of any image library within the current process that was not linked with the /guard:cf option.	This is set through SetProcessMitigationPolicy or with the PROCESS_CREATION_MITIGATION_POLICY2_STRICT_CONTROL_FLOW_GUARD_ALWAYS_ON process-creation attribute flag.
Disable Non System Fonts	This prevents the loading of any font files that have not been registered by Winlogon at user logon time, after being installed in the C:\windows\fonts directory.	This is set through SetProcessMitigationPolicy or the PROCESS_CREATION_MITIGATION_POLICY_FONT_DISABLE_ALWAYS_ON process-creation attribute flag.
Microsoft-Signed Binaries Only	This prevents the loading of any image library within the current process that has not been signed by a Microsoft CA-issued certificate.	This is set through the PROCESS_CREATION_MITIGATION_POLICY_BLOCK_NON_MICROSOFT_BINARIES_ALWAYS_ON process-attribute flag at startup time.
Store-Signed Binaries Only	This prevents the loading of any image library within the current process that has not been signed by the Microsoft Store CA.	This is set through the PROCESS_CREATION_MITIGATION_POLICY_BLOCK_NON_MICROSOFT_BINARIES_ALLOW_STORE process attribute flag at startup time.

No Remote Images	This prevents the loading of any image library within the current process that is present on a non-local (UNC or WebDAV) path.	This is set through SetProcessMitigationPolicy or the PROCESS_CREATION_MITIGATION_POLICY_IMAGE_LOAD_NO_REMOTE_ALWAYS_ON process-creation attribute flag.
No Low IL Images	This prevents the loading of any image library within the current process that has a mandatory label below medium (NX2000).	This is set through SetProcessMitigationPolicy or the PROCESS_CREATION_MITIGATION_POLICY_IMAGE_LOAD_NO_LOW_LABEL_ALWAYS_ON process-creation flag. It can also be set through a resource claim ACE called IMAGELOAD on the file of the process being loaded.
Prefer System32 Images	This modifies the loader's search path to always look for the given image library being loaded (through a relative name) in the %SystemRoot%\System32 directory, regardless of the current search path.	This is set through SetProcessMitigationPolicy or the PROCESS_CREATION_MITIGATION_POLICY_IMAGE_LOAD_PREFER_SYSTEM32_ALWAYS_ON process-creation attribute flag.
Return Flow Guard (RFG)	This helps prevent additional classes of memory-corruption vulnerabilities that affect control flow by validating, before the execution of a RET instruction, that the function was not called through a return-oriented programming (ROP) exploit by not having begun its execution correctly or by executing on an invalid stack. This is part of the Control Flow Integrity (CFI) mechanisms.	Currently still being implemented in a robust and performant way, this mitigation is not yet available, but is included here for completeness.
Restrict Set Thread Context	This restricts the modification of the current thread's context.	Currently disabled pending the availability of RFG, which makes the mitigation more robust, this mitigation may appear in a future version of Windows. It is included here for completeness.
Loader Continuity	This prohibits the process from dynamically loading any DLLs that do not have the same integrity level as the process, in cases where a signature policy mitigation above could not be enabled at startup time due to compatibility concerns. This specifically targets cases of DLL planting attacks.	This is set through SetProcessMitigationPolicy or the PROCESS_CREATION_MITIGATION_POLICY2_LOADER_INTEGRITY_CONTINUITY_ALWAYS_ON process-creation attribute flag.
Heap Terminate On Corruption	This disables the Fault Tolerant Heap (FTH) and the raising of a continuable exception in the case of heap corruption by terminating the process instead. This prevents the use of heap corruption as a way to force an attacker-controlled exception handler from executing, or in cases where the program ignores the heap exception, or cases where the exploit only sometimes causes heap corruption (limiting its universal effectiveness or reliability).	This is set through HeapSetInformation or by using the PROCESS_CREATION_MITIGATION_POLICY_HEAP_TERMINATE_ALWAYS_ON process-creation attribute flag.
Disable Child Process Creation	This prohibits the creation of child processes by marking the token with a special restriction, which should stop any other component from creating a process while impersonating the token of this process (for example, WMI process creation, or a kernel component creating the process).	This is set through the PROCESS_CREATION_CHTID_PROCESS_RESTRICTED process-creation attribute flag. It can be overridden to allow packaged (UWP) applications with the PROCESS_CREATION_DESKTOP_APPX_OVERRIDE flag.
All Application Packages Policy	This makes an application running under an AppContainer unable to access resources that have an ALL_APPLICATION_PACKAGES_SID present, as was explained earlier in the "AppContainers" section. The presence of an ALL_RESTRICTED_APPLICATION_PACKAGES_SID will be required instead. Sometimes referred to as Less Privileged App Container (LPAC).	This is set through the PROCESS_THREAD_ATTRIBUTE_ALL_APPLICATION_PACKAGES_POLICY process-creation attribute.

TABLE 7-20 Process mitigation options

Note that it is also possible to some of these mitigations on a per-application or per-system basis without the cooperation of the application developer. To do so, open the Local Group Policy Editor. Then

expand **Computer Configuration**, then **Administrative Templates**, then **System**, and finally **Mitigation Options** (see [Figure 7-30](#)). In the Process Mitigation Options dialog box, enter the appropriate bit-number value that corresponds to the mitigations being enabled, using **1** to enable a mitigation, **0** to disable it, or **?** to leave it to its default or process-requested value (again, see [Figure 7-30](#)). The bit numbers are taken from the `PROCESS_MITIGATION_POLICY` enumeration found in the `Winnt.h` header file. This will result in the appropriate registry value being written in the Image File Execution Options (IFEO) key for the entered image name. Unfortunately the current version of Windows 10 Creators Update and earlier will strip out many of the newer mitigations. You can avoid this by manually setting the `REG_DWORD MitigationOptions` registry value.

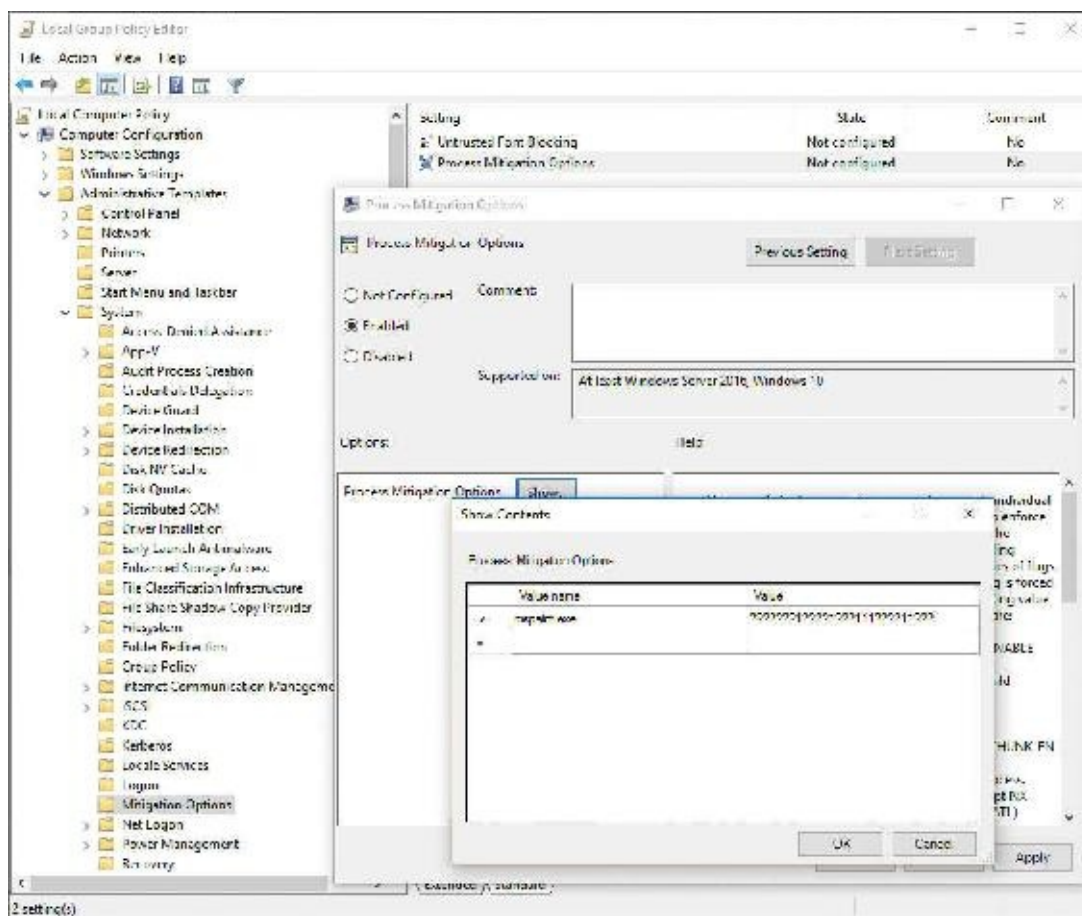


FIGURE 7-30 Customizing process-mitigation options.

Control Flow Integrity

Data Execution Prevention (DEP) and Arbitrary Code Guard (ACG) make it hard for exploits to place executable code on the heap or stack, to allocate new executable code, or to change existing executable code. As a result, memory/data-only attacks have become more interesting. Such attacks allow the modification of portions of memory to redirect control flow, such as modifying return addresses on the stack or indirect function pointers stored in memory. Techniques such as return-oriented-programming (ROP) and jump-oriented-programming (JOP) are often used to violate the regular code flow of the program and redirect it to known locations of interesting code snippets (“gadgets”).

Because such snippets are often present in the middle or end of various functions, when control flow is redirected in this way, it must be redirected into the middle or end of a legitimate function. By employing Control Flow Integrity (CFI) technologies—which can, for example, validate that the target of an indirect JMP or CALL instruction is the beginning of a real function, or that a RET instruction is pointing to an expected location, or that a RET instruction is issued after the function was entered through its beginning

—the operating system and compiler can detect and prevent most classes of such exploits.

Control Flow Guard

Control Flow Guard (CFG) is an exploit-mitigation mechanism first introduced in Windows 8.1 Update 3 that exists in enhanced version in Windows 10 and Server 2016, with further improvements released on various updates (up to and including the latest Creators Update). Originally implemented only for user-mode code, CFG now also exists as Kernel CFG (KCFG) on the Creators Update. CFG addresses the indirect CALL/JMP part of CFI by verifying that the target of an indirect call is at the start of a known function (more on that momentarily). If the target is not at the start of a known function, the process is simply terminated. [Figure 7-31](#) shows the conceptual operation of CFG.

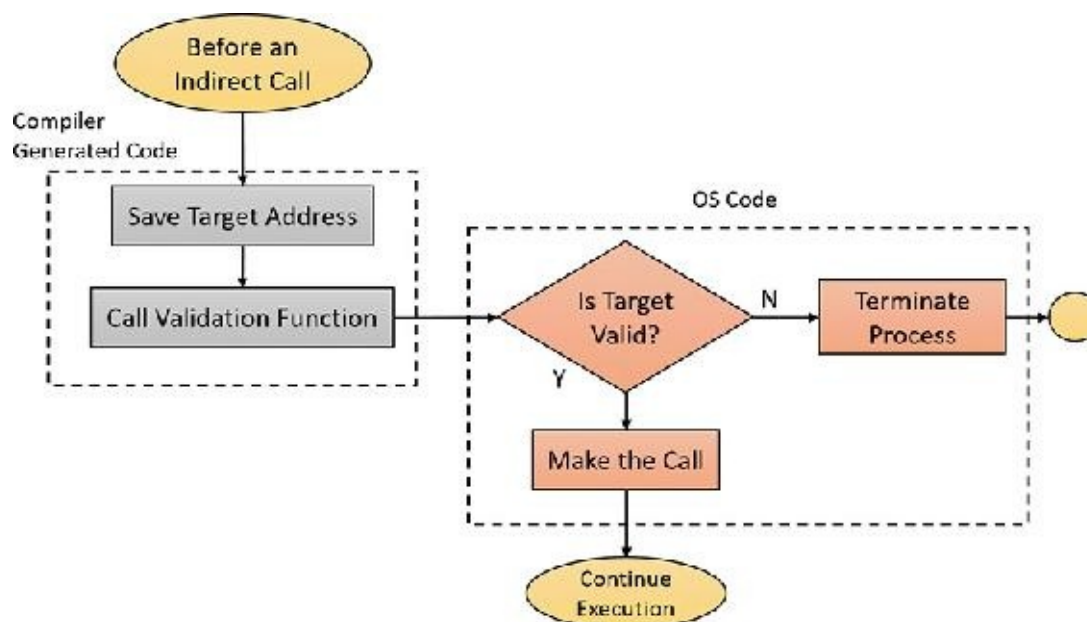


FIGURE 7-31 Conceptual view of Control Flow Guard.

CFG requires the cooperation of a supported compiler that will add the call to the validation code before indirect changes in control flow. The Visual C++ compiler has an option, `/guard:cf`, that must be set for images (or even on a C/C++ source file level) to be built with CFG support (this option is also available in Visual Studio's GUI in the C/C++/Code Generation/Control Flow Guard setting in the project's properties). This setting should also be set in the linker settings, as both components of Visual Studio are required to collaborate to support CFG.

Once those settings are present, images (EXEs and DLLs) that are compiled with CFG-enabled indicate this in their PE header. In addition, they contain a list of functions that are the valid indirect control flow targets in a `.gids` PE section (by default merged by the linker with the `.rdata` section). This list is built by the linker and contains the relative virtual address (RVA) of all functions in the image. This includes those that might not be called by an indirect call by the code present in the image because there's no way of knowing if outside code does not somehow legitimately know the address of a function and is attempting to call it. This can be especially true of exported functions, which can be called after obtaining their pointer through `GetProcAddress`.

That being said, programmers can use a technique called *CFG suppression*, which is supported through the `DECLSPEC_GUARD_SUPPRESS` annotation, and which marks the function in the table of valid functions with a special flag indicating that the programmer never expects such a function to be the target of any indirect call or jump.

Now that a table of valid function targets exists, all that a simple validation function would need to do is to compare the target of the `CALL` or `JMP` instruction with one of the functions in the table.

Algorithmically, this would result in an $O(n)$ algorithm, where the number of functions needed to check would be equivalent, in the worst case, to the number of functions in the table. Clearly, linearly scanning an entire array during every single indirect change in control flow would bring a program to its knees, so operating system support is needed to perform CFG checks efficiently. We'll see in the next section how Windows can achieve this.

EXPERIMENT: Control Flow Guard information

The DumpBin Visual Studio tool can show some basic CFG information. The following dumps header and loader configuration information for Smss:

[Click here to view code image](#)

```
c:\> dumpbin /headers /loadconfig c:\windows\system32\smss.exe
Microsoft (R) COFF/PE Dumper Version 14.00.24215.1
Copyright (C) Microsoft Corporation. All rights reserved.
Dump of file c:\windows\system32\smss.exe
PE signature found
File Type: EXECUTABLE IMAGE
FILE HEADER VALUES
      8664 machine (x64)
        6 number of sections
57899A7D time date stamp Sat Jul 16 05:22:53 2016
        0 file pointer to symbol table
        0 number of symbols
F0 size of optional header
22 characteristics
    Executable
    Application can handle large (>2GB) addresses

OPTIONAL HEADER VALUES
      20B magic # (PE32+)
14.00 linker version
12800 size of code
EC00 size of initialized data
      0 size of uninitialized data
1080 entry point (00000000140001080) NtProcessStartupW
1000 base of code
140000000 image base (00000000140000000 to 00000000140024FFF)
1000 section alignment
200 file alignment
10.00 operating system version
10.00 image version
10.00 subsystem version
      0 Win32 version
25000 size of image
400 size of headers
270FD checksum
      1 subsystem (Native)
```

4160 DLL characteristics
High Entropy Virtual Addresses
Dynamic base
NX compatible
Control Flow Guard

...
Section contains the following load config:

000000D0 size
0 time date stamp
0.00 Version
0 GlobalFlags Clear
0 GlobalFlags Set
0 Critical Section Default Timeout
0 Decommit Free Block Threshold
0 Decommit Total Free Threshold
000000000000000000 Lock Prefix Table
0 Maximum Allocation Size
0 Virtual Memory Threshold
0 Process Heap Flags
0 Process Affinity Mask
0 CSD Version
0800 Dependent Load Flag
000000000000000000 Edit List
0000000140020660 Security Cookie
00000001400151C0 Guard CF address of check-function pointer
00000001400151C8 Guard CF address of dispatch-function pointer
00000001400151D0 Guard CF function table
2A **Guard CF function count**
00010500 **Guard Flags**
CF Instrumented
FID table present
Long jump target table present
0000 Code Integrity Flags
0000 Code Integrity Catalog
00000000 Code Integrity Catalog Offset
00000000 Code Integrity Reserved
000000000000000000 **Guard CF address taken IAT entry table**
0 **Guard CF address taken IAT entry count**
000000000000000000 **Guard CF long jump target table**
0 **Guard CF long jump target count**
000000000000000000 Dynamic value relocation table

Guard CF Function Table

Address

■ ■ ■

The CFG-related information is marked in bold in the preceding output. We will discuss that shortly. For now, open Process Explorer, right-click the process column header, choose **Select Columns**. Then, in the Process Image tab, select the **Control Flow Guard** check box. Also select **Virtual Size** in the Process Memory tab. You should see something like this:

Process Explorer - Sysinternals: www.sysinternals.com [ZOD-AC\CS3500]

File Options View Process Find Users Help

Process	PID	CPU	Control Flow Guard	Virtual Size	Private Bytes	Working Set
Architect Manager.exe	4360			100,208 K	3,743 K	6,768 K
armory.exe	4164			50,212 K	1,280 K	1,176 K
audiodg.exe	9560		CFG	2,147,535,004 K	6,128 K	11,016 K
BcomMgmtAgent.exe	4444			57,836 K	4,864 K	4,864 K
Calculator.exe	15864	suspend...		330,980 K	28,464 K	580 K
CfgTest.exe	16284		CFG	5,442,435,355 K	780 K	224 K
conhost.exe	2512		CFG	2,147,545,712 K	1,156 K	956 K
conhost.exe	9284		CFG	2,147,574,075 K	1,904 K	1,764 K
conhost.exe	14420	< 0.01	CFG	2,147,574,275 K	1,924 K	2,252 K
conhost.exe	6656		CFG	2,147,574,275 K	1,944 K	1,804 K
conhost.exe	12652		CFG	2,147,575,500 K	2,012 K	2,720 K
conhost.exe	19064		CFG	2,147,562,792 K	1,412 K	5,516 K
creator-ws.exe	4388	< 0.01		72,084 K	2,535 K	4,320 K
csrss.exe	860		CFG	2,147,549,060 K	1,680 K	1,012 K
csrss.exe	960	0.21	CFG	2,147,535,701 K	2,815 K	3,948 K
CxMonProc.exe	3564			535,915 K	26,000 K	1,280 K
CxUtilSvc.exe	4112			84,412 K	1,668 K	2,240 K
dxsHost.exe	3468		CFG	2,147,531,188 K	1,992 K	2,940 K
devenv.exe	7000	0.28		1,140,960 K	420,575 K	178,516 K
devenv.exe	14900	1.15		1,575,600 K	739,588 K	427,936 K
devenv.exe	1900	0.93		1,203,272 K	456,780 K	314,560 K
devenv.exe	22018	0.96		1,330,848 K	555,288 K	408,700 K
devenv.exe	22760	0.90		1,417,760 K	577,475 K	607,016 K
dlhost.exe	3210		CFG	2,147,829,064 K	5,060 K	6,976 K
DPAgent.exe	9512			277,984 K	26,775 K	9,880 K
DPAgentProc.exe	10008			40,512 K	634 K	660 K

CPU Usage: 14.25% Commit Charge: 50.80% Processes: 207 Physical Usage: 37.24%

You should see most Microsoft-provided processes were built with CFG (including Smss, Csrss, Audiodg, Notepad, and many others). The virtual size for CFG-built processes is surprisingly high. Recall that the virtual size indicates the total address space used in the process, whether that memory is committed or reserved. In contrast, the Private Bytes column shows the private committed memory and is not even remotely close to the virtual size (although the virtual size includes non-private memory as well). For 64-bit processes, the virtual size is at least 2 TB, which we will shortly be able to rationalize.

The CFG bitmap

As you saw earlier, forcing the program to iterate through a list of function calls every few instructions would not be practical. Therefore, instead of an algorithm that requires linear time $O(n)$, performance requirements dictate that an $O(1)$ algorithm be used instead—one where a constant lookup time is used, regardless of how many functions are present in the table. This constant lookup time should be as small as possible. A clear winner of such a requirement would be an array that is indexable by the target function's address, which is an indication if this address is valid or not (such as a simple `BOOL`). With a 128 TB of possible addresses, though, such an array would itself have to be $128 \text{ TB} * \text{sizeof}(\text{BOOL})$, which is an unacceptable storage size—bigger than the address space itself. Can we do better?

First, we can leverage the fact that compilers ought to generate x64 function code on 16-byte boundaries. This reduces the size to the required array to only $8 \text{ TB} * \text{sizeof}(\text{BOOL})$. But using an entire `BOOL` (which is 4 bytes in the worst case or 1 byte in the best) is extremely wasteful. We only need one state, valid or invalid, which only needs to use 1 bit. This makes the calculation $8 \text{ TB} / 8$, or simply 1 TB. Unfortunately, however, there's a snag. There's no guarantee that the compiler will generate *all* functions on a 16-byte binary. Hand-crafted assembly code and certain optimizations might violate this rule. As such, we'll have to figure out a solution. One possible option is to simply use another bit to indicate if the function begins *somewhere* on the next 15 bytes instead of on the 16-byte boundary itself. Thus, we have the following possibilities:

- **{0, 0}** No valid function begins inside this 16-byte boundary.
- **{1, 0}** A valid function begins exactly on this aligned 16-byte address.
- **{1, 1}** A valid function begins somewhere inside of this 16-byte address.

Thanks to this setup, if the attacker attempts to call inside a function that was marked as 16-byte aligned by the linker, the 2-bit state will be `{1, 0}`, while the required bits (that is, bits 3 and 4) in the address will be `{1, 1}` as the address won't be 16-byte aligned. Therefore, an attacker will only be able to call an arbitrary instruction in the first 16 bytes of the function if the linker did not generate the function aligned in the first place (bits would then be `{1, 1}`, as shown above). Even then, this instruction must somehow be useful to the attacker without crashing the function (typically some sort of stack pivot or gadget that ends in a `ret` instruction).

With this understanding in mind, we can apply the following formulas to compute the size of the CFG bitmap:

- **32-bit application on x86 or x64** $2 \text{ GB} / 16 * 2 = 32 \text{ MB}$
- **32-bit application with /LARGEADDRESSAWARE, booted in 3 GB mode on x86** $3 \text{ GB} / 16 * 2 = 48 \text{ MB}$
- **64-bit application** $128 \text{ TB} / 16 * 2 = 2 \text{ TB}$
- **32-bit application with /LARGEADDRESSAWARE, on x64** $4 \text{ GB} / 16 * 2 = 64 \text{ MB}$, plus the size of the 64-bit bitmap, which is needed to protect 64-bit Ntdll.dll and WoW64 components, so $2 \text{ TB} + 64 \text{ MB}$

Allocating and filling out 2 TB of bits on every single process execution is still a tough performance overhead to swallow. Even though we have fixed the execution cost of the indirect call itself, process startup cannot be allowed to take so long, and 2 TB of committed memory would exhaust the commit limit instantly. Therefore, two memory-saving and performance-helping tricks are used.

First, the memory manager will only reserve the bitmap, basing itself on the assumption that the CFG validation function will treat an exception during CFG bitmap access as an indication that the bit state is

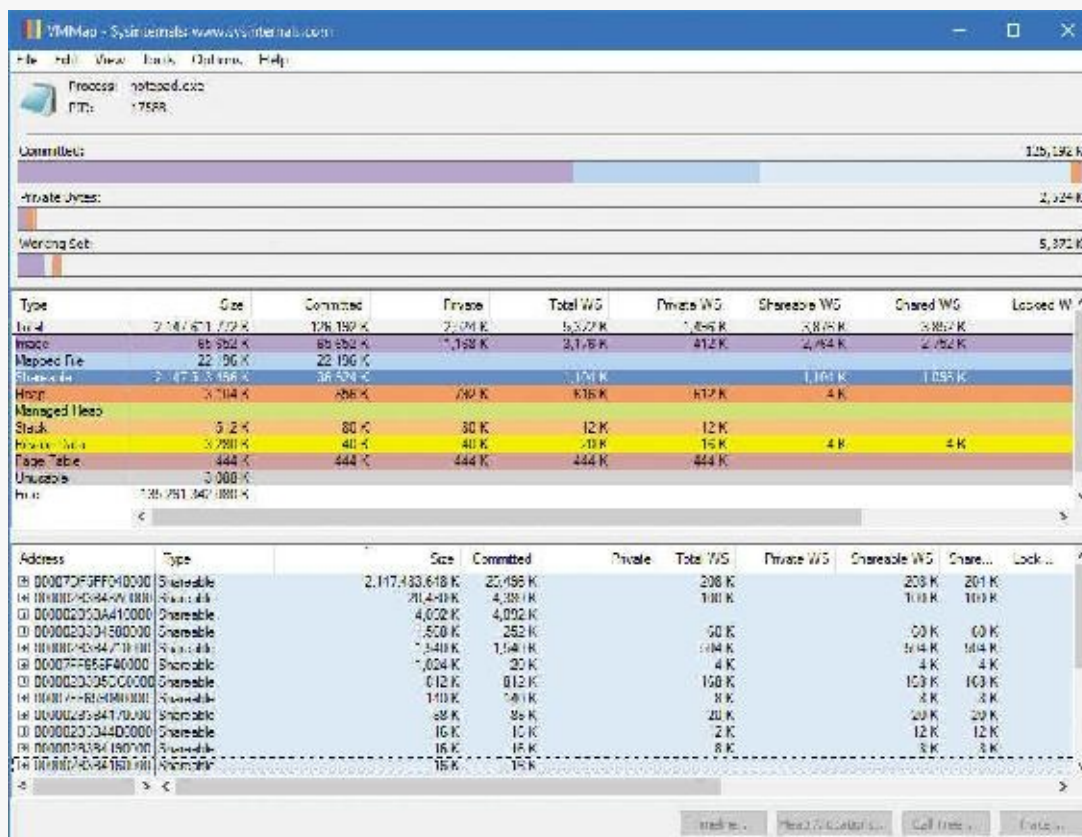
{0,0}. As such, as long as the region contains 4 KB of bit states that are all {0, 0}, it can be left as reserved, and only pages with at least one bit set {1, X} need to be committed.

Next, as described in the ASLR section of [Chapter 5](#), “[Memory management](#),” the system performs the randomization/relocation of libraries typically only once at boot, as a performance-saving measure to avoid repeated relocations. As such, after a library that supports ASLR has been loaded once at a given address, it will always be loaded at that same address. This also therefore means that once the relevant bitmap states have been calculated for the functions in that library, they will be identical in all other processes that also load the same binary. As such, the memory manager treats the CFG bitmap as a region of pagefile-backed shareable memory, and the physical pages that correspond to the shared bits only exist in RAM once.

This reduces the cost of the committed pages in RAM and means that only the bits corresponding to private memory need to be calculated. In regular applications, private memory is not executable except in the copy-on-write case where someone has patched a library (but this will not happen at image load), so the cost of loading an application, if it shares the same libraries as other previously launched applications, is almost nil. The next experiment demonstrates this.

EXPERIMENT: Control Flow Guard bitmap

Open the VMMap tool and select a Notepad process. You should see a large reserved block in the Sharable section like so:



You can sort the lower pane by size and quickly locate the large chunk used for the CFGBitmap, as shown. Additionally, if you attach to the process and use the `!address` command on the process, you will see WinDBG identifying the CFG bitmap for you:

[Click here to view code image](#)

```
+      7df5'ff530000 7df6'0118a000 0'01c5a000  MEM_MAPPED
MEM_RESERVE
Other  [CFG Bitmap]
      7df6'0118a000 7df6'011fb000 0'00071000  MEM_MAPPED MEM_COMMIT
PAGE_NOACCESS
Other  [CFG Bitmap]
      7df6'011fb000 7ff5'df530000 1ff'de335000 MEM_MAPPED
MEM_RESERVE
Other  [CFG Bitmap]
      7ff5'df530000 7ff5'df532000 0'00002000  MEM_MAPPED MEM_COMMIT
PAGE_READONLY
Other  [CFG Bitmap]
```

Note how large regions are marked as MEM_RESERVE, in between regions that are MEM_COMMIT, representing that at least one valid bit state {1, X} is set. Also, all (or almost all) the regions will be MEM_MAPPED, since they belong to the shared bitmap.

CFG bitmap construction

Upon system initialization, the `MiInitializeCfg` function is called to initialize support for CFG. The function creates one or two section objects (`MmCreateSection`) as reserved memory with size appropriate for the platform, as shown earlier. For 32-bit platforms, one bitmap is enough. For x64 platforms, two bitmaps are required—one for 64-bit processes and the other for Wow64 processes (32-bit applications). The section objects' pointers are stored in a substructure within the `MiState` global variable.

After a process is created, the appropriate section is securely mapped into the process's address space. *Securely* here means that the section cannot be unmapped by code running within the process or have its protection changed. (Otherwise, malicious code could just unmap the memory, reallocate, and fill everything with 1 bits, effectively disabling CFG, or simply modify any bits by marking the region read/write.)

The user mode CFG bitmap(s) are populated in the following scenarios:

- During image mapping, images that have been dynamically relocated due to ASLR (see [Chapter 5](#), for more on ASLR) will have their indirect call target metadata extracted. If an image does not have indirect call target metadata, meaning it was not compiled with CFG, it is assumed that every address within the image can be called indirectly. As explained, because dynamically relocated images are expected to load at the same address in every process, their metadata is used to populate the shared section that is used for the CFG bitmap.
- During image mapping, special care is needed for non-dynamically relocated images and images not being mapped at their preferred base. For these image mappings, the relevant pages of the CFG bitmap are made private and are populated using the CFG metadata from the image. For images whose CFG bits are present in the shared CFG bitmap, a check is made to ensure that all the relevant CFG bitmap pages are still shared. If this is not the case, the bits of the private CFG bitmap pages are populated using the CFG metadata from the image.
- When virtual memory is allocated or re-protected as executable, the relevant pages of the CFG bitmap are made private and initialized to all 1s by default. This is needed for cases such as just-in-time (JIT) compilation, where code is generated on the fly and then executed (for example, .NET or Java).

Strengthening CFG protection

Although CFG does an adequate job to prevent types of exploits that leverage indirect calls or jumps, it could be bypassed through the following ways:

- If the process can be tricked or an existing JIT engine abused to allocate executable memory, all the corresponding bits will be set to {1, 1}, meaning that all memory is considered a valid call target.
- For 32-bit applications, if the expected call target is `__stdcall` (standard calling convention), but an attacker is able to change the indirect call target to `__cdecl` (C calling convention), the stack will become corrupt, as the C call function will not perform cleanup of the caller's arguments, unlike a standard call function. Because CFG cannot differentiate between the different calling conventions, this results in a corrupt stack, potentially with an attacker-controlled return address, bypassing the CFG mitigation.
- Similarly, compiler-generated `setjmp/longjmp` targets behave differently from true indirect calls. CFG cannot differentiate between the two.
- Certain indirect calls are harder to protect, such as the Import Address Table (IAT) or Delay-Load

Address Table, which is typically in a read-only section of the executable.

- Exported functions may not be desirable indirect function calls.

Windows 10 introduces advancements to CFG that address all these issues. The first is to introduce a new flag to the `VirtualAlloc` function called `PAGE_TARGETS_INVALID` and one to `VirtualProtect` called `PAGE_TARGETS_NO_UPDATE`. With these flags set, JIT engines that allocate executable memory will not see all their allocations' bits set to the {1, 1} state. Instead, they must manually call the `SetProcess-ValidCallTargets` function (which calls the native `NtSetInformationVirtualMemory` function), which will allow them to specify the actual function start addresses of their JITed code. Additionally, this function is marked as a suppressed call with `DECLSPEC_GUARD_SUPPRESS`, making sure that attackers cannot use an indirect `CALL` or `JMP` to redirect into it, even at its function start. (Because it's an inherently dangerous function, calling it with a controlled stack or registers could result in the bypassing of CFG.)

Next, improved CFG changes the default flow you saw in the beginning of this section with a more refined flow. In this flow, the loader does not implement a simple “verify target, return” function, but rather a “verify target, call target, check stack, return” function, which is used in a subset of places on 32-bit applications (and/or running under WoW64). This improved execution flow is shown in [Figure 7-32](#).

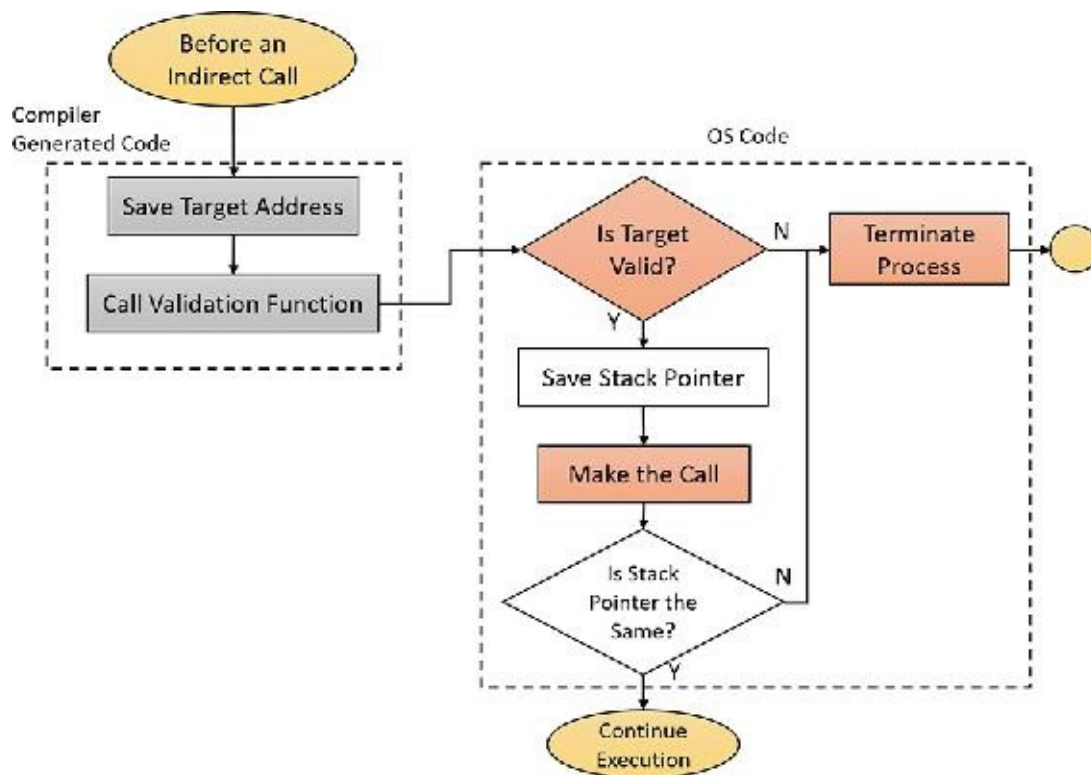


FIGURE 7-32 Improved CFG.

Next, improved CFG adds additional tables inside of the executable, such as the Address Taken IAT table and the Long Jump Address table. When `longjmp` and IAT CFG protection are enabled in the compiler, these tables are used to store destination addresses for these specific types of indirect calls, and the relevant functions are *not* placed in the regular function table, therefore not figuring in the bitmap. This means that if code is attempting to indirect jump/call to one of these functions, it will be treated as an illegal transition. Instead, the C Runtime and linker will validate the targets of, say, the `longjmp` function, by manually checking this table. Although it's more inefficient than a bitmap, there should be little to no functions in these tables, making the cost bearable.

Finally, improved CFG implements a feature called *export suppression*, which must be supported by the compiler and enabled by process-mitigation policy. (See the section "[Process-mitigation policies](#)" for

more on process level mitigations.) With this feature enabled, a new bit state is implemented (recall that bulleted list had {0, 1} as an undefined state). This state indicates that the function is valid but export-suppressed, and it will be treated differently by the loader.

You can determine which features are present in a given binary by looking at the guard flags in the Image Load Configuration Directory, which the DumpBin application used earlier can decode. For reference, they are listed in [Table 7-21](#).

Flag Symbol	Value	Description
IMAGE_GUARD_CF_INSTRUMENTED	0x100	This indicates CFG support is present for this module.
IMAGE_GUARD_CF_WRITE_INSTRUMENTED	0x200	This module performs CFG and write integrity checks.
IMAGE_GUARD_CF_FUNCTION_TABLE_PRESENT	0x400	This module contains CFG-aware function lists.
IMAGE_GUARD_SECURITY_COOKIE_UNUSED	0x800	This module does not make use of the security cookie emitted with the compiler /GS flag.
IMAGE_GUARD_PROTECT_DELAYLOAD_IAT	0x1000	This module supports read-only delay-load Import Address Tables (IATs).
IMAGE_GUARD_DELAYLOAD_IAT_IN_ITS_OWN_SECTION	0x2000	Delay-load IAT is its own section, so it can be re-protected if desired.
IMAGE_GUARD_CF_EXPORT_SUPPRESSION_INFO_PRESENT	0x4000	This module contains suppressed export information.
IMAGE_GUARD_CF_ENABLE_EXPORT_SUPPRESSION	0x8000	This module enables suppression of exports.
IMAGE_GUARD_CF_LONGJUMP_TABLE_PRESENT	0x10000	This module contains longjmp target information.

TABLE 7-21 Control Flow Guard flags

Loader interaction with CFG

Although it is the memory manager that builds the CFG bitmap, the user-mode loader (see [Chapter 3](#) for more information) serves two purposes. The first is to dynamically enable CFG support only if the feature is enabled (for example, the caller may have requested no CFG for the child process, or the process itself might not have CFG support). This is done by the `LdrpCfgProcessLoadConfig` loader function, which is called to initialize CFG for each loaded module. If the module `DllCharacteristics` flags in the optional header of the PE does not have the CFG flag set (`IMAGE_DLLCHARACTERISTICS_GUARD_CF`), the `GuardFlags` member of `IMAGE_LOAD_CONFIG_DIRECTORY` structure does not have the `IMAGE_GUARD_CF_INSTRUMENTED` flag set, or the kernel has forcibly turned off CFG for this module, then there is nothing to do.

Second, if the module is indeed using CFG, `LdrpCfgProcessLoadConfig` gets the indirect checking function pointer retrieved from the image (the `GuardCfCheckFunctionPointer` member of `IMAGE_LOAD_CONFIG_DIRECTORY` structure) and sets it to either `LdrpValidateUserCallTarget` or `LdrpValidateUserCallTargetES` in `Ntdll`, depending on whether export suppression is enabled. Additionally, the function first makes sure the indirect pointer has not been somehow modified to point outside the module itself.

Furthermore, if improved CFG was used to compile this binary, a second indirect routine is available, called the *dispatch CFG routine*. It is used to implement the enhanced execution flow described earlier. If the image includes such a function pointer (in the `GuardCfDispatchFunctionPointer` member of the abovementioned structure), it is initialized to `LdrpDispatchUserCallTarget`, or

LdrpDispatchUserCallTargetES if export suppression is enabled.



Note

In some cases, the kernel itself can emulate or perform indirect jumps or calls on behalf of user mode. In situations where this is a possibility, the kernel implements its own `MmValidateUserCallTarget` routine, which performs the same work as `LdrpValidate-UserCallTarget`.

The code generated by the compiler when CFG is enabled issues an indirect call that lands in the `LdrpValidateCallTarget(ES)` or `LdrpDispatchUserCallTarget(ES)` functions in `Ntdll`. This function uses the target branch address and checks the bit state value for the function:

- If the bit state is {0, 0}, the dispatch is potentially invalid.
- If the bit state is {1, 0}, and the address is 16-byte aligned, the dispatch is valid. Otherwise, it is potentially invalid.
- If the bit state is {1, 1}, and the address is not 16-byte aligned, the dispatch is valid. Otherwise, it is potentially invalid.
- If the bit state is {0, 1}, the dispatch is potentially invalid.

If the dispatch is potentially invalid, the `RtlpHandleInvalidUserCallTarget` function will execute to determine the appropriate action. First, it checks if suppressed calls are allowed in the process, which is an unusual application-compatibility option that might be set if Application Verifier is enabled, or through the registry. If so, it will check if the address is suppressed, which is why it was not inserted into the bitmap (recall that a special flag in the guard function table entry indicates this). If this is the case, the call is allowed through. If the function is not valid at all (meaning it's not in the table), then the dispatch is aborted and the process terminated.

Second, a check is made to see if export suppression is enabled. If it is, the target address is checked against the list of export-suppressed addresses, which is once again indicated with another flag that is added in the guard function table entry. If this is the case, the loader validates that the target address is a forwarder reference to the export table of another DLL, which is the only allowed case of an indirect call toward an image with suppressed exports. This is done by a complex check that makes sure the target address is in a different image, that its image load directory has enabled export suppression, and that this address is in the import directory of that image. If these checks match, the kernel is called through the `NtSetInformationVirtualMemory` call described earlier, to change the bit state to {1, 0}. If any of these checks fail, or export suppression is not enabled, then the process is terminated.

For 32-bit applications, an additional check is performed if DEP is enabled for the process. (See [Chapter 5](#) for more on DEP.) Otherwise, because there are no execution guarantees to begin with, the incorrect call is allowed, as it may be an older application calling into the heap or stack for legitimate reasons.

Finally, because large sets of {0, 0} bit states are not committed to save space, if checking the CFG bitmap lands on a reserved page, an access violation exception occurs. On x86, where exception handling setup is expensive, instead of being handled as part of the verification code, it is left to propagate normally. (See Chapter 8 in Part 2 for more on exception dispatching.) The user-mode dispatcher handler, `KiUserExceptionDispatcher`, has specific checks for recognizing CFG bitmap access violation exceptions within the validation function and will automatically resume execution if the exception code

was `STATUS_IN_PAGE_ERROR`. This simplifies the code in `LdrpValidateUserCallTarget(ES)` and `LdrpDispatchUserCallTarget(ES)`, which don't have to include exception handling code. On x64, where exception handlers are simply registered in tables, the `LdrpICallHandler` handler runs instead, with the same logic as above.

Kernel CFG

Although drivers compiled with Visual Studio and `/guard:cf` also ended up with the same binary properties as user-mode images, the first versions of Windows 10 did not do anything with this data. Unlike the user-mode CFG bitmap, which is protected by a higher, more trusted entity (the kernel), there is nothing that can truly “protect” the kernel CFG bitmap if one were to be created. A malicious exploit could simply edit the PTE that corresponded to the page containing the desired bits to modify, mark it as read/write, and proceed with the indirect call or jump. Therefore, the overhead of setting up such a trivially bypassable mitigation was simply not worth it.

With a greater number of users enabling VBS features, once again, the higher security boundary that VTL 1 provides can be leveraged. The SLAT page table entries come to the rescue by providing a second boundary against PTE page protection changes. While the bitmap is readable to VTL 0 because the SLAT entries are marked as read only, if a kernel attacker attempts to change the PTEs to mark them read/write, they cannot do the same to the SLAT entries. As such, this will be detected as an invalid KCFG bitmap access which HyperGuard can act on (for telemetry reasons alone—since the bits can't be changed anyway).

KCFG is implemented almost identically to regular CFG, except that export suppression is not enabled, nor is `longjmp` support, nor is the ability to dynamically request additional bits for JIT purposes. Kernel drivers should not be doing any of these things. Instead, the bits are set in the bitmap based on the “address taken IAT table” entries, if any are set; by the usual function entries in the guard table each time a driver image is loaded; and for the HAL and kernel during boot by `MiInitializeKernelCfg`. If the hypervisor is not enabled, and SLAT support is not present, then none of this will be initialized, and Kernel CFG will be kept disabled.

Just like in the user-mode case, a dynamic pointer in the load configuration data directory is updated, which in the enabled case will point to `__guard_check_icall` for the check function and `__guard_dispatch_icall` for the dispatch function in enhanced CFG mode. Additionally, a variable named `guard_icall_bitmap` will hold the virtual address of the bitmap.

One last detail on Kernel CFG is that unfortunately, dynamic Driver Verifier settings will not be configurable (for more information on Driver Verifier, see [Chapter 6, “I/O system”](#)), as this would require adding dynamic kernel hooks and redirecting execution to functions that may not be in the bitmap. In this case, `STATUS_VRF_CFG_ENABLED (0xC000049F)` will be returned, and a reboot is required (at which time the bitmap can be built with the Verifier Driver hooks in place).

Security assertions

Earlier, we described how Control Flow Guard will terminate the process. We also explained how certain other mitigations or security features will raise an exception to kill the process. It is important to be accurate with what *exactly* happens during these security violations because both these descriptions hide important details about the mechanism.

In fact, when a security-related breach occurs, such as when CFG detects an incorrect indirect call or jump, terminating the process through the standard `TerminateProcess` mechanism would not be an adequate path. There would be no crash generated, and no telemetry sent to Microsoft. These are both

important tools for the administrator to understand that a potential exploit has executed or that an application compatibility issue exists, as well as for Microsoft to track zero-day exploitation in the wild. On the flip side, while raising an exception would achieve the desired result, exceptions are callbacks, which can be:

- Potentially hooked by attackers if `/SAFESEH` and `SEHOP` mitigations are not enabled, causing the security check to be the one that gives control to an attacker in the first place—or an attacker can simply “swallow” the exception.
- Potentially hooked by legitimate parts of the software through an unhandled exception filter or vectored exception handler, both of which might accidentally swallow the exception.
- Same as above, but intercepted by a third-party product that has injected its own library into the process. Common to many security tools, this can also lead in the exception not being correctly delivered to Windows Error Reporting (WER).
- A process might have an application recovery callback registered with WER. This might then display a less clear UI to the user, and might restart the process in its current exploited state, leading anywhere from a recursive crash/start loop to the exception being swallowed whole.
- Likely in a C++-based product, caught by an outer exception handler as if “thrown” by the program itself, which, once again, might swallow the exception or continue execution in an unsafe manner.

Solving these issues requires a mechanism that can raise an exception that cannot be intercepted by any of the process’s components outside of the WER service, which must itself be guaranteed to receive the exception. This is where security assertions come into play.

Compiler and OS support

When Microsoft libraries, programs, or kernel components encounter unusual security situations, or when mitigations recognize dangerous violations of security state, they now use a special compiler intrinsic supported by Visual Studio, called `__fastfail`, which takes one parameter as input. Alternatively, they can call a runtime library (Rtl) function in Ntdll called `RtlFailFast2`, which itself contains a `__fastfail` intrinsic. In some cases, the WDK or SDK contain inline functions that call this intrinsic, such as when using the `LIST_ENTRY` functions `InsertTailList` and `RemoveEntryList`. In other situations, it is the Universal CRT (uCRT) itself that has this intrinsic in its functions. In yet others, APIs will do certain checks when called by applications and may use this intrinsic as well.

Regardless of the situation, when the compiler sees this intrinsic, it generates assembly code that takes the input parameter, moves it into the RCX (x64) or ECX (x86) register, and then issues a software interrupt with the number 0x29. (For more information on interrupts, see Chapter 8 in Part 2.)

In Windows 8 and later, this software interrupt is registered in the Interrupt Dispatch Table (IDT) with the handler `KiRaiseSecurityCheckFailure`, which you can verify on your own by using the `!idt 29` command in the debugger. This will result (for compatibility reasons) in `KiFastFailDispatch` being called with the `STATUS_STACK_BUFFER_OVERRUN` status code (0xC0000409). This will then do regular exception dispatching through `KiDispatchException`, but treat this as a second-chance exception, which means that the debugger and process won’t be notified.

This condition will be specifically recognized and an error message will be sent to the WER error ALPC port as usual. WER will claim the exception as non-continuable, which will then cause the kernel to terminate the process with the usual `ZwTerminateProcess` system call. This, therefore, guarantees that once the interrupt is used, no return to user mode will ever be performed within this process again, that WER will be notified, and that the process will be terminated (additionally, the error code will be the

exception code). When the exception record is generated, the first exception argument will be the input parameter to `__fastfail`.

Kernel-mode code can also raise exceptions, but in this case `KiBugCheckDispatch` will be called instead, which will result in a special kernel mode crash (bugcheck) with code `0x139` (`KERNEL_SECURITY_CHECK_FAILURE`), where the first argument will be the input parameter to `__fastfail`.

Fast fail/security assertion codes

Because the `__fastfail` intrinsic contains an input argument that is bubbled up to the exception record or crash screen, it allows the failing check to identify what part of the system or process is not working correctly or has encountered a security violation. [Table 7-22](#) shows the various failure conditions and their meaning or significance.

Code	Meaning
Legacy OS Violation (0x0)	An older buffer security check present in a legacy binary has failed, and has been converted to a security assertion instead.
V-Table Guard Failure (0x1)	The Virtual Table Guard Mitigation in Internet Explorer 10 and higher has encountered a corrupted virtual function table pointer.
Stack Cookie Check Failure (0x2)	The stack cookie generated with the <code>/GS: compiler</code> option (also called a stack canary) has been corrupted.
Corrupt List Entry (0x3)	One of the macros for manipulating <code>LIST_ENTRY</code> structures has detected an inconsistent linked list, where the grandparent or grandchild entry does not point to the parent or child entry of the item being manipulated.
Incorrect Stack (0x4)	A user-mode or kernel-mode API that is often potentially called from ROP-based exploits while operating on an attacker-controlled stack has been called, and the stack is therefore not the expected one.
Invalid Argument (0x5)	A user-mode CRT API (typically) or other sensitive function has been called with an invalid argument, suggesting potential ROP-based use or an otherwise corrupted stack.
Stack Cookie Init Failure (0x6)	The initialization of the stack cookie has failed, suggesting image patching or corruption.
Fatal App Exit (0x7)	The application has used the <code>FatalAppExit</code> user-mode API, which has been converted into a security assertion to grant it the advantages this has.
Range Check Failure (0x8)	Additional validation checks in certain fixed array buffers to check if the array element index is within expected bounds.
Unsafe Registry Access (0x9)	A kernel-mode driver is attempting to access registry data from a user-controllable hive (such as an application hive or user profile hive) and is not using the <code>RTL_QUERY_REGISTRY_TYPECHECK</code> flag to protect itself.

CFG Indirect Call Failure (0xA)	Control Flow Guard has detected an indirect CALL or JMP instruction to a target address that is not a valid dispatch per the CFG bitmap.
CFG Write Check Failure (0xB)	Control Flow Guard with write protection has detected an invalid write to protected data. This feature (/guard:cfw) is not supported outside of testing at Microsoft.
Invalid Fiber Switch (0xC)	The SwitchToFiber API was used on an invalid fiber or from a thread which has not been converted to a fiber.
Invalid Set of Context (0xD)	An invalid context record structure was detected while attempting to restore it (due to an exception or SetThreadContext API), in which the stack pointer is not valid. Checked only when CFG is active on the process.
Invalid Reference Count (0xE)	A reference counted object (such as the OBJECT_HEADER in kernel-mode or a Win32k.sys GDI object) has underflowed its reference count below 0 or overflowed beyond its maximum capacity back to 0.
Invalid Jump Buffer (0x12)	A Jongjmp attempt is being made with a jump buffer that contains an invalid stack address or invalid instruction pointer. Checked only when CFG is active on the process.
MRDATA Modified (0x13)	The mutable read-only data heap/section of the loader has been modified. Checked only when CFG is active on the process.
Certification Failure (0x14)	One or more Cryptographic Services APIs has encountered an issue parsing a certificate or an invalid ASN.1 stream.
Invalid Exception Chain (0x15)	An image linked with /SAFESEH, or with the SEHOP mitigation, has encountered an invalid exception handler dispatch.
Crypto Library (0x16)	CNG.SYS, KSECCOD.SYS, or their equivalent APIs in user mode have encountered some critical failure.
Invalid Call in DLL Callout (0x17)	An attempt to call dangerous functions while in the user-mode loader's notification callback has occurred.
Invalid Image Base (0x18)	An invalid value for __ImageBase (IMAGE_DOS_HEADER structure) was detected by the user-mode image loader.
Delay Load Protection Failure (0x19)	The delay-loaded IAT has been found to be corrupted while delay-loading an imported function. Checked only when CFG is active on the process, and delay-load IAT protection is enabled.
Unsafe Extension Call (0x1A)	Checked when certain kernel-mode extension APIs are called, and the caller state is incorrect.
Deprecated Service Called (0x1B)	Checked when certain no-longer supported, and undocumented system calls, are called.
Invalid Buffer Access (0x1C)	Checked by the runtime library functions in Ntdll and the kernel when a generic buffer structure is corrupt in some way.
Invalid Balanced Tree (0x1D)	Checked by the runtime library functions in Ntdll and the kernel when an RTL_RB_TREE or RTL_AVL_TABLE structure has invalid nodes (where siblings and/or parent nodes do not match up with the grandparent's, similar to the LIST_ENTRY checks).
Invalid Next Thread (0x1E)	Checked by the kernel scheduler when the next thread to schedule in the KPCR is invalid in some way.
CFG Call Suppressed (0x1F)	Checked when CFG is allowing a suppressed call due to compatibility concerns. In this situation, WER will mark the error as handled, and the kernel will not terminate the process, but telemetry will still be sent to Microsoft.
APCs Disabled (0x20)	Checked by the kernel when returning to user-mode and kernel APCs are still disabled.
Invalid Idle State (0x21)	Checked by the kernel power manager when the CPU is attempting to enter an invalid C state.
MRDATA Protection Failure (0x22)	Checked by the user-mode loader when the Mutable Read-Only Heap Section has already been unprotected outside of the expected code path.
Unexpected Heap Exception (0x23)	Checked by the heap manager whenever the heap is corrupted in ways that indicate potential exploitation attempts.

Invalid Lock State (0x24)	Checked by the kernel when certain locks are not in their expected state, such as if an acquired lock is already in a released state.
Invalid Longjmp (0x25)	Checked by Longjmp when called, and CFG is active on the process with Longjmp Protection enabled, but the Longjmp Table is corrupt or missing in some way.
Invalid Longjmp Target (0x26)	Same conditions as above, but the Longjmp Table indicates that this is not a valid Longjmp target function.
Invalid Dispatch Context (0x27)	Checked by the exception handler in kernel-mode when an exception is attempted to be dispatched with an incorrect CONTEXT record.
Invalid Thread (0x28)	Checked by the scheduler in kernel-mode when the KTHREAD structure is corrupt during certain scheduling operations.
Invalid System Call Number (0x29)	Similar to Deprecated Service Called, but WER will mark the exception as handled, resulting in the process continuing and therefore only used for telemetry.
Invalid File Operation (0x2A)	Used by the I/O Manager and certain file systems, as another telemetry-type failure as above.
LPAC Access Denied (0x2B)	Used by the SRM's access check function when a lower-privilege AppContainer attempts to access an object that does not have the ALL_RESTRICTED_APPLICATON_PACKAGE_SID and tracing of such failures is enabled. Once more, results only in telemetry data, not a process crash.
RFG Stack Failure (0x2C)	Used by Return Flow Guard (RFG), although this feature is currently disabled.
Loader Continuity Failure (0x2D)	Used by the process-mitigation policy of the same name, shown earlier, to indicate that an unexpected image with a different signature or no signature has been loaded.
CFG Export Suppression Failure (0x2D)	Used by CFG when enabled with export suppression to indicate that a suppressed export has been the target of an indirect branch.
Invalid Control Stack (0x2E)	Used by RFG, although this feature is currently disabled.
Set Context Denied (0x2F)	Used by the process-mitigation policy of the same name, shown earlier, although this feature is currently disabled.

TABLE 7-22 __fastfail failure codes

Application Identification

Historically, security decisions in Windows have been based on a user's identity (in the form of the user's SID and group membership), but a growing number of security components (AppLocker, firewall, antivirus, anti-malware, Rights Management Services, and others) need to make security decisions based on what code is to be run. In the past, each of these security components used their own proprietary method for identifying applications, which led to inconsistent and overly complicated policy authoring. The purpose of Application Identification (AppID) is to bring consistency to how the security components recognize applications by providing a single set of APIs and data structures.



Note

This is not the same as the AppID used by DCOM/COM+ applications, where a GUID represents a process that is shared by multiple CLSIDs, nor is it related to UWP application ID.

Just as a user is identified when she logs in, an application is identified just before it is started by generating the main program's AppID. An AppID can be generated from any of the following attributes of the application:

- **Fields** Fields within a code-signing certificate embedded within the file allow for different combinations of publisher name, product name, file name, and version. **APPID://FQBN** is a fully qualified binary name, and it is a string in the following form: **{Publisher\Product\Filename,Version}**. **Publisher** is the Subject field of the x.509 certificate used to sign the code, using the following fields:

- **O** Organization
 - **L** Locality
 - **S** State or province
 - **C** Country
- **File hash** there are several methods that can be used for hashing. The default is `APPID://SHA256HASH`. However, for backward compatibility with SRP and most x.509 certificates, SHA-1 (`APPID://SHA1HASH`) is still supported. `APPID://SHA256HASH` specifies the SHA-256 hash of the file.
 - **The partial or complete path to the file** `APPID://Path` specifies a path with optional wildcard characters (*).
-



Note

An AppID does not serve as a means for certifying the quality or security of an application. An AppID is simply a way of identifying an application so that administrators can reference the application in security policy decisions.

The AppID is stored in the process access token, allowing any security component to make authorization decisions based on a single consistent identification. AppLocker uses conditional ACEs (described earlier) for specifying whether a particular program is allowed to be run by the user.

When an AppID is created for a signed file, the certificate from the file is cached and verified to a trusted root certificate. The certificate path is reverified daily to ensure the certificate path remains valid. Certificate caching and verification are recorded in the system event log at Application and Services Logs\Microsoft\Windows\AppID\Operational.

AppLocker

Windows 8.1 and Windows 10 (Enterprise editions) and Windows Server 2012/R2/2016 support a feature known as *AppLocker*, which allows an administrator to lock down a system to prevent unauthorized programs from being run. Windows XP introduced Software Restriction Policies (SRP), which was the first step toward this capability, but SRP was difficult to manage, and it couldn't be applied to specific users or groups. (All users were affected by SRP rules.) AppLocker is a replacement for SRP, and yet coexists alongside SRP, with AppLocker's rules being stored separately from SRP's rules. If both AppLocker and SRP rules are in the same Group Policy object (GPO), only the AppLocker rules will be applied.

Another feature that makes AppLocker superior to SRP is AppLocker's auditing mode, which allows an administrator to create an AppLocker policy and examine the results (stored in the system event log) to determine whether the policy will perform as expected—without actually performing the restrictions. AppLocker auditing mode can be used to monitor which applications are being used by one or more users on a system.

AppLocker allows an administrator to restrict the following types of files from being run:

- Executable images (EXE and COM)
- Dynamic-link libraries (DLL and OCX)
- Microsoft Software Installer (MSI and MSP) for both install and uninstall

- Scripts
- Windows PowerShell (PS1)
- Batch (BAT and CMD)
- VisualBasic Script (VBS)
- Java Script (JS)

AppLocker provides a simple GUI rule-based mechanism, which is very similar to network firewall rules, for determining which applications or scripts are allowed to be run by specific users and groups, using conditional ACEs and AppID attributes. There are two types of rules in AppLocker:

- Allow the specified files to run, denying everything else.
- Deny the specified files from being run, allowing everything else. Deny rules take precedence over allow rules.

Each rule can also have a list of exceptions to exclude files from the rule. Using an exception, you could create a rule to, for example, allow everything in the C:\Windows or C:\Program Files directories to be run except RegEdit.exe.

AppLocker rules can be associated with a specific user or group. This allows an administrator to support compliance requirements by validating and enforcing which users can run specific applications. For example, you can create a rule to allow users in the Finance security group to run the finance line-of-business applications. This blocks everyone who is not in the Finance security group from running finance applications (including administrators) but still provides access for those who have a business need to run the applications. Another useful rule would be to prevent users in the Receptionists group from installing or running unapproved software.

AppLocker rules depend upon conditional ACEs and attributes defined by AppID. Rules can be created using the following criteria:

- **Fields within a code-signing certificate embedded within the file, allowing for different combinations of publisher name, product name, file name, and version** For example, a rule could be created to allow all versions greater than 9.0 of Contoso Reader to run or allow anyone in the Graphics group to run the installer or application from Contoso for GraphicsShop as long as the version is 14.*. For example, the following SDDL string denies execute access to any signed programs published by Contoso for the RestrictedUser user account (identified by the user's SID):

[Click here to view code image](#)

```
D: (XD;;;FX;;;S-1-5-21-3392373855-1129761602-2459801163-1028;
((Exists APPID://FQBN)
&& ((APPID://FQBN) >= ({ "0=CONTOSO, INCORPORATED, L=REDMOND,
S=CWASHINGTON, C=US\*\*", 0 })))
```

- **Directory path, allowing only files within a particular directory tree to run** This can also be used to identify specific files. For example, the following SDDL string denies execute access to the programs in the directory C:\Tools for the RestrictedUser user account (identified by the user's SID):

[Click here to view code image](#)

```
D: (XD;;;FX;;;S-1-5-21-3392373855-1129761602-2459801163-1028;
(APPID://PATH
Contains "%OSDRIVE%\TOOLS\*"))
```

- **File hash** Using a hash will also detect if a file has been modified and prevent it from running. This can also be a weakness if files are changed frequently because the hash rule will need to be updated frequently. File hashes are often used for scripts because few scripts are signed. For example, this SDDL string denies execute access to programs with the specified hash values for the RestrictedUser user account (identified by the user's SID):

[Click here to view code image](#)

```
D: (XD;;;FX;;;S-1-5-21-3392373855-1129761602-2459801163-1028;  
(APPID://SHA256HASH  
Any_of  
{#7a334d2b99d48448eedd308dfca63b8a3b7b44044496ee2f8e236f5997f1b647  
#2a782f76cb94ece307dc52c338f02edbbfdca83906674e35c682724a8a92a76b}
```

AppLocker rules can be defined on the local machine using the Security Policy MMC snap-in (secpol.msc, see [Figure 7-33](#)) or a Windows PowerShell script, or they can be pushed to machines within a domain using Group Policy. AppLocker rules are stored in multiple locations within the registry:

- **HKLM\Software\Policies\Microsoft\Windows\SrpV2** This key is also mirrored to HKLM\SOFTWARE\Wow6432Node\Policies\Microsoft\Windows\SrpV2. The rules are stored in XML format.
- **HKLM\SYSTEM\CurrentControlSet\Control\Srp\Gp\Exe** The rules are stored as SDDL and a binary ACE.
- **HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Group Policy Objects\{GUID}\Machine\Software\Policies\Microsoft\Windows\SrpV2** AppLocker policy pushed down from a domain as part of a GPO are stored here in XML format.

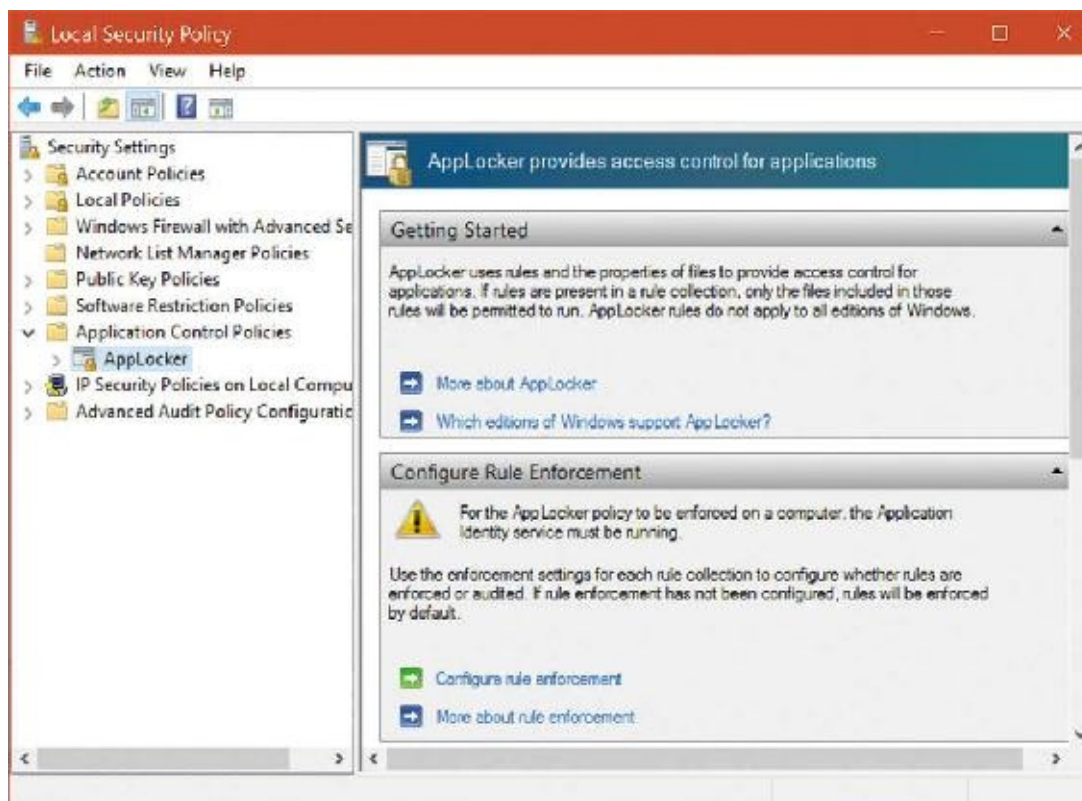


FIGURE 7-33 AppLocker configuration page in Local Security Policy.

Certificates for files that have been run are cached in the registry under the key HKLM\SYSTEM\CurrentControlSet\Control\AppID\CertStore. AppLocker also builds a certificate chain

(stored in HKLM\SYSTEM\CurrentControlSet\Control\AppID\CertChainStore) from the certificate found in a file back to a trusted root certificate.

There are also AppLocker-specific PowerShell commands (cmdlets) to enable deployment and testing via scripting. After using the Import-Module AppLocker to get AppLocker cmdlets into PowerShell, several cmdlets are available. These include Get-AppLockerFileInformation, Get-AppLockerPolicy, New-AppLockerPolicy, Set-AppLockerPolicy, and Test-AppLockerPolicy.

The AppID and SRP services coexist in the same binary (AppIdSvc.dll), which runs within an Svchost process. The service requests a registry change notification to monitor any changes under that key, which is written by either a GPO or the AppLocker UI in the Local Security Policy MMC snap-in. When a change is detected, the AppID service triggers a user-mode task (AppIdPolicyConverter.exe), which reads the new rules (described with XML) and translates them into binary format ACEs and SDDL strings, which are understandable by both the user-mode and kernel-mode AppID and AppLocker components. The task stores the translated rules under HKLM\SYSTEM\CurrentControlSet\Control\Srp\Gp. This key is writable only by System and Administrators, and it is marked read-only for authenticated users. Both user-mode and kernel-mode AppID components read the translated rules from the registry directly. The service also monitors the local machine trusted root certificate store, and it invokes a user-mode task (AppIdCertStoreCheck.exe) to reverify the certificates at least once per day and whenever there is a change to the certificate store. The AppID kernel-mode driver (%SystemRoot%\System32\drivers\AppId.sys) is notified about rule changes by the AppID service through an APPID_POLICY_CHANGED DeviceIoControl request.

An administrator can track which applications are being allowed or denied by looking at the system event log using Event Viewer (once AppLocker has been configured and the service started). See [Figure 7-34](#).

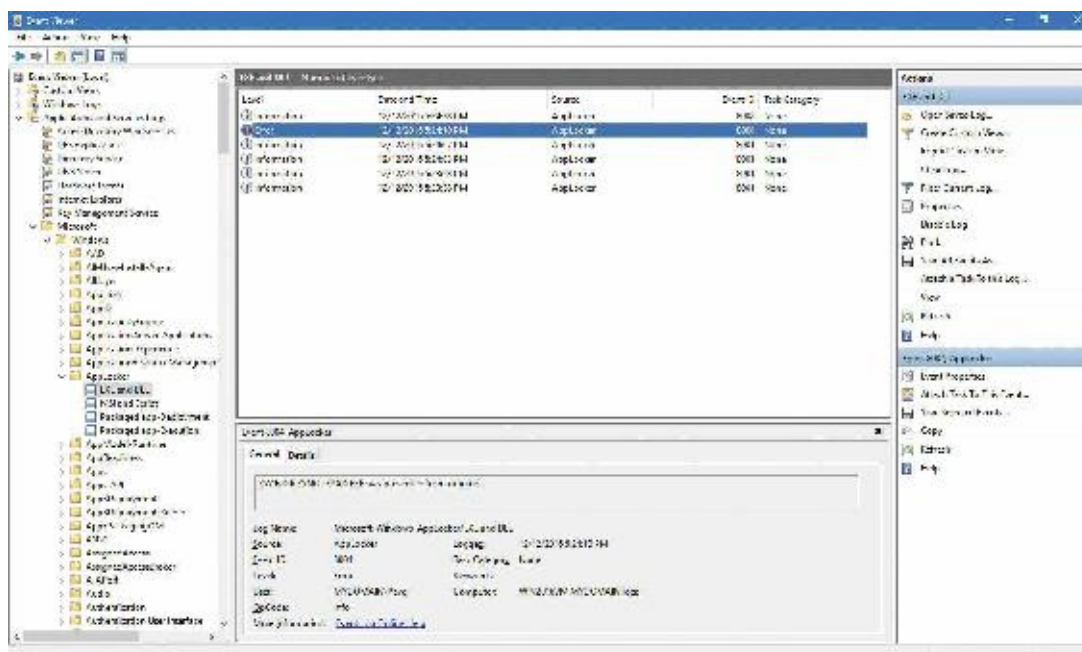


FIGURE 7-34 Event Viewer showing AppLocker allowing and denying access to various applications. Event ID 8004 is denied; 8002 is allowed.

The implementations of AppID, AppLocker, and SRP are somewhat blurred and violate strict layering, with various logical components coexisting within the same executables, and the naming is not as consistent as one would like.

The AppID service runs as `LocalService` so that it has access to the Trusted Root Certificate Store on the system. This also enables it to perform certificate verification. The AppID service is responsible for the following:

- Verification of publisher certificates
- Adding new certificates to the cache
- Detecting AppLocker rule updates and notifying the AppID driver

The AppID driver performs the majority of the AppLocker functionality and relies on communication (via `DeviceIoControl` requests) from the AppID service, so its device object is protected by an ACL, granting access only to the NT SERVICE\AppIDSvc, LOCAL SERVICE and BUILTIN\Administrators groups. Thus, the driver cannot be spoofed by malware.

When the AppID driver is first loaded, it requests a process-creation callback by calling `PsSetCreateProcessNotifyRoutineEx`. When the notification routine is called, it is passed a `PPS_CREATE_NOTIFY_INFO` structure (describing the process being created). It then gathers the AppID attributes that identify the executable image and writes them to the process's access token. Then it calls the undocumented routine `SeSrpAccessCheck`, which examines the process token and the conditional ACE AppLocker rules, and determines whether the process should be allowed to run. If the process should not be allowed to run, the driver writes `STATUS_ACCESS_DISABLED_BY_POLICY_OTHER` to the `Status` field of the `PPS_CREATE_NOTIFY_INFO` structure, which causes the process creation to be canceled (and sets the process's final completion status).

To perform DLL restriction, the image loader sends a `DeviceIoControl` request to the AppID driver whenever it loads a DLL into a process. The driver then checks the DLL's identity against the AppLocker conditional ACEs, just like it would for an executable.



Note

Performing these checks for every DLL load is time-consuming and might be noticeable to end users. For this reason, DLL rules are normally disabled, and they must be specifically enabled via the Advanced tab in the AppLocker properties page in the Local Security Policy snap-in.

The scripting engines and the MSI installer have been modified to call the user-mode SRP APIs whenever they open a file, to check whether a file is allowed to be opened. The user-mode SRP APIs call the AuthZ APIs to perform the conditional ACE access check.

Software Restriction Policies

Windows contains a user-mode mechanism called *Software Restriction Policies* (SRP) that enables administrators to control what images and scripts execute on their systems. The Software Restriction Policies node of the Local Security Policy editor, shown in [Figure 7-35](#), serves as the management interface for a machine's code execution policies, although per-user policies are also possible using domain group policies.

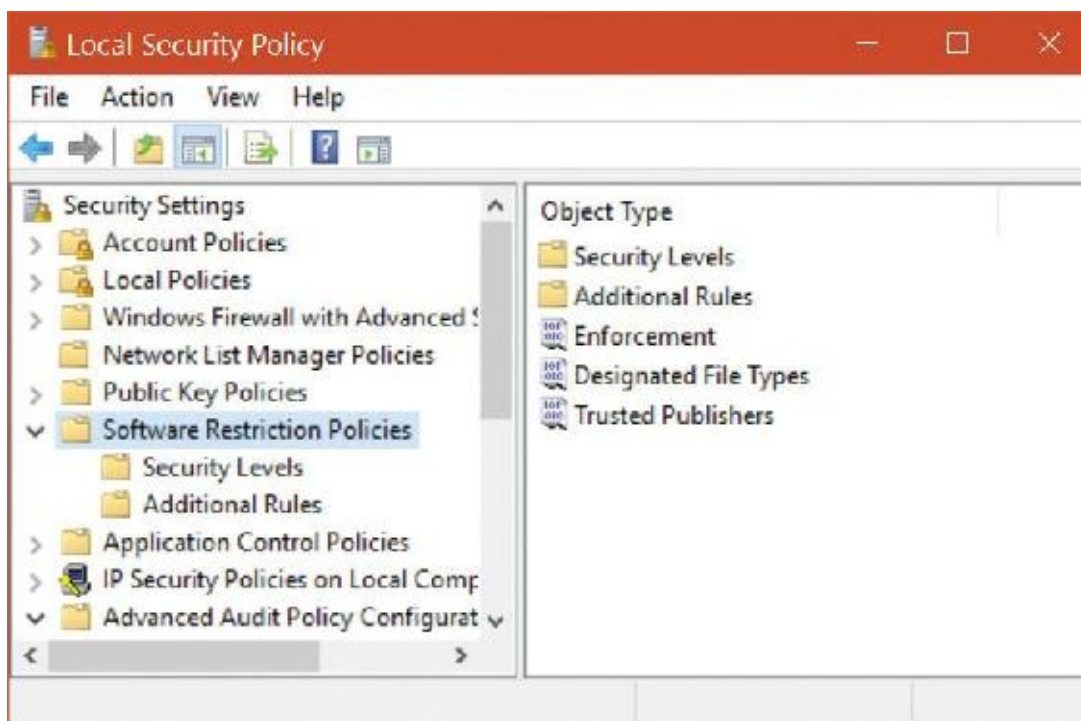


FIGURE 7-35 Software Restriction Policy configuration.

Several global policy settings appear beneath the Software Restriction Policies node:

- **Enforcement** This policy configures whether restriction policies apply to libraries, such as DLLs, and whether policies apply to users only or to administrators as well.
- **Designated File Types** This policy records the extensions for files that are considered executable code.
- **Trusted Publishers** This policy controls who can select which certificate publishers are trusted.

When configuring a policy for a particular script or image, an administrator can direct the system to recognize it using its path, its hash, its Internet zone (as defined by Internet Explorer), or its cryptographic certificate, and can specify whether it is associated with the Disallowed or Unrestricted security policy.

Enforcement of SRPs takes place within various components where files are treated as containing executable code. Some of these components are listed here:

- The user-mode Windows `CreateProcess` function in `Kernel32.dll` enforces it for executable images.
- The DLL loading code in `Ntdll` enforces it for DLLs.
- The Windows command prompt (`Cmd.exe`) enforces it for batch file execution.
- Windows Scripting Host components that start scripts—`Cscript.exe` (for command-line scripts), `Wscript.exe` (for UI scripts), and `Scrobj.dll` (for script objects)—enforce it for script execution.
- The PowerShell host (`PowerShell.exe`) enforces it for PowerShell script execution.

Each of these components determines whether the restriction policies are enabled by reading the `TransparentEnabled` registry value in the `HKLM\Software\Policies\Microsoft\Windows\Safer\CodeIdentifiers` key, which if set to 1 indicates that policies are in effect. Then it determines whether the code it's about to execute matches one of the rules specified in a subkey of the `CodeIdentifiers` key and, if so, whether the execution should be allowed. If there is no match, the default policy, as specified in the `DefaultLevel` value of the `CodeIdentifiers` key, determines whether the execution is allowed.

Software Restriction Policies are a powerful tool for preventing the unauthorized access of code and scripts, but only if properly applied. Unless the default policy is set to disallow execution, a user can make minor changes to an image that's been marked as disallowed so that he can bypass the rule and execute it. For example, a user can change an innocuous byte of a process image so that a hash rule fails to recognize it, or copy a file to a different location to avoid a path-based rule.

EXPERIMENT: Watching Software Restriction Policy enforcement

You can indirectly see SRPs being enforced by watching accesses to the registry when you attempt to execute an image that you've disallowed.

1. Run `secpol.msc` to open the Local Security Policy editor and navigate to the Software Restriction Policies node.
2. Choose **Create New Policies** from the context menu if no policies are defined.
3. Create a path-based disallow restriction policy (under the Additional Rules node) for `%SystemRoot%\System32\notepad.exe`.
4. Run Process Monitor and set an include a Path filter for Safer.
5. Open a command prompt and run Notepad from the prompt.

Your attempt to run Notepad should result in a message telling you that you cannot execute the specified program, and Process Monitor should show the command prompt (`cmd.exe`) querying the local machine restriction policies.

Kernel Patch Protection

Some device drivers modify the behavior of Windows in unsupported ways. For example, they patch the system call table to intercept system calls or patch the kernel image in memory to add functionality to specific internal functions. Such modifications are inherently dangerous and can reduce system stability and security. Additionally, it is also possible for such modifications to be made with malicious intent, either by rogue drivers or through exploits due to vulnerabilities in Windows drivers.

Without the presence of a more privileged entity than the kernel itself, detecting and protecting against kernel-based exploits or drivers from within the kernel itself is a tricky game. Because both the detection/protection mechanism and the unwanted behavior operate in ring 0, it is not possible to define a security boundary in the true sense of the word, as the unwanted behavior could itself be used to disable, patch, or fool the detection/prevention mechanism. That being said, even in such conditions, a mechanism to react to such unwanted operations can still be useful in the following ways:

- By crashing the machine with a clearly identifiable kernel-mode crash dump, both users and administrators can easily see that an unwanted behavior has been operating inside of their kernel and they can take action. Additionally, it means that legitimate software vendors will not want to take the risk of crashing their customers' systems and will find supported ways of extending kernel functionality (such as by using the filter manager for file system filters or other callback-based mechanisms).
- Obfuscation (which is not a security boundary) can make it costly—either in time or in complexity—for the unwanted behavior to disable the detection mechanism. This added cost means that the unwanted behavior is more clearly identified as potentially malicious, and that its complexity results in additional costs to a potential attacker. By shifting the obfuscation techniques, it means that legitimate vendors will be better off taking the time to move away from their legacy extension

- mechanisms and implement supported techniques instead, without the risk of looking like malware.
- Randomization and non-documentation of which specific checks the detection/prevention mechanism makes to monitor kernel integrity, and non-determinism of when such checks are executed, cripple the ability of attackers to ensure their exploits are reliable. It forces them to account for every possible non-deterministic variable and state transition that the mechanism has through static analysis, which obfuscation makes nearly impossible within the timeframe required before another obfuscation shift or feature change is implemented in the mechanism.
 - Because kernel mode crash dumps are automatically submitted to Microsoft, it allows the company to receive telemetry of in-the-wild unwanted code, and to either identify software vendors whose code is unsupported and is crashing systems, or to track the progress of malicious drivers in the wild, or even zero-day kernel-mode exploitations, and fix bugs that may not have been reported, but are actively exploited.

PatchGuard

Shortly after the release of 64-bit Windows for x64 and before a rich third-party ecosystem had developed, Microsoft saw an opportunity to preserve the stability of 64-bit Windows, and to add telemetry and exploit-crippling patch detection to the system, through a technology called *Kernel Patch Protection (KPP)*, also referred to as *PatchGuard*. When Windows Mobile was released, which operates on a 32-bit ARM processor core, the feature was ported to such systems, too, and it will be present in 64-bit ARM (AArch64) systems as well. Due to the existence of too many legacy 32-bit drivers that still use unsupported and dangerous hooking techniques, however, this mechanism is not enabled on such systems, even on Windows 10 operating systems. Fortunately, usage of 32-bit systems is almost coming to an end, and server versions of Windows no longer support this architecture at all.

Although both *Guard* and *Protection* imply that this mechanism will protect the system, it is important to realize that the only guard/protection offered is the crashing of the machine, which prevents further execution of the unwanted code. The mechanism does not *prevent* the attack in the first place, nor mitigate against it, nor undo it. Think of KPP as an Internet-connected video security system, or CCTV, with a loud alarm (the crash) inside the vault (the kernel), not as an impenetrable lock on the vault.

KPP has a variety of checks that it makes on protected systems, and documenting them all would both be impractical (due to the difficulty of static analysis) and valuable to potential attackers (reducing their research time). However, Microsoft does document certain checks, which we generalize in [Table 7-23](#). When, where, and how KPP makes these checks, and which specific functions or data structures are affected, is outside of the scope of this analysis.

Component	Legitimate Usage	Potential Unwanted Usage
Executable code in the kernel, its dependencies, and core drivers, as well as the Import Address Table (IAT) of these components	Standard Windows components key to operation of kernel-mode usage.	Patching code in these components can modify their behavior and introduce unwanted back doors to the system, hide data or unwanted communications from the system, as well as reduce the stability of the system, or even add additional vulnerabilities through buggy third-party code.
Global Descriptor Table (GDT)	CPU hardware protection for the implementation of ring privilege levels (ring 0 versus ring 3).	Modification of expected permissions and mappings between code and ring levels, allowing ring 3 code ring 0 access.
Interrupt Descriptor Table (IDT) or Interrupt Vector Table	Table read by the CPU to deliver interrupt vectors to the correct handling routine.	Hooking of keystrokes, network packets, paging mechanism, system calls, hypervisor communication, and more, which can be used for back-dooring, hiding malicious data or communications, or accidentally adding vulnerabilities through buggy third-party code.
System Service Descriptor Table (SSDT)	Table containing the array of pointers for each system call handler.	Hooking of all user-mode communications with the kernel. Same issues as above.
Critical CPU registers such as Control Registers, Vector Base Address Register, and Model Specific Registers	Used for system calls, virtualization, enabling CPU security features such as SMEP, and more.	Same as above, plus disabling of key CPU security features or hypervisor protection.
Various function pointers in the kernel	Used as indirect calls to various internal functionality.	Can be used to hook certain internal kernel operations, leading to back doors and/or instability.
Various global variables in the kernel	Used to configure various parts of the kernel, including certain security features.	Malicious code would disable these security features, such as through an exploit from user mode allowing arbitrary memory overwrites.
Process and module list	Used to show the user, in tools such as Task Manager, Process Explorer, and the Windows Debugger, which processes are active, and which drivers are loaded.	Malicious code can hide the existence of certain processes or drivers on the machine, making them invisible to the user and most applications such as security software.
Kernel stacks	Store function arguments, the call stack (where a function should return), and variables.	Operating on a non-standard kernel stack is often the sign of a return-oriented programming (ROP) exploit operating on a pivoted stack as part of the attack.
Window Manager, graphical system calls, callbacks, and more	Provides the GUI, GDI, and DirectX services.	Same hooking abilities as described earlier, but specifically targeting the graphics and window-management stack. Same issues as other types of hooks.
Object types	Definitions for the various objects (such as processes and files) that the system supports through the object manager.	Can be used as another hooking technique, which does not target indirect function pointers in binaries' data sections, nor patching code directly. Same issues.
Local APIC	Used to receive hardware interrupts on the processor, receive timer interrupts, and inter-processor interrupts (IPI).	Can be used to hook timer execution, IPIs, or interrupts, or as a way for persistent code to covertly maintain liveness on the machine, executing on a periodic basis.
Filter and third-party notification callbacks	Used by legitimate third-party security software (and Windows Defender) to receive notifications about system actions, and in some cases even block/defend against certain actions. Exists as the supported way to achieve much of what KPP prevents.	Could be used by malicious code to hook all the filterable operations, as well as maintain liveness on a machine, executing on a periodic basis.
Specialized configuration and flags	Various data structures, flags, and elements of legitimate components that provide security and/or mitigation guarantees to them.	Could be used by malicious code to bypass certain mitigations or violate certain guarantees or expectations that user-mode processes might have, such as unprotecting a protected process.
KPP engine itself	Code related to bug-checking the system during a KPP violation, executing the callbacks associated with KPP, and more.	By modifying certain parts of the system used by KPP, unwanted components could attempt to silence, ignore, or otherwise cripple KPP.

TABLE 7-23 Generalized description of elements protected by KPP

As mentioned, when KPP detects unwanted code on the system, it crashes the system with an easily identifiable code. This corresponds to bugcheck code `0x109`, which stands for `CRITICAL_STRUCTURE_CORRUPTION`, and the Windows Debugger can be used to analyze this crash dump. (See Chapter 15, “Crash dump analysis,” in Part 2 for more information.) The dump information will contain *some* information about the corrupted or scrumptiously modified part of the kernel, but any additional data must be analyzed by Microsoft’s Online Crash Analysis (OCA) and/or Windows Error Reporting (WER) teams and is not exposed to users.

For third-party developers who use techniques that KPP deters, the following supported techniques can be used:

- **File system (mini) filters** Use these to hook all file operations, including loading image files and DLLs, that can be intercepted to purge malicious code on-the-fly or block reading of known bad executables or DLLs. (See Chapter 13, “File systems,” in Part 2 for more information on these.)
- **Registry filter notifications** Use these to hook all registry operations. (See Chapter 9 in Part 2 for more information on these notifications.) Security software can block modification of critical parts of the registry, as well as heuristically determine malicious software by registry access patterns or known bad registry keys.
- **Process notifications** Security software can monitor the execution and termination of all processes and threads on the system, as well as DLLs being loaded or unloaded. With the enhanced notifications added for antivirus and other security vendors, they also can block process launch. (See [Chapter 3](#) for more information on these notifications.)
- **Object manager filtering** Security software can remove certain access rights being granted to processes and/or threads to defend their own utilities against certain operations. (These are discussed in Chapter 8 in Part 2.)
- **NDIS Lightweight Filters (LWF) and Windows Filtering Platform (WFP) filters** Security software can intercept all socket operations (accept, listen, connect, close, and so on) and even the packets themselves. With LWF, security vendors have access to the raw Ethernet frame data that is going from the network card (NIC) to the wire.
- **Event Tracing for Windows (ETW)** Through ETW, many types of operations that have interesting security properties can be consumed by a user-mode component, which can then react to data in near real-time. In certain cases, special secure ETW notifications are available to anti-malware-protected processes under NDA with Microsoft and participation in various security programs, which give access to a greater set of tracing data. (ETW is discussed in Chapter 8 in Part 2.)

HyperGuard

On systems that run with virtualization-based security (described earlier in this chapter in the section “[Virtualization-based security](#)”), it is no longer true that attackers with kernel-mode privileges are essentially running at the same security boundary as a detection/prevention mechanism. In fact, such attackers would operate at VTL 0, while a mechanism could be implemented in VTL 1. In the Anniversary Update of Windows 10 (version 1607), such a mechanism does indeed exist, which is appropriately named *HyperGuard*. HyperGuard has a few interesting properties that set it apart from PatchGuard:

- It does not need to rely on obfuscation. The symbol files and function names that implement HyperGuard are available for anyone to see, and the code is not obfuscated. Complete static analysis is possible. This is because HyperGuard is a true security boundary.

- It does not need to operate non-deterministically because this would provide no advantage due to the preceding property. In fact, by operating deterministically, HyperGuard can crash the system at the precise time unwanted behavior is detected. This means crash data will contain clear and actionable data for the administrator (and Microsoft's analysis teams), such as the kernel stack, which will show the code that performed the undesirable behavior.
- Due to the preceding property, it can detect a wider variety of attacks, because the malicious code does not have the chance to restore a value back to its correct value during a precise time window, which is an unfortunate side-effect of PatchGuard's non-determinism.

HyperGuard is also used to extend PatchGuard's capabilities in certain ways, and to strengthen its ability to run undetected by attackers trying to disable it. When HyperGuard detects an inconsistency, it too will crash the system, albeit with a different code: 0x18C (HYPERGUARD_VIOLATION). As before, it might be valuable to understand, at a generic level, what kind of things HyperGuard will detect, which you can see in [Table 7-24](#).

Component	Legitimate Usage	Potential Unwanted Usage
Executable code in the kernel, its dependencies, and core drivers, as well as the Import Address Table (IAT) of these components	Refer to Table 7-23.	Refer to Table 7-23.
Global Descriptor Table (GDT)	Refer to Table 7-23.	Refer to Table 7-23.
Interrupt Descriptor Table (IDT) or Interrupt Vector Table	Refer to Table 7-23.	Refer to Table 7-23.
Critical CPU registers such as Control Registers, GDTR, IDTR, Vector Base Address Register, and Model Specific Registers	Refer to Table 7-23.	Refer to Table 7-23.
Executable code, callbacks, and data regions in the Secure Kernel and its dependencies, including HyperGuard itself	Standard Windows components key to operation of VTL 1 and secure kernel-mode usage.	Patching code in these components implies the attacker has access to some sort of vulnerability in VTL 1, either through hardware or the hypervisor. Could be used to subvert Device Guard, HyperGuard, and Credential Guard.
Structures and features used by Trustlets	Sharing data between one Trustlet to another, or Trustlets and the kernel, or Trustlets and VTL 0.	Implies that some vulnerability might exist in one or more Trustlets, which could be used to hamper features such as Credential Guard or Shielded Fabric/TPM.
Hypervisor structures and regions	Used by the hypervisor to communicate with VTL 1.	Implies a potential vulnerability in a VTL 1 component or the hypervisor itself, which may be accessible from ring 0 in VTL 0.
Kernel CFG bitmap	Used to identify valid kernel functions that are the subject of indirect function calls or jumps, as described earlier.	Implies that an attacker has been able to perform a modification to the VTL 1-protected KCFG bitmap through some sort of hardware or hypervisor exploit.
Page verification	Used to implement HVCI-related work for Device Guard.	Implies that an attacker has somehow attacked SKCI, which could result in Device Guard compromise or non-authorized IUM Trustlets.
NULL page	None.	Implies that an attacker has somehow coerced the kernel and/or secure kernel to allocate virtual page 0, which can be used to exploit NULL-page vulnerabilities in either VTL 0 or VTL 1.

TABLE 7-24 Generalized description of elements protected by HyperGuard

On systems with VBS enabled, there is another security-related feature that is worth describing, which is implemented in the hypervisor itself: Non-Privileged Instruction Execution Prevention (NPIEP). This mitigation targets specific x64 instructions that can be used to leak the kernel-mode addresses of the GDT, IDT, and LDT, which are SGDT, SIDT, and SLDT. With NPIEP, these instructions are still allowed to execute (due to compatibility concerns), but will return a per-processor unique number that is not actually the kernel address of these structures. This serves as a mitigation against Kernel ASLR (KASLR) information leaks from local attackers.

Finally, note that there is no way to disable PatchGuard or HyperGuard once they are enabled. However, because device-driver developers might need to make changes to a running system as part of debugging, PatchGuard is not enabled when the system boots in debugging mode with an active remote kernel-debugging connection. Similarly, HyperGuard is disabled if the hypervisor boots in debugging mode with a remote debugger attached.

Conclusion

Windows provides an extensive array of security functions that meet the key requirements of both government agencies and commercial installations. In this chapter, we've taken a brief tour of the internal components that are the basis of these security features. In Chapter 8 of Part 2, we'll look at various mechanisms that are spread out throughout the Windows system.